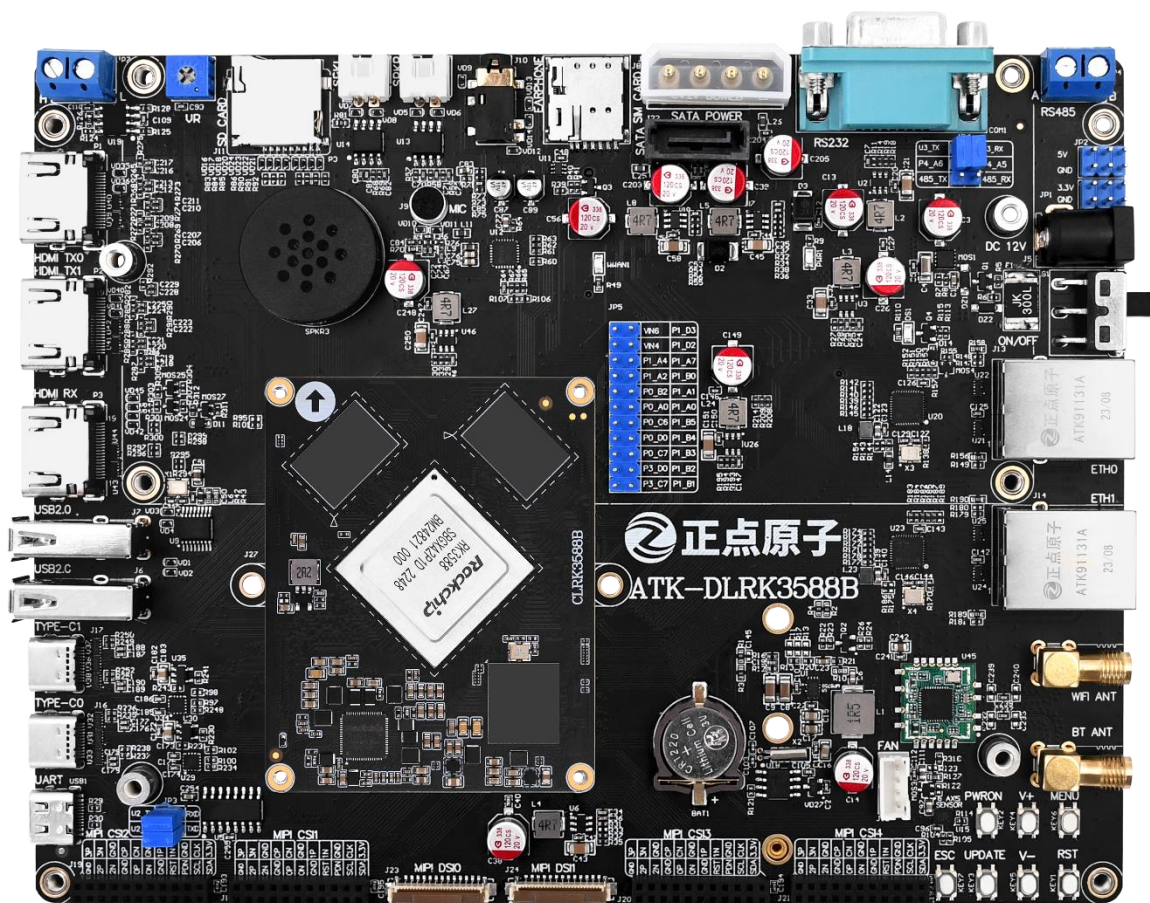


ATK-DLRK3588

Android 系统开发手册

V1.1

-正点原子 ATK-DLRK3588 开发板文档



修订历史:

版本	日期	修改内容
V1.0	2024/06/25	第一次发布
V1.1	2024/07/10	4.3 小节添加内容: 编译前选择屏幕



正点原子公司名称 : 广州市星翼电子科技有限公司
原子哥在线教学平台 : www.yuanzige.com
开源电子网 / 论坛 : www.openedv.com
正点原子官方网站 : www.alientek.com
正点原子淘宝店铺 : <https://openedv.taobao.com>
正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。
请关注正点原子公众号, 资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

前言	1
嵌入式开发环境概述	2
第一章 安装 Ubuntu 系统.....	3
1.1 安装 VMware 虚拟机软件	3
1.1.1 下载 VMware 软件	3
1.1.2 安装 VMware 软件	5
1.2 安装 Ubuntu 系统.....	11
1.2.1 下载 Ubuntu 系统镜像.....	11
1.2.2 创建虚拟机.....	13
1.2.3 安装 Ubuntu 系统.....	23
第二章 开发环境搭建.....	37
2.1 Ubuntu 系统设置	37
2.1.1 设置 root 用户密码	37
2.1.2 更换软件下载源.....	38
2.1.3 关闭自动更新.....	42
2.2 Ubuntu 与 Windows 之间文件互传	42
2.2.1 Ubuntu 系统下搭建 FTP 服务器.....	42
2.2.2 Windows 下安装 FTP 客户端.....	43
2.2.3 FileZilla 使用方法.....	47
2.3 Ubuntu 系统搭建 tftp 服务器	52
2.4 Ubuntu 系统搭建 nfs 服务器	53
2.5 Ubuntu 系统搭建 ssh 服务器	53
2.6 CH343 串口驱动安装	53
2.7 MobaXterm 软件安装	55
2.7.1 MobaXterm 软件下载	55
2.7.2 MobaXterm 软件安装	56
2.7.3 MobaXterm 软件的使用	60
2.8 Rockchip USB 驱动安装	66
2.9 镜像烧录工具的使用.....	67
2.9.1 烧写模式介绍.....	67
2.9.2 Windows 烧录工具 RKDevTool	68
2.9.3 update.img 镜像的烧录方法.....	73
2.9.4 擦除操作	74
2.9.5 Ubuntu 下 Linux_Upgrade_Tool 工具的使用	75
2.10 ADB 工具安装	76
第三章 正点原子 ATK-DLRK3588 平台简介	77
3.1 RK3588 简介	77
3.2 正点原子 ATK-DLRK3588 开发板硬件资源简介	77
第四章 RK3588 Android SDK 软件包	78
4.1 安装 RK3588 Android SDK	78
4.1.1 安装依赖软件包.....	78
4.1.2 安装 repo (跳过、无需安装)	78
4.1.3 Git 配置	79
4.1.4 安装 SDK	79



4.1.5 SDK 更新	81
4.1.6 SDK 问题反馈	81
4.2 SDK 软件架构介绍	81
4.2.1 Android SDK 工程目录介绍	82
4.2.2 SDK 软件框图	83
4.2.3 SDK 版本查询	86
4.3 SDK 全自动编译	86
4.4 镜像介绍	92
4.5 单独编译	93
4.5.1 单独编译 U-Boot	93
4.5.2 单独编译 Linux 内核	95
4.5.3 单独编译 Android 源码	98
4.5.4 制作 update.img 固件	100
4.6 SDK 清理	101
第五章 镜像烧录	102
5.1 烧写模式介绍	103
5.2 Windows 系统下烧写	103
5.2.1 分区表 parameter.txt 介绍	104
5.2.2 配置	105
5.2.3 烧录	106
5.2.4 启动系统	108
5.2.5 烧写 update.img	108
5.3 Ubuntu 系统下烧写	108
5.3.1 将 USB 设备连接到 Ubuntu	109
5.3.2 使用 upgrade_tool 工具烧写	111
5.3.3 烧写 update.img	113
5.3.4 擦除操作	114
第六章 Android SDK 开发	115
6.1 U-Boot 开发	116
6.1.1 U-Boot 的设备树	116
6.1.2 U-Boot 编译	116
6.1.3 U-Boot 的配置	118
6.1.4 快捷键	119
6.2 kernel 开发	119
6.2.1 Linux 内核编译	119
6.2.2 kernel 的配置	121
6.2.3 内核设备树和 defconfig 配置文件	122
6.2.5 替换 logo	123
6.2.6 内核模块开发文档汇总	124
6.3 Android 系统开发	126
6.3.1 Android 系统架构	126
6.3.2 Android 产品配置	129
6.3.3 Android 源码编译	140
6.3.4 关闭 SELinux	151
6.3.5 应用预安装功能	153
6.3.6 修改默认语言和时区信息	155
6.3.7 设置系统默认不锁屏、不息屏	156
6.3.8 修改默认背光亮度	156



6.3.9 修改启动 logo	156
6.3.10 修改开机动画.....	157
6.3.11 修改主屏/副屏.....	161
6.3.12 修改屏幕方向.....	166
6.3.13 添加自定义模块.....	170
6.3.14 获取 root 权限	172
6.3.15 多屏异显异触.....	173
6.3.16 多屏异声	173
6.3.17 多屏显示	173
6.4 Android 调试	173
6.4.1 ADB	173
6.4.2 logcat.....	173
6.5 Android 应用开发	175
6.7 安全启动/安全机制开发	175
6.8 A/B 系统开发	176
6.9 系统升级开发	176

前言

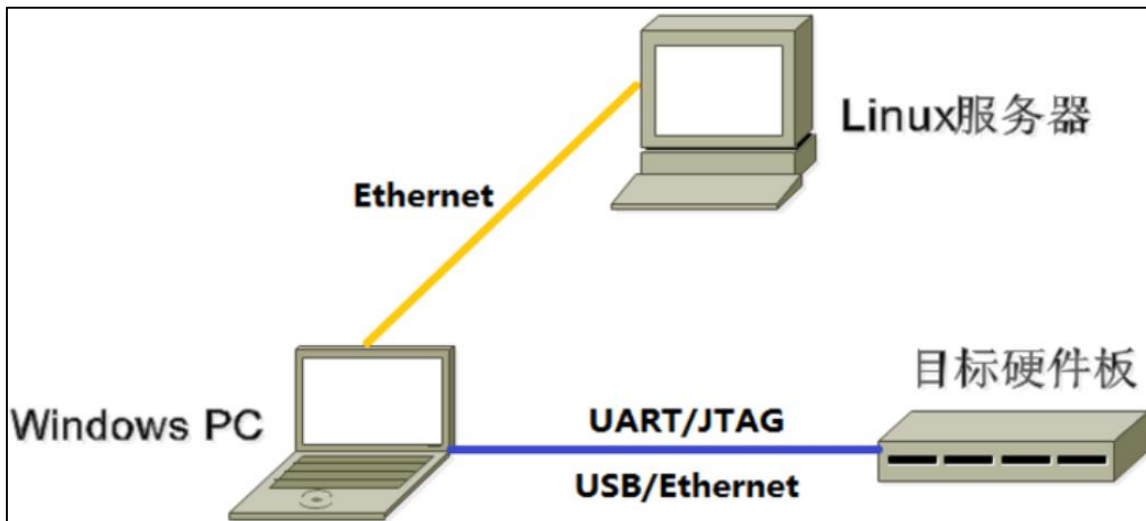
本文档主要记录了正点原子 ATK-DLRK3588 开发平台 Android 系统开发相关内容, 包括如下几个章节:

- 1, 安装 Ubuntu 操作系统;
- 2, 搭建开发环境;
- 3, 正点原子 ATK-DLRK3588 开发平台简介;
- 4, RK3588 Android SDK 软件包安装与使用;
- 5, SDK 镜像烧录;
- 6, SDK 开发。

本文档内容适用于 RK3588 平台 Android12 & Android13, 正点原子将向用户提供 ATK-DLRK3588 开发板对应的 Android12 以及 Android13 SDK。

嵌入式开发环境概述

一个典型的嵌入式开发环境通常包括 Linux 服务器、Windows PC 和目标硬件板，典型开发环境如下图所示：



- Linux 服务器上建立交叉编译环境，为软件开发提供代码更新下载、代码交叉编译服务。
- Windows PC 安装远程终端软件，譬如 Putty、SecureCRT 或 MobaXterm 等，通过网络远程登录（ssh 远程登录）到 Linux 服务器，进行交叉编译（将编译后生成的镜像文件通过网络拷贝到 Windows PC），及代码的开发调试。
- Windows PC 通过串口和 USB 与目标硬件板连接，可将编译后生成的镜像文件烧写到目标硬件板（一般都是通过 USB 或者 JTAG 烧录器进行烧录），并调试系统或应用程序。

◆ 虚拟机 Linux 系统+Windows PC+目标硬件板

对于很多开发者来说，可能并没有 Linux 服务器或者公司并未提供专用于代码交叉编译的 Linux 服务器，你只有一台 Windows 电脑；那么这个时候可以在 Windows PC 上安装虚拟机软件（譬如 VMware、Virtual Box 或 Virtual PC），使用虚拟机软件在 Windows PC 上创建虚拟机，虚拟机安装 Linux 操作系统；使用虚拟机 Linux 系统来代替 Linux 服务器。

现在很多芯片平台都支持直接在 Linux 系统环境下烧写镜像文件至目标硬件设备，譬如 Rockchip 平台；使用虚拟机 Linux 系统会更加方便，因为虚拟机 Linux 系统与 Windows 系统本就处于同一台实体机上，共享实体机的硬件资源；目标硬件设备通过串口和 USB 连接到 PC，Windows 系统能检测到设备接入，同样虚拟机 Linux 系统也能检测到设备接入（通常可以选择设备连接到 Windows 系统还是虚拟机 Linux 系统），所以可以直接在虚拟机 Linux 系统环境下烧写镜像文件至目标硬件设备，而不用将镜像文件从 Linux 系统拷贝至 Windows 系统、再由 Windows 系统烧写镜像文件，极大方便开发！

第一章 安装 Ubuntu 系统

嵌入式开发一般都是基于 Windows+Ubuntu 双系统开发环境,本章向大家介绍如何在虚拟机上安装 Ubuntu 操作系统。

本章将分为如下几个小节:

- 1.1 安装 VMware 虚拟机软件
- 1.2 安装 Ubuntu 系统

1.1 安装 VMware 虚拟机软件

VMware (Vmware Workstation) 是一款功能非常强大的虚拟机软件,通过该软件我们可以在 Windows 系统中创建一个或多个虚拟机。所谓虚拟机 (Virtual Machine), 它是指通过软件模拟出来的具有实体计算机完整硬件系统功能 (譬如内存、硬盘、处理器等) 的计算机系统。可以在虚拟机中安装实体机所支持的各种操作系统,譬如 Windows 操作系统、DOS、Linux 操作系统 (Ubuntu、Debian、CentOS 等),在实体计算机中能够完成的工作在虚拟机中都能够实现,可以像使用实体机一样对虚拟机进行操作。

目前流行的虚拟机软件有 VMware(Vmware Workstation)、Virtual Box 和 Virtual PC 等,本文档将向用户介绍如何通过 VMware 软件创建虚拟机并安装 Ubuntu 操作系统。

1.1.1 下载 VMware 软件

开发板资料包中已经给大家提供了 VMware 虚拟机软件的安装包文件,路径为: **开发板光盘 A 盘-基础资料→04、软件→VMware-workstation-full-16.2.1-18811642.exe**,如果用户不想浪费时间下载,那么可以直接使用该安装包文件,进入下一步 1.1.2 小节。

用户也可以通过链接地址: <https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html>, 下载当前最新版本的 VMware Workstation 安装包文件:



图 1.1.1.1 VMware 虚拟机软件最新版本安装包下载

上述链接地址只能下载当前最新版本的安装包文件,如果用户需要下载旧版本的安装包文件,可以通过链接地址: https://customerconnect.vmware.com/cn/downloads/#all_products, 进行下载:



图 1.1.1.2 旧版本 VMware 虚拟机软件安装包下载 1



图 1.1.1.3 旧版本 VMware 虚拟机软件安装包下载 2

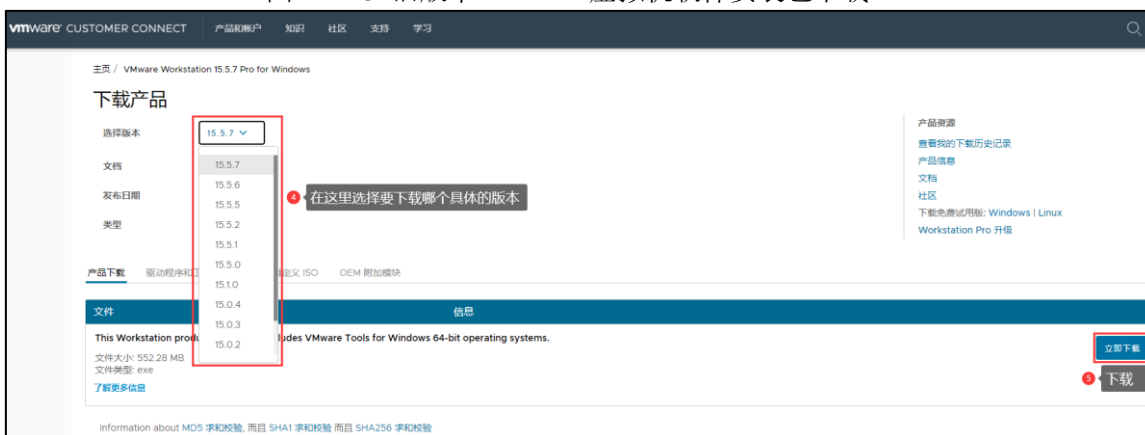


图 1.1.1.4 旧版本 VMware 虚拟机软件安装包下载 3

下载过程需要用户登录 VMware 账号，如果没有 VMware 账号、可以通过 VMware 官网 <https://www.vmware.com/cn.html> 进行注册。

对 VMware 虚拟机软件的版本没有什么要求，最新版本也行、旧的版本（譬如 15.X.X 或 14.X.X）也可以，本文档将以 16.2.1 版本为例，其对应的安装包文件名为 VMware-workstation-full-16.2.1-18811642.exe。

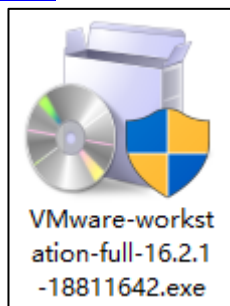


图 1.1.1.5 16.2.1 版本的 VMware 虚拟机软件安装包文件

1.1.2 安装 VMware 软件

Windows 下安装 VMware 软件非常简单, 直接双击运行安装包文件 VMware-workstation-full-16.2.1-18811642.exe, 按照图 1.1.2.1~1.1.2.8 所示操作步骤安装 VMware 虚拟机软件:

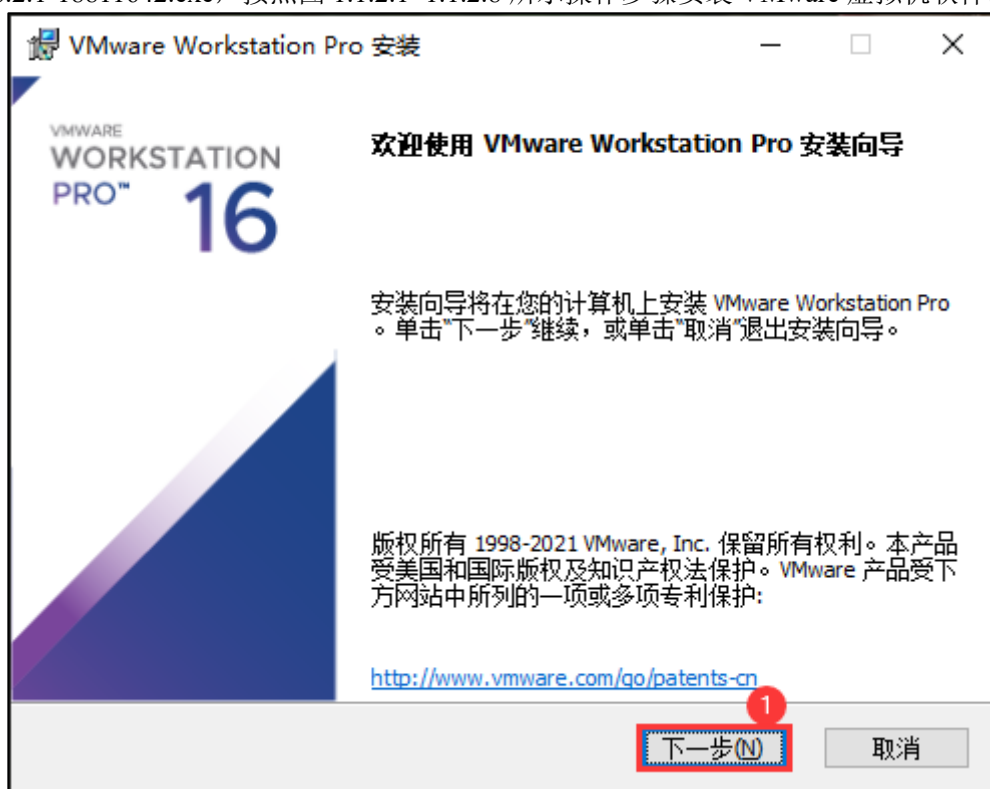


图 1.1.2.1 安装 VMware 软件 1

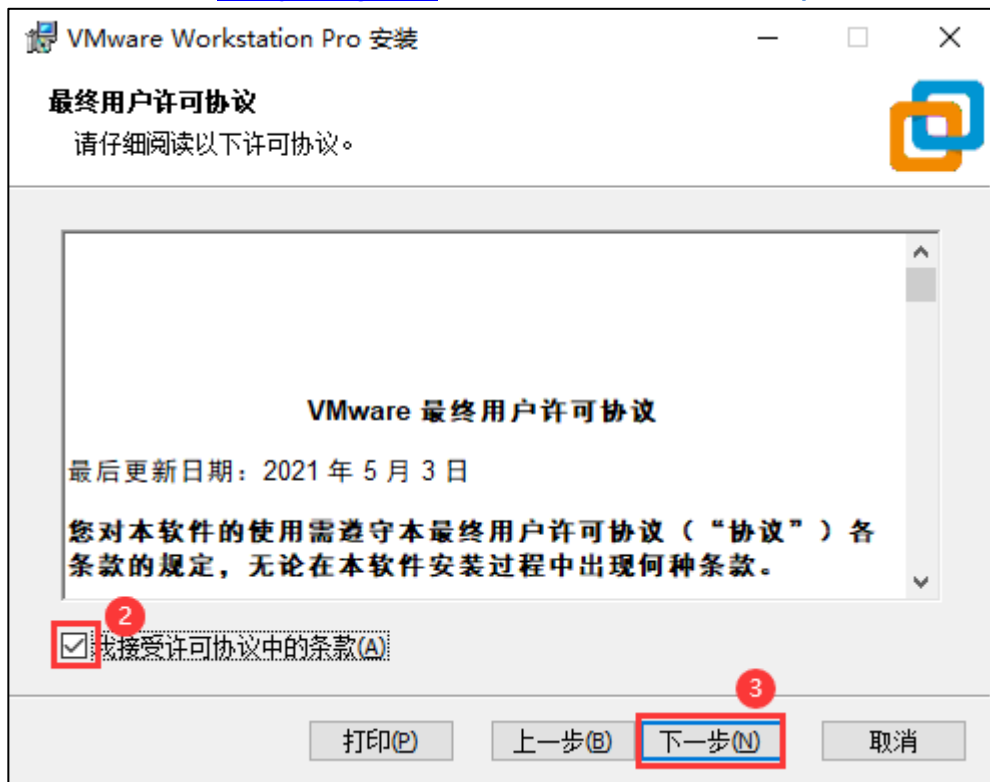


图 1.1.2.2 安装 VMware 软件 2

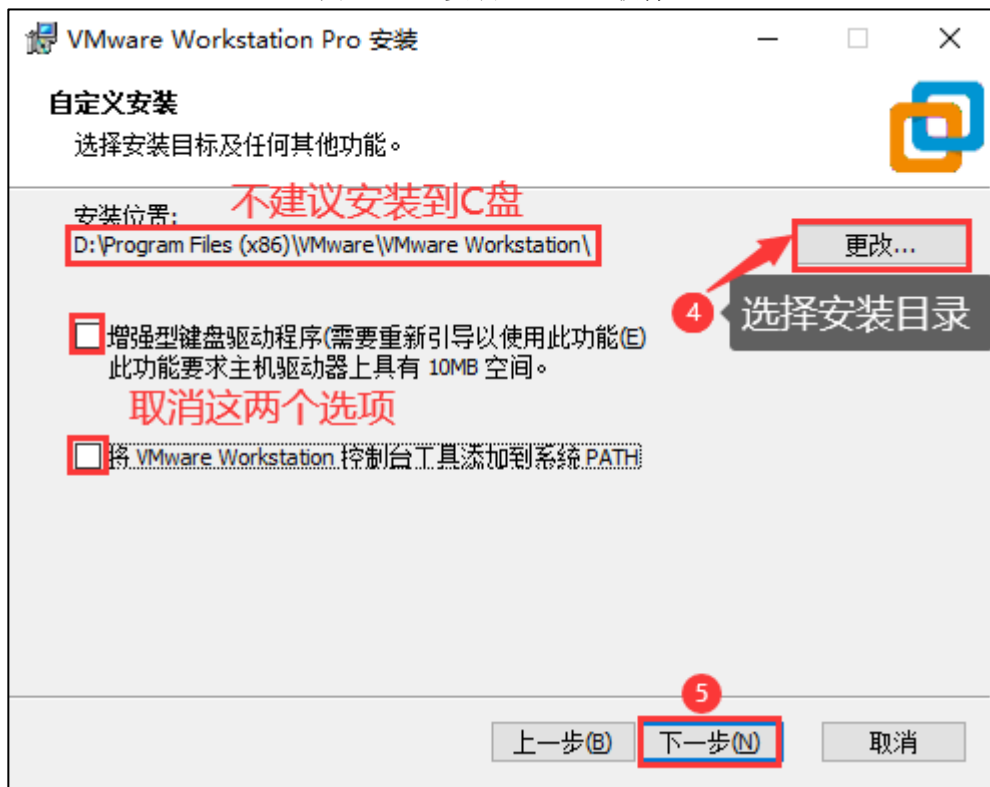


图 1.1.2.3 安装 VMware 软件 3

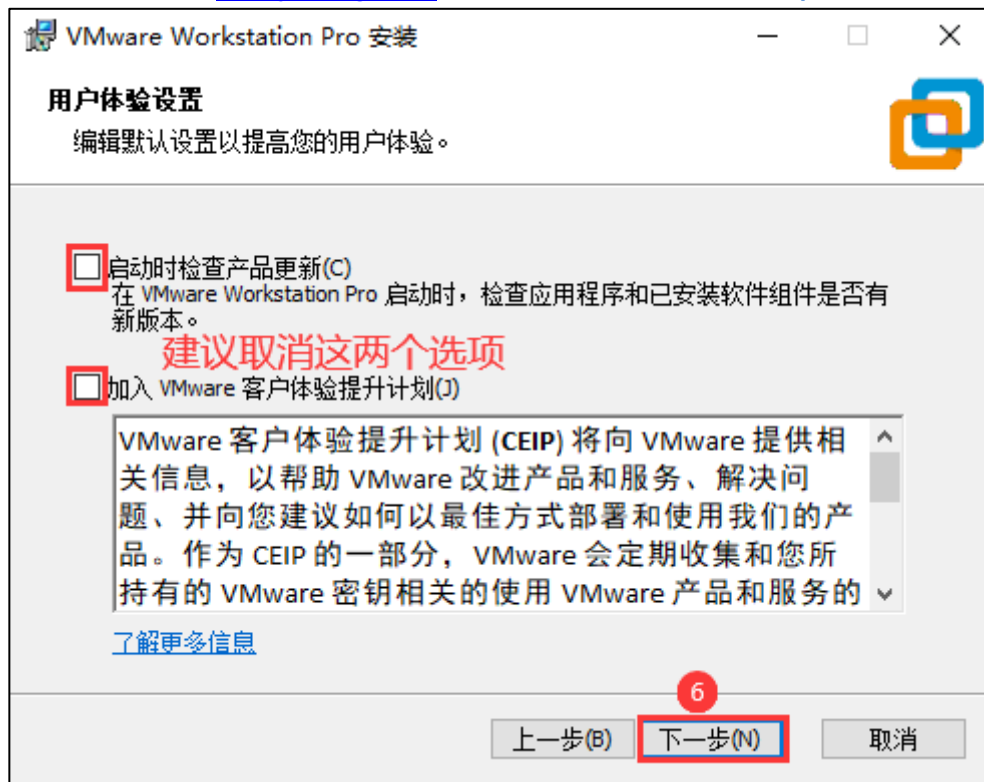


图 1.1.2.4 安装 VMware 软件 4

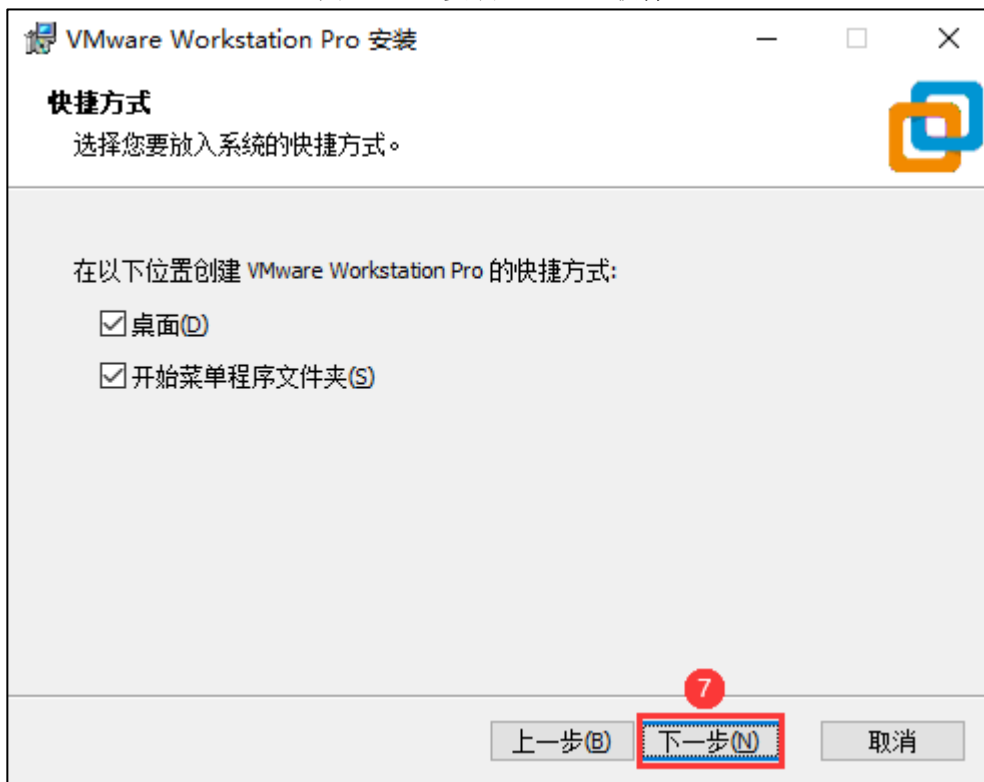


图 1.1.2.5 安装 VMware 软件 5

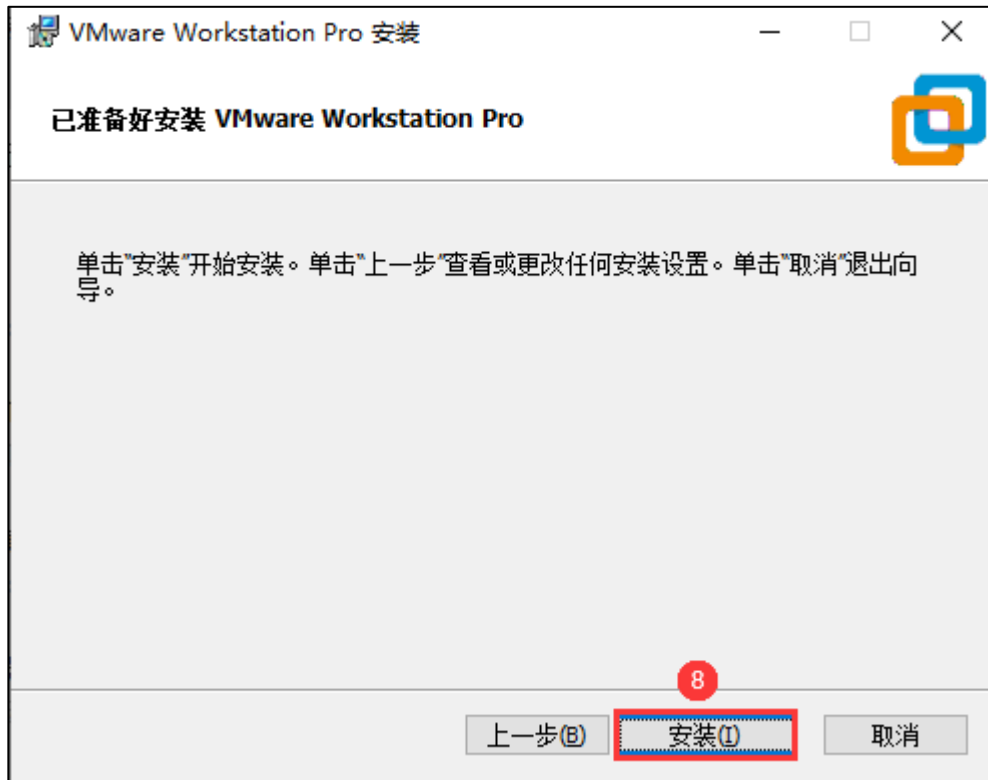


图 1.1.2.6 安装 VMware 软件 6

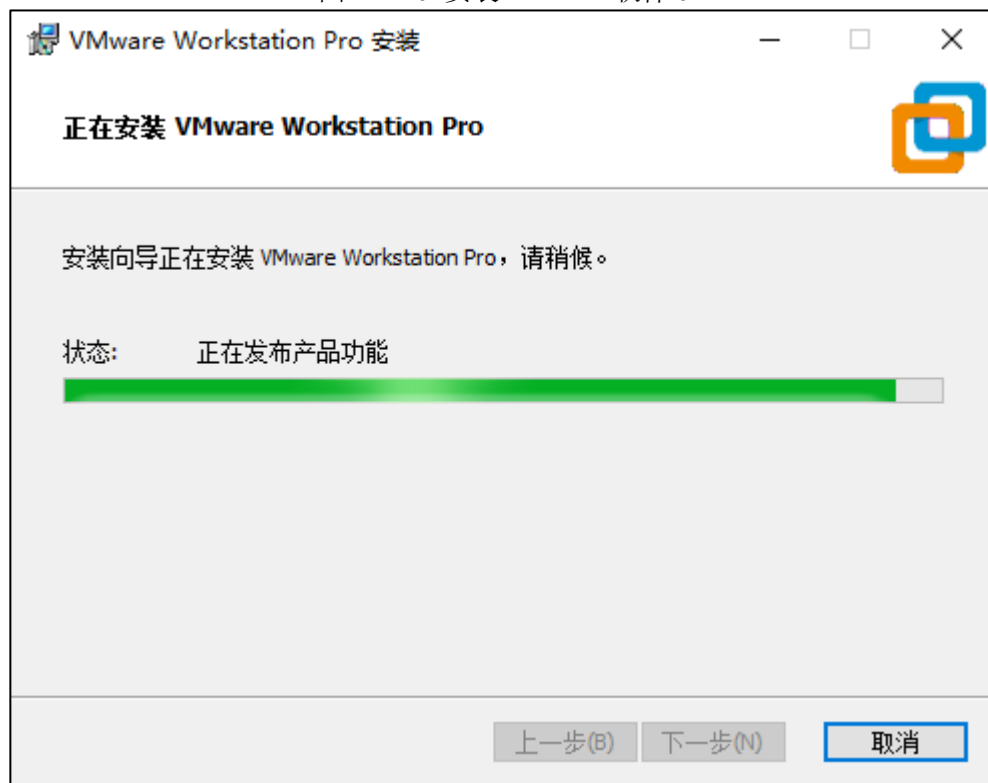


图 1.1.2.7 安装 VMware 软件 7



图 1.1.2.8 安装 VMware 软件 8

点击“完成”按钮，至此，VMware 虚拟机软件安装完成。

安装完成后，会在 Windows 系统桌面生成 VMware 虚拟机软件快捷方式图标，如图 1.1.2.9 所示：



图 1.1.2.9 VMware 软件桌面快捷方式图标

双击图标打开 VMware 虚拟机软件，首次打开软件会提示用户输入许可证密钥，如图 1.1.2.10 所示：



图 1.1.2.10 输入许可证密钥

VMware 虚拟机软件是付费软件，需要用户购买才能使用；如果用户购买了 VMware 软件使用许可，那么将会得到一串许可证密钥，将许可证密钥填写至“**我有 VMware Workstation 16 的许可证密钥(H)**”所对应的输入框中进行激活，激活成功后便可以正常使用 VMware 软件了。

如果用户没有购买软件使用许可，那么可以选择“**我希望使用 VMware Workstation 16 30 天(W)**”选项，这样便可获得 VMware 软件的 30 天试用期，点击“继续”按钮：



图 1.1.2.11 欢迎使用 VMware 软件

点击“完成”按钮，接着进入到 VMware 虚拟机软件主界面，如图 1.1.2.12 所示：

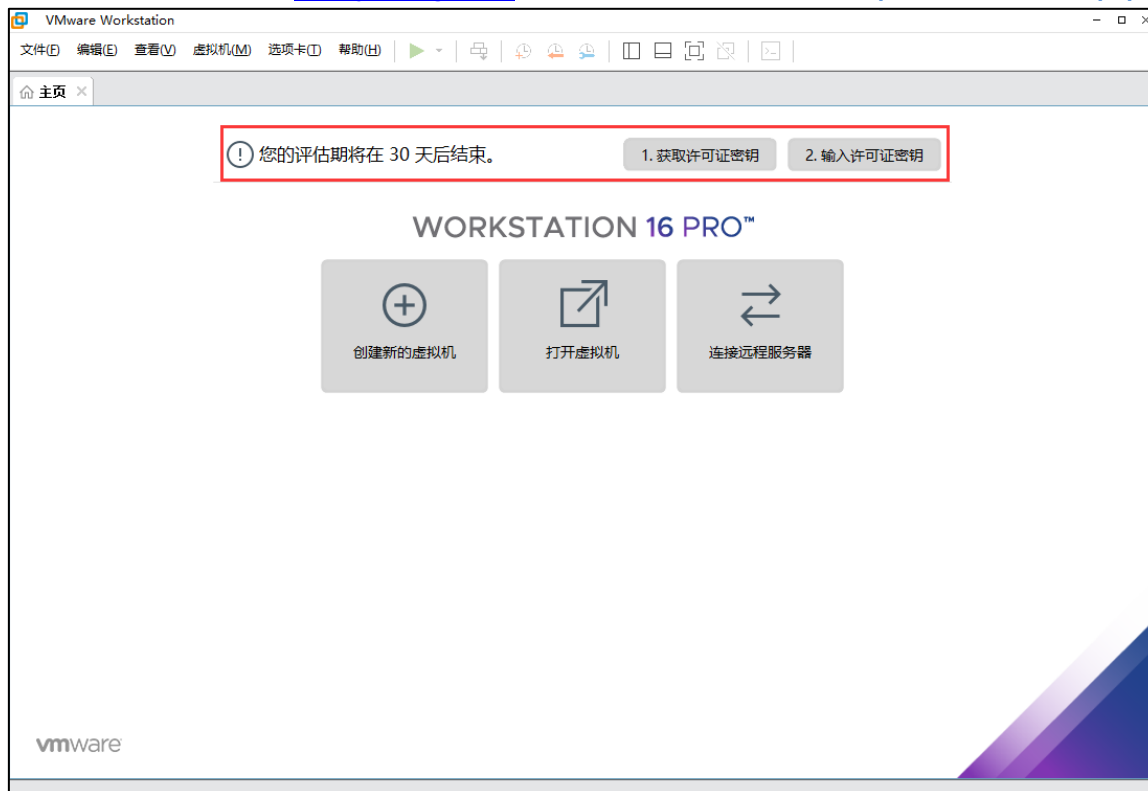


图 1.1.2.12 VMware 虚拟机软件主界面

1.2 安装 Ubuntu 系统

本小节向用户介绍如何在 VMware 虚拟机中安装 Ubuntu 操作系统。推荐用户使用 Ubuntu 20.04（**强烈建议大家使用 20.04 版本，我们提供的文档、视频，包括开发人员在开发过程中所使用的都是 Ubuntu 20.04 版本**）或者是 Ubuntu 18.04（**不推荐!**），本文档将以 Ubuntu 20.04 版本为例进行介绍，对于使用 Ubuntu 18.04 的用户，可对比参考本文档，如果后续使用过程中出现了任何问题，请自行解决！

1.2.1 下载 Ubuntu 系统镜像

开发板资料包中已经给用户提供了 Ubuntu 18.04 以及 Ubuntu 20.04 系统镜像文件，路径为：**开发板光盘 A 盘-基础资料→04、软件→ubuntu-18.04.6-desktop-amd64.iso** 和 **开发板光盘 A 盘-基础资料→04、软件→ubuntu-20.04.2-desktop-amd64.iso**。如果用户不想浪费时间下载，可以直接使用资料包中提供的 Ubuntu 系统镜像，进入下一步 1.2.2 小节。

用户也可以通过链接地址：<https://old-releases.ubuntu.com/releases/>，下载 Ubuntu 系统镜像（以 Ubuntu 20.04.2 版本为例）：

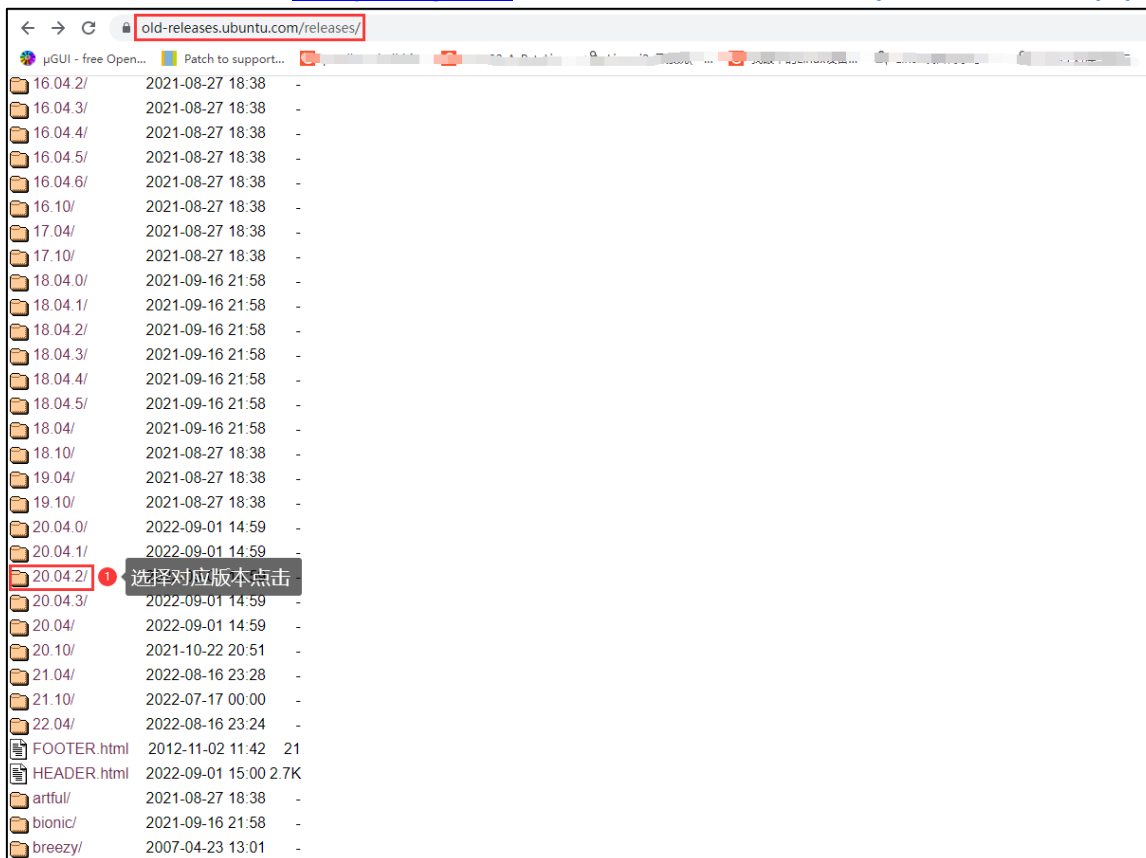


图 1.2.1.1 下载 Ubuntu 系统镜像 1

	ubuntu-20.04.1-live-server-s390x.manifest	2020-07-31 16:38	15K
	ubuntu-20.04.1-preinstalled-server-arm64+raspi.img.xz	2020-07-31 16:58	676M
	ubuntu-20.04.1-preinstalled-server-arm64+raspi.img.xz.zsync	2020-08-06 15:27	1.3M
	ubuntu-20.04.1-preinstalled-server-arm64+raspi.manifest	2020-07-31 16:58	15K
	ubuntu-20.04.1-preinstalled-server-armhf+raspi.img.xz	2020-07-31 17:17	646M
	ubuntu-20.04.1-preinstalled-server-armhf+raspi.img.xz.zsync	2020-08-06 15:27	1.3M
	ubuntu-20.04.1-preinstalled-server-armhf+raspi.manifest	2020-07-31 17:17	15K
	ubuntu-20.04.2-desktop-amd64.iso	2021-02-04 02:52	2.7G
	ubuntu-20.04.2-desktop-amd64.iso.torrent	2021-02-04 17:49	215K
	ubuntu-20.04.2-desktop-amd64.iso.zsync	2021-02-04 17:49	5.4M
	ubuntu-20.04.2-desktop-amd64.list	2021-02-04 02:52	26K
	ubuntu-20.04.2-desktop-amd64.manifest	2021-02-04 02:48	56K
	ubuntu-20.04.2-live-server-amd64.iso	2021-02-01 17:58	1.1G
	ubuntu-20.04.2-live-server-amd64.iso.torrent	2021-02-04 17:46	91K
	ubuntu-20.04.2-live-server-amd64.iso.zsync	2021-02-04 17:46	2.3M

图 1.2.1.2 下载 Ubuntu 系统镜像 2

下载完成之后便会得到 Ubuntu 20.04.2 系统镜像文件 ubuntu-20.04.2-desktop-amd64.iso:

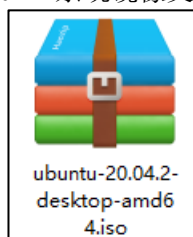


图 1.2.1.3 Ubuntu 20.04.2 系统镜像文件

1.2.2 创建虚拟机

打开 VMware 虚拟机软件，如图所示：

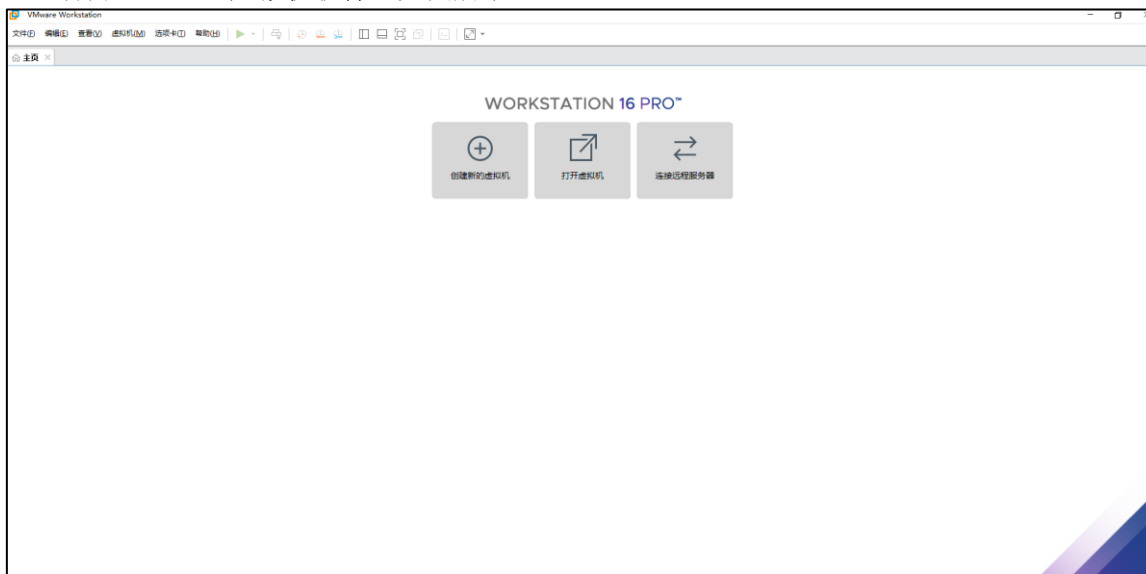


图 1.2.2.1 VMware 虚拟机软件

然后按照图 1.2.2.2~1.2.2.17 所示操作步骤创建一个新的虚拟机：

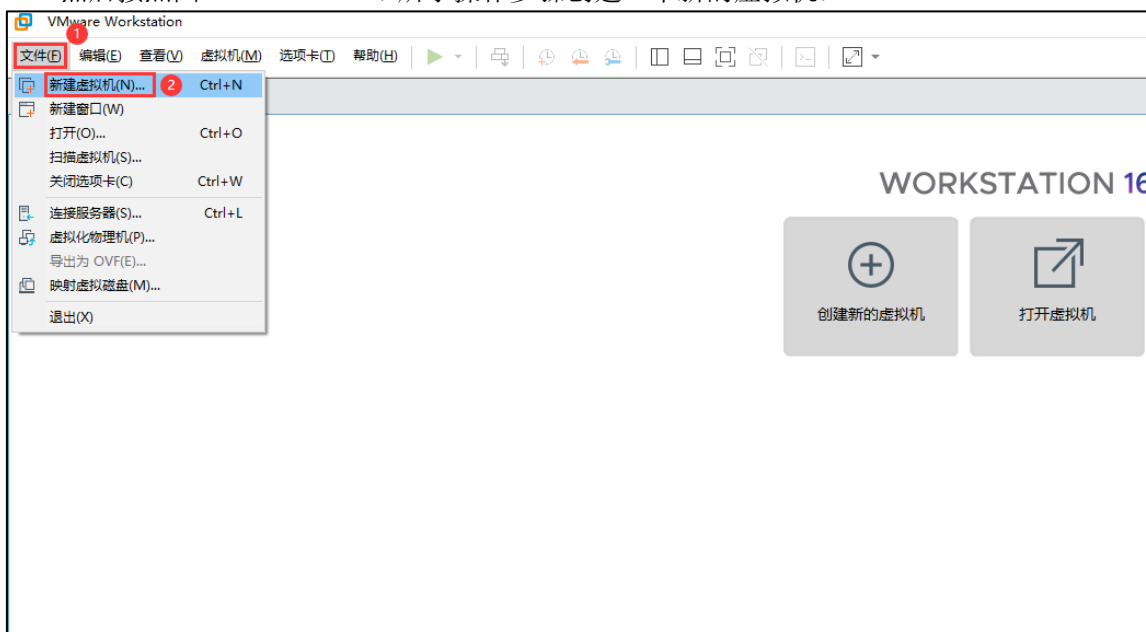


图 1.2.2.2 创建虚拟机(1)

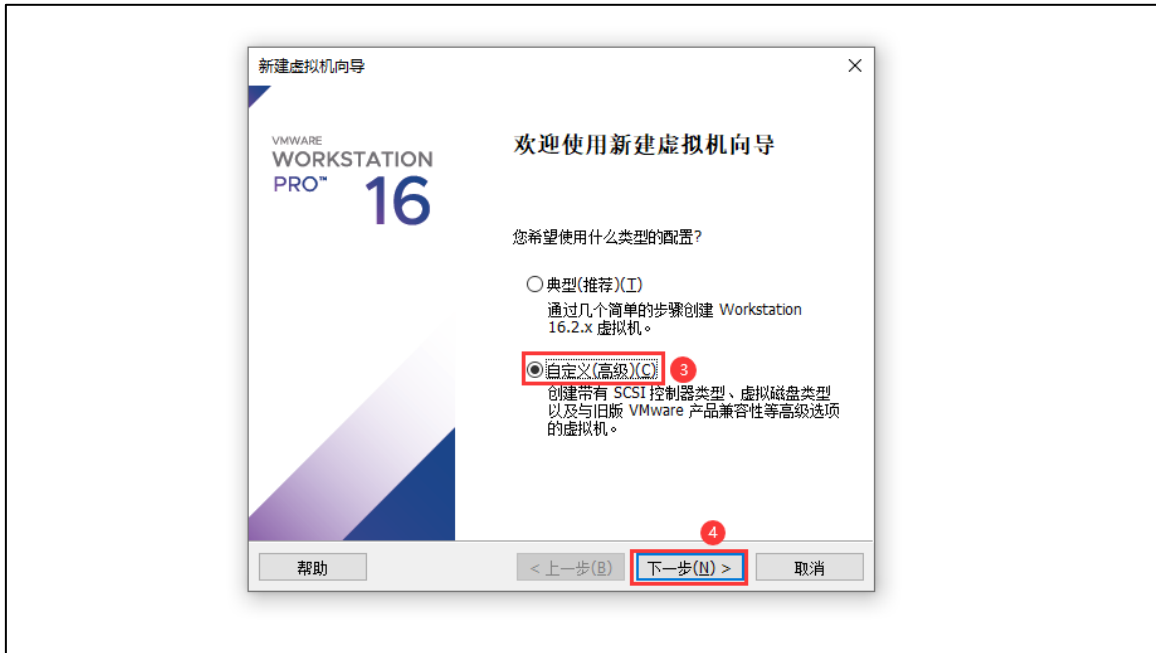


图 1.2.2.3 创建虚拟机(2)



图 1.2.2.4 创建虚拟机(3)

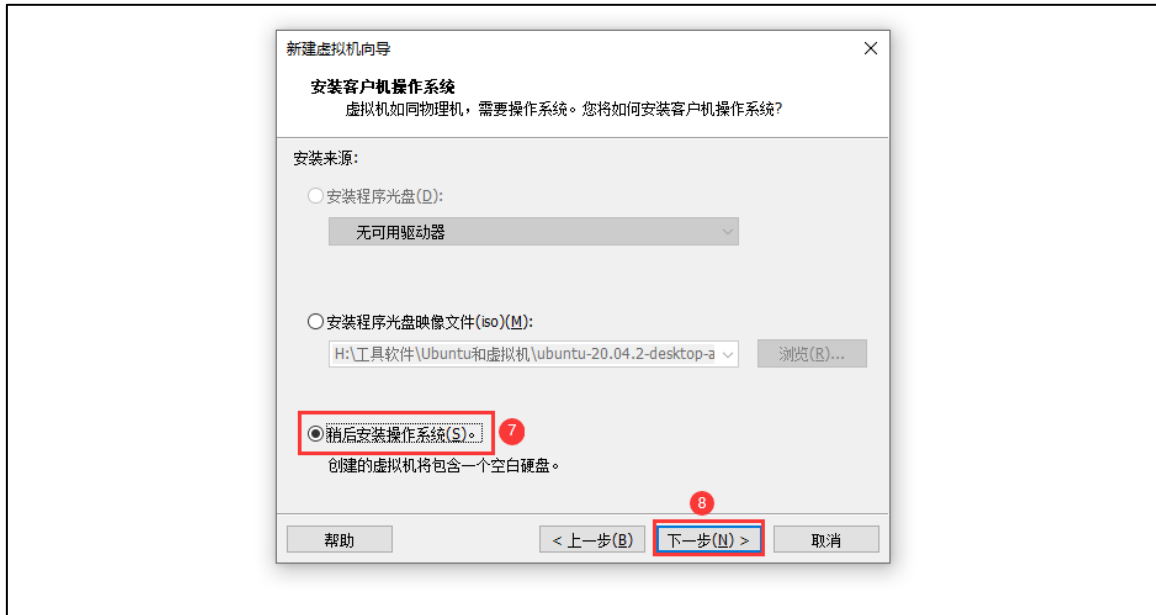


图 1.2.2.5 创建虚拟机(4)



图 1.2.2.6 创建虚拟机(5)

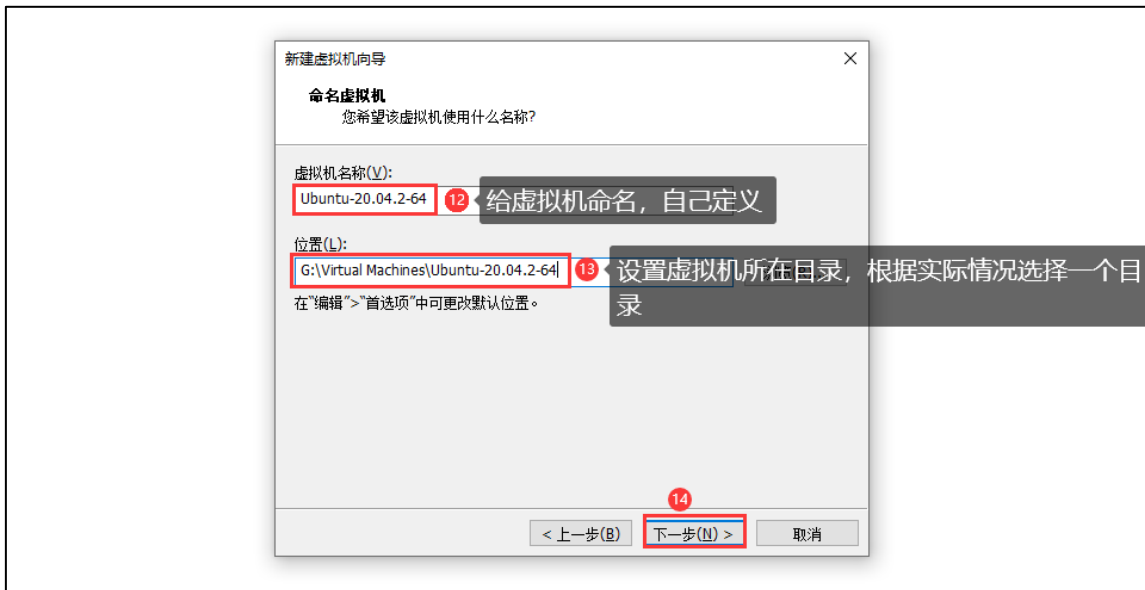


图 1.2.2.7 创建虚拟机(6)

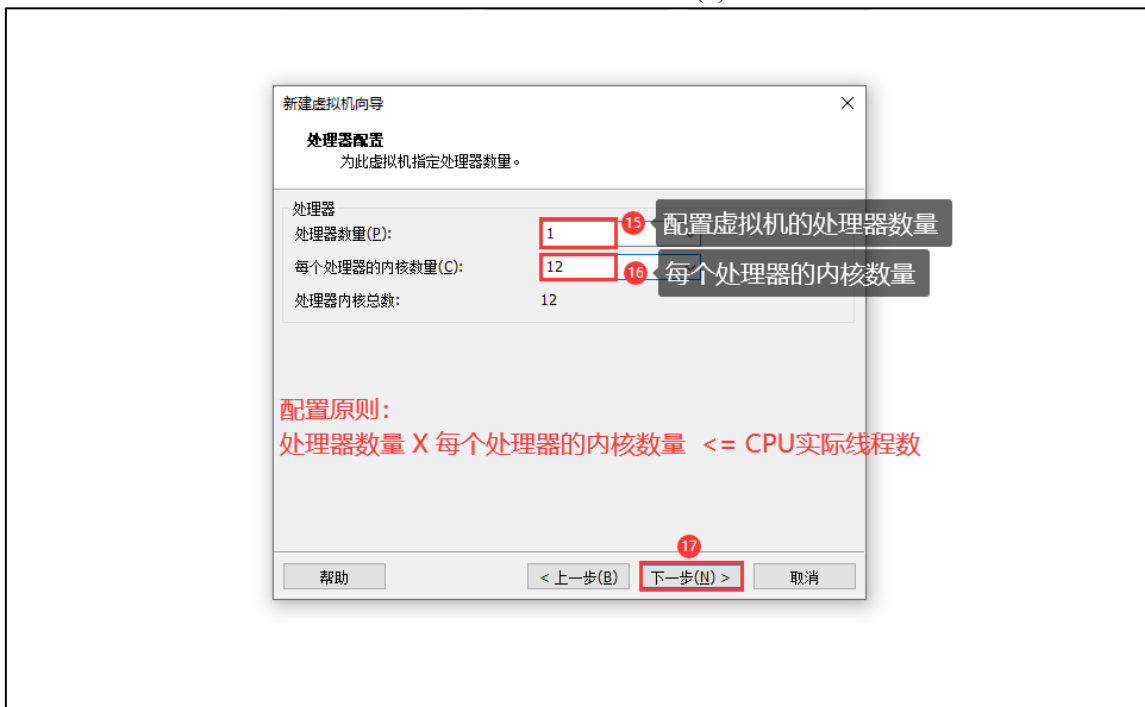


图 1.2.2.8 创建虚拟机(7)

为虚拟机配置处理器数量以及每个处理器的内核数量:

处理器内核总数 = 处理器数量 * 每个处理器的内核数量;

用户需根据自己的电脑配置 (CPU 配置) 情况来为虚拟机分配处理器数量以及每个处理器的内核数量, 只需满足如下要求:

虚拟机处理器内核总数小于或等于 ($\leq$) CPU 实际线程数;

可通过 Windows 任务管理器来查询电脑的 CPU 配置情况, 如图 1.2.2.9 所示:

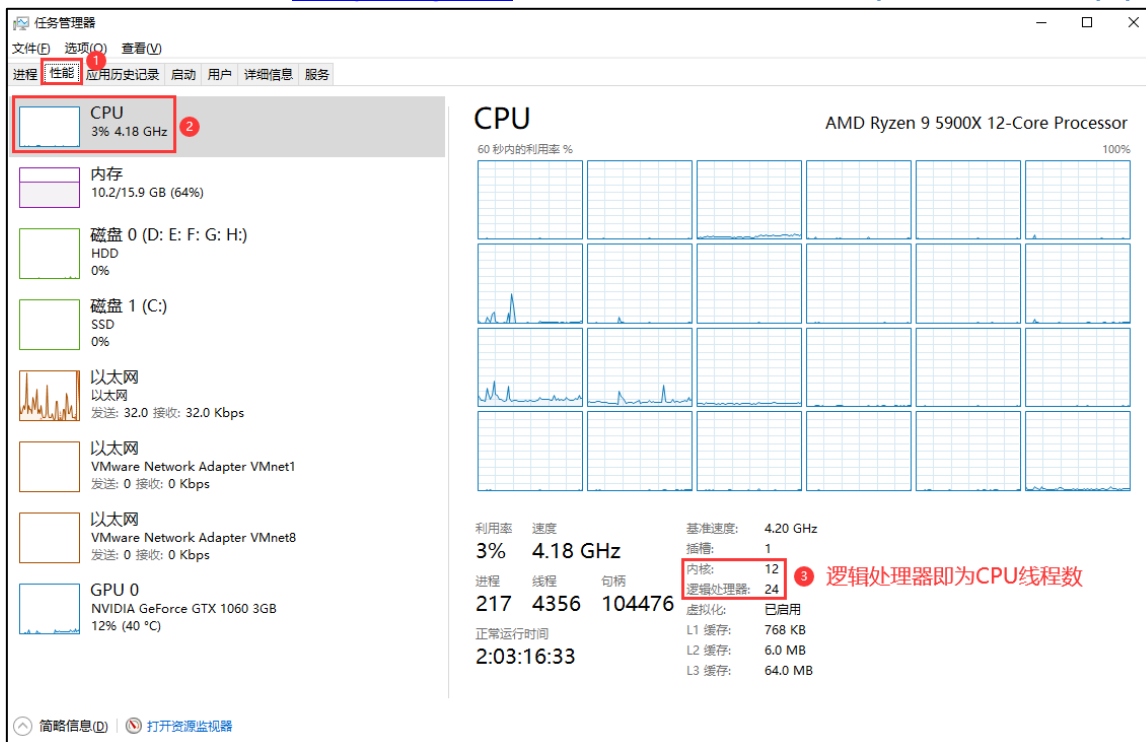


图 1.2.2.9 查询 CPU 配置情况

图中逻辑处理器数量即为 CPU 实际线程数，所以我的电脑 CPU 实际线程数为 24，按照配置要求，虚拟机的处理器内核总数需小于或等于 ($\leq$) CPU 实际线程数 24，那么我们可以有多种不同的配置方式，譬如：

- 处理器数量=1、每个处理器的内核数量=12;
- 处理器数量=1、每个处理器的内核数量=16;
- 处理器数量=1、每个处理器的内核数量=18;
- 处理器数量=4、每个处理器的内核数量=4;
- 处理器数量=2、每个处理器的内核数量=6;
- 处理器数量=6、每个处理器的内核数量=3;
- 处理器数量=8、每个处理器的内核数量=2;
- 等等.....

总之具体的配置方式多种多样，只需满足要求即可！

当然，在条件允许的情况下（譬如不影响实体机 Windows 系统的正常使用），尽可能使得虚拟机的处理器内核总数稍多一些，这样我们在虚拟机 Ubuntu 系统下编译源码时可以开启更多的线程进行编译，自然编译速度也会更快！

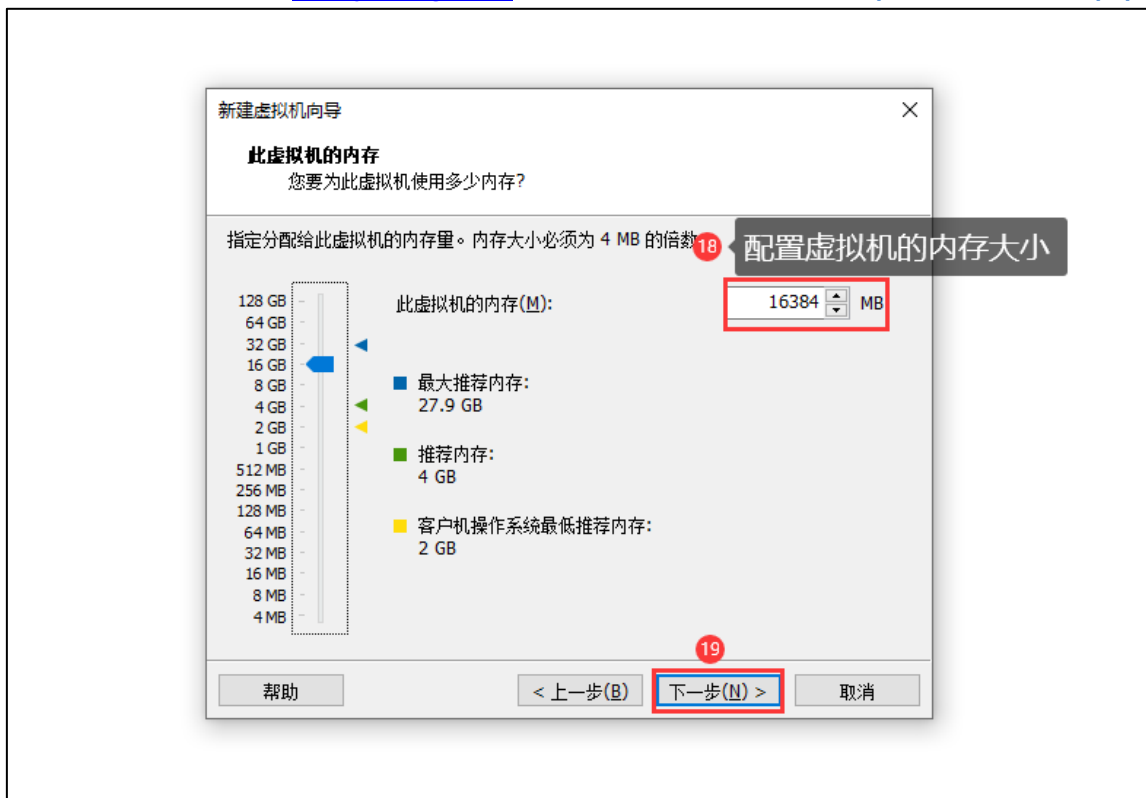


图 1.2.2.10 创建虚拟机(8)

可以通过上下拖动左边的蓝色滑块来调整虚拟机的内存大小，也可以直接在右边的输入框中填写数字来指定虚拟机的内存大小，单位是 MB。同样，虚拟机的内存大小也需要根据用户的电脑配置（内存配置）情况来设置，比如我的电脑内存大小为 32GB，因此我可以将 16GB 内存分配给虚拟机使用。

当然，分配给虚拟机的内存容量不能太小，如果内存太小，可能会导致虚拟机 Ubuntu 系统下编译源码时报错（由于内存不足而出错）；对于 RK3588 平台 Android 系统开发来说，建议分配给虚拟机的内存量至少大于或等于 16GB、甚至更大。

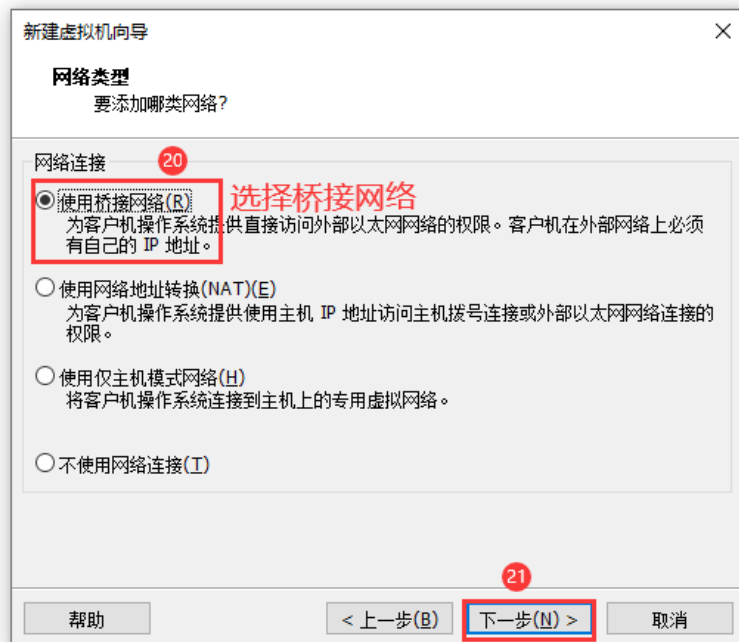


图 1.2.2.11 创建虚拟机(9)



图 1.2.2.12 创建虚拟机(10)



图 1.2.2.13 创建虚拟机(11)

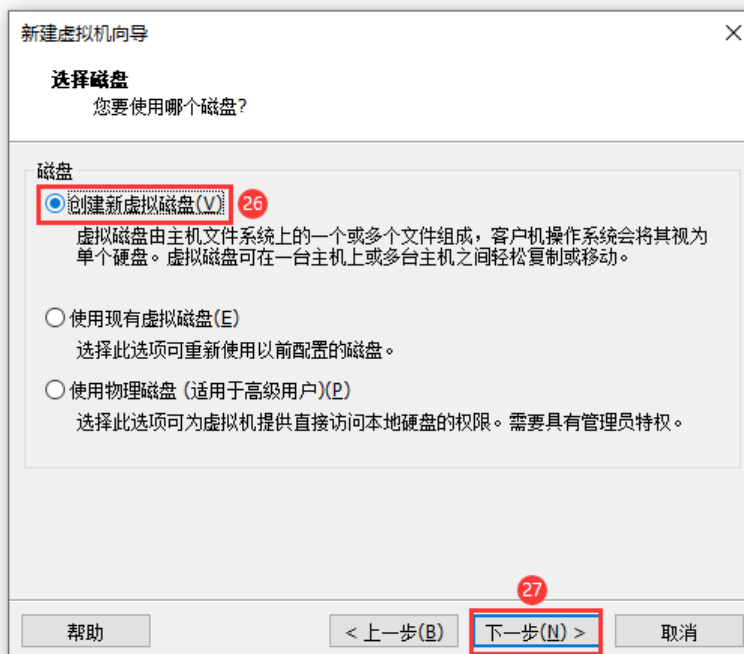


图 1.2.2.14 创建虚拟机(12)

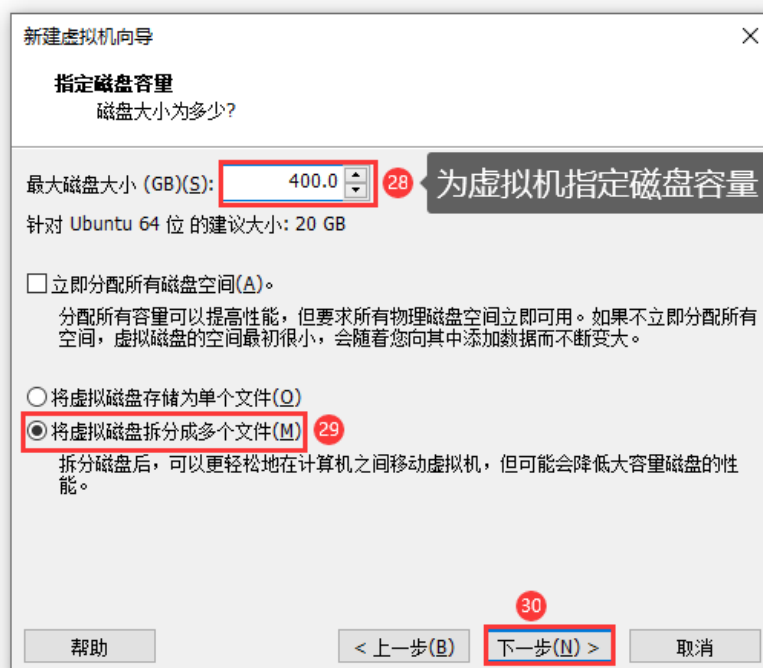


图 1.2.2.15 创建虚拟机(13)

同样，虚拟机磁盘容量也需要根据用户的电脑配置（磁盘配置）情况来设置，建议至少 300~500GB。

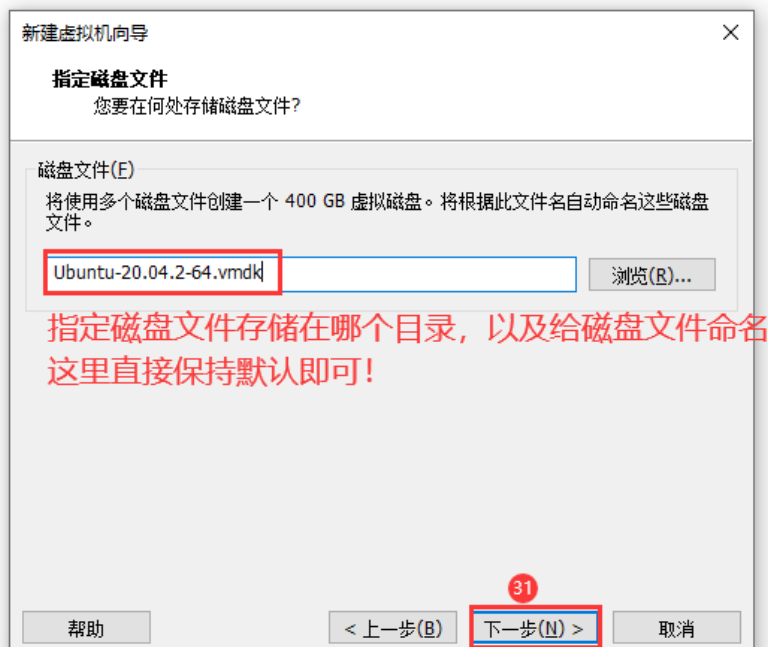


图 1.2.2.16 创建虚拟机(14)



图 1.2.2.17 创建虚拟机(15)

至此，虚拟机便创建完成，如图 1.2.2.18 所示：

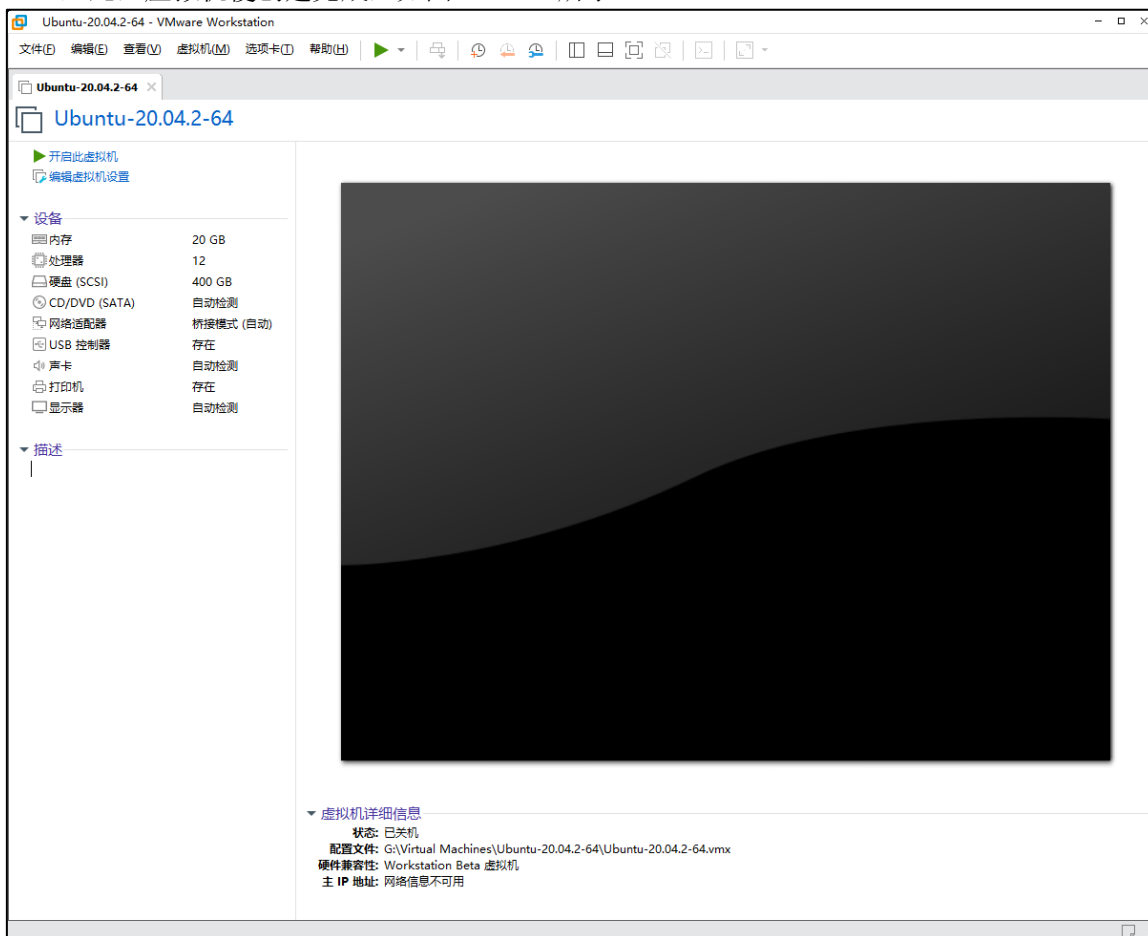


图 1.2.2.18 虚拟机创建完成

1.2.3 安装 Ubuntu 系统

接下来我们在虚拟机上安装 Ubuntu 20.04 操作系统，在创建好的虚拟机上点击“虚拟机(M)→设置”打开虚拟机设置界面：

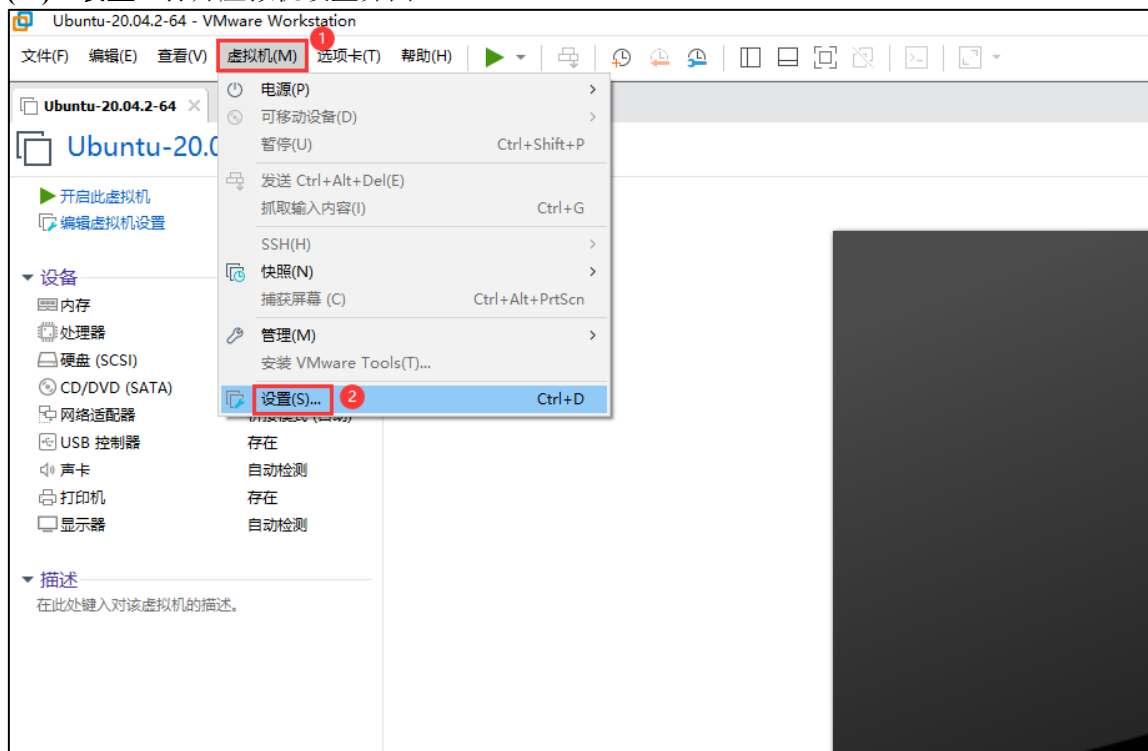


图 1.2.3.1 打开虚拟机设置界面

首先指定 Ubuntu 20.04 系统镜像文件 ubuntu-20.04.2-desktop-amd64.iso 所在路径：

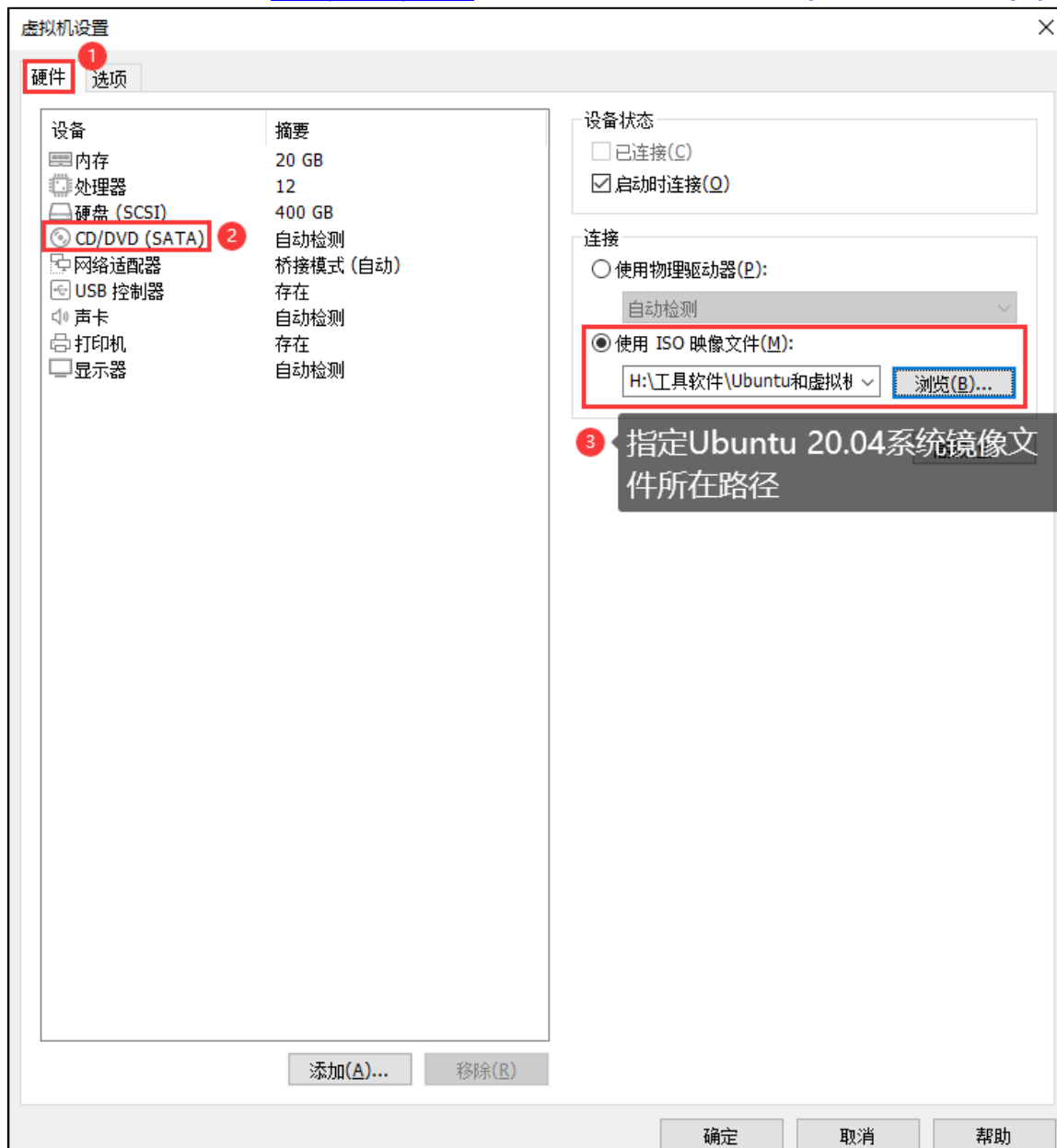


图 1.2.3.2 指定 Ubuntu 系统镜像

点击“浏览(B)”按钮，选择 Ubuntu 20.04 系统镜像文件 `ubuntu-20.04.2-desktop-amd64.iso`。
接下来对虚拟机 USB 控制器进行设置：



图 1.2.3.3 设置 USB 控制器兼容性

USB 控制器的 USB 兼容性默认为 USB 2.0, 当我们使用 USB 3.0 设备时, Ubuntu 系统可能识别不出来, 因此我们需要调整 USB 兼容性为 USB 3.1, 最后点击“确定”按钮退出窗口。

接下来开始安装 Ubuntu 操作系统, 按照图 1.2.3.4~1.2.3.13 所示步骤安装 Ubuntu 系统:

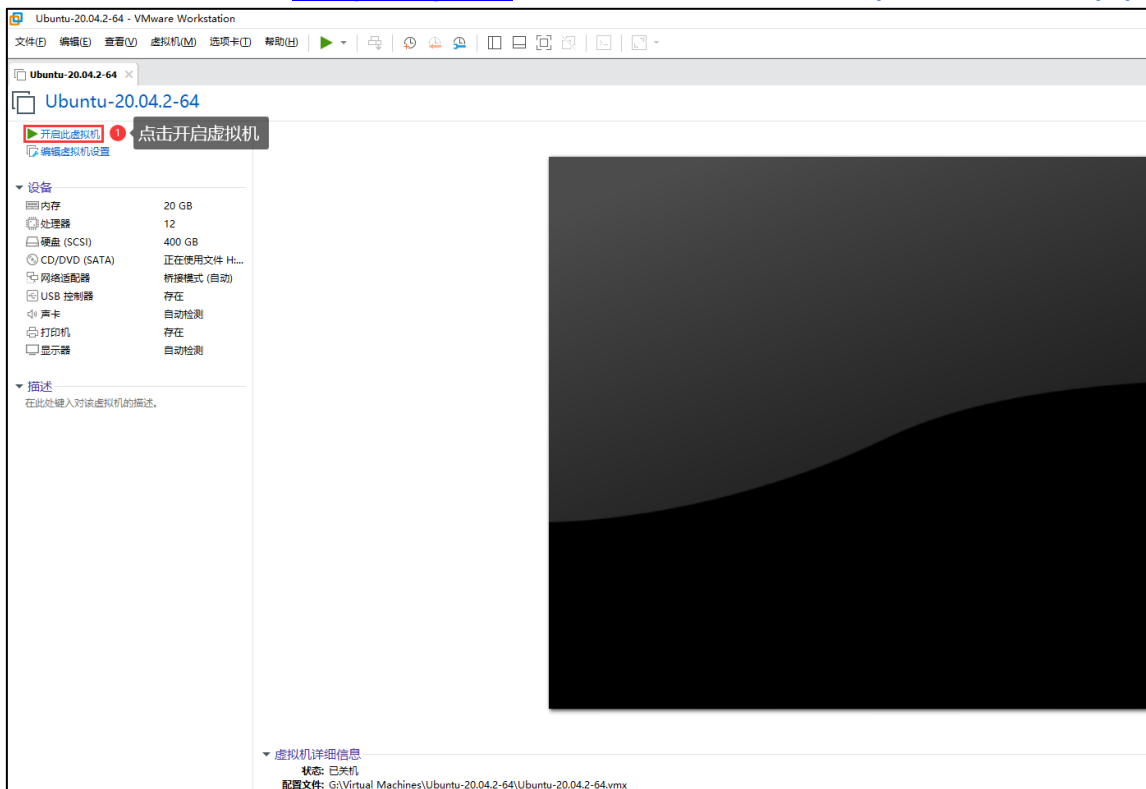


图 1.2.3.4 安装 Ubuntu 操作系统(1)

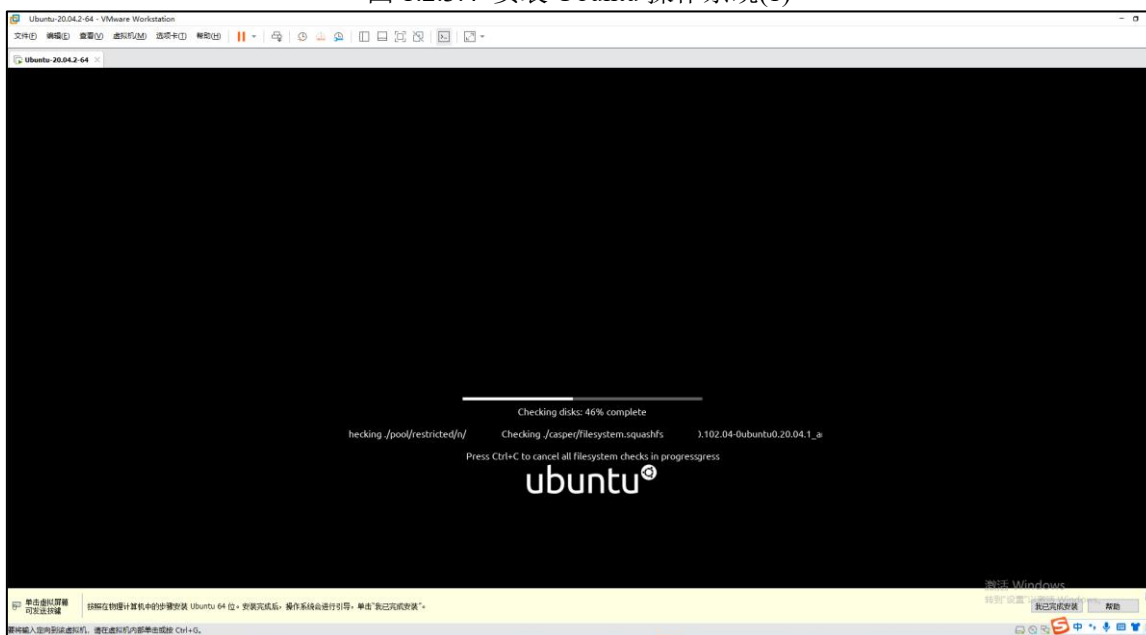


图 1.2.3.5 安装 Ubuntu 操作系统(2)



图 1.2.3.6 安装 Ubuntu 操作系统(3)

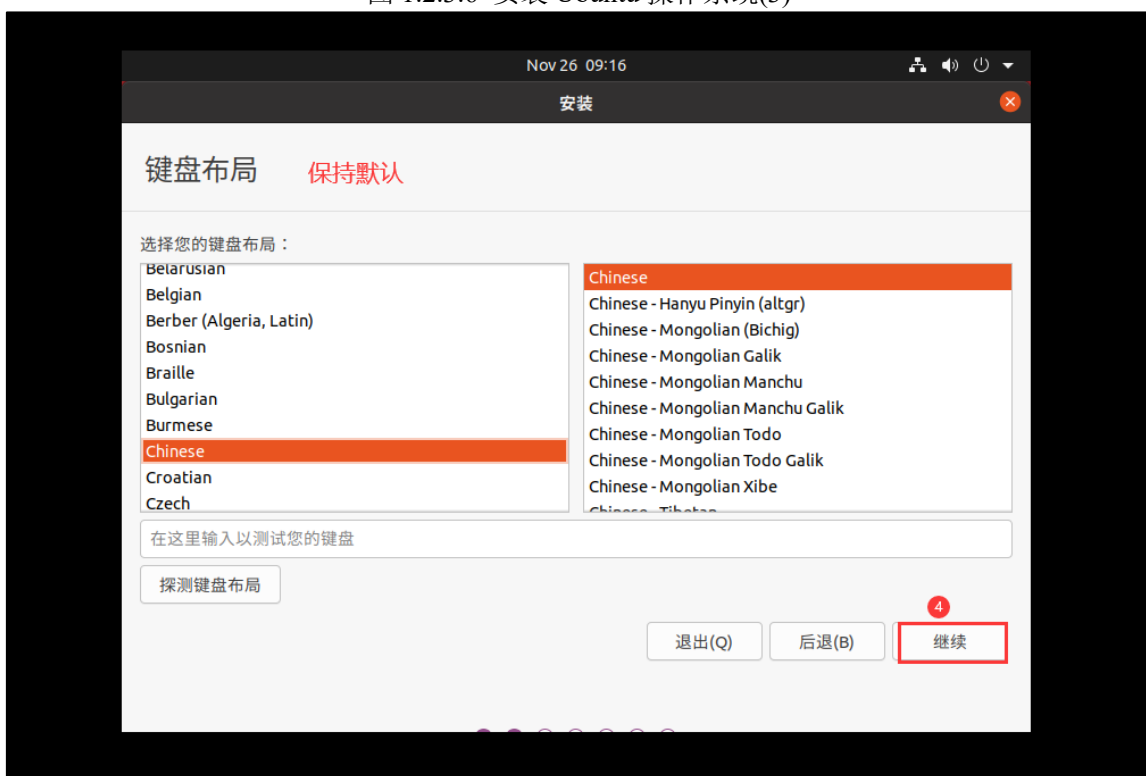


图 1.2.3.7 安装 Ubuntu 操作系统(4)

如果用户在安装过程中,由于窗口显示的问题,无法看到下面的三个按钮(退出、后退以及继续这三个按钮),导致无法点击按钮;这个时候可以按住 Win 键(Ctrl 键和 Alt 键中间的那个按键),然后使用鼠标左键拖动窗口。

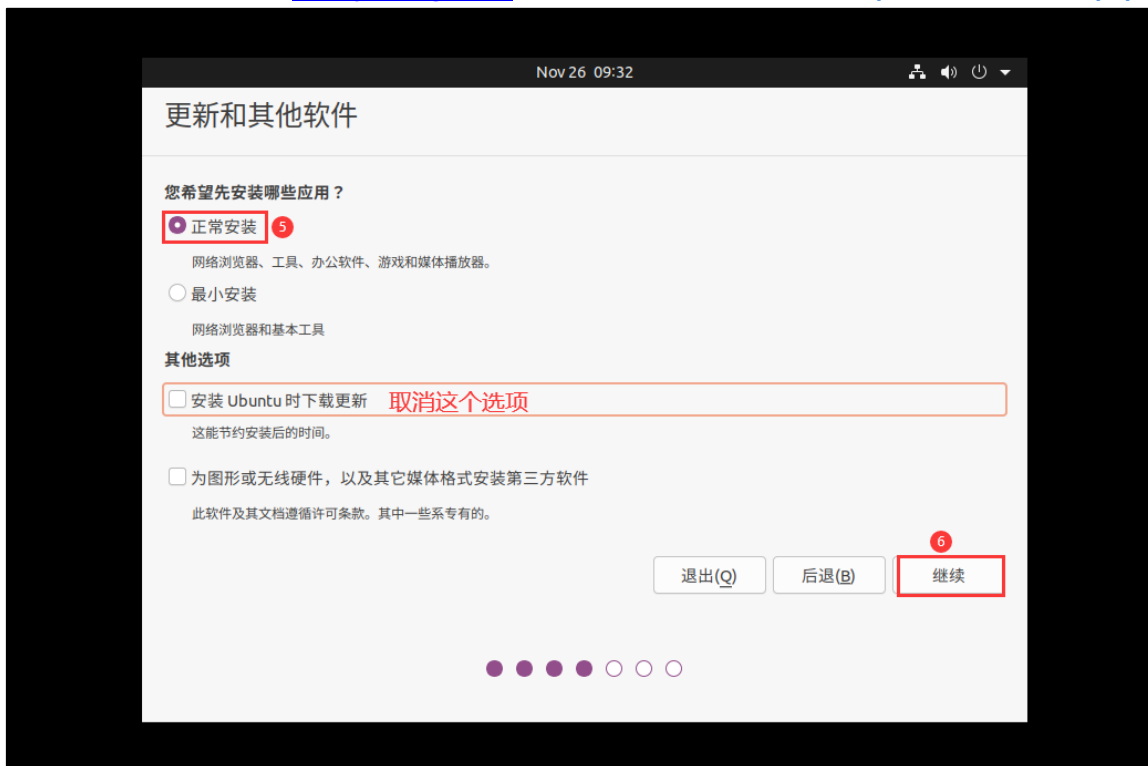


图 1.2.3.8 安装 Ubuntu 操作系统(5)

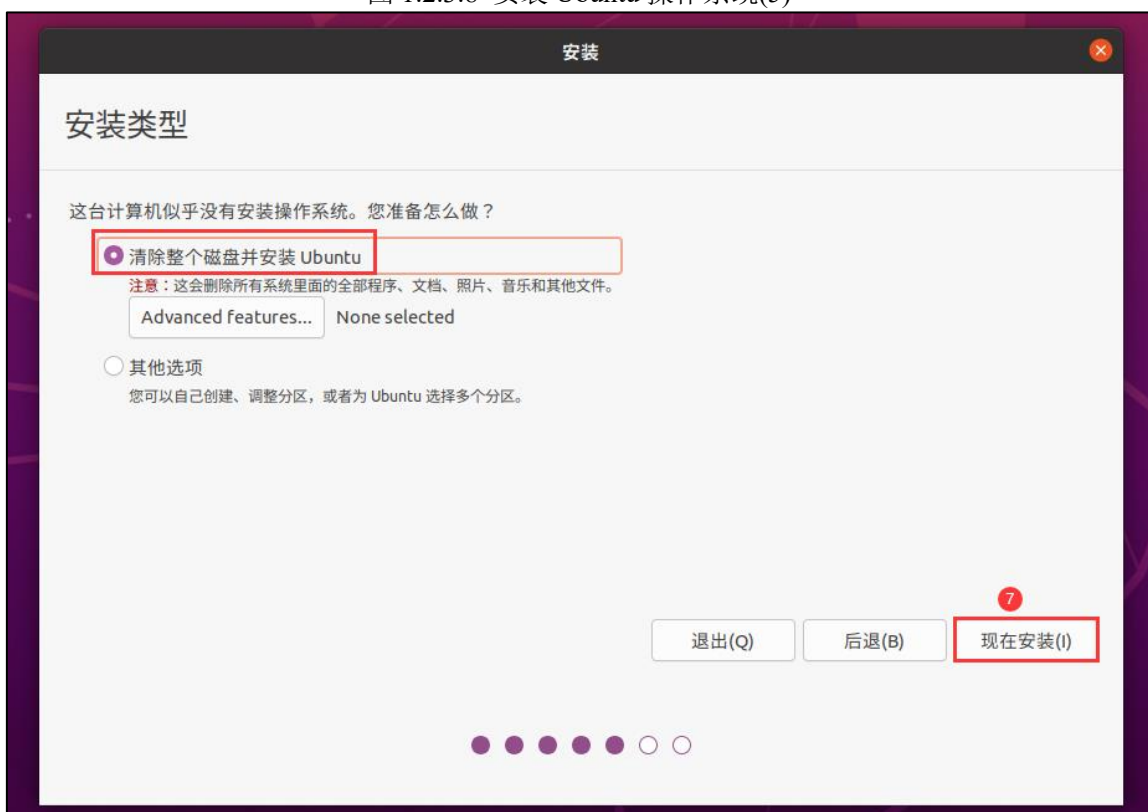


图 1.2.3.9 安装 Ubuntu 操作系统(6)

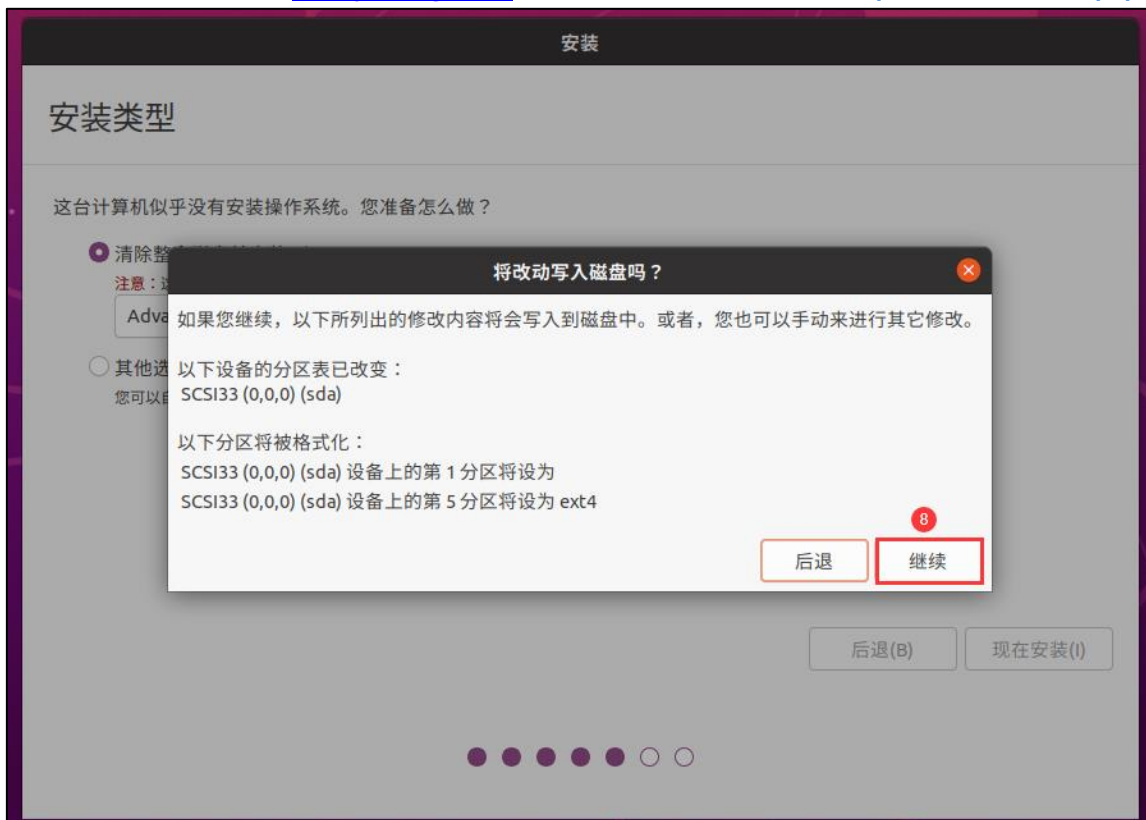


图 1.2.3.10 安装 Ubuntu 操作系统(7)



图 1.2.3.11 安装 Ubuntu 操作系统(8)

输入用户所在城市名称，这个信息主要用于确定用户所在地所处时区。



图 1.2.3.12 安装 Ubuntu 操作系统(9)

点击“继续”按钮后，开始正式安装：

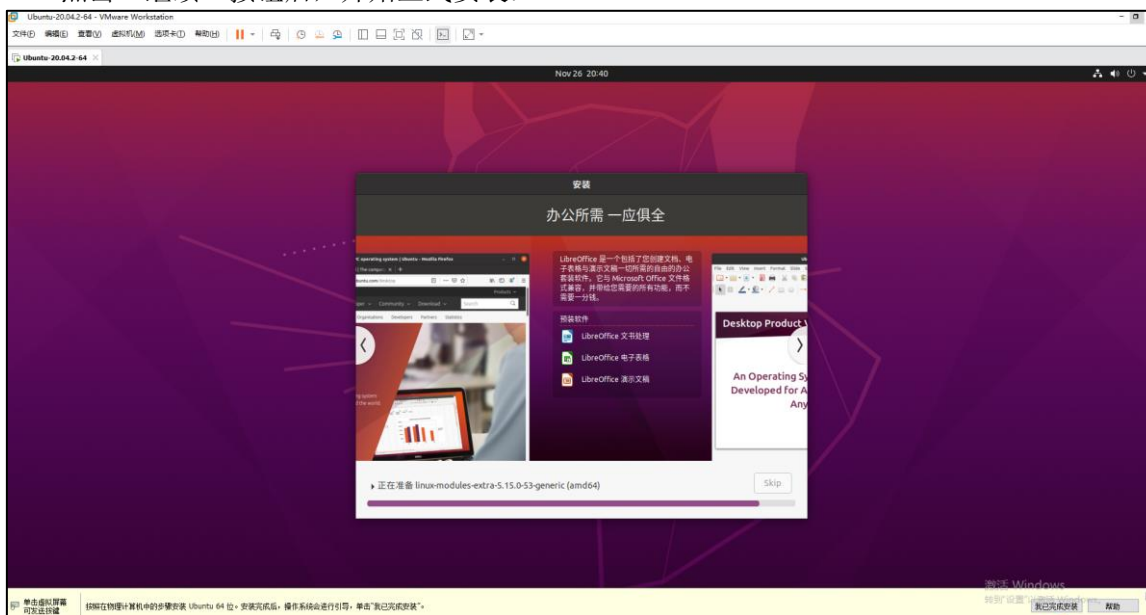


图 1.2.3.13 安装 Ubuntu 操作系统(10)

然后等待系统安装完成，安装完成后将会提示用户重启系统，如图 1.2.3.14 所示：

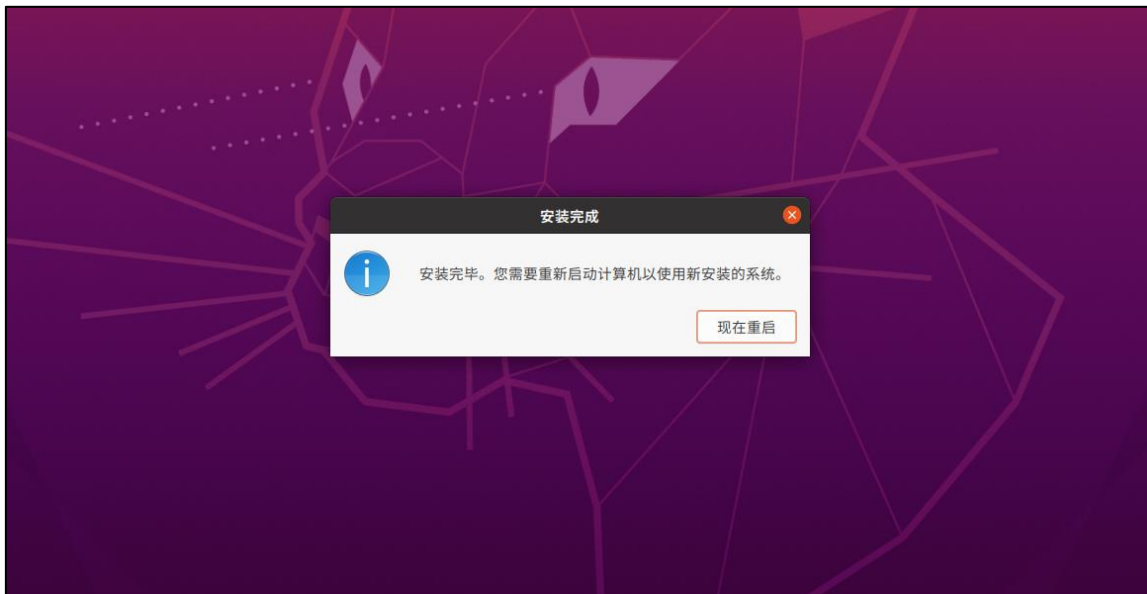


图 1.2.3.14 Ubuntu 系统安装完成

此时先别着急点击“现在重启”按钮，我们需要先关闭虚拟机，移除 Ubuntu 20.04 系统镜像文件。按照图 1.2.3.15~1.2.3.16 所示步骤关闭虚拟机：

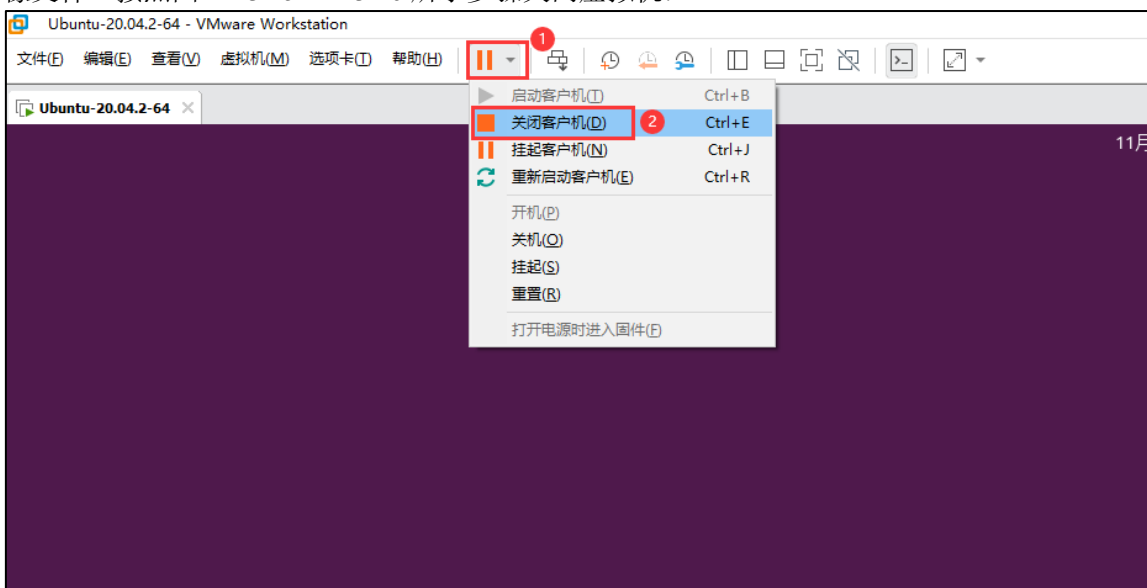


图 1.2.3.15 关闭虚拟机(1)

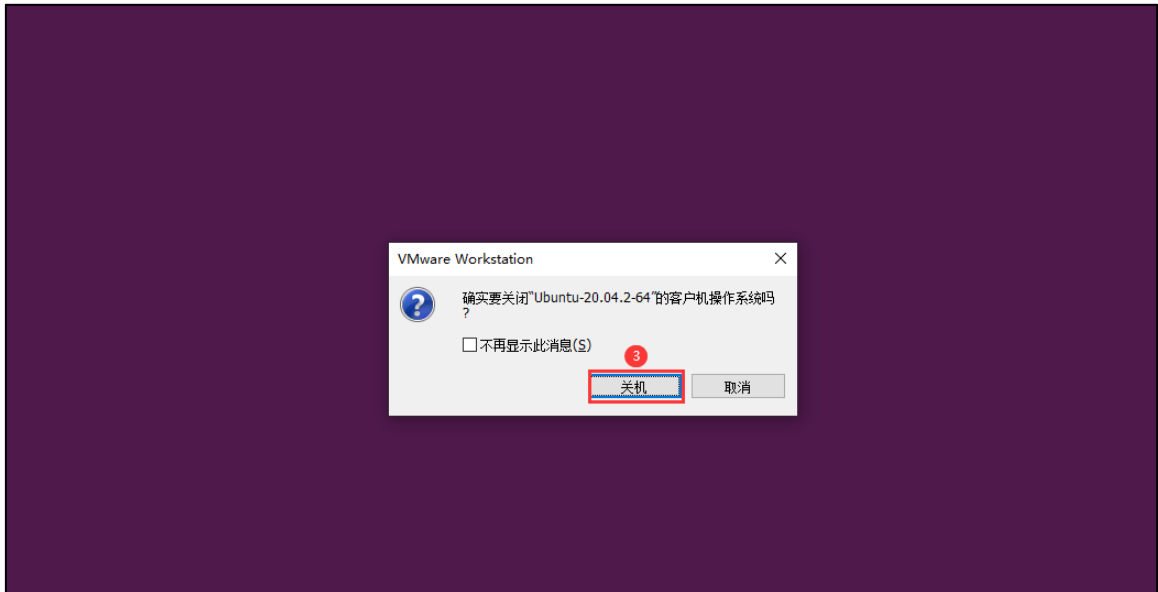


图 1.2.3.16 关闭虚拟机(2)

虚拟机关闭之后, 点击菜单栏“**虚拟机(M)→设置(S)**”进入虚拟机设置界面, 按照图 1.2.3.17 所示移除 Ubuntu 系统镜像文件:

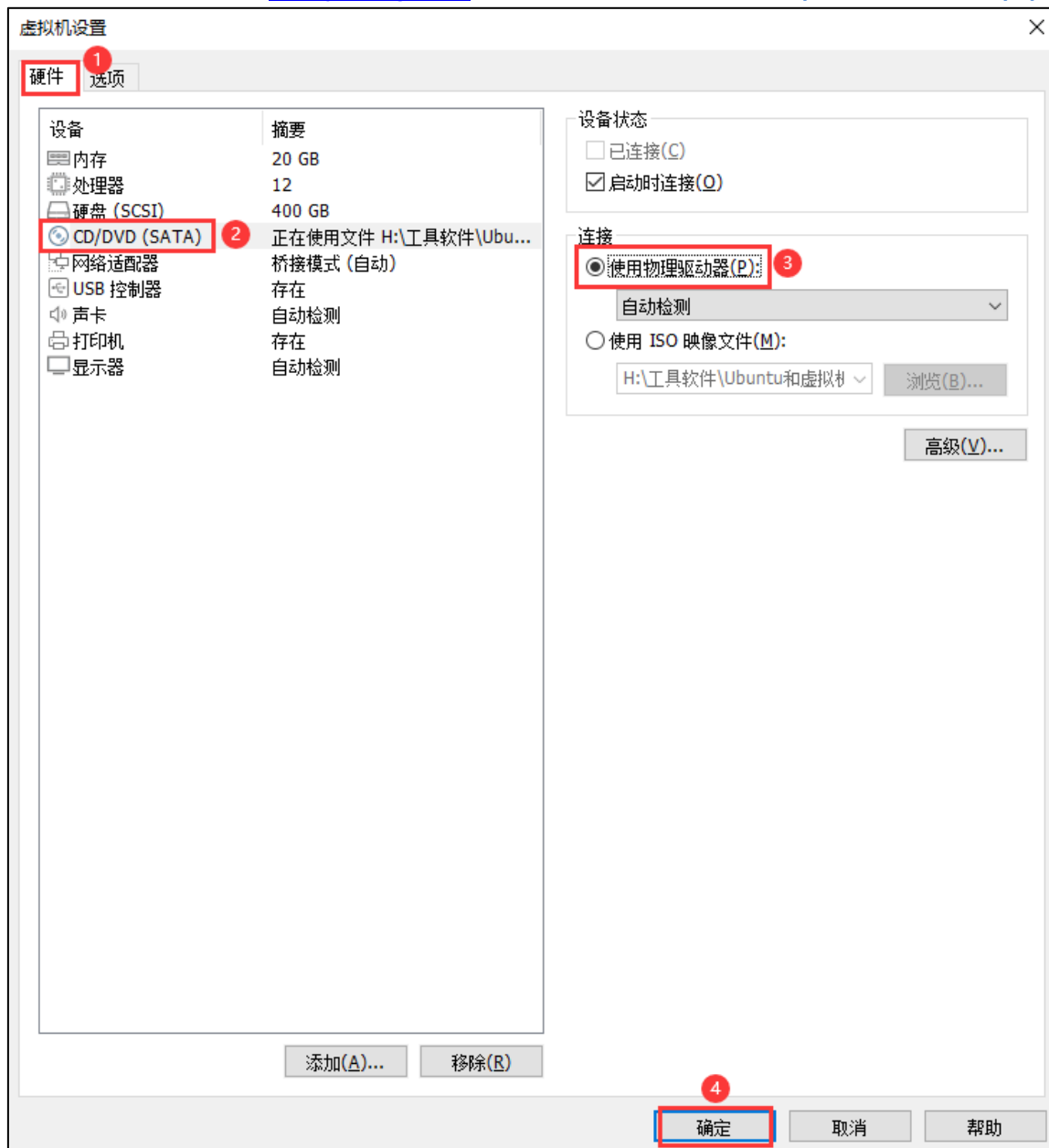
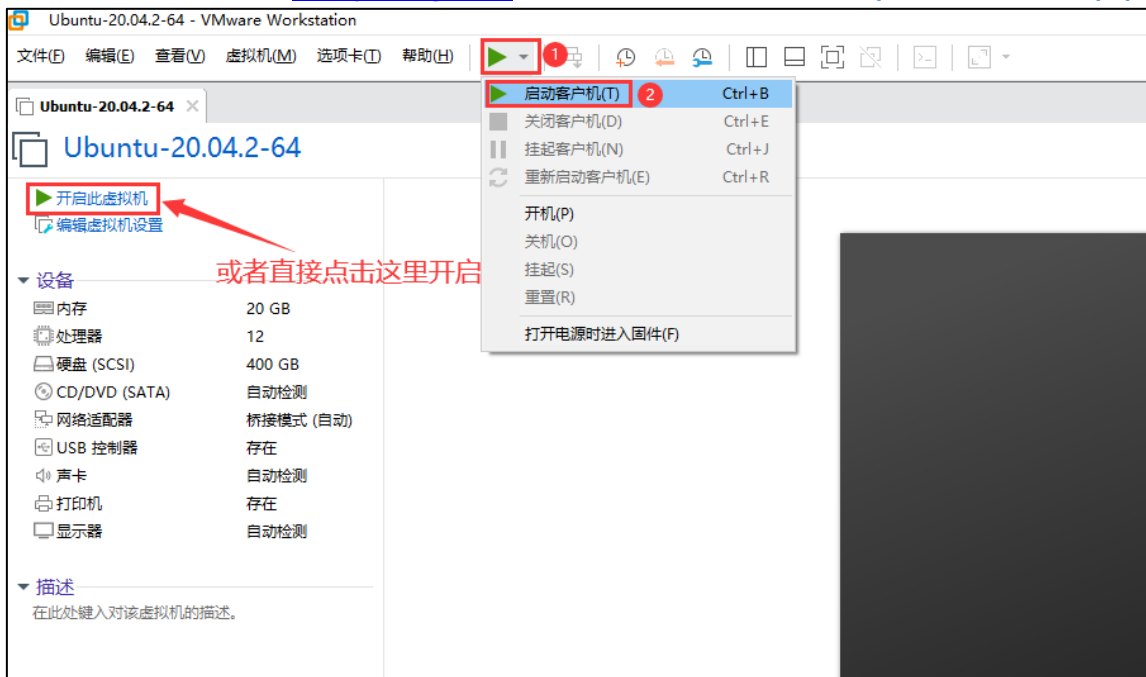
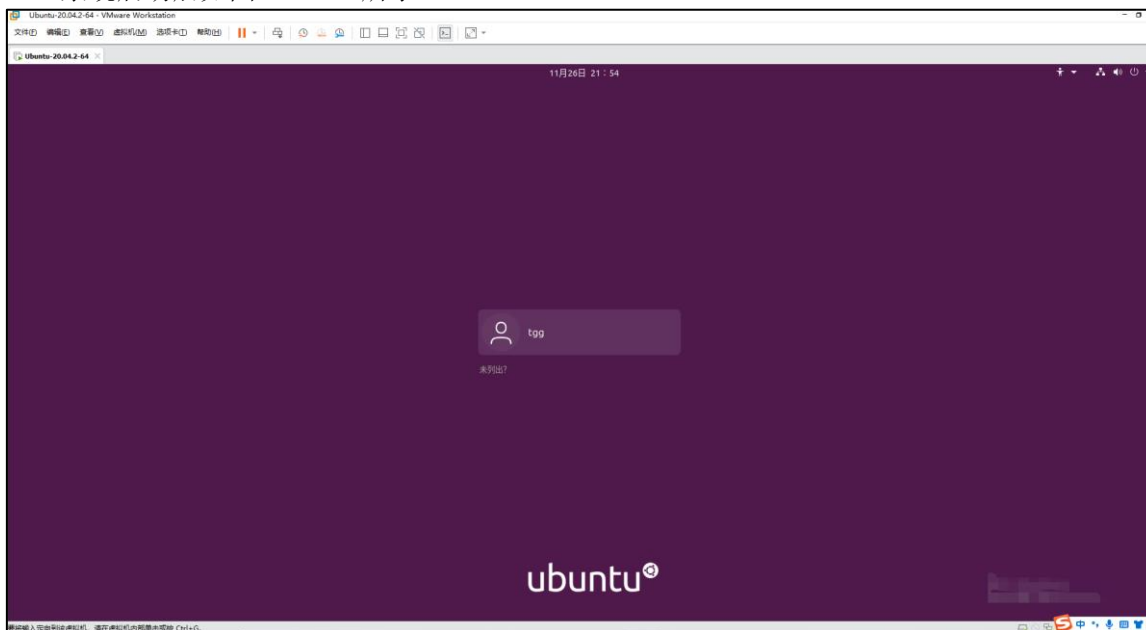


图 1.2.3.17 移除 Ubuntu 系统镜像文件

点击“确定”按钮后退出设置窗口，之后便可以开启虚拟机了，按照图 1.2.3.18 所示步骤开启虚拟机（也就是启动 Ubuntu 系统）：



系统启动后如图 1.2.3.19 所示:



点击接着用户名、输入密码后按回车键登录系统，进入 Ubuntu 系统主界面:

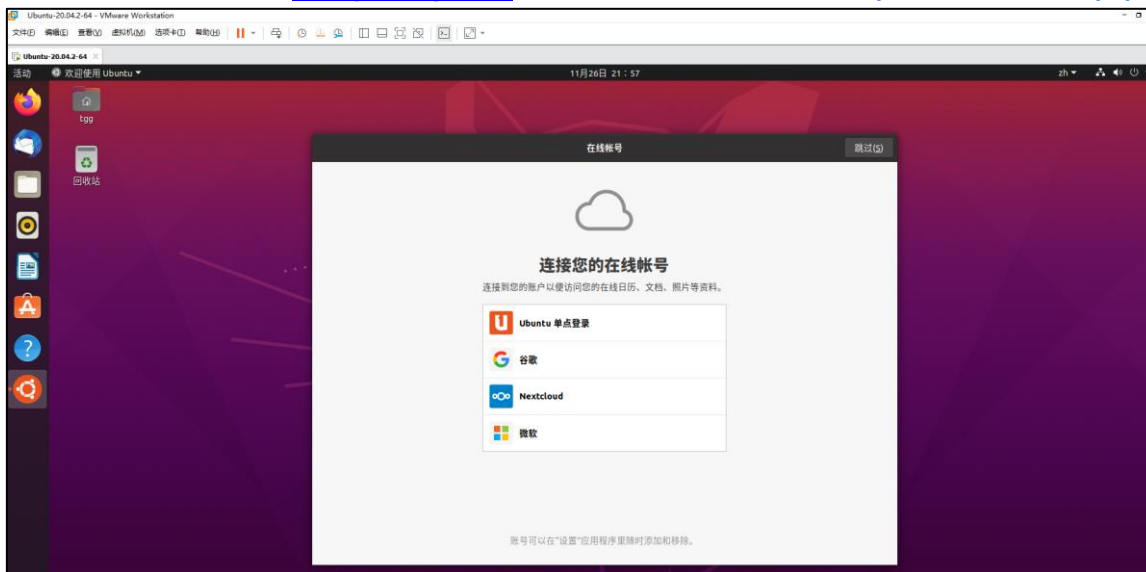


图 1.2.3.20 Ubuntu 系统主界面

首次进入 Ubuntu 20.04 系统桌面, 会弹出“在线帐号”窗口(也就是一个简易的使用引导), 如图 1.2.3.20 所示; 大家可以按照指引一步一步观看, 也可以直接退出, 如图 1.2.3.21 所示:

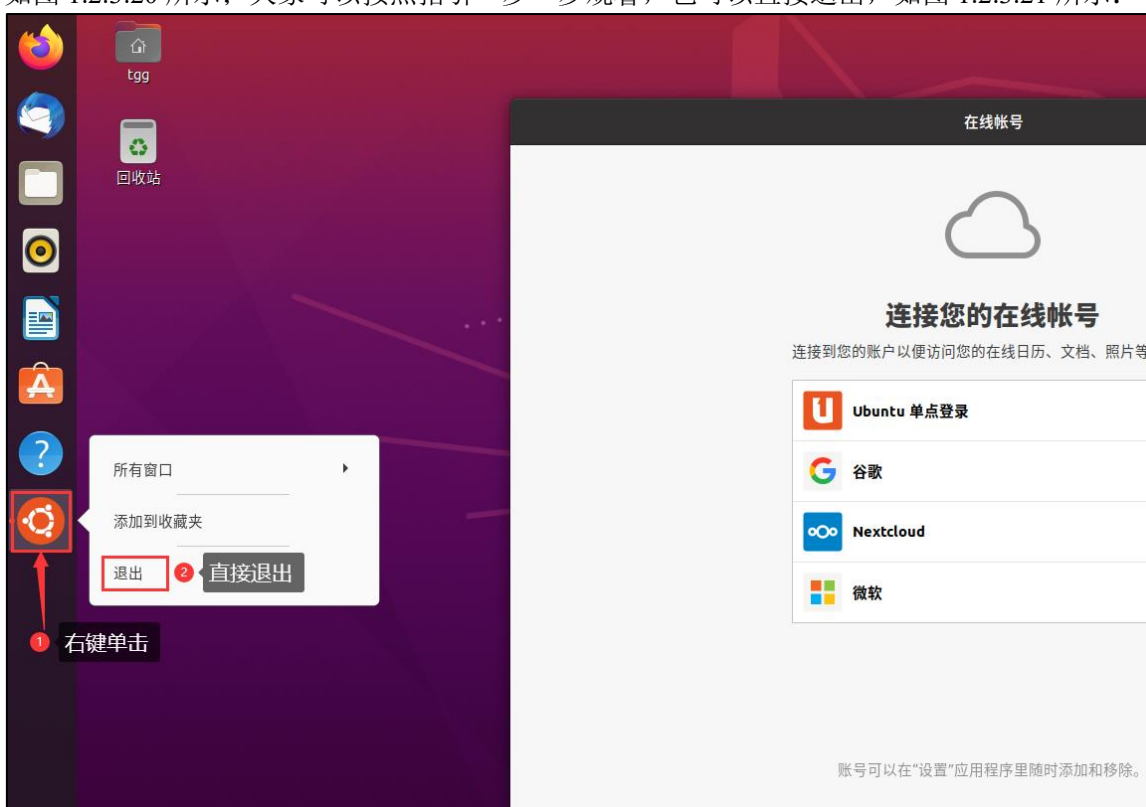


图 1.2.3.21 退出使用指引窗口

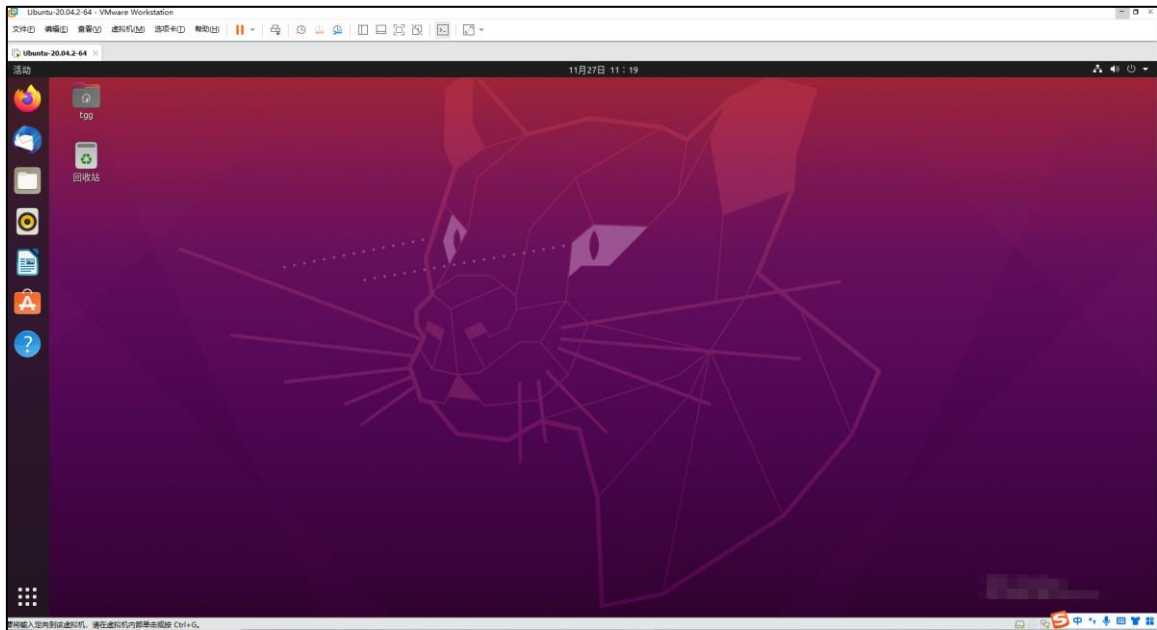


图 1.2.3.22 Ubuntu 系统桌面

至此，Ubuntu 20.04 操作系统安装完成！

第二章 开发环境搭建

在做 RK3588 Android 系统开发之前,我们需要先搭建好开发环境;包括 Windows 和 Ubuntu 这两种操作系统下的环境搭建,因为嵌入式开发一般都是基于 Windows+Ubuntu 双系统开发环境。

本章将分为如下几个小节:

- 2.1 Ubuntu 系统设置
- 2.2 Ubuntu 与 Windows 之间文件互传
- 2.3 Ubuntu 系统下搭建 tftp 服务器
- 2.4 Ubuntu 系统下搭建 nfs 服务器
- 2.5 Ubuntu 系统下搭建 ssh 服务器
- 2.6 CH343 串口驱动安装
- 2.7 MobaXterm 软件安装
- 2.8 Rockchip USB 驱动安装
- 2.9 ADB 工具安装

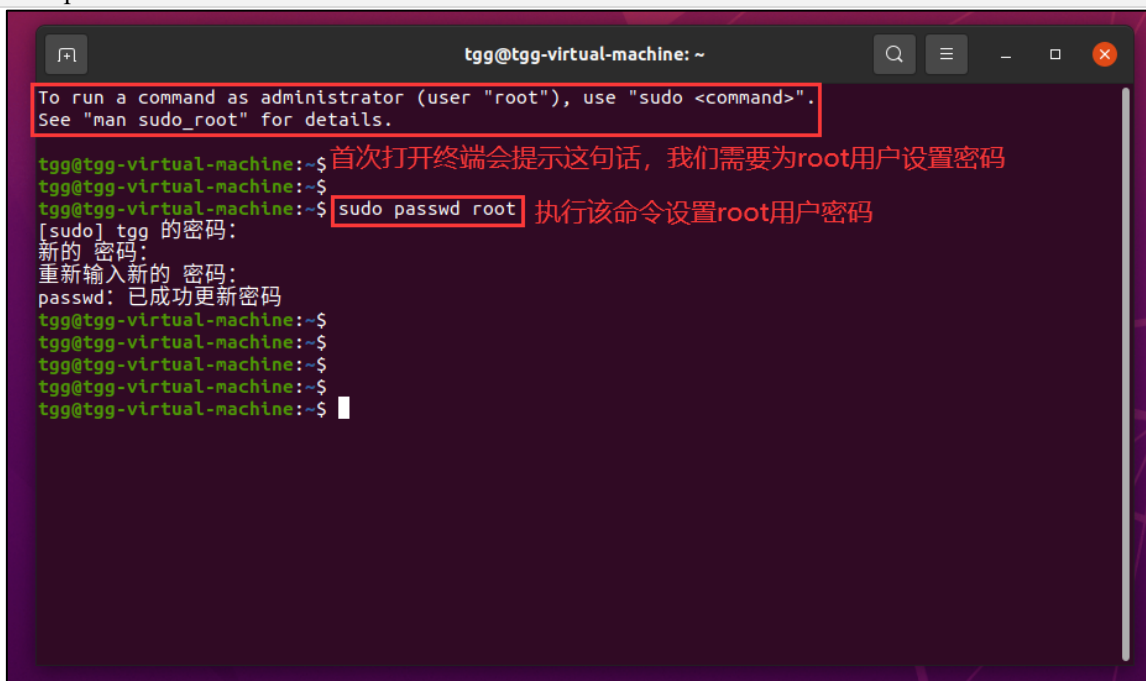
2.1 Ubuntu 系统设置

本小节对 Ubuntu 系统进行一个简单的设置。

2.1.1 设置 root 用户密码

刚安装好的 Ubuntu 系统没有设置 root 用户密码,我们需要打开终端(Ctrl+T)执行如下命令设置 root 用户密码:

```
sudo passwd root
```



```
tgg@tgg-virtual-machine: ~  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
tgg@tgg-virtual-machine:~$ sudo passwd root  
[sudo] tgg 的密码:  
新的 密码:  
重新输入新的 密码:  
passwd: 已成功更新密码  
tgg@tgg-virtual-machine:~$  
tgg@tgg-virtual-machine:~$  
tgg@tgg-virtual-machine:~$  
tgg@tgg-virtual-machine:~$  
tgg@tgg-virtual-machine:~$
```

图 2.1.1.1 设置 root 用户密码

2.1.2 更换软件下载源

Ubuntu 系统默认的软件下载源由于服务器的原因, 在国内的下载速度往往比较慢, 这时我们可以将 Ubuntu 系统的软件下载源更改为国内软件源, 譬如阿里源、中科大源、清华源等等, 下载速度相比 Ubuntu 官方软件源会快很多!

按照图 2.1.2.1~2.1.2.7 所示步骤进行更改:



图 2.1.2.1 更改软件下载源(1)



图 2.1.2.2 更改软件下载源(2)



图 2.1.2.3 更改软件下载源(3)

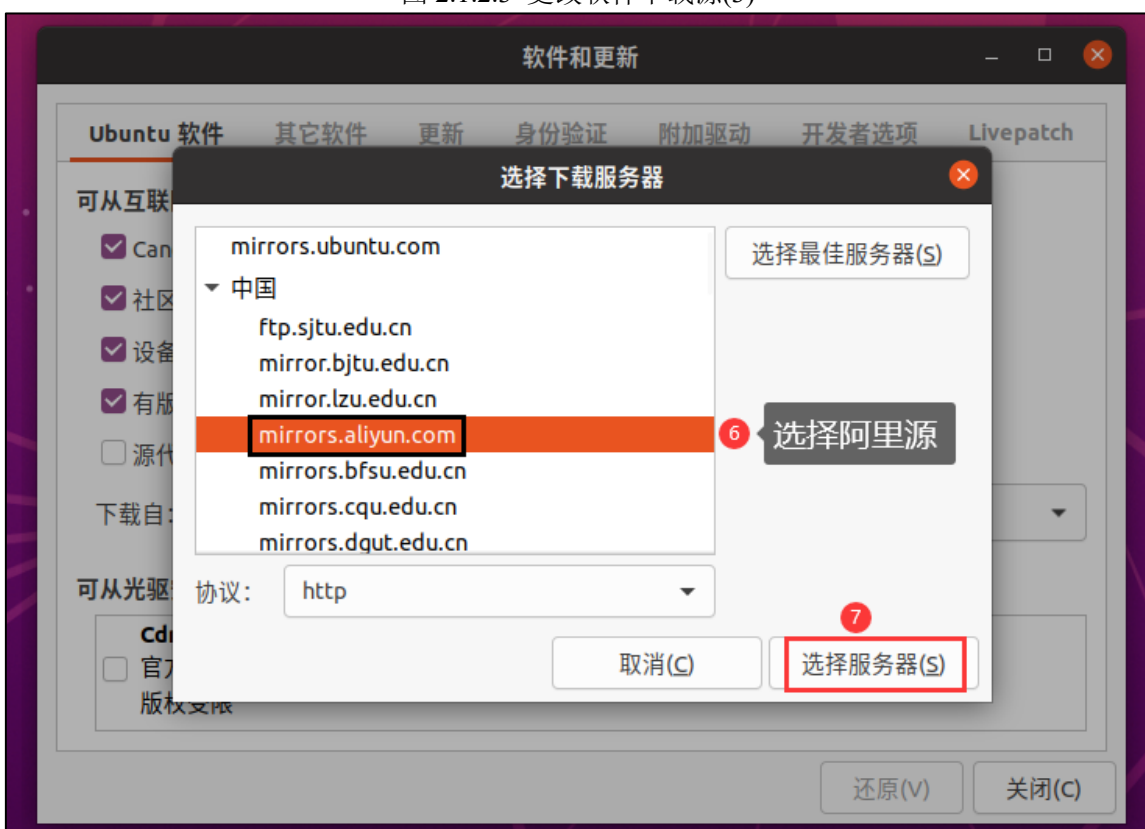


图 2.1.2.4 更改软件下载源(4)

从列表可知,有很多国内的下载源供我们选择,这里我们以阿里源 mirrors.aliyun.com 为例,当然也可以选择其它下载源。



图 2.1.2.5 更改软件下载源(5)



图 2.1.2.6 更改软件下载源(6)



图 2.1.2.7 更改软件下载源(7)

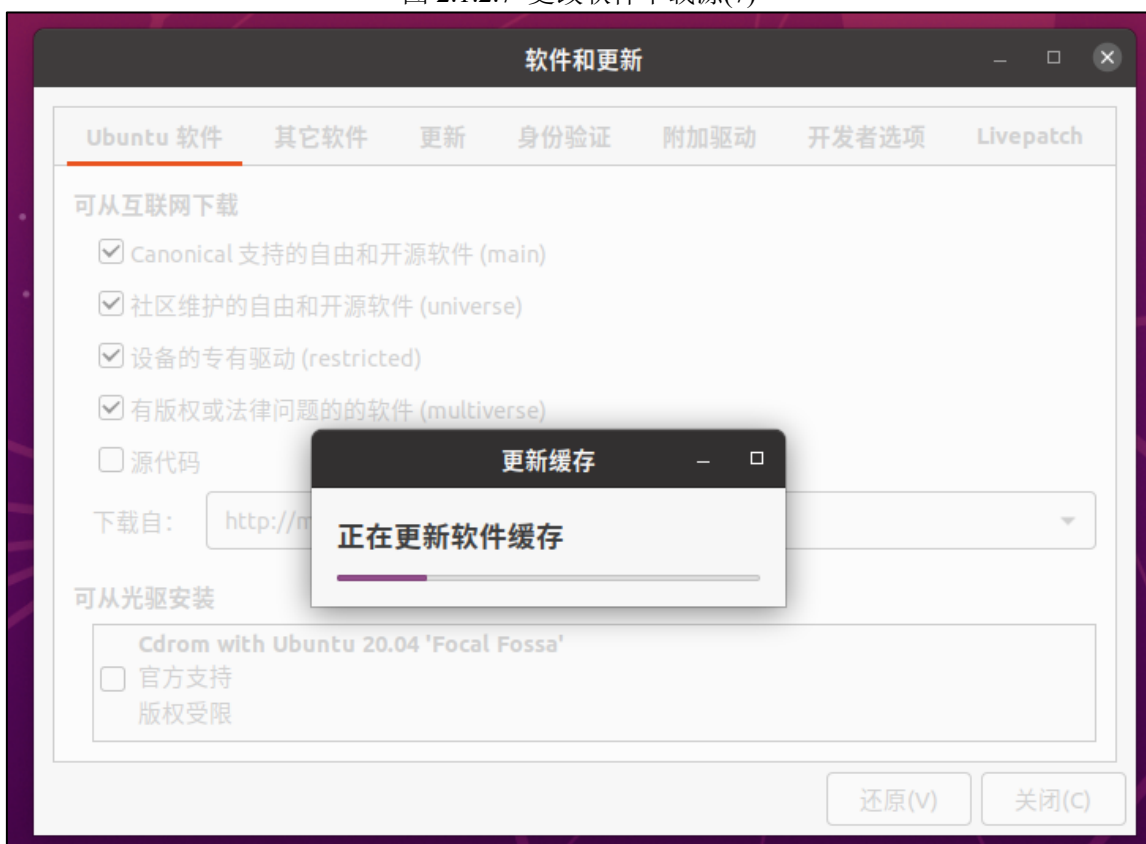


图 2.1.2.8 更改软件下载源(8)

2.1.3 关闭自动更新

同样，打开 Ubuntu 系统“软件与更新”窗口，按照图 2.1.3.1 所示步骤关闭自动更新功能：



图 2.1.3.1 关闭自动更新功能

2.2 Ubuntu 与 Windows 之间文件互传

我们在开发过程中，经常需要在 Windows 系统与 Ubuntu 系统之间进行文件传输，文件互传的方式比较多，譬如共享文件夹、Samba、FTP、scp 等等，本小节向用户介绍通过 FTP 方式进行文件传输。

通过 FTP（File Transfer Protocol，文件传输协议）在 Windows 与 Ubuntu 之间进行文件传输，需要完成以下两件事情。

2.2.1 Ubuntu 系统下搭建 FTP 服务器

在 Ubuntu 系统下打开终端，执行如下命令安装 FTP 服务：

```
sudo apt-get update
sudo apt-get install vsftpd
```

vsftpd 安装完成后，使用 vi 命令打开/etc/vsftpd.conf 配置文件，如果没有安装 vim 软件，则需要先通过如下命令安装 vim 编辑器：

```
sudo apt-get install vim
```

然后再执行如下命令打开/etc/vsftpd.conf 配置文件：

```
sudo vi /etc/vsftpd.conf
```

打开配置文件后，找到如下两行，确保其前面没有“#”符号（“#”符号表示注释，我们要取消注释，删除这个符号）：

```
26 #
27 # Uncomment this to allow local users to log in.
28 local_enable=YES
29 #
30 # Uncomment this to enable any form of FTP write command.
31 write_enable=YES
32 #
```

图 2.2.1.1 修改/etc/vsftpd.conf 配置文件

默认情况下，“write_enable=YES”前面有一个“#”号，我们需要将其去掉，使能该配置。修改完成后保存退出，然后执行如下命令重启 FTP 服务：

```
sudo /etc/init.d/vsftpd restart
```

可通过如下命令确认 FTP 服务是否开启：

```
ps -aux | grep vsftpd | grep -v grep
```

```
tgg@tgg-virtual-machine:~$ ps -aux | grep vsftpd | grep -v grep
root      5469  0.0  0.0  6816  3076 ?        Ss   11:15   0:00 /usr/sbin/vsftpd /etc/vsftpd.conf
tgg@tgg-virtual-machine:~$
```

图 2.2.1.2 确认 FTP 服务是否开启

2.2.2 Windows 下安装 FTP 客户端

我们需要在 Windows 系统下安装一个 FTP 客户端软件，这里选择 FileZilla 作为 FTP 客户端软件，这是一个免费的 FTP 客户端软件。

ATK-RK3588 开发板资料盘中已经给用户提供了 FileZilla 软件安装包，路径为：**开发板光盘 A 盘-基础资料→04、软件→FileZilla_3.61.0_win64-setup.exe**，用户也可以通过链接地址：<https://www.filezilla.cn/download/client>，自己下载安装包文件。

The screenshot shows the FileZilla download page. It has a navigation bar with links like 首页, 功能, 下载, 新闻公告, 更新日志, 常见问题. Below the navigation bar, there's a section for '客户端' (Client). Under 'Windows 平台' (Windows platform), there's a table with download links for 32-bit and 64-bit versions. A red box highlights the 'Windows 平台' section. Under 'Linux 平台' (Linux platform), there's a table with download links for 1686 and 86_64 versions.

文件名	FILEZILLA 32位 安装包	FILEZILLA 32位 绿色版	FILEZILLA 64位 安装包	FILEZILLA 64位 绿色版
软件版本	3.62.0	3.62.0	3.62.0	3.62.0
更新时间	2022-10-11	2022-10-11	2022-10-11	2022-10-11
支持系统	支持 64 位版本的 Windows 8.1 和 10。	支持 64 位版本的 Windows 8.1 和 10。	支持 64 位版本的 Windows 8.1 和 10。	支持 64 位版本的 Windows 8.1 和 10。
	立即下载	立即下载	立即下载	立即下载

文件名	FILEZILLA 1686	FILEZILLA 86_64
软件版本	3.62.0	3.62.0
更新时间	2022-10-11	2022-10-11

图 2.2.2.1 下载 FileZilla 软件安装包

下载完成如图 2.2.2.2 所示：



图 2.2.2.2 FileZilla 软件安装包

接着双击安装包文件 FileZilla_3.61.0_win64-setup.exe, 按照图 2.2.2.3~2.2.2.8 所示步骤进行安装:

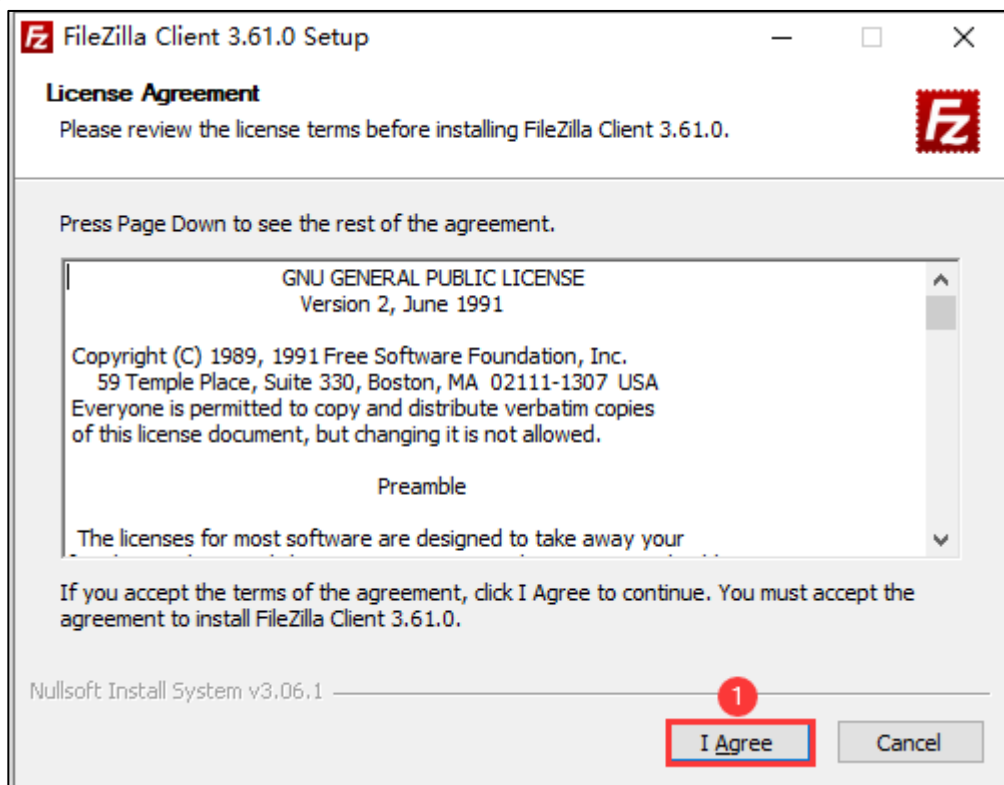


图 2.2.2.3 安装 FileZilla 软件(1)

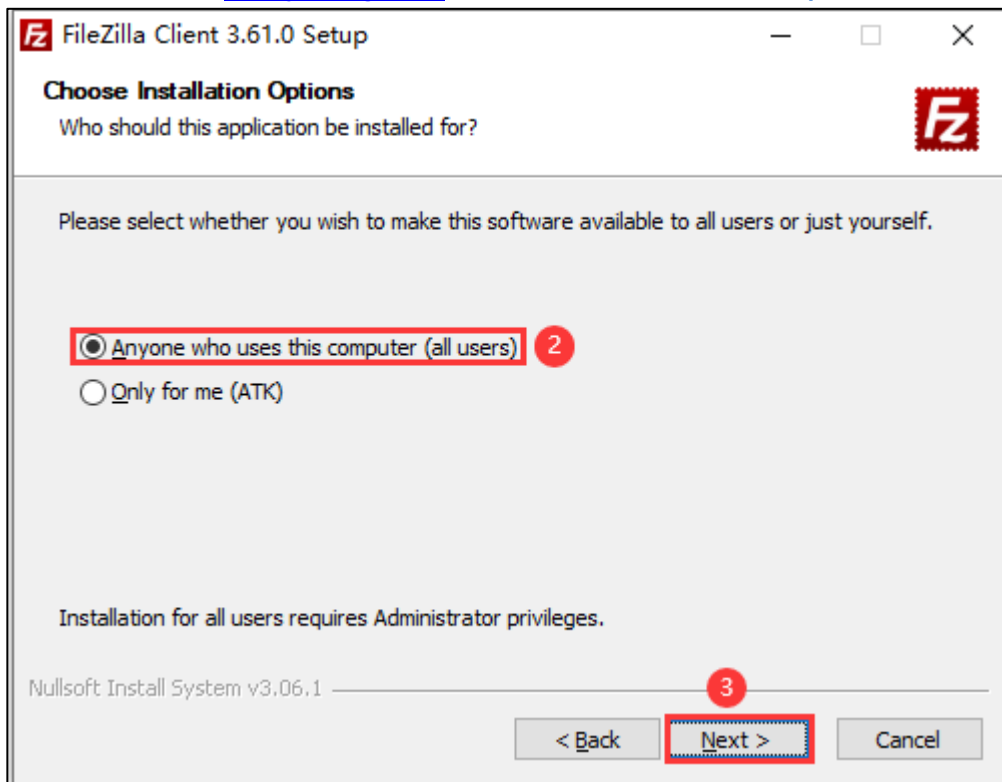


图 2.2.2.4 安装 FileZilla 软件(2)

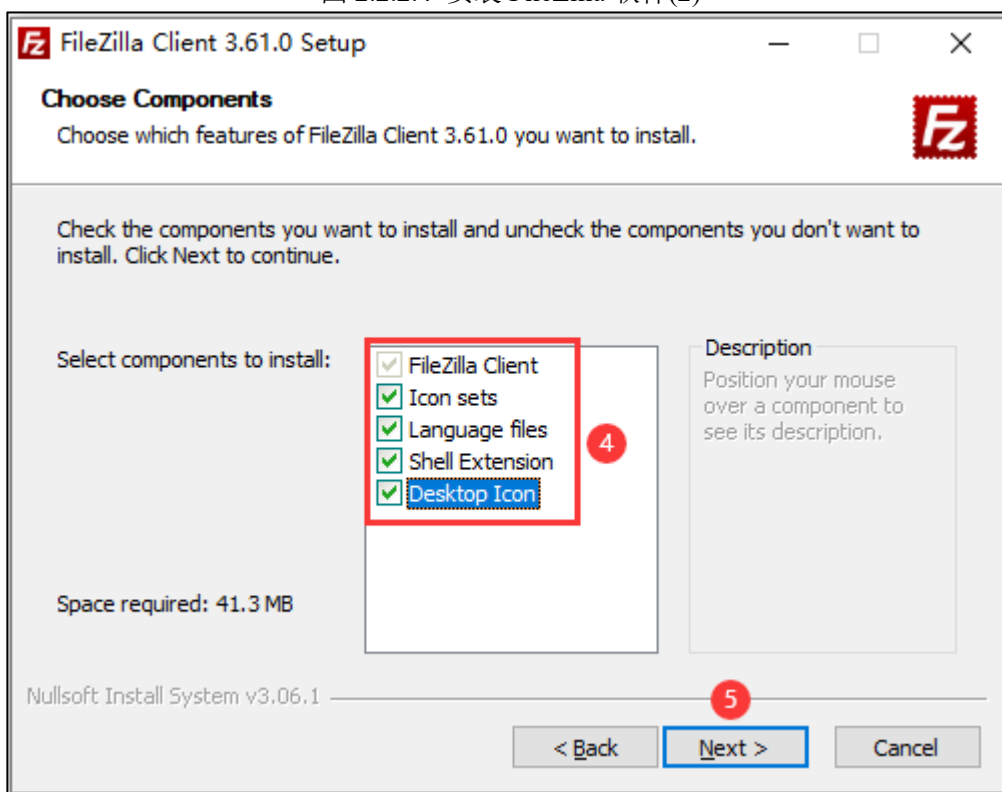


图 2.2.2.5 安装 FileZilla 软件(3)

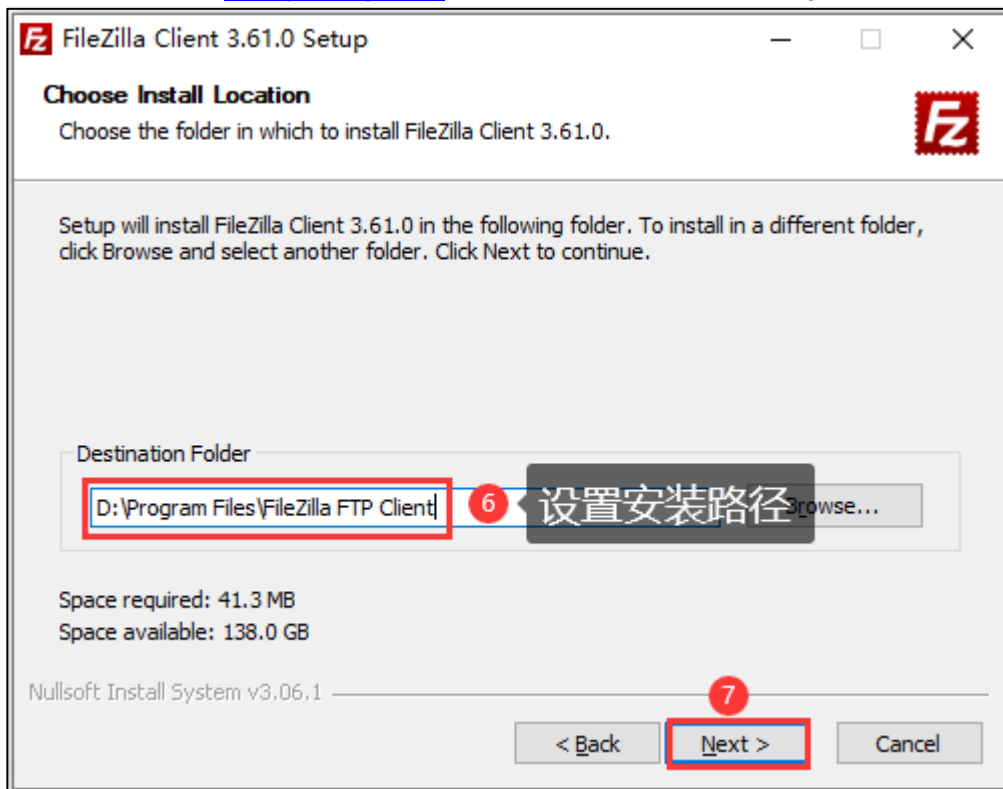


图 2.2.2.6 安装 FileZilla 软件(4)

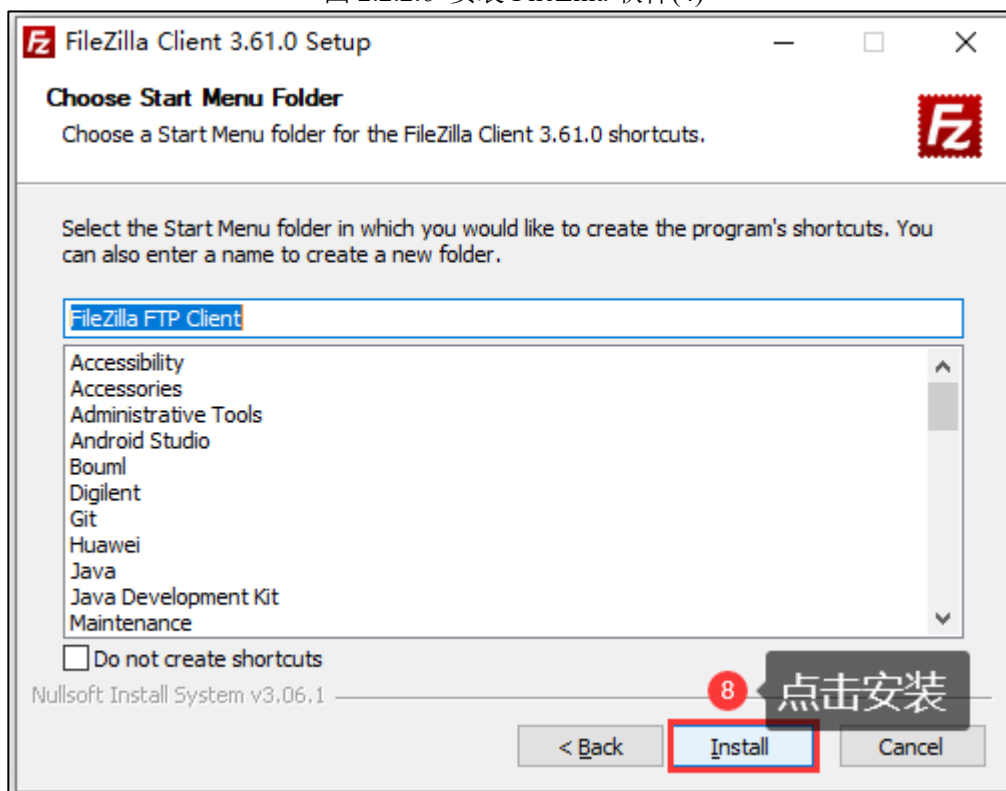


图 2.2.2.7 安装 FileZilla 软件(5)

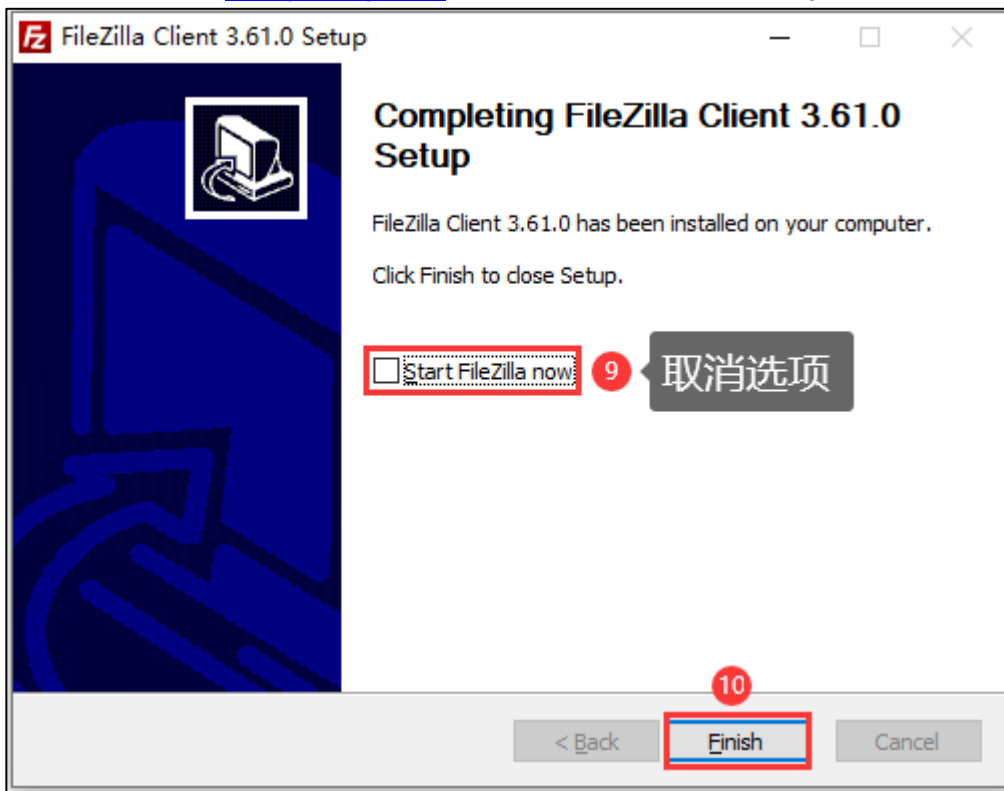


图 2.2.2.8 安装 FileZilla 软件(6)

至此，软件安装完成，桌面会生成对应的快捷方式：

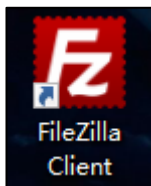


图 2.2.2.9 FileZilla 桌面图标

2.2.3 FileZilla 使用方法

接下来介绍一下 FileZilla 的使用方法，首先双击 FileZilla 桌面图标打开该软件：

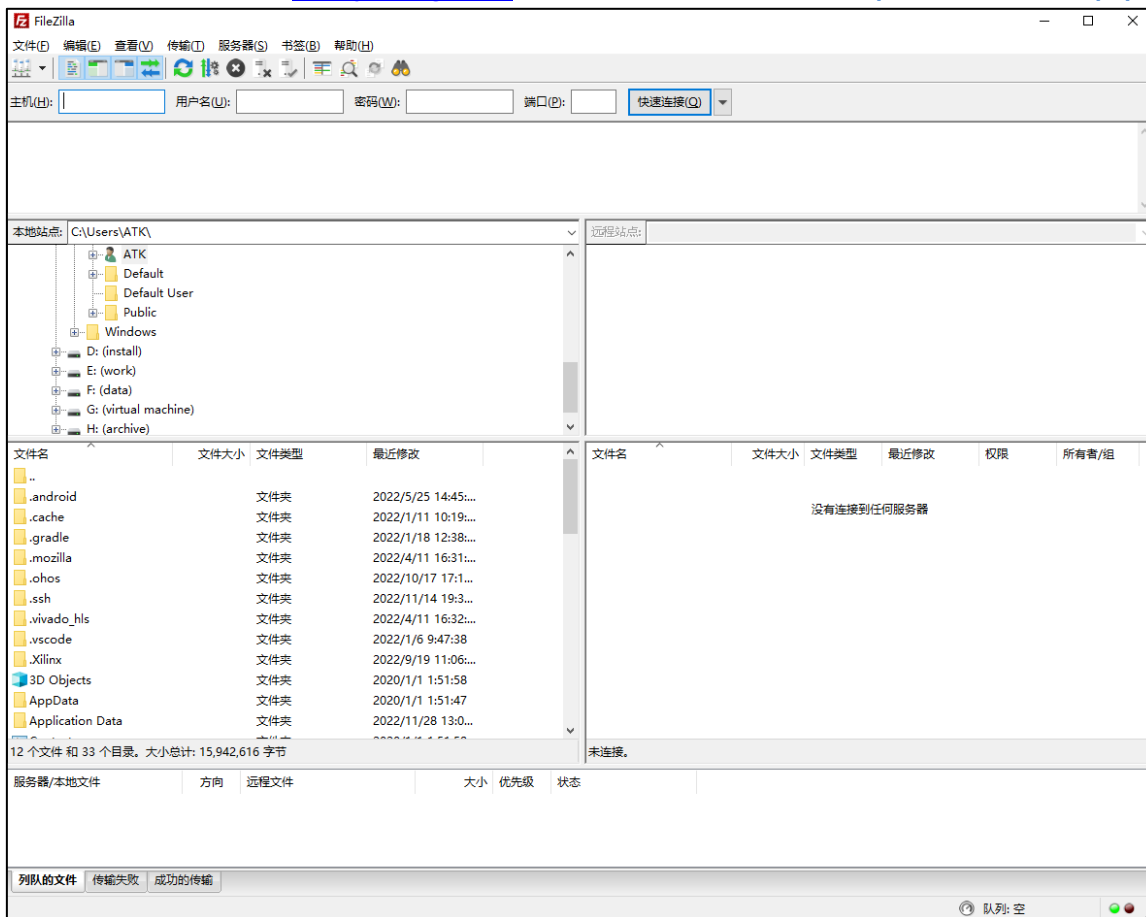


图 2.2.3.1 FileZilla 软件界面

Ubuntu 作为 FTP 服务器, Windows 作为 FTP 客户端, 在进行文件传输之前, 客户端需要先连接到服务器。按照图 2.2.3.2~2.2.3.6 所示步骤连接 FTP 服务器:

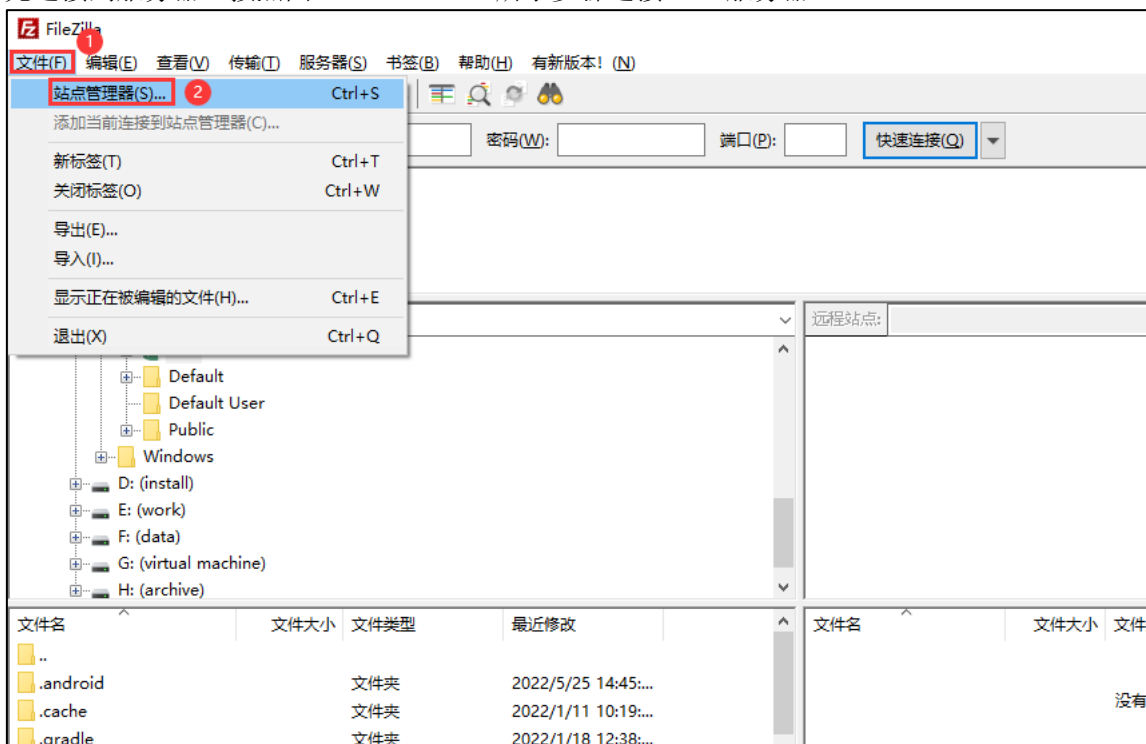


图 2.2.3.2 FTP 客户端连接 FTP 服务器(1)



图 2.2.3.3 FTP 客户端连接 FTP 服务器(2)



图 2.2.3.4 FTP 客户端连接 FTP 服务器(3)

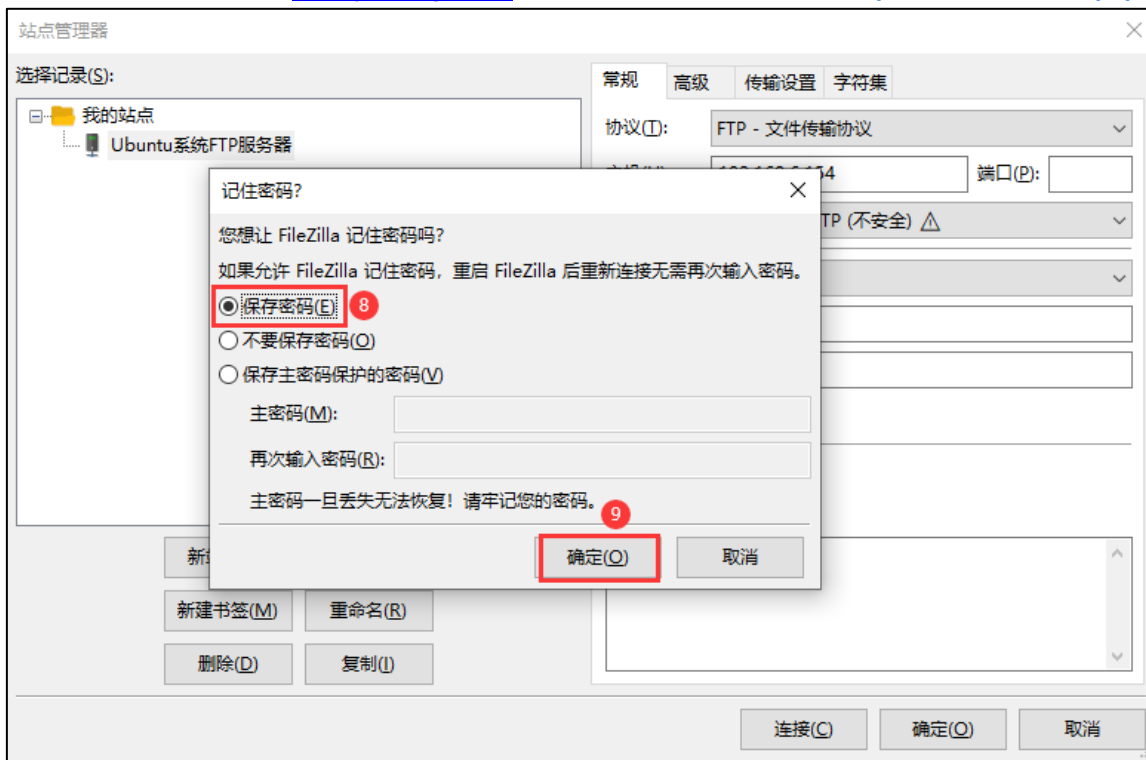


图 2.2.3.5 FTP 客户端连接 FTP 服务器(4)

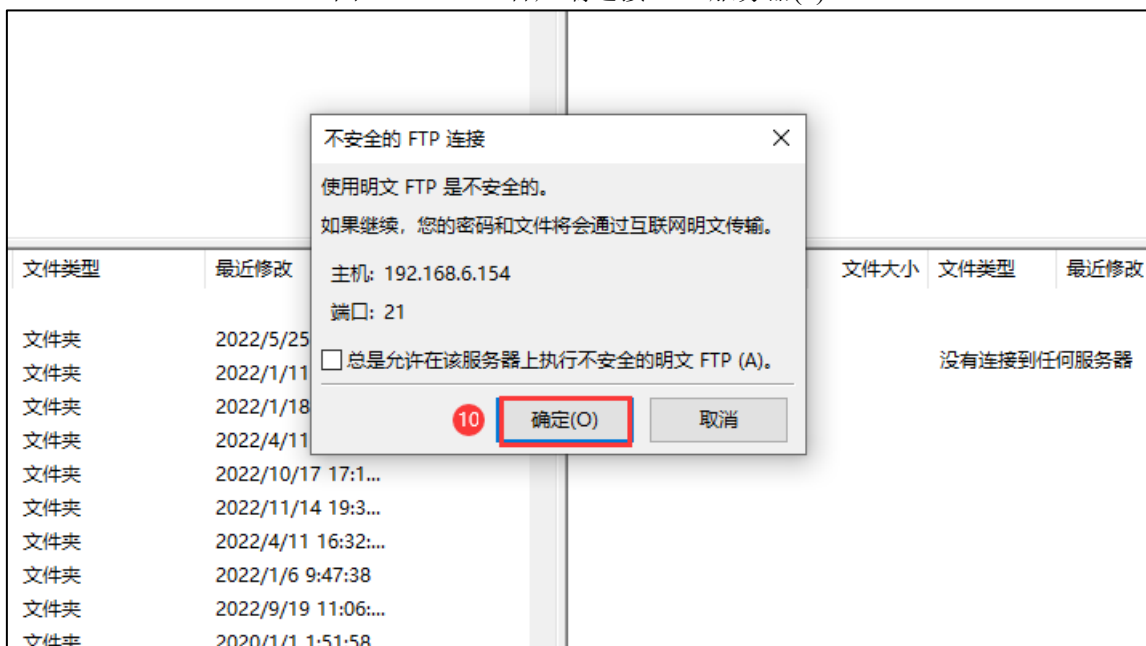


图 2.2.3.6 FTP 客户端连接 FTP 服务器(5)

连接成功如图 2.2.3.7 所示:

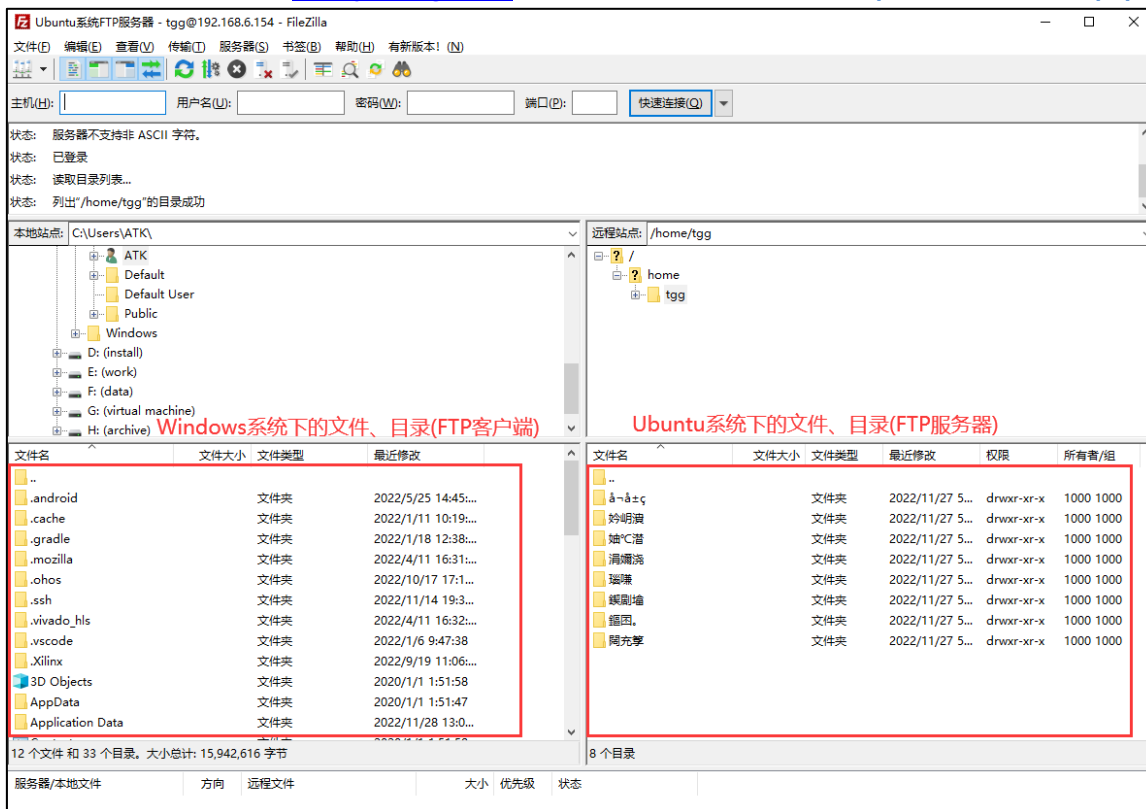


图 2.2.3.7 连接成功

其中左边是 Windows 系统下的文件、目录（FTP 客户端），右边是 Ubuntu 系统下的文件、目录（FTP 服务器，默认会进入到用户家目录，譬如“/home/tgg”）。从上图可知，Ubuntu 系统下的文件列表名称全是乱码，这是因为编码的问题导致的，我们需要修改变码方式，再次打开“站点管理器”，按照图 2.2.3.8 所示步骤设置编码方式：



图 2.2.3.8 设置字符编码方式(1)

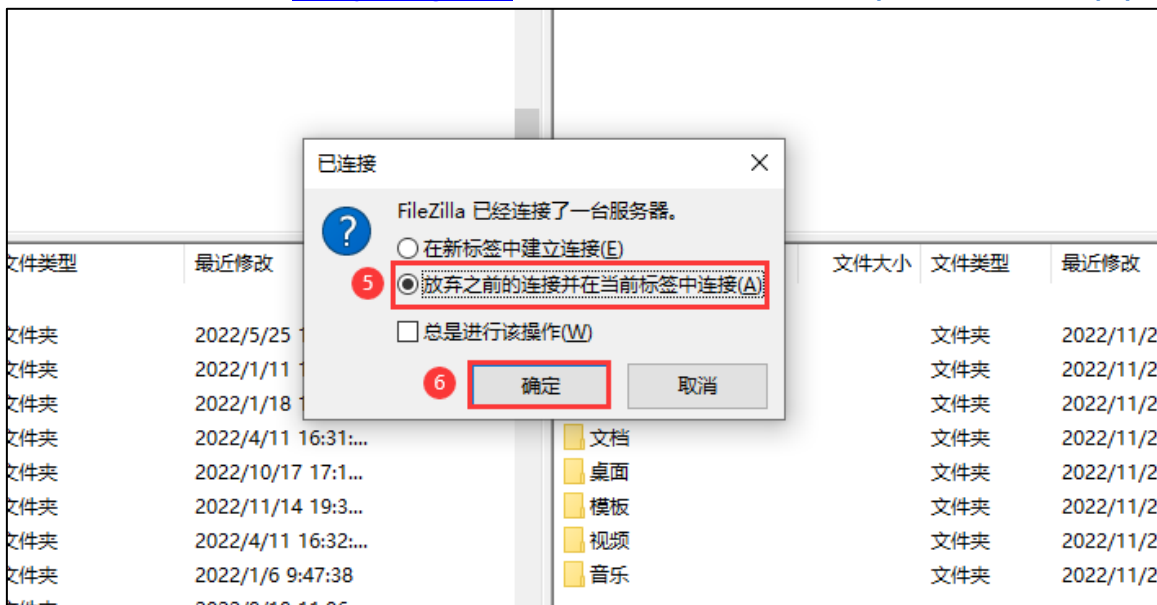


图 2.2.3.9 设置字符编码方式(2)

再次连接之后, 会发现文件名已经显示正常了, 如图 2.2.3.10 所示:

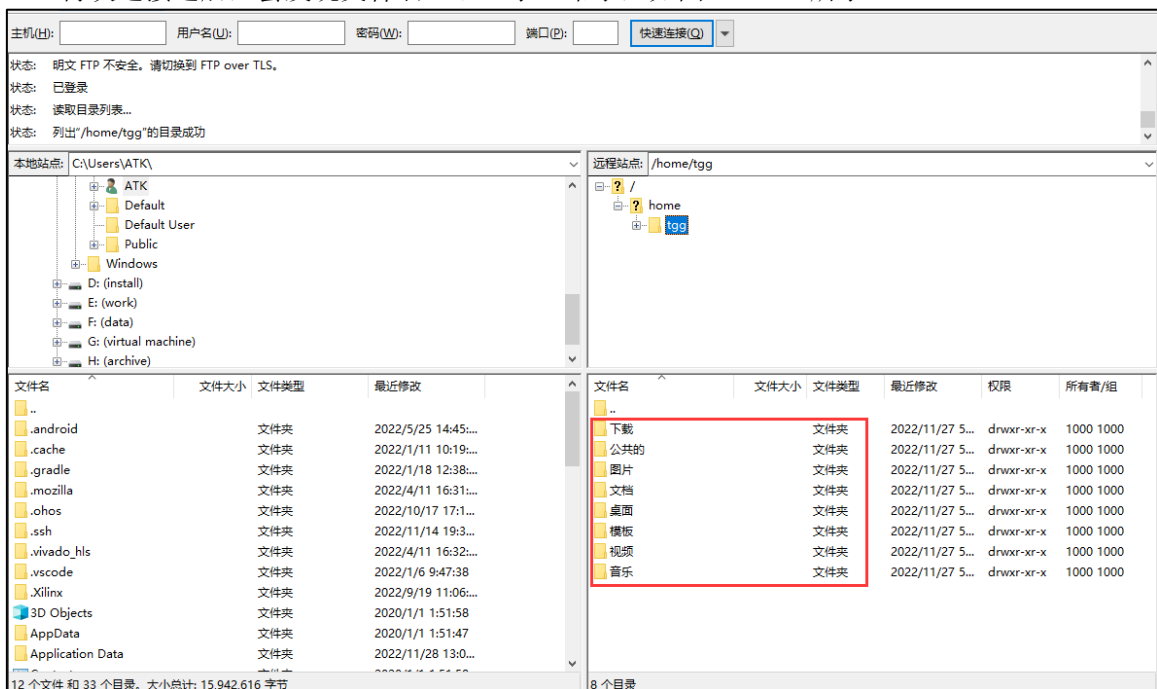


图 2.2.3.10 文件名显示正常

客户端成功连接上服务器后, 便可以进行文件传输了。传输的方式也非常简单, 选择要传输的文件, 直接使用鼠标左键将其拖动到 Windows 目录区域或者 Ubuntu 系统目录区域即可! 比如, 将 Windows 系统下的 test.txt 文件拷贝到 Ubuntu 系统, 首先在左侧 Windows 目录区域找到该文件 test.txt, 然后使用鼠标左键将其拖动至右边 Ubuntu 目录区域释放即可; 同理, Ubuntu 系统下的文件拷贝到 Windows 系统也是如此!

2.3 Ubuntu 系统搭建 tftp 服务器

后续有需求再考虑要不要加上!

2.4 Ubuntu 系统搭建 nfs 服务器

后续有需求再考虑要不要加上!

2.5 Ubuntu 系统搭建 ssh 服务器

在开发过程中,有时需要通过 ssh 远程连接、登录 Ubuntu 系统,为了实现这个功能,我们需要在 Ubuntu 系统下开启 ssh 服务。执行如下命令安装 openssh-server:

```
sudo apt-get install openssh-server
```

安装 openssh-server 后,ssh 服务将自动开启,可通过如下命令确认 ssh 服务是否开启:

```
ps -aux | grep ssh | grep -v grep
```

```
tgg@tgg-virtual-machine:~$ ps -aux | grep ssh | grep -v grep
tgg      1793  0.0  0.0   6040   452 ?        Ss   09:53   0:00 /usr/bin/ssh-agent /usr/bin/im-launch env GNOME_SHELL_SESSION_PATH=/usr/share/gnome-shell/gnome-shell-session-
root     7414  0.0  0.0  12184  6972 ?        Ss   15:22   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
tgg@tgg-virtual-machine:~$
```

图 2.5.1 确认 ssh 服务是否开启

2.6 CH343 串口驱动安装

正点原子 ATK-DLRK3588 开发板使用了国产芯片 CH343 来实现 USB 转串口功能,可以直接通过 USB 线将开发板的调试串口连接到电脑、而无需使用 USB 转串口线,方便用户使用;但是需要在 Windows 下安装 CH343 驱动才能识别到开发板的调试串口。

开发板资料盘中已经给用户提供了 CH343 驱动安装文件,路径为: **开发板光盘 A 盘-基础资料→04、软件→CH343SER.EXE**,双击 CH343SER.EXE 可执行文件,按照图 2.6.1~2.6.2 所示步骤安装 CH343 驱动 (**安装之前先不要连接开发板调试串口**):

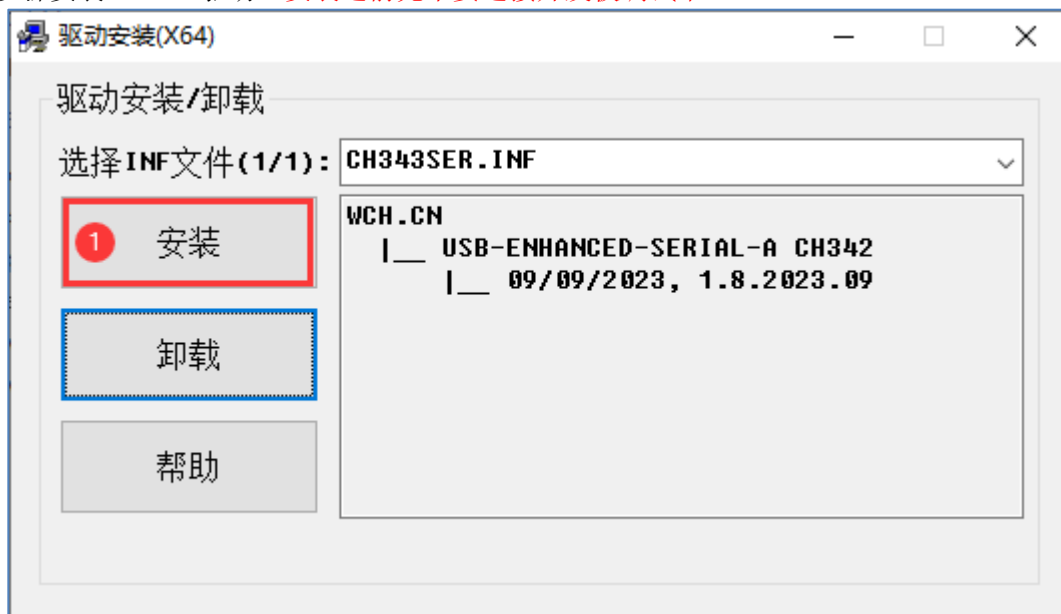


图 2.6.1 安装 CH343 驱动(1)

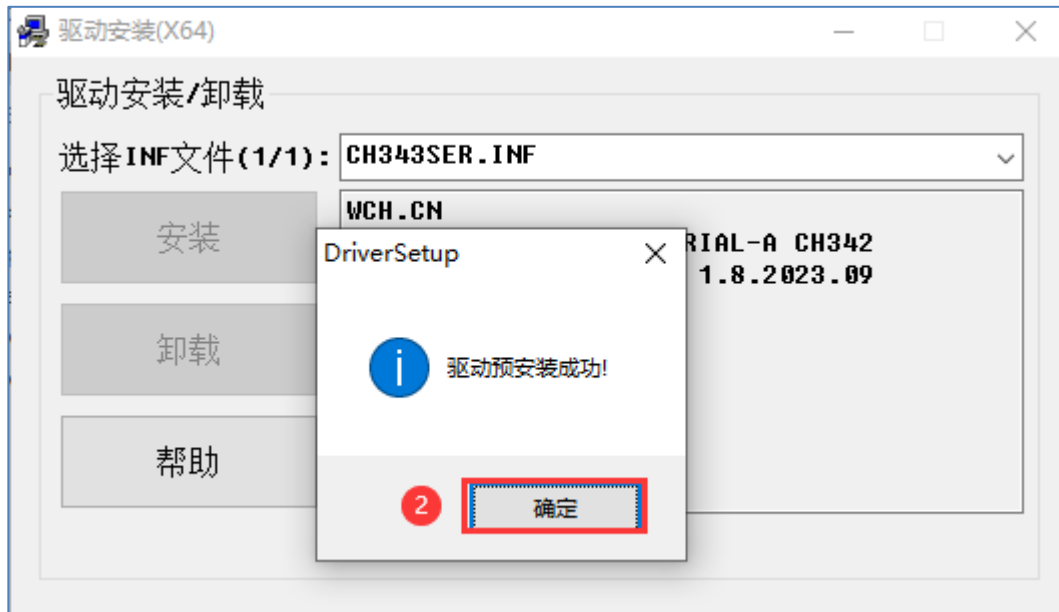


图 2.6.2 安装 CH343 驱动(2)

安装成功后，通过 USB 线将开发板调试串口与电脑相连，连接方式如图 2.6.3 所示：

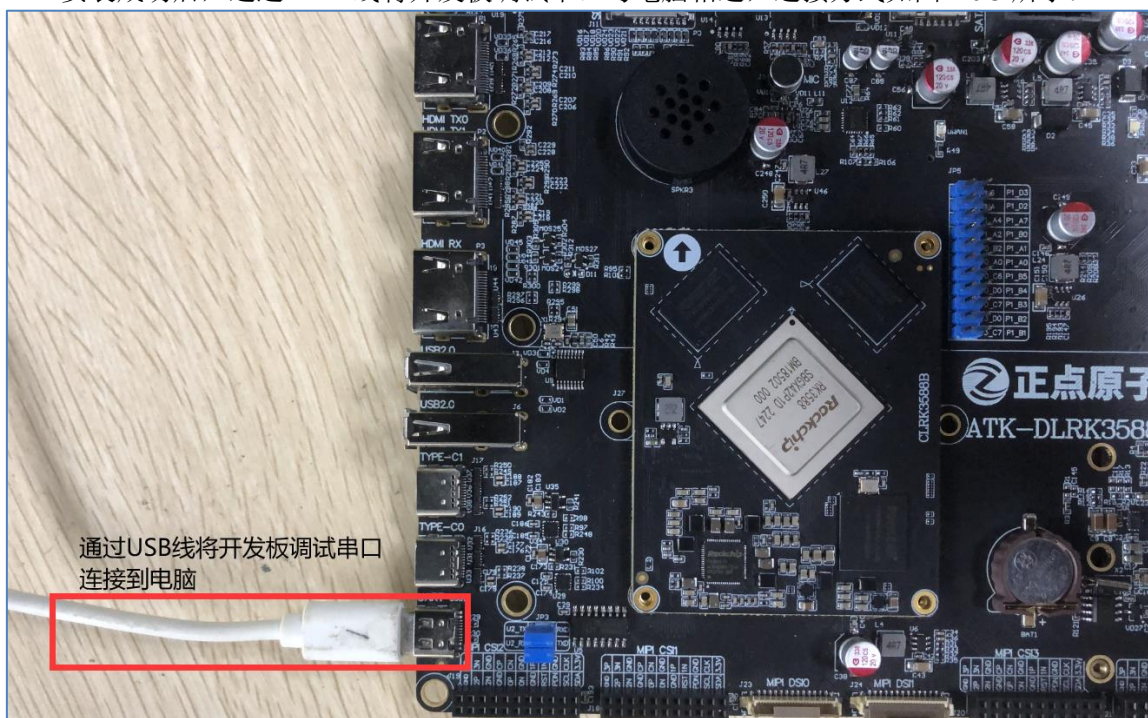


图 2.6.3 开发板调试串口与电脑相连

连接成功后，此时电脑会检测到一个 USB 设备，打开 Windows 设备管理器：

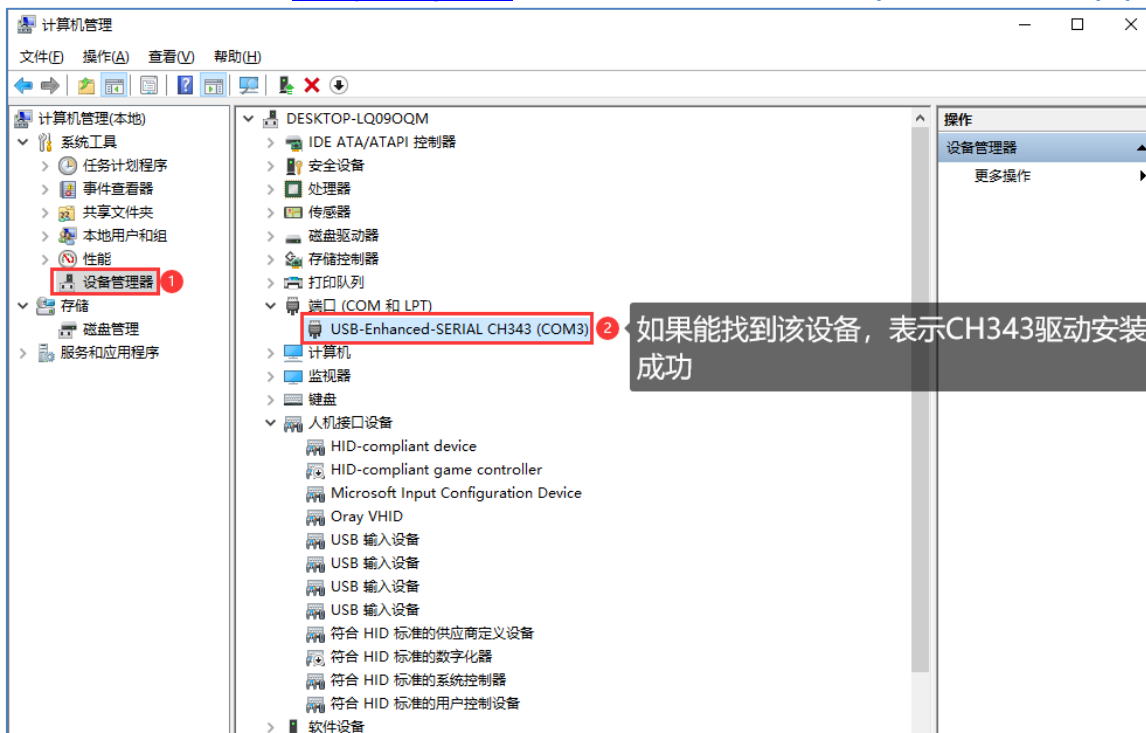


图 2.6.4 Windows 设备管理器

如果在“设备管理器→端口（COM 和 LPT）”下能找到一个名为“USB-SERIALCH343”的设备，则表示 CH343 驱动安装成功！如果找不到名为“USB-SERIALCH343”的设备，请用户自行检查硬件连接是否有误！可拔掉 USB 线、重新连接，或者连接电脑的其它 USB 口试试。

2.7 MobaXterm 软件安装

MobaXterm 是一款多功能远程终端软件，功能强大、而且免费（也有收费版本），支持创建 SSH、Telnet、Rsh、Xdmcp、RDP、VNC、FTP、SFTP、串口(Serial COM)等超多远程连接功能。MobaXterm 提供了人性化的操作界面，功能十分强大，所以推荐用户使用 MobaXterm 这款终端软件。

2.7.1 MobaXterm 软件下载

开发板资料盘中已经给用户提供了 MobaXterm 软件安装包，路径为：**开发板光盘 A 盘-基础资料→04、软件→MobaXterm_Installer_v12.3.zip**；用户也可以通过链接地址：<https://mobaxterm.mobatek.net/download.html>，自己下载：

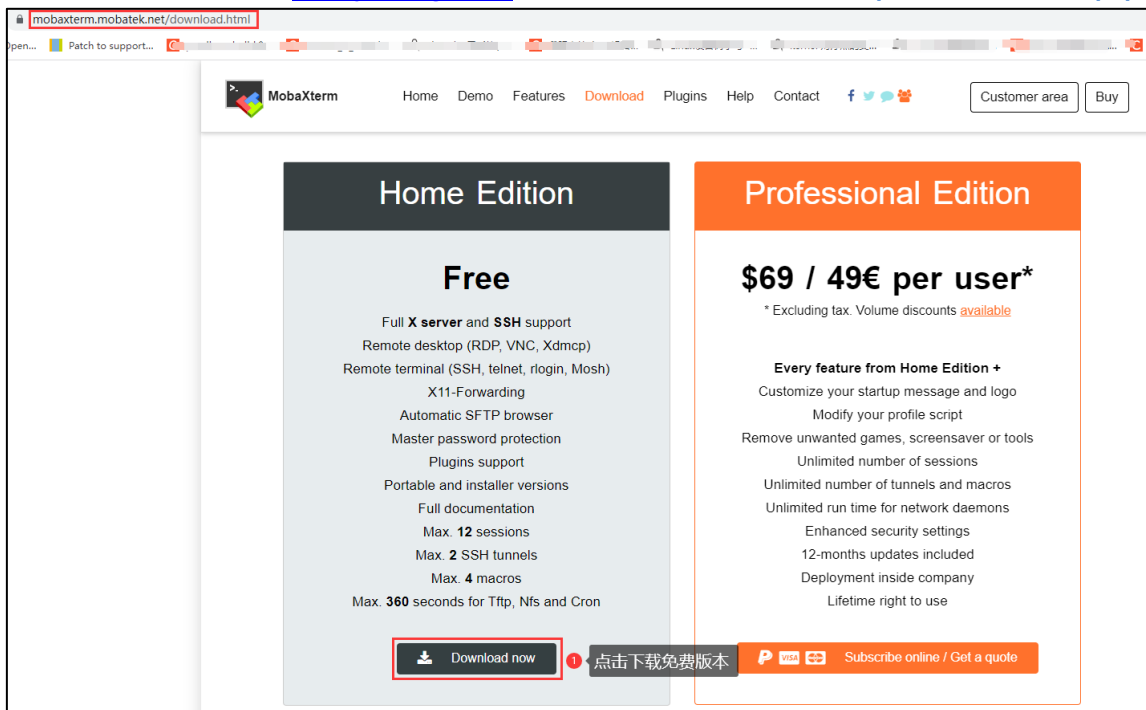


图 2.7.1.1 下载 MobaXterm 软件(1)



图 2.7.1.2 下载 MobaXterm 软件(2)

下载完成后会得到一个名为 MobaXterm_Installer_vxxx.zip 的压缩包文件 (xxx 为版本号), 目前最新版本为 22.2, 资料包中给用户提供的安装包对应的版本为 12.3, 用新的版本也行, 旧的版本也可以, 这个都没什么关系, 这里我们以 12.3 版本为例。

2.7.2 MobaXterm 软件安装

将 MobaXterm_Installer_v12.3.zip 压缩包文件解压, 解压之后如图所示:

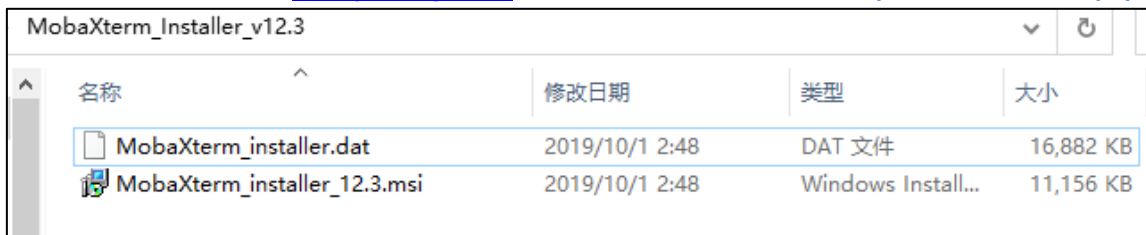


图 2.7.2.1 MobaXterm_Installer_v12.3.zip 解压后的文件

接着双击运行 MobaXterm_Installer_v12.3.msi 文件，按照图 2.7.2.2~2.7.2.6 所示步骤安装 MobaXterm 软件：

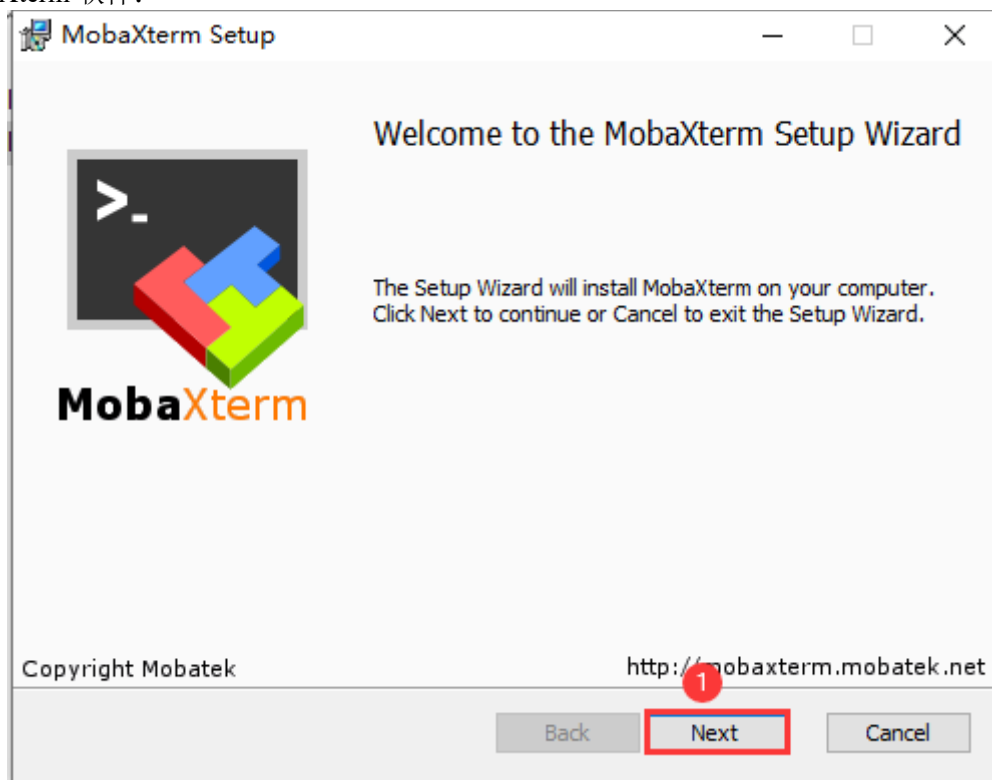


图 2.7.2.2 安装 MobaXterm 软件(1)

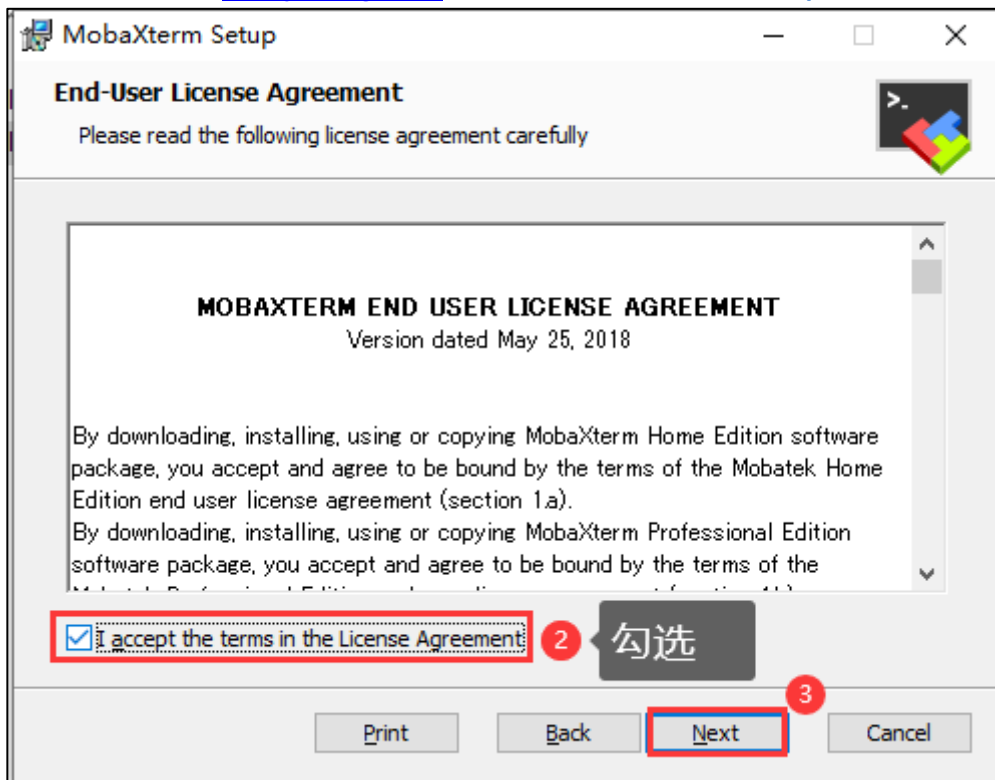


图 2.7.2.3 安装 MobaXterm 软件(2)

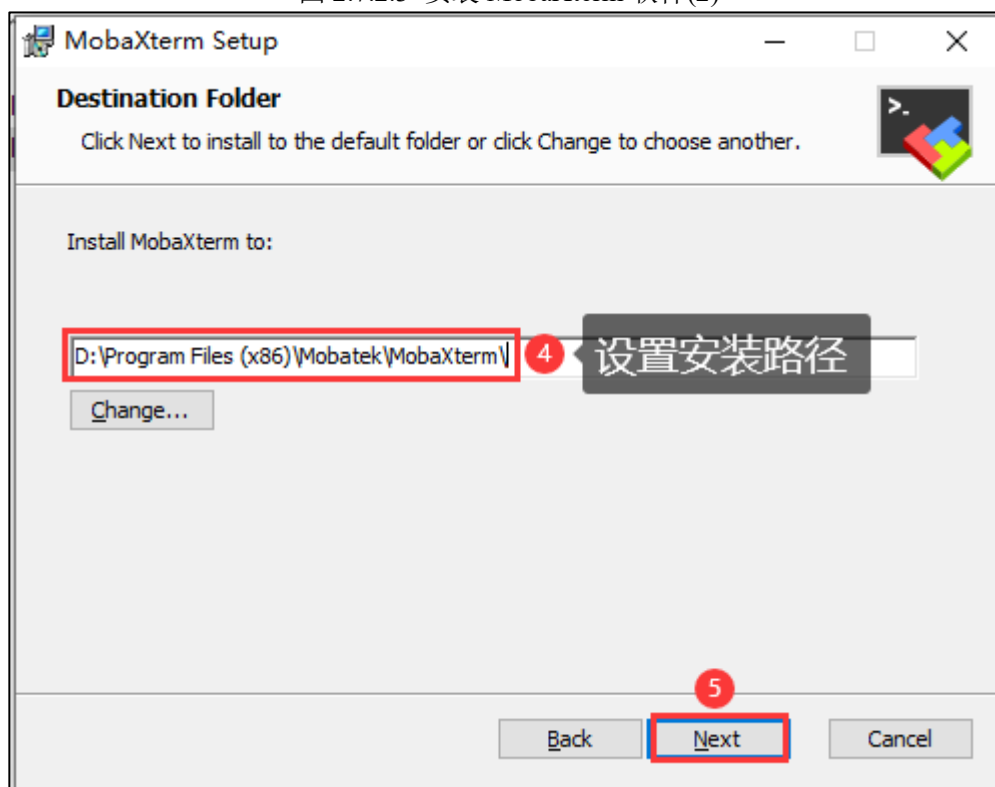


图 2.7.2.4 安装 MobaXterm 软件(3)

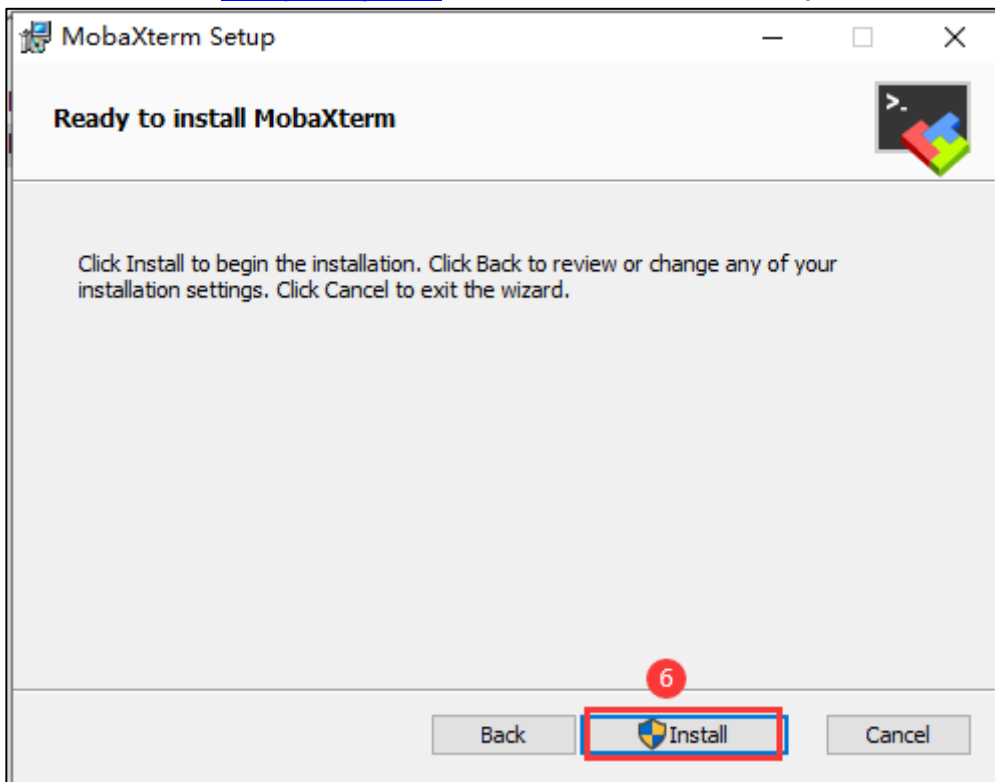


图 2.7.2.5 安装 MobaXterm 软件(4)

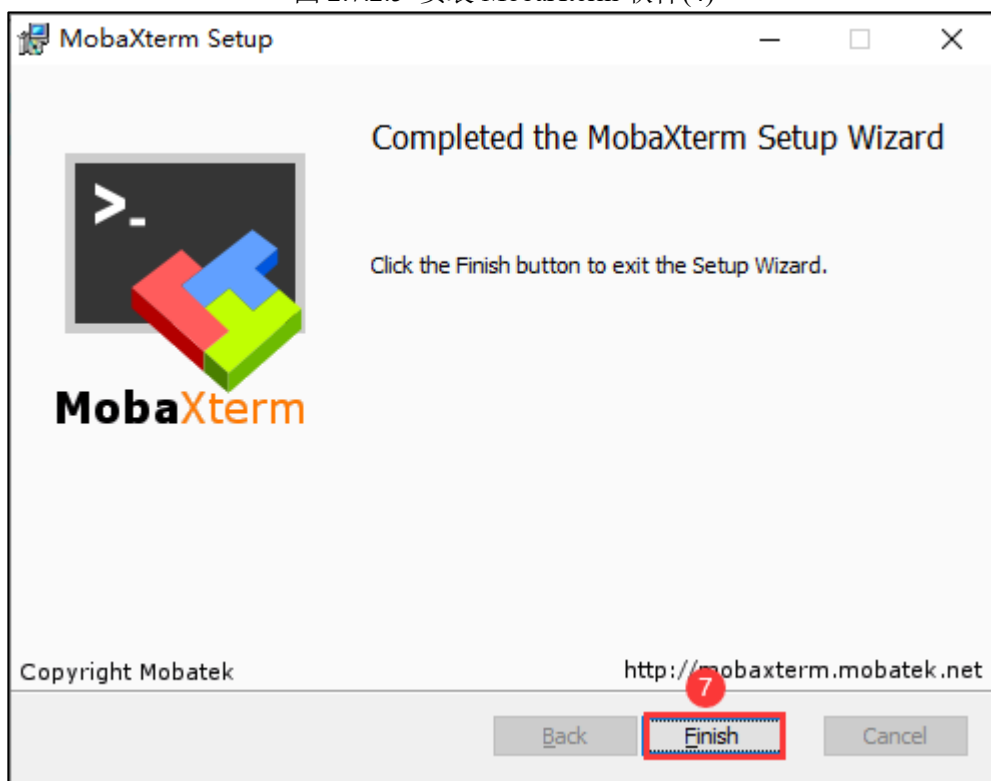


图 2.7.2.6 安装 MobaXterm 软件(5)

至此，软件安装完成，桌面会自动生成 MobaXterm 软件快捷方式图标：

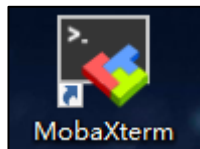


图 2.7.2.7 MobaXterm 软件桌面图标

2.7.3 MobaXterm 软件的使用

双击 MobaXterm 桌面图标打开该软件，如图 2.7.3.1 所示：

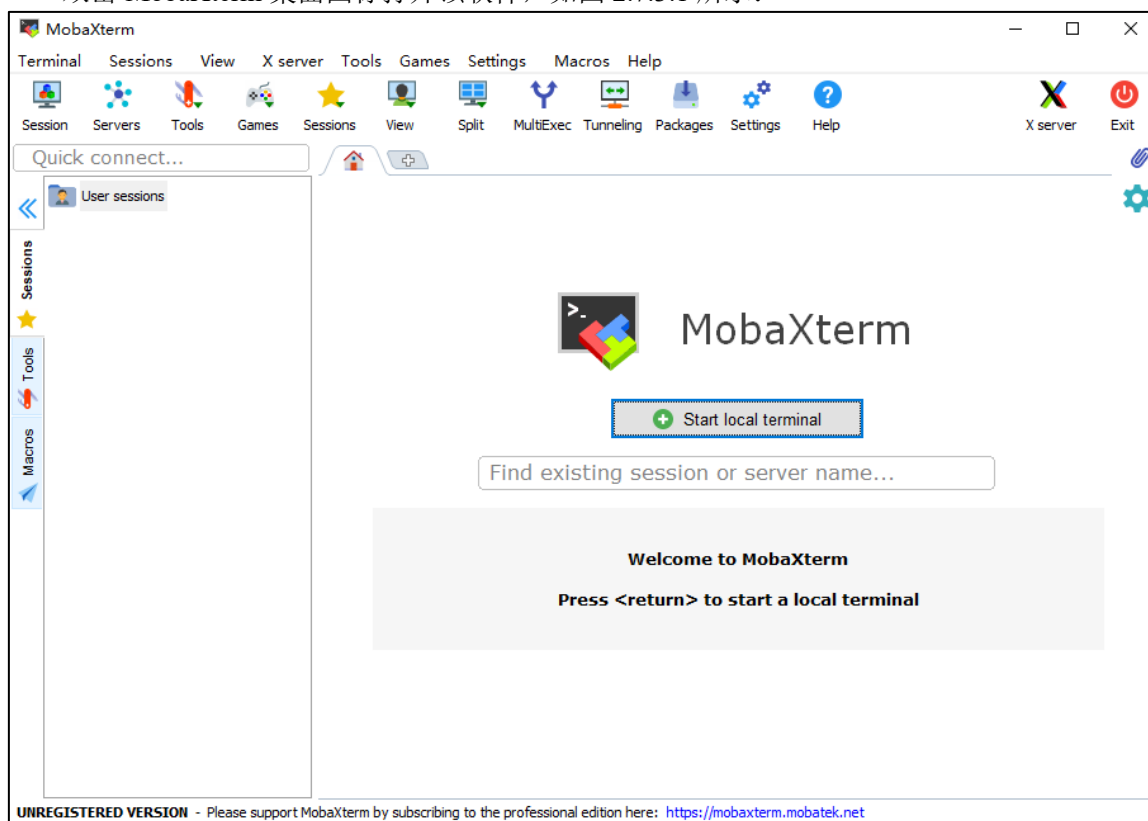


图 2.7.3.1 MobaXterm 软件主界面

接下来向大家介绍如何建立 Serial（串口）连接以及 ssh 远程连接。

1. 串口连接

ATK-RK3588 开发板上有一个调试串口，可直接通过 USB 线将其连接到电脑。串口作为嵌入式设备最为常见的通信接口之一，不但能实现计算机与嵌入式设备之间的数据传输，而且还能实现计算机对嵌入式设备的控制，嵌入式开发过程中，通常将其作为调试接口（串口调试），用于调试嵌入式设备。

按照图 2.7.3.2~2.7.3.3 所示操作步骤建立一个 Serial（串口）连接（在建立连接之前，需要通过 USB 线将开发板的调试串口与电脑相连、并且已经安装了 CH343 驱动）：

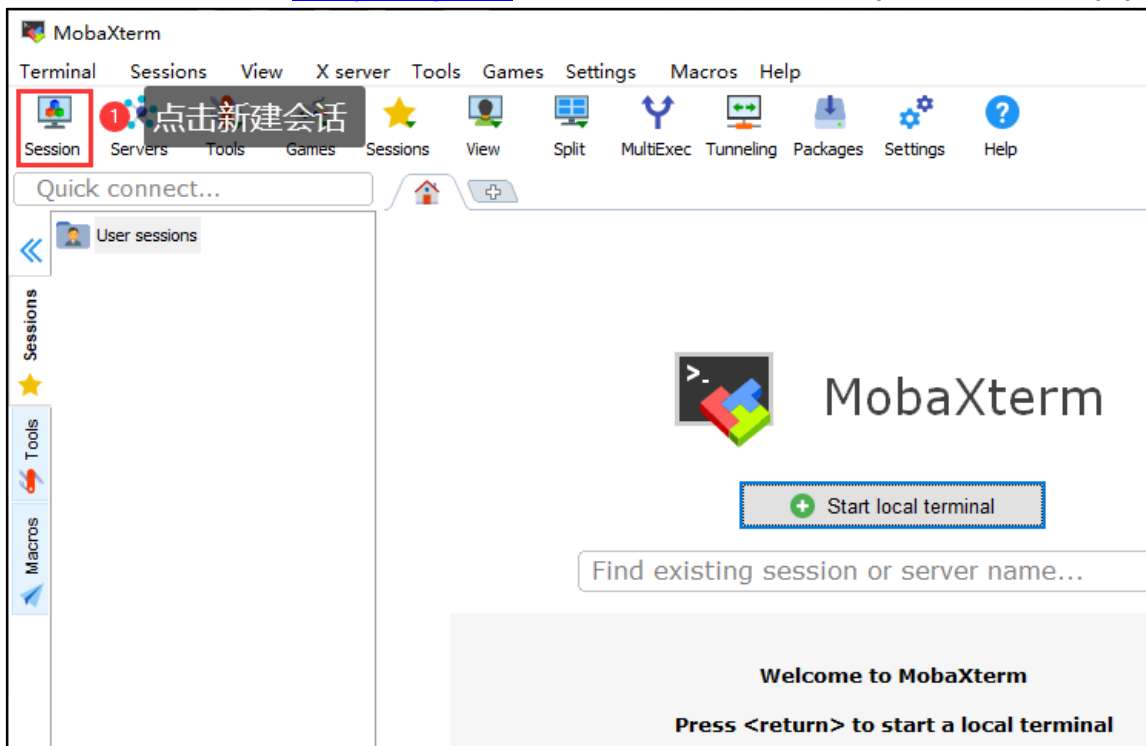


图 2.7.3.2 建立串口连接(1)

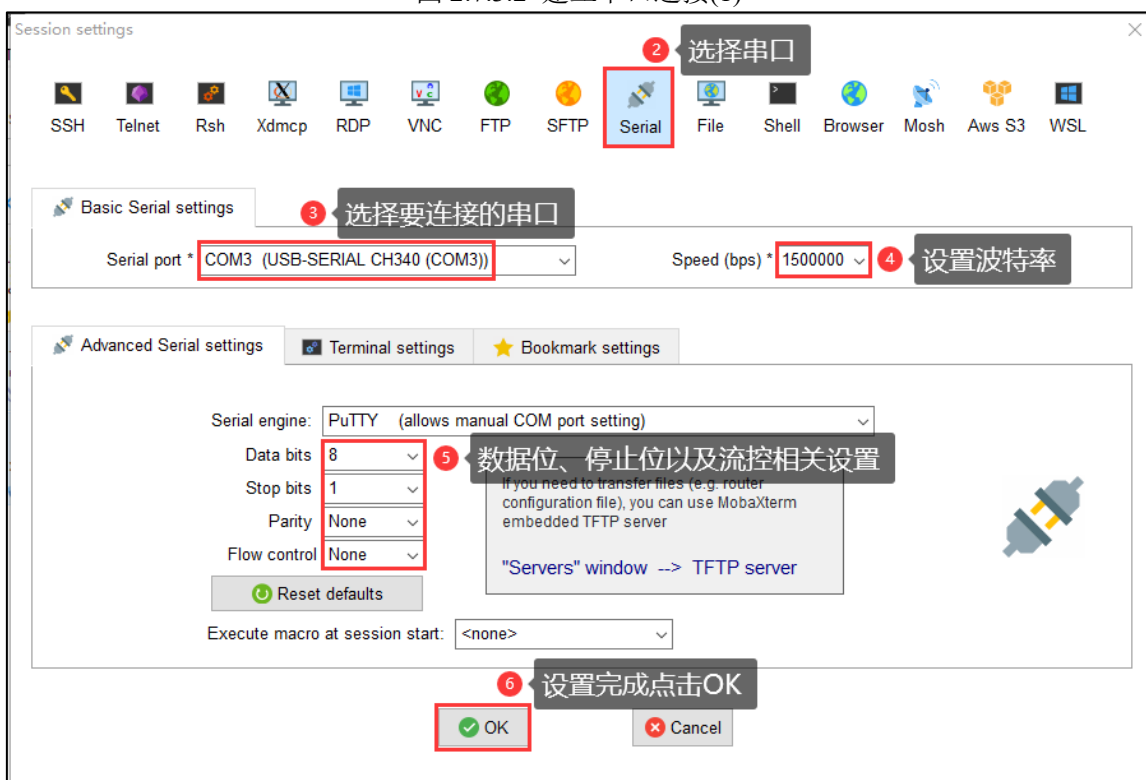


图 2.7.3.3 建立串口连接(2)

首先选择需要进行连接的串口, 确保开发板的调试串口与电脑已经通过 USB 线相连、并且 CH343 驱动已经安装成功 (“设备管理器→端口 (COM 和 LPT)” 下能找到一个名为 “USB-SERIAL CH343” 的设备), 那么 MobaXterm 软件才能识别到开发板的调试串口, 我们便可以在 “Serial port” 下拉列表中找到开发板对应的串口 (USB-SERIAL CH343), 然后选择它即可!

接着设置串口通信波特率, 根据实际情况进行设置, RK3588 平台的调试串口, 其默认波特率为 1500000; 然后设置数据位、停止位以及流控等, 设置完成后点击 “OK” 按钮。

Serial 连接就建立成功了, 如图 2.7.3.4 所示:

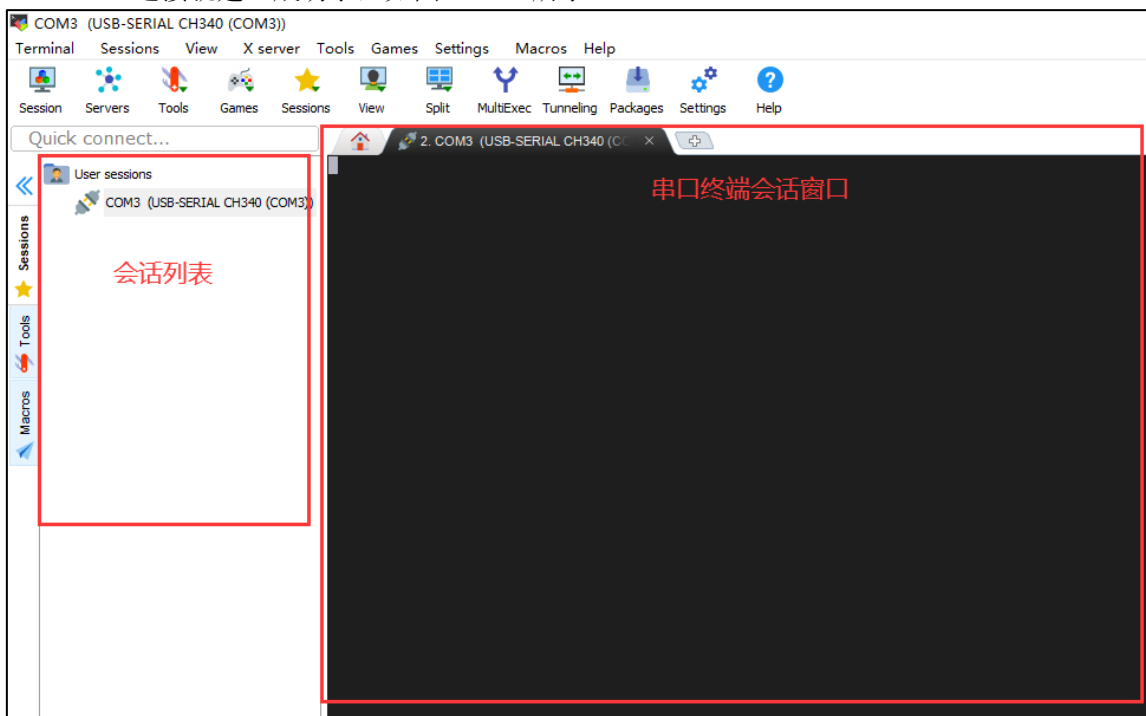


图 2.7.3.4 建立串口连接(3)

开发板上电启动, 运行 Linux 系统或 Android 系统, 系统启动过程中的 log 信息将会在串口终端会话窗口显示出来, 如图 2.7.3.5 所示:

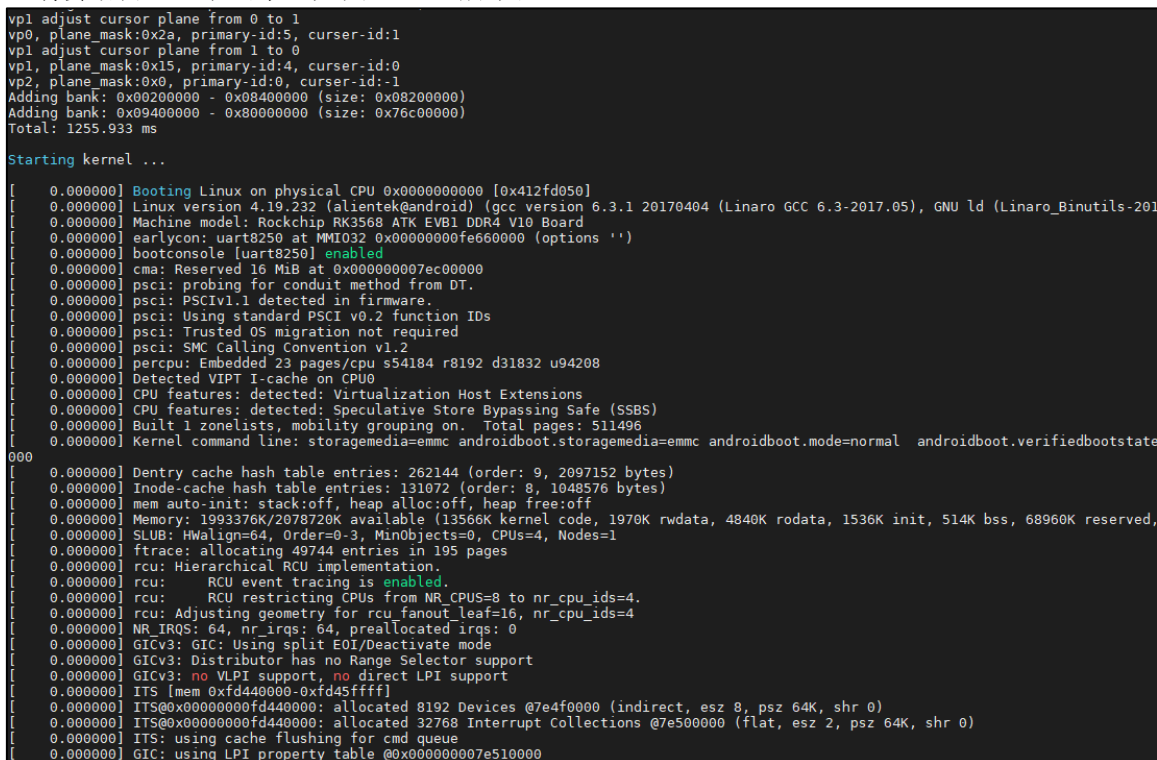


图 2.7.3.5 系统启动 log 信息

系统启动成功后, 用户可以通过串口终端执行命令、命令执行结果也会通过串口终端显示出来。

2. ssh 连接

接下来介绍如何建立 ssh 远程连接, 通过 ssh 可以实现远程登录, 譬如远程登录 Ubuntu 系统、远程登录开发板 Linux 系统; 按照图 2.7.3.5~2.7.3.6 所示操作步骤建立 ssh 远程连接、实现远程登录操作 (以远程登录 Ubuntu 系统为例):

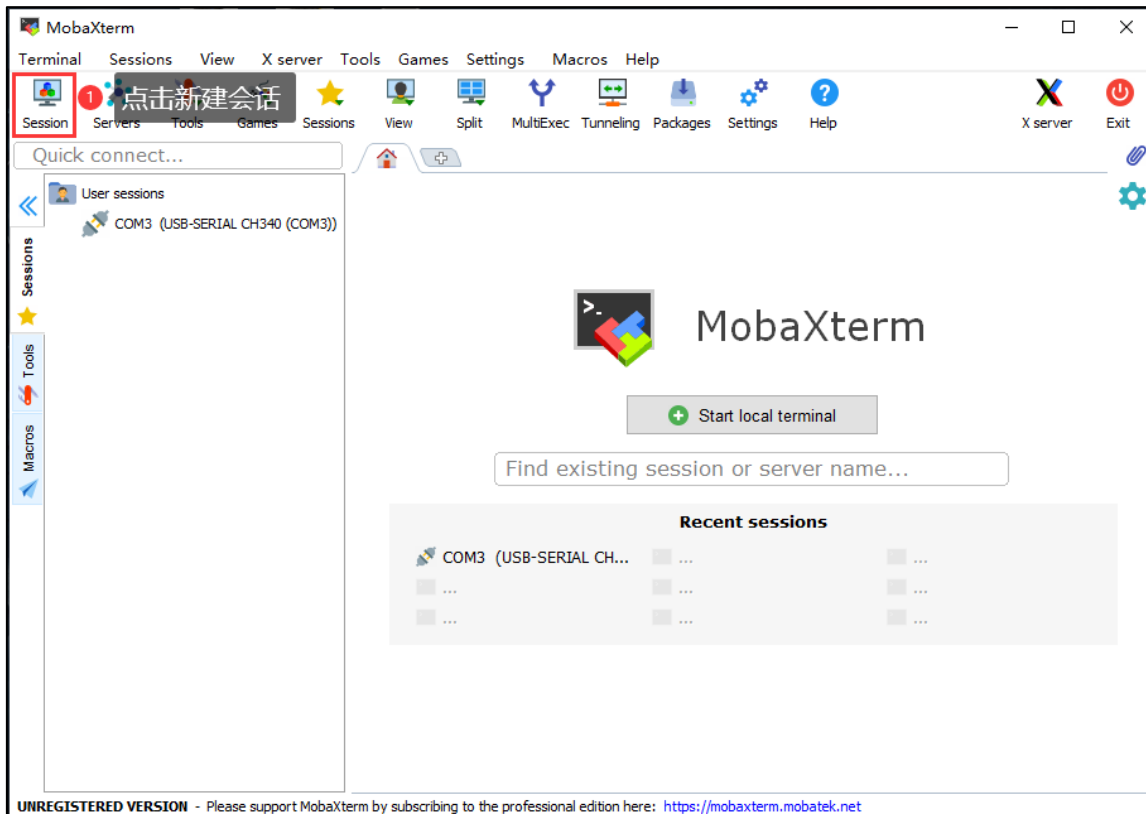


图 2.7.3.6 建立 ssh 远程连接(1)

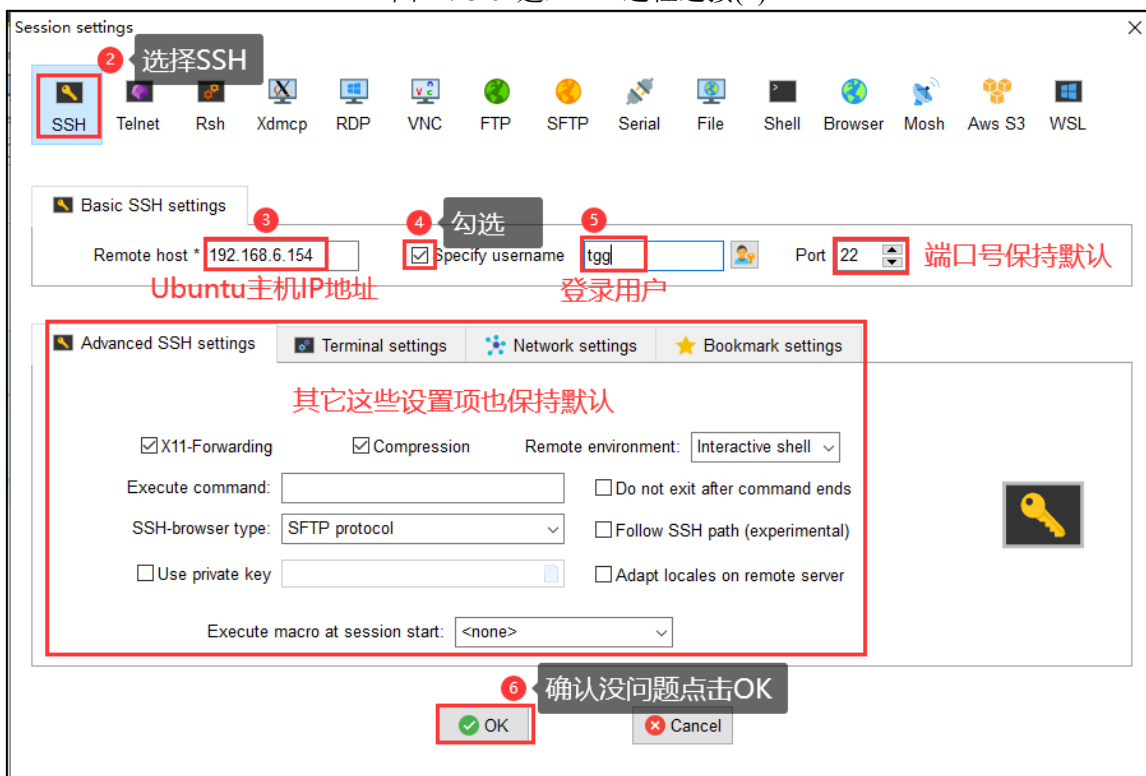


图 2.7.3.7 建立 ssh 远程连接(2)

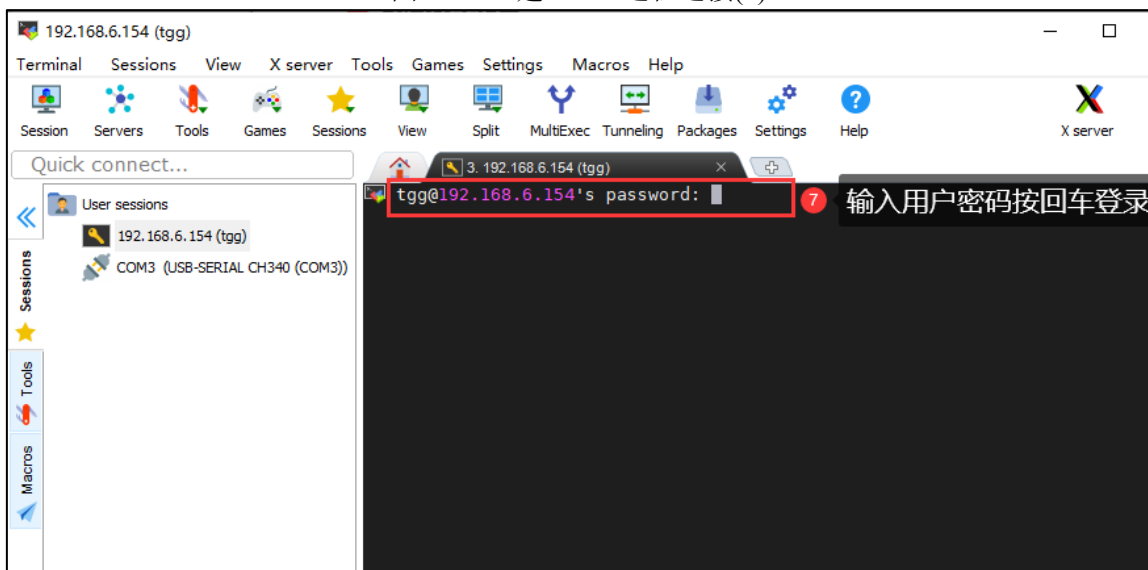


图 2.7.3.8 建立 ssh 远程连接(3)



图 2.7.3.9 建立 ssh 远程连接(4)

弹出窗口询问用户是否需要保存密码，我们可以选择“**Yes**”保存密码、也可以选择“**No**”不保存，取决于个人。至此，ssh 远程连接就创建成功了，如图 2.7.3.9 所示：

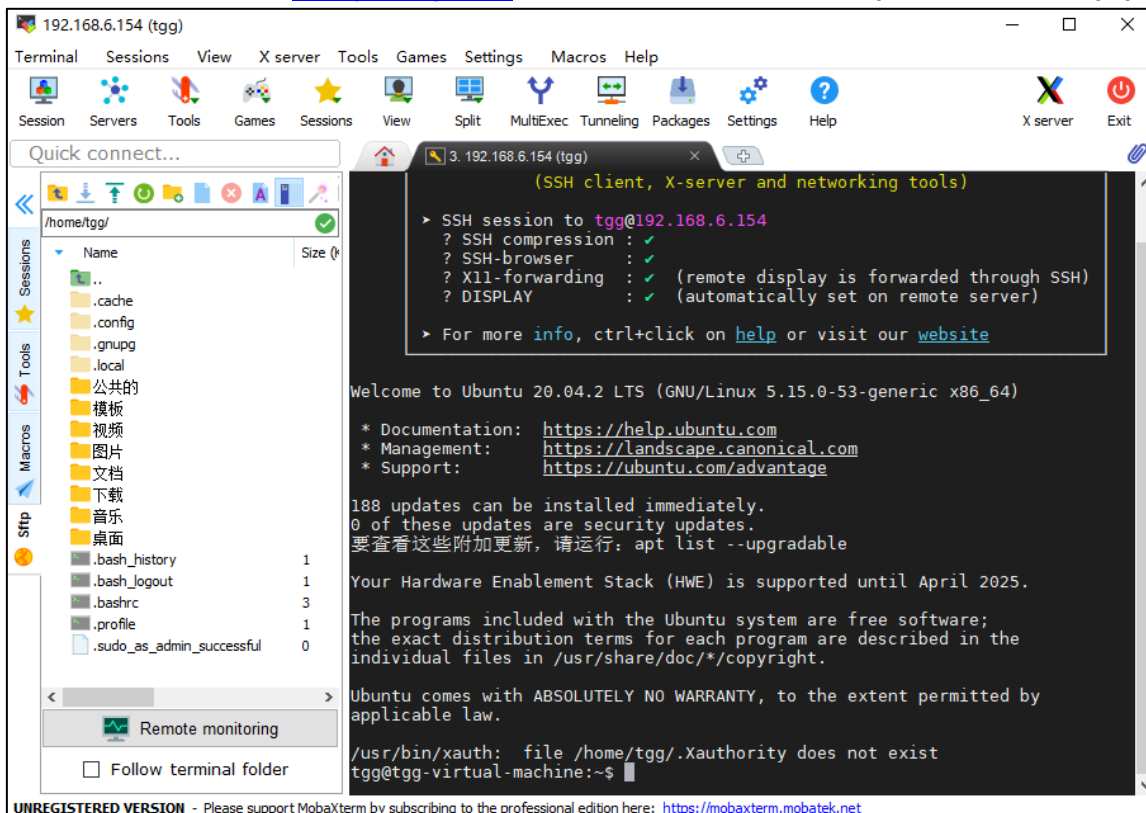


图 2.7.3.10 ssh 远程登录成功

在 ssh 远程登录的情况下，我们也可通过 MobaXterm 软件来实现 Windows 与 Ubuntu 之间的文件互传（通过 sftp 协议进行文件传输），具体方式如图 2.7.3.11 所示：

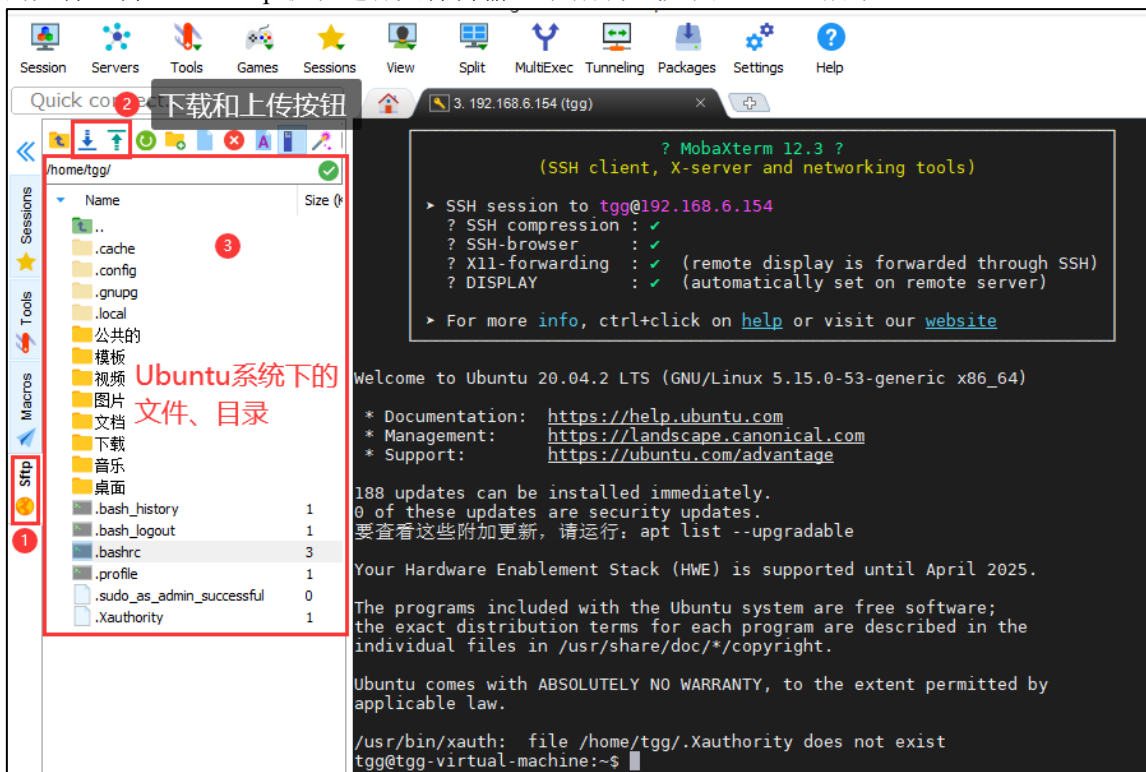


图 2.7.3.11 MobaXterm 实现 Windows 与 Ubuntu 之间文件互传

首先需要选中①“Sftp”选项卡，通过“下载”按钮可以将 Ubuntu 系统下的文件拷贝到 Windows 系统，通过“上传”按钮可以将 Windows 系统下的文件拷贝到 Ubuntu 系统。也可

以直接使用鼠标左键拖动, ③所示区域便是 Ubuntu 系统下的文件、目录, 可以直接使用鼠标左键将 Ubuntu 系统下的文件拖动至 Windows 系统、也可以将 Windows 系统下的文件拖动至 Ubuntu 系统。

2.8 Rockchip USB 驱动安装

在开发调试阶段, 经常需要将设备切换至 **Loader** 模式或者 **Maskrom** 模式, 譬如使用瑞芯微开发工具(RKDevTool)烧录镜像时(需连接 USB 线), 设备需要工作在 Loader 模式或 Maskrom 模式, 此时才可进行烧写; 需要在 Windows 下安装 Rockchip USB 驱动才能识别到设备。

开发板资料盘中已经给用户提供了 Rockchip USB 驱动安装包, 路径为: **开发板光盘 A 盘-基础资料→05、开发工具→RKTools→windows→DriverAssitant_v5.12.zip**, 支持 xp、win7_32、win7_64、win10_32、win10_64 等操作系统; 将 DriverAssitant_v5.12.zip 压缩文件解压, 解压之后得到如图 2.8.1 所示文件、目录列表:

名称	修改日期	类型	大小
ADBDriver	2020/11/10 14:13	文件夹	
bin	2020/11/10 14:14	文件夹	
Driver	2022/2/28 14:14	文件夹	
config.ini	2014/6/3 15:38	配置设置	1 KB
DriverInstall.exe	2022/2/28 14:11	应用程序	491 KB
Readme.txt	2018/1/31 17:44	文本文档	1 KB
revision.log	2022/2/28 14:14	文本文档	1 KB

图 2.8.1 DriverAssitant_v5.12.zip 解压

直接双击 DriverInstall.exe 文件, 按照图 2.8.2~2.8.4 所示步骤安装 Rockchip USB 驱动:

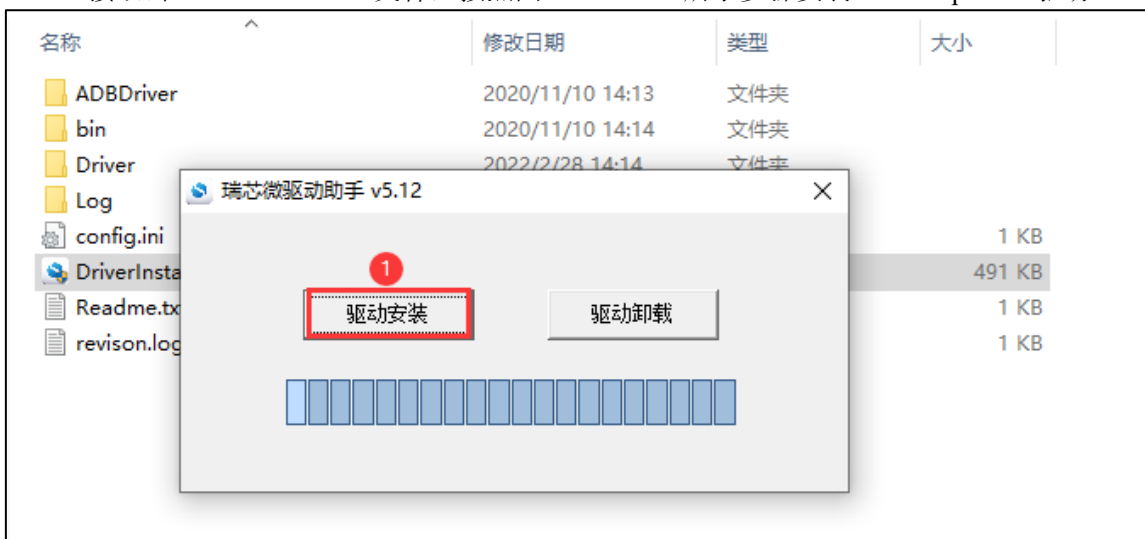


图 2.8.2 安装 Rockchip USB 驱动(1)

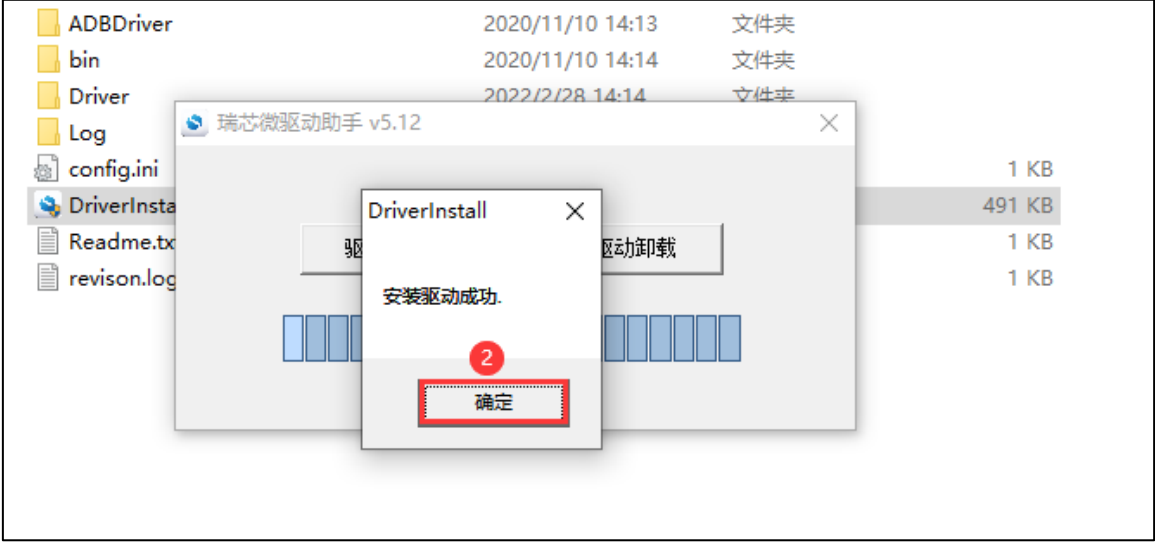


图 2.8.3 安装 Rockchip USB 驱动(2)

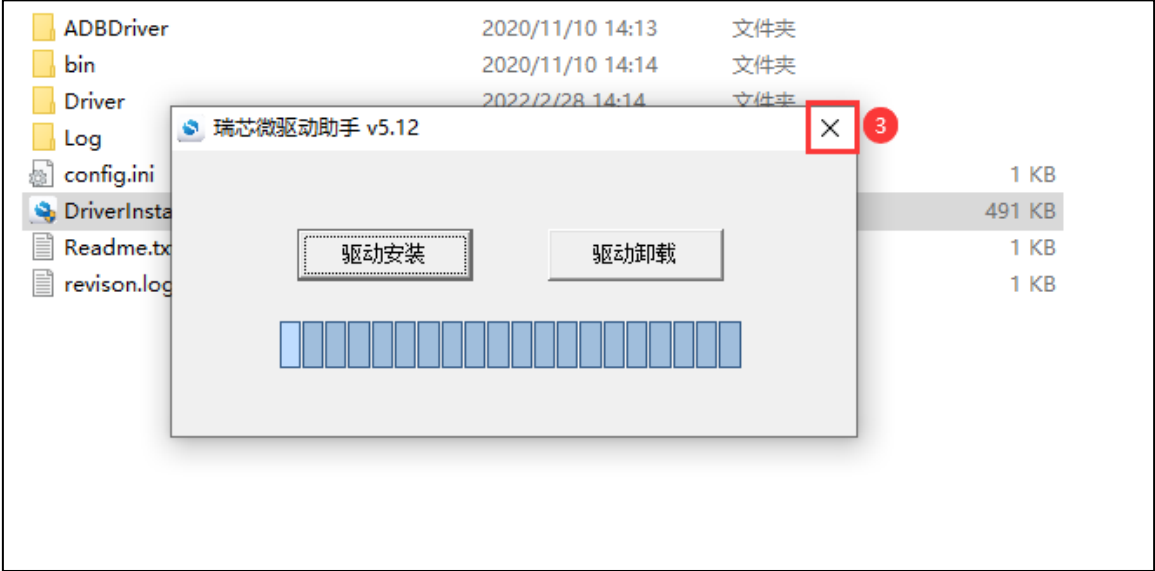


图 2.8.4 安装 Rockchip USB 驱动(3)

2.9 镜像烧录工具的使用

Rockchip 平台提供了多种镜像烧写方式，譬如在 **Windows** 下通过 **RKDevTool**（瑞芯微开发工具）工具烧写、通过 **SD 卡** 方式烧写、通过 **FactoryTool** 工具批量烧写（量产烧写工具、支持 **USB 一拖多** 烧写）以及在 **Ubuntu** 系统下通过 **Linux_Upgrade_Tool** 工具烧写等等，总之，烧写镜像的方式有很多种，用户可以选择合适的烧写方式进行烧写，将镜像文件烧写至开发板。

本小节简单介绍下 **Windows** 下的烧写工具 **RKDevTool** 以及 **Linux** 下的烧写工具 **Linux_Upgrade_Tool**，在我们开发调试过程中，这两个应该是使用最多的烧写工具。

2.9.1 烧写模式介绍

Rockchip 平台硬件设备运行的几种模式如下表所示，只有当设备处于 **Maskrom** 以及 **Loader** 模式时（**必须要通过 USB 线将开发板的 TYPE-C0 口连接到电脑**），才能够烧写镜像，或对板上镜像进行更新操作。

模式	是否支持烧录	描述
Maskrom	支持	Flash 在未烧录镜像时，芯片会引导进入 Maskrom 模式，可以进行初次镜像的烧写；开发调试过程中若遇到无法

		正常进入 Loader 模式的情况, 也可进入 Maskrom 模式烧写镜像。
Loader	支持	Loader 模式下可以进行镜像烧写、更新升级, 可以通过烧写工具单独烧写某一个分区镜像文件, 方便调试。
Recovery	不支持	系统引导 recovery 启动, 主要作用是升级、恢复出厂设置类操作。
Normal Boot	不支持	系统正常启动后进入的模式, 通过引导 rootfs 启动, 加载 rootfs, 大多数的开发都是在这个模式下调试的。

表 2.9.1.1 Rockchip 平台运行模式说明

进入 Maskrom 烧写模式的方法: (先连接电源适配器和 TYPE-C0)

- 开发板未烧录过镜像, 上电之后就会进入到 Maskrom 模式;
- 开发板烧录过镜像, 按住开发板上的 **UPDATE 按键**, 然后开发板上电或复位, 系统将会进入到 Maskrom 模式;
- 开发板烧录过镜像, 在 U-Boot 命令行下, 执行 “**rbrom**” 命令进入到 Maskrom 模式。

进入 Loader 烧写模式的方法: (先连接电源适配器和 TYPE-C0)

- 开发板烧录过镜像, 按住 **V+按键** (音量+) 按键, 然后开发板上电或复位, 系统将会进入到 Loader 模式;
- 开发板烧录过镜像, 在 U-Boot 命令行下, 执行 “**download**” 命令进入到 Loader 模式;
- 开发板烧录过镜像, 在 Android 系统下, 通过串口或 ADB 执行命令 “**reboot loader**” 重启进入到 Loader 模式;
- 开发板烧录过镜像, 上电或复位后开发板正常启动进入到系统, 瑞芯微开发工具 (也就是 RKDevTool 工具) 会显示 “发现一个 ADB 设备”, 然后点击 “**切换**” 按钮, 进入到 Loader 模式, 如下图。

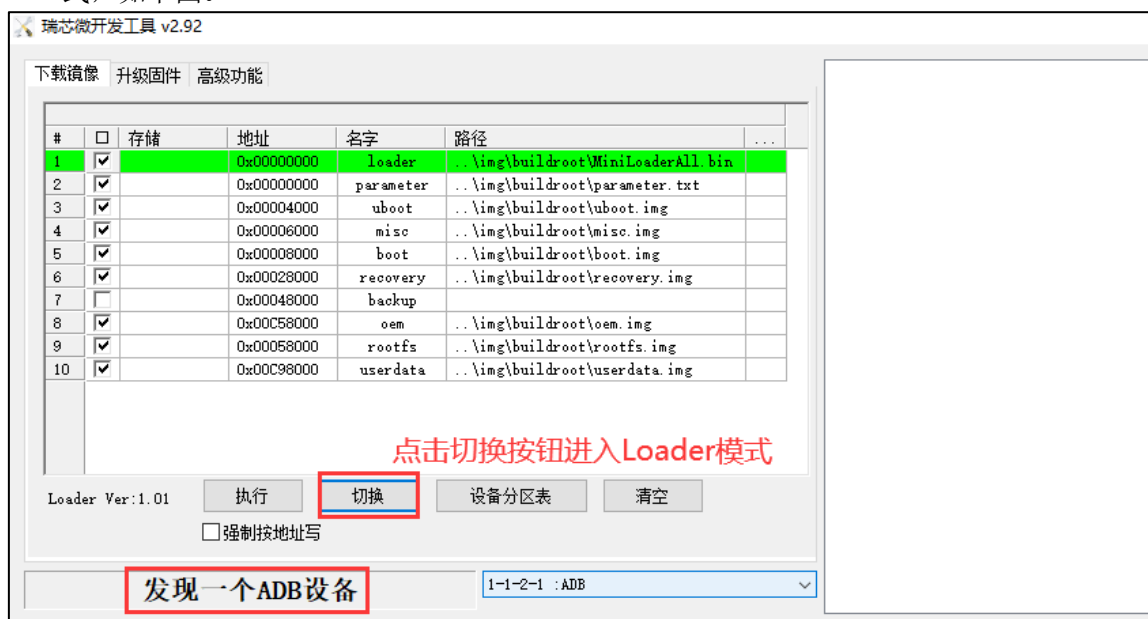


图 2.9.1.1 通过瑞芯微开发工具进入 Loader 模式

以上给大家介绍了开发板如何进入到 Maskrom 和 Loader 这两种烧写模式, 只有进入 Maskrom 模式或 Loader 模式后才可进行烧录。

2.9.2 Windows 烧录工具 RKDevTool

在 Windows 下烧写镜像使用 RKDevTool 工具, RK 的文档中一般也将其称为瑞芯微开发工具, 该工具除了用于烧录镜像之外, 还有其它的一些功能, 是我们开发、调试过程中最常用的工具。

正点原子 ATK-DLRK3588 开发板资料盘中已经给用户提供了该工具, 路径为: **开发板光盘 A 盘-基础资料→05、开发工具→RKTools→windows→RKDevTool_Release_v2.92.zip**, 当然该工具在 RK3588 Android12/13 SDK 中也可以找到。

首先将 RKDevTool_Release_v2.92.zip 压缩文件解压开来, 解压之后的内容如下所示:

RKDevTool_Release_v2.92 >				
名称	修改日期	类型	大小	
bin	2017/9/11 15:07	文件夹		
Language	2017/9/11 15:07	文件夹		
config.cfg	2021/11/30 11:06	Configuration 源...	7 KB	
config.ini	2021/11/30 11:04	配置设置	2 KB	
revision.txt	2021/11/30 11:06	文本文档	2 KB	
RKDevTool.exe	2021/12/7 16:34	应用程序	1,205 KB	
RKDevTool_manual_v1.2_cn.pdf	2023/7/8 10:39	Foxit PDF Reade...	530 KB	
RKDevTool_manual_v1.2_en.pdf	2023/7/8 10:39	Foxit PDF Reade...	448 KB	
开发工具使用文档_v1.0.pdf	2021/8/27 10:28	Foxit PDF Reade...	450 KB	

图 2.9.2.1 RKDevTool_Release_v2.92 目录

该目录下有两份瑞芯微官方提供的文档:《**开发工具使用文档_v1.0.pdf**》、《**RKDevTool_manual_v1.2_xx.pdf**》, 向用户详细介绍了如何使用 RKDevTool 工具(瑞芯微开发工具), 该工具的详细使用方法大家可以参考这两份文档, 我们这里只是做一个简单的介绍。

双击 **RKDevTool.exe** 可执行文件打开瑞星微开发工具, 如下图所示:

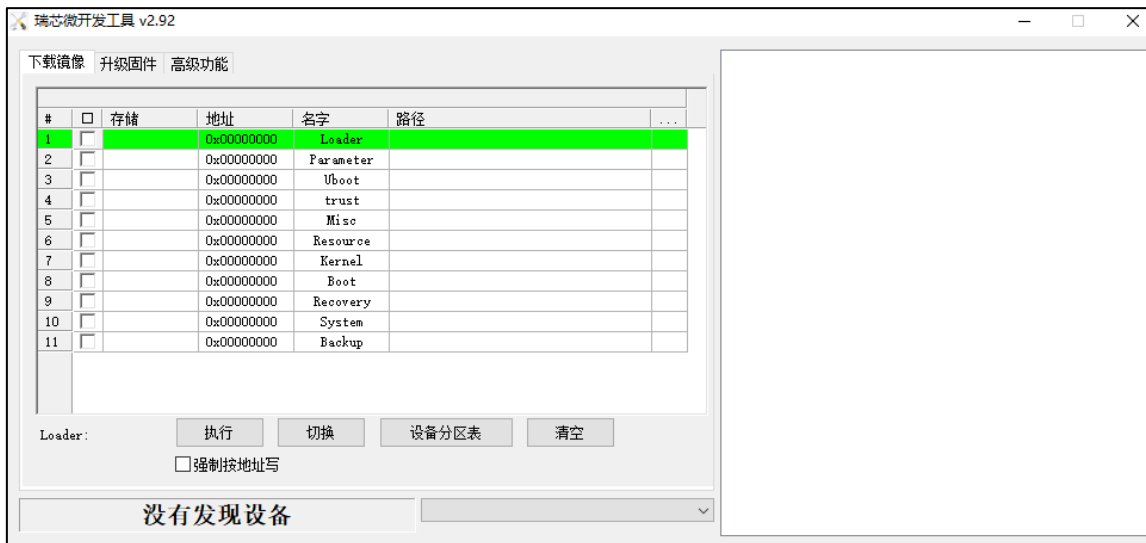


图 2.9.2.4 瑞星微开发工具界面

上图便是瑞芯微开发工具的一个主界面, RKDevTool 工具有两种镜像烧录方式:

- **单独烧录各个分立镜像**。Android SDK 编译完成后会生成多个镜像文件, 包括 boot.img、baseparameter.img、MiniLoaderAll.bin、misc.img、dtbo.img、recovery.img、super.img、uboot.img、vbmeta.img 等多个镜像文件, 其中还包括一个分区表文件 parameter.txt; 使用 RKDevTool 工具可单独烧录这些镜像。
- **烧录完整 update.img 固件**。update.img 并非编译得到, 它其实是由多个镜像打包而成, 包括 uboot.img、boot.img、dtbo.img、misc.img、vbmeta.img、recovery.img、super.img、MiniLoaderAll.bin 等, 是这些分立镜像的集合体; 通过 RK 提供的工具可将各个分立镜像打包成一个 update.img。

接下来讲一下如何以分立镜像的形式进行烧录，2.9.3 小节将向用户介绍如何烧录 update.img 固件。

按照图 2.9.2.5~2.9.2.8 所示操作步骤，导入一个.cfg 配置文件（配置文件中包含了烧录时的配置信息，包括烧录地址、镜像的路径等信息，当然也可自己手动配置这些信息，后面会讲）：

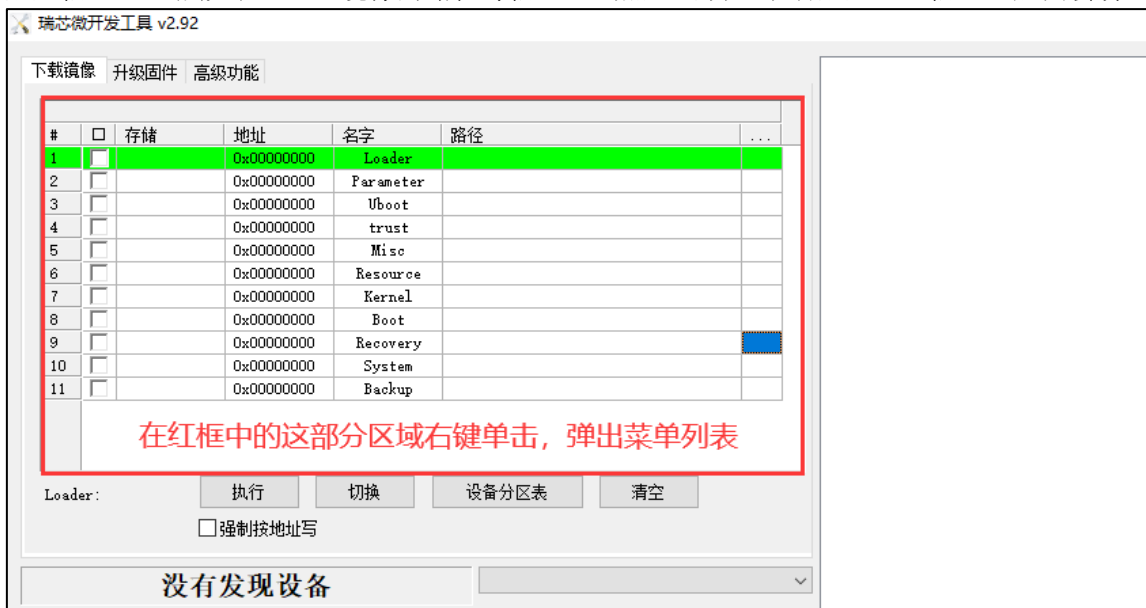


图 2.9.2.5 导入配置文件(1)

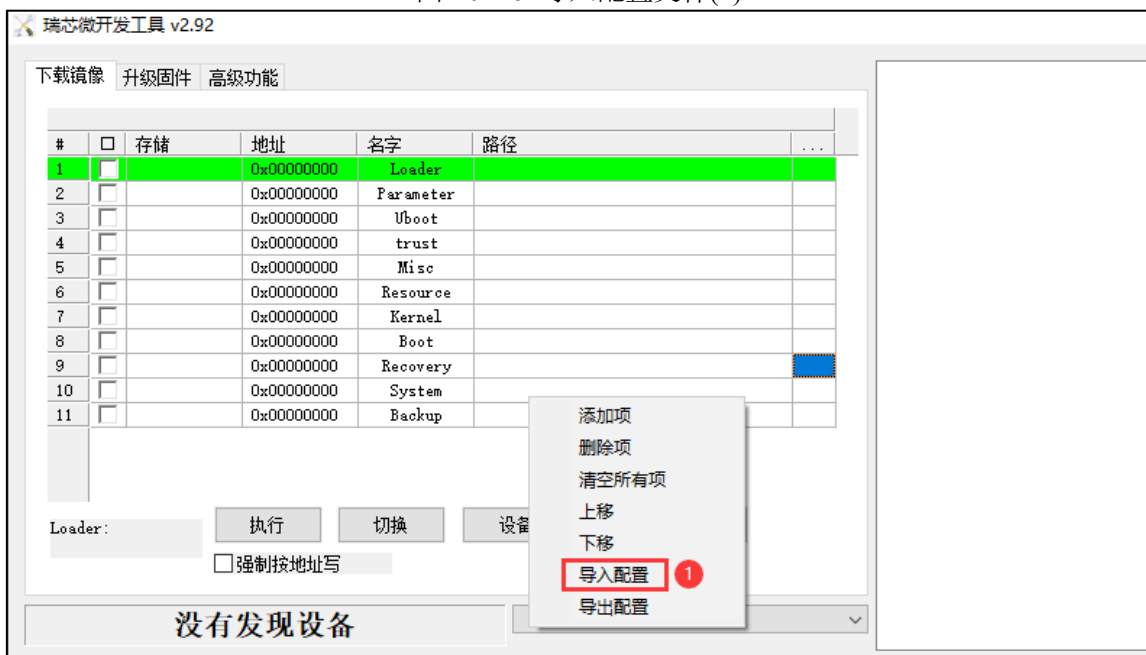


图 2.9.2.6 导入配置文件(2)

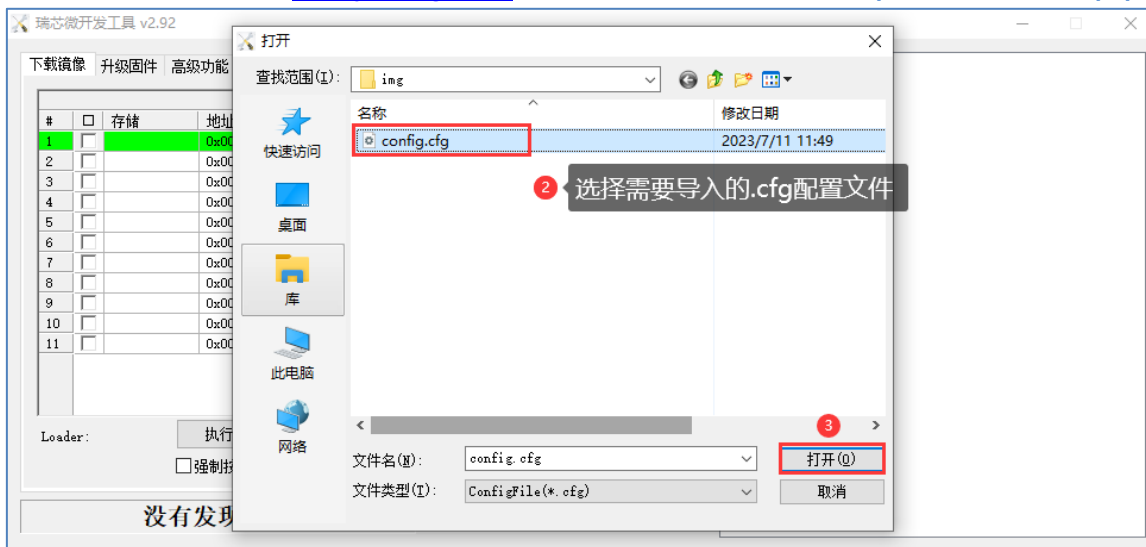


图 2.9.2.7 导入配置文件(3)

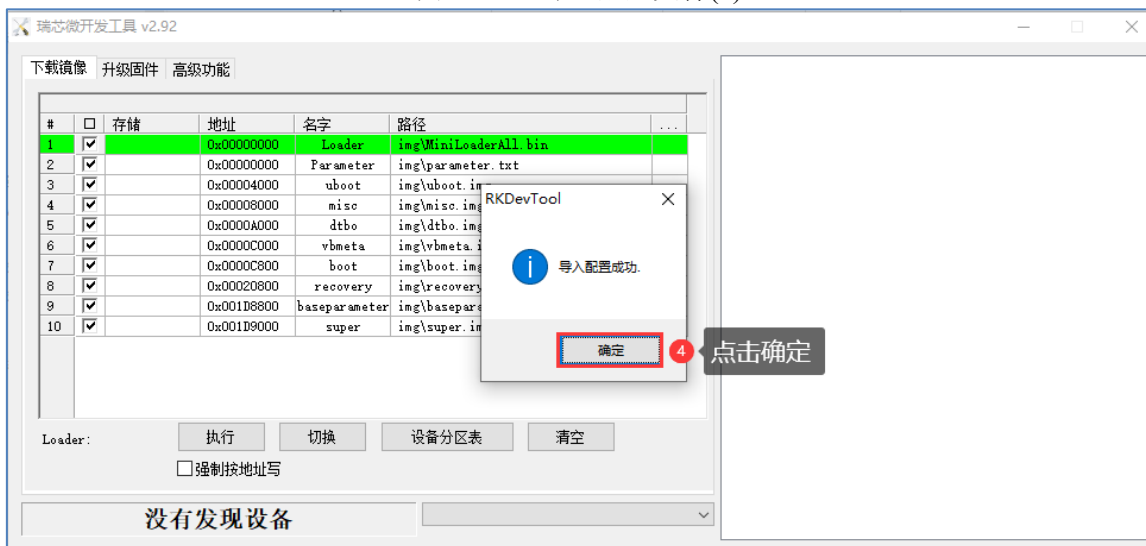


图 2.9.2.8 导入配置文件(4)

至此，我们就导入了一个烧录配置文件：

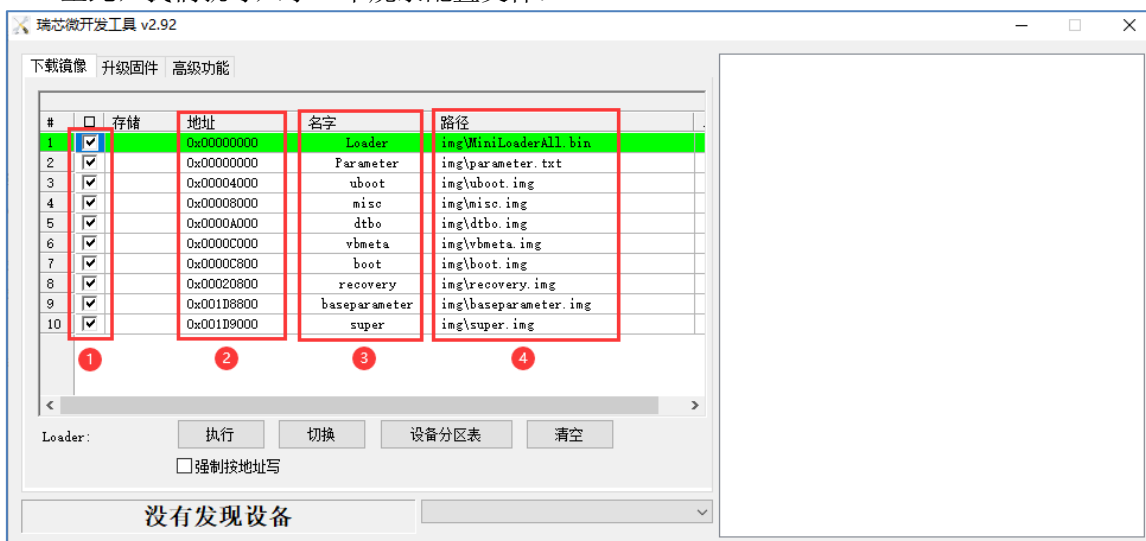


图 2.9.2.9 配置信息划分说明

①部分用于控制是否烧写对应的镜像，勾选表示需要烧写、不勾选表示不烧写；

②部分表示烧写的地址（也就是镜像的烧录地址）；

③部分表示该烧录项的名字；

④部分用于指定镜像的所在路径。

这些配置信息都是可以手动进行更改的，改完之后可以重新导出到配置文件中进行保存，方便下次导入该配置。

导入配置后，接下来便可以烧写了。执行烧写操作之前，**开发板必须处于 Maskrom 模式或 Loader 模式**，客户拿到手的开发板默认都是烧录过镜像的，可以进入到 Loader 模式，当然也可以进入到 Maskrom 模式。首先开发板需要先连接电源适配器以及 TYPE-C0 口，如下图所示：

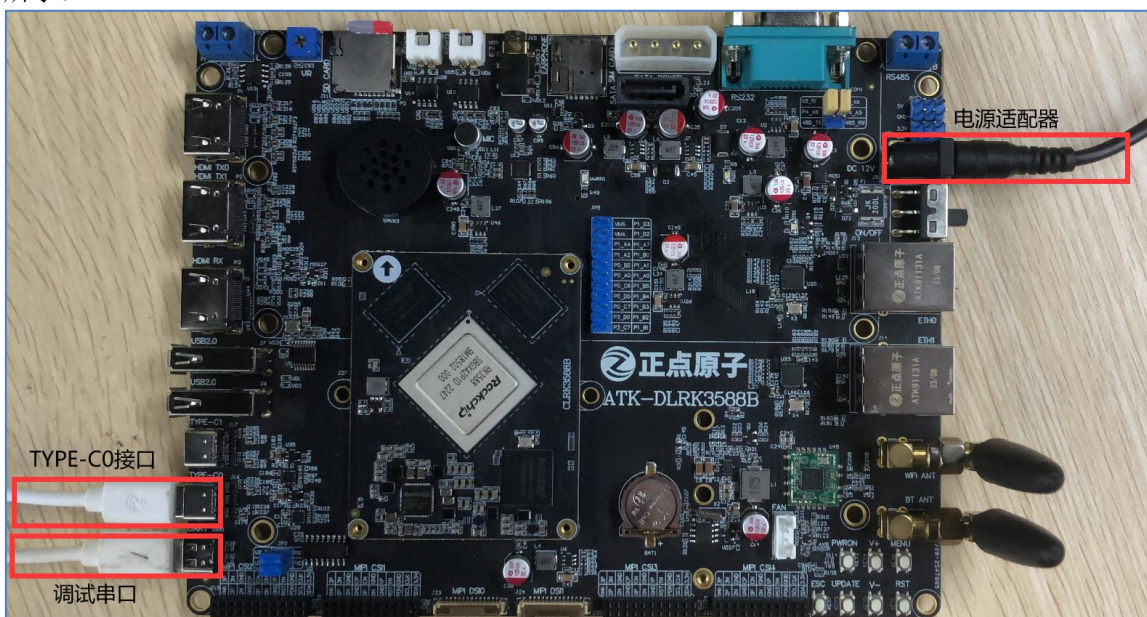


图 2.9.2.10 开发板硬件连接示意图

电源适配器使用 ATK-DLRK3588 开发板配套的 12V-2.5A 电源适配器；通过 USB 线将开发板的调试串口 UART 连接到电脑（先安装好 CH343 驱动），同样使用 USB 线将开发板的 TYPE-C0 接口连接到电脑。

硬件连接好之后，按住开发板上的 **V+按键**（音量+）按键，然后给开发板上电或复位，系统将会进入到 Loader 模式；此时瑞芯微开发工具界面上便会显示“**发现一个 LOADER 设备**”：



图 2.9.2.11 瑞芯微开发工具检测到 loader 设备

表示我们的开发板当前处于 Loader 模式，直接点击“**执行**”按钮即可烧写镜像，如下所示：

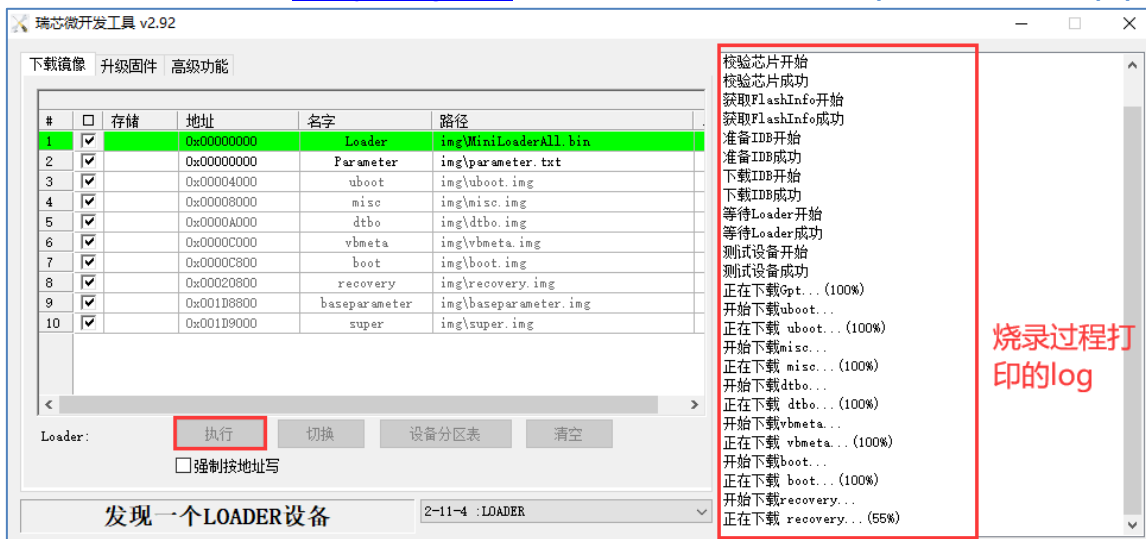


图 2.9.2.12 执行烧写操作

当然也可以在 Maskrom 模式下烧写, 2.9.1 小节中已经介绍了如何进入 Maskrom 模式, 进入 Maskrom 模式后, 同样也是点击“执行”按钮进行烧写。

如果没有出现意外, 那么烧写将会成功, 也就意味着这些镜像已经成功烧写到了开发板 Flash 存储器中 (ATK-DLRK3588 开发板使用了 eMMC 作为板载存储器, 所以默认情况下镜像会烧录到开发板 eMMC 中, 然后通过 eMMC 启动开发板; 当然也可以将镜像烧写到 SD 卡, 通过 SD 卡启动); 如果不幸, 烧写过程中出现了错误、导致烧写失败, 那么大家可以参考 RK 官方提供的文档《RKDevTool_manual_v1.2_cn.pdf》, 该文档中有针对几种常见的错误进行说明。

2.9.3 update.img 镜像的烧录方法

update.img 是多个镜像的集合体 (由多个镜像打包合并而成), 使用 RK 提供的工具可以将各个分立镜像 (uboot.img、boot.img、MiniLoaderAll.bin、parameter.txt、misc.img、super.img、dtbo.img、vbmeta.img、recovery.img 等) 打包成一个 update.img 固件, 方便用户烧录、升级。

本小节介绍如何使用瑞芯微开发工具烧写 update.img, 首先打开瑞芯微开发工具, 选择“升级固件”:

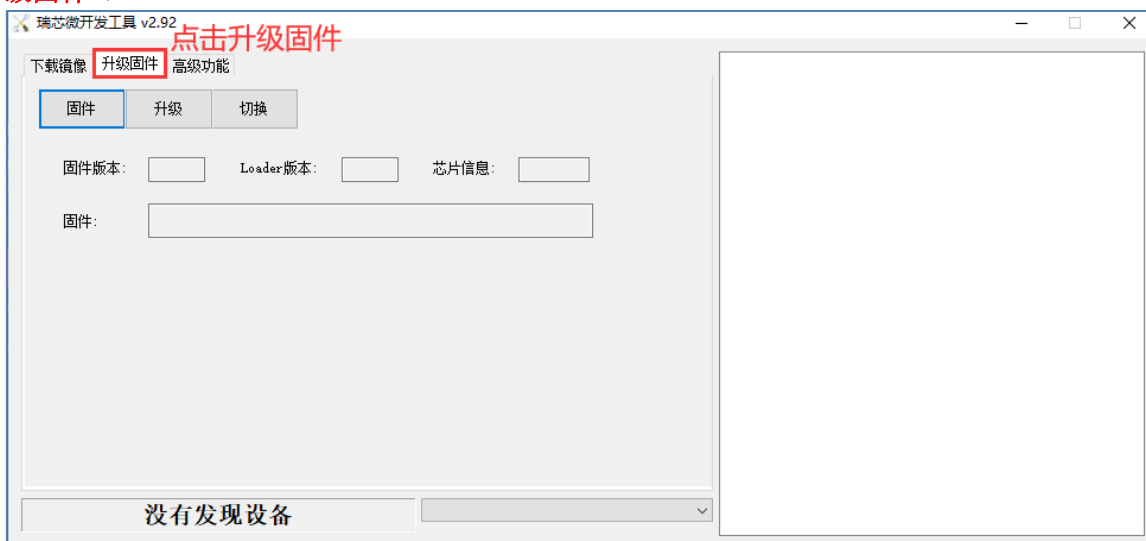


图 2.9.3.1 升级固件选项卡

然后点击“**固件**”按钮可以选择我们需要进行升级、烧录的 update.img 固件（开发板资料盘中并未给用户提供 update.img 固件，这里只是演示烧写 update.img 固件的方法），加载镜像之后会显示出该固件的一些信息：

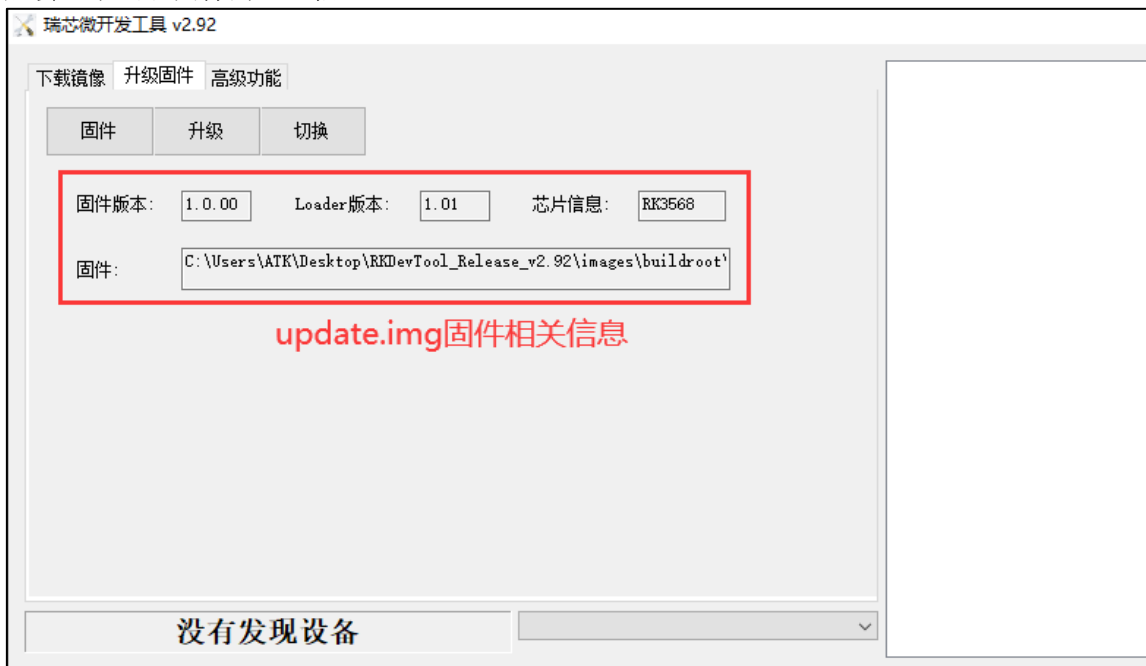


图 2.9.3.2 导入 update.img 镜像

首先让设备进入 Maskrom 或 Loader 模式，然后点击“**升级**”按钮进行固件升级、更新：

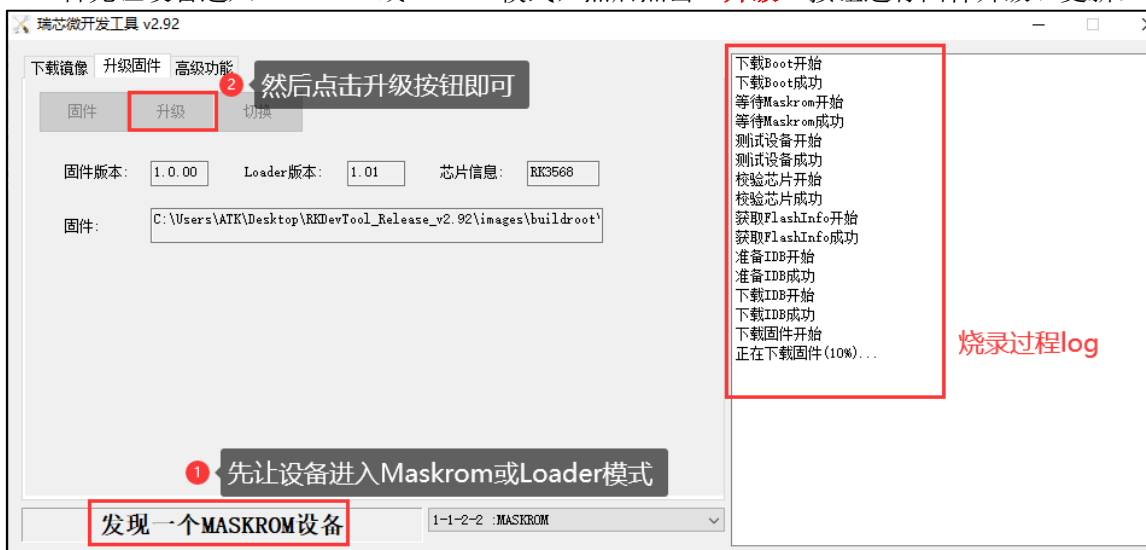


图 2.9.3.3 update.img 升级

2.9.4 擦除操作

通过瑞芯微开发工具可以擦除 Flash，将烧录到 Flash 中的镜像擦除。打开瑞芯微开发工具，点击“**高级功能**”选项卡，如下所示：

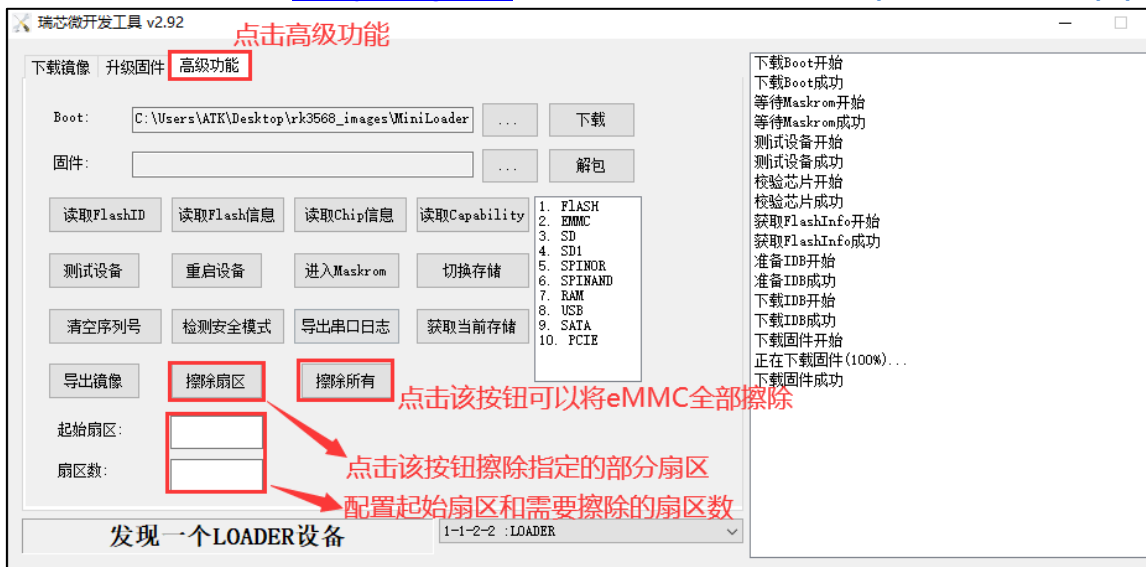


图 2.9.4.1 高级功能页面

需要注意，擦除操作需要在 Loader 模式下进行（Maskrom 模式也可以执行擦除操作，但需要先下载 MiniLoaderAll.bin，首先选择 MiniLoaderAll.bin 镜像，然后点击上图中的“下载”按钮执行下载操作，详情请参考<开发工具使用文档_v1.0.pdf>文档）。点击“擦除所有”按钮可以将开发板 eMMC 中的数据全部擦除；点击“擦除扇区”按钮可以擦除用户指定的部分扇区，将擦除的起始扇区填写至“起始扇区”输入框，将需要擦除的总扇区数填写至“扇区数”输入框即可。

譬如点击“擦除所有”按钮来擦除开发板 eMMC 中的全部数据：



图 2.9.4.2 擦除所有操作

2.9.5 Ubuntu 下 Linux_Upgrade_Tool 工具的使用

开发、调试阶段我们通常是在 Ubuntu 下编译源码、编译生成的镜像也存在于 Ubuntu 系统目录下；使用瑞芯微开发工具（RKDevTool）烧录之前需要先将这些镜像文件拷贝到 Windows 系统，然后再进行烧写，所以确实会有些麻烦。

那么除了可以在 Windows 下烧录之外，我们还可以在 Ubuntu 下烧录镜像，这样就避免了镜像的拷贝过程；在 Ubuntu 系统下，可以使用 RK 提供的工具 Linux_Upgrade_Tool 进行烧录。

该工具在 SDK 中，ATK-DLRK3588 开发板 Android SDK 自带了该烧录工具，第五章再给大家介绍。



2.10 ADB 工具安装

adb 全称 Android Debug Bridge, 直译过来就是 Android 调试桥, 它是一个通用的命令行工具。adb 做为 Android 设备与 PC 端连接的一个桥梁, 它可以让开发者通过网络或 USB 与 Android 设备进行通信, 从而作为 Android 设备的调试工具; 在 Android 开发中, adb 是必不可少的一个工具。

用户可以通过 adb 在电脑上对 Android 模拟器或者真实的 Android 设备进行一系列操作, 比如安装、卸载和调试应用 (apk), 拷贝推送文件 (在电脑与 Android 设备之间拷贝文件), 查看设备信息, 抓取 log 等操作。它的主要功能包括:

- 运行设备的 shell (命令行);
- 管理模拟器或设备的端口映射;
- 计算机与设备之间的文件上传/下载;
- 将本地 apk 软件安装至模拟器或 Android 设备。

关于 ADB 工具的安装以及使用方法, 本文档不做介绍, 用户可以参考正点原子提供的另一份文档《[adb 工具使用说明.pdf](#)》, 路径为: **开发板光盘 A 盘-基础资料→10、用户手册→03、辅助文档→【正点原子】adb 工具使用说明.pdf**, 在这份文档中会向用户介绍如何在 Windows 系统以及 Ubuntu 系统下安装 adb 工具, 并且会向用户介绍如何使用 adb 工具。

第三章 正点原子 ATK-DLRK3588 平台简介

参考文档:

《【正点原子】ATK-DLRK3588 开发板硬件参考手册.pdf》

3.1 RK3588 简介

参考文档:

《【正点原子】ATK-DLRK3588 开发板硬件参考手册.pdf》第一章

3.2 正点原子 ATK-DLRK3588 开发板硬件资源简介

参考文档:

《【正点原子】ATK-DLRK3588 开发板硬件参考手册.pdf》第二章



第四章 RK3588 Android SDK 软件包

本章向用户介绍正点原子 ATK-DLRK3588 硬件平台 Android SDK 软件包的安装以及使用方法, 该 SDK 适用于正点原子 ATK-DLRK3588 开发板及基于此开发板进行二次开发的所有 Android 系统产品; 基于本 SDK, 可以有效实现系统定制和应用移植开发, 帮助用户快速开发、提高开发效率!

对于 ATK-DLRK3588 开发板, 正点原子将向用户提供 Android12 以及 Android13 SDK, 用户可根据自己的需求进行选择, 建议客户使用 Android12, 因为 Android12 更为稳定、可靠!

RK3588 Android SDK 集成了 Android 源码、Linux Kernel 源码、U-Boot 源码以及 RK 提供的各种开发工具, Linux 内核版本为 5.10、U-Boot 版本为 2017.09。

本章将分为如下几个小节:

- 4.1 安装 RK3588 Android SDK, 包括 Android12 和 Android13
- 4.2 SDK 软件架构介绍
- 4.3 SDK 一键全自动编译
- 4.4 单独编译
- 4.5 SDK 清理
- 4.6 镜像介绍

本文档参考了 RK 官方提供的文档:

Android12:

<SDK>/RKDocs/android/RK3588_Android12_SDK_Developer_Guide_CN.pdf

Android13:

<SDK>/RKDocs/android/Rockchip_Android13_SDK_Developer_Guide_CN.pdf

强烈建议大家去看看!

4.1 安装 RK3588 Android SDK

本小节向用户介绍如何在 Ubuntu 系统下安装正点原子 ATK-DLRK3588 开发板 Android SDK 软件包。

4.1.1 安装依赖软件包

首先需要在 Ubuntu 系统下安装 SDK 编译环境所依赖的软件包, 执行如下命令安装软件包:

```
sudo apt-get update
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl zlib1g-dev gcc-multilib \
g++-multilib libc6-dev-i386 lib32ncurses5-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev \
libxml2-utils xsltproc unzip python-pyelftools python3-pyelftools device-tree-compiler libfdt-dev \
libfdt1 libssl-dev liblz4-tool python-dev libncurses5 make
```

安装过程中确保 Ubuntu 系统网络连接正常, 安装过程需要一定的时间, 请用户耐心等待!
将 python2 设置为系统默认 python 版本:

```
sudo rm -rf /usr/bin/python
sudo ln -s /usr/bin/python2 /usr/bin/python
```

4.1.2 安装 repo (跳过、无需安装)

repo 是建立在 Git 之上的一个多仓库管理工具, 可以组织多个仓库的上传和下载, 用于管理多个 Git 存储仓库。RK3588 Android SDK 的代码和相关文档被划分成了若干个 git 仓库分别进行版本管理 (SDK 包含了若干个 git 仓库), 开发者可以使用 repo 工具对这些 git 仓库进行统一下载、提交、切换分支等操作。

根据下面的步骤安装 repo。

首先在用户家目录下创建一个 bin 文件夹, 并将其导出到 PATH 环境变量:

```
mkdir ~/bin
export PATH=~/bin:$PATH
```

如果可以访问 Google, 可通过如下命令下载 repo, 并赋予其可执行权限:

```
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
chmod a+x ~/bin/repo
```

国内网络环境下如果执行上述命令后发现~/bin/repo 文件为空, 此时可以访问国内的站点来下载 repo 工具, 并赋予其可执行权限:

```
curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo -o ~/bin/repo
chmod a+x ~/bin/repo
```

```
tgg@tgg-virtual-machine:~$ curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo -o ~/bin/repo
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 45769 100 45769 0 0 108k 0 --:--:-- --:--:-- --:--:-- 107k
tgg@tgg-virtual-machine:~$ chmod a+x ~/bin/repo
tgg@tgg-virtual-machine:~$
```

图 4.1.2.1 下载 repo 工具

repo 工具其实就是一个 python 脚本。

4.1.3 Git 配置

使用 repo 之前需要用户配置自己的 git 信息, 否则后面的操作可能会遇到 hook 检查的障碍:

```
git config --global user.name "your name"
git config --global user.email "your email"
```

请用户根据实际情况配置。可通过如下命令查看 git 配置信息:

```
git config --list
```

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ git config --list
user.name=713588_20240601
user.email=713588_20240601@tgg.com
tgg@tgg-virtual-machine:~$
```

图 4.1.3.1 查看 git 配置信息

4.1.4 安装 SDK

接下来开始安装 RK3588 Android SDK, 开发板资料盘中已经给用户提供了 SDK 的压缩包 (包括 Android12 以及 Android13), Android12 SDK 路径为: **开发板光盘 B 盘-开发环境及 SDK→02、ATK-DLRK3588 开发板 SDK→02、android12_sdk→atk-rk3588_androidS_release_v1.0_20240601.tgz**, 随着版本的更新, SDK 压缩包文件的名称也将会发生改变, 但均以 **atk-rk3588_androidS_release_版本_发布日期.tgz** 方式进行命名。每次发布新版本 SDK 时, 将会替换网盘中的旧版本 SDK, 如果用户需要获取旧版本 SDK, 请联系正点原子 Linux 技术支持获取!

Android13 SDK 路径为: **开发板光盘 B 盘-开发环境及 SDK→02、ATK-DLRK3588 开发板 SDK→03、android13_sdk→atk-rk3588_androidT_release_v1.0_20240601.tgz**, SDK 压缩包文件的名称同样也会随着版本更新发生改变, 但均以 **atk-rk3588_androidT_release_版本_发布日期.tgz** 方式进行命名。

用户可根据自己的需求选择安装其中一个 SDK 即可, 如果用户对 Android 版本没有要求, 建议选择 Android12, 原因前面已经提到过。将 tgz 压缩文件拷贝到 Ubuntu 系统的用户家目录下, 并执行如下命令将其解压到家目录下的 rk3588_android_sdk 目录, 下面以 Android12 为例:

```
mkdir ~/rk3588_android_sdk
tar xf atk-rk3588_androidS_release_v1.0_20240601.tgz -C ~/rk3588_android_sdk
```

```
dengtao@dengtao-virtual-machine:~$
dengtao@dengtao-virtual-machine:~$ ls
公共的 模板 视频 图片 文档 下载 音乐 桌面 atk-rk3588_android5_release_v1.0_20240601.tgz bin
dengtao@dengtao-virtual-machine:~$
dengtao@dengtao-virtual-machine:~$ mkdir ~/rk3588_android_sdk
dengtao@dengtao-virtual-machine:~$ tar xf atk-rk3588_android5_release_v1.0_20240601.tgz -C ~/rk3588_android_sdk
```

图 4.1.4.1 SDK 包解压

解压时间会比较长,请耐心等待!解压完成后,~/rk3588_android_sdk/目录下会存在一个.repo 文件夹,如图 4.1.4.2 所示:

```
dengtao@dengtao-virtual-machine:~$
dengtao@dengtao-virtual-machine:~$ ls -lha ~/rk3588_android_sdk/
总用量 12K
drwxrwxr-x 3 dengtao dengtao 4.0K 3月 7 17:00 .
drwxr-xr-x 17 dengtao dengtao 4.0K 3月 7 17:00 ..
drwxrwxr-x 7 dengtao dengtao 4.0K 3月 7 16:01 .repo
dengtao@dengtao-virtual-machine:~$
```

图 4.1.4.2 .repo 文件夹

执行如下命令可检出源码:

```
cd ~/rk3588_android_sdk/
.repo/repo/repo sync -l -j16
```

```
dengtao@dengtao-virtual-machine:~$ cd ~/rk3588_android_sdk/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ .repo/repo/repo sync -l -j16
正在更新文件: 100% (425/425), 完成.
正在更新文件: 100% (564/564), 完成.
正在更新文件: 100% (300/300), 完成.
正在更新文件: 100% (1465/1465), 完成.
正在更新文件: 100% (153/153), 完成.
正在更新文件: 100% (4664/4664), 完成.
正在更新文件: 100% (68/68), 完成.
正在更新文件: 100% (561/561), 完成.
正在更新文件: 100% (4377/4377), 完成.
正在更新文件: 100% (19/19), 完成.
正在更新文件: 100% (560/560), 完成.
正在更新文件: 100% (608/608), 完成.
正在更新文件: 100% (909/909), 完成.
正在更新文件: 100% (451/451), 完成.
正在更新文件: 100% (13/13), 完成.
正在更新文件: 100% (415/415), 完成.
正在更新文件: 100% (709/709), 完成.
正在更新文件: 100% (3097/3097), 完成.
正在更新文件: 100% (34/34), 完成.
正在更新文件: 100% (79/79), 完成.
正在更新文件: 100% (10/10), 完成.
正在更新文件: 100% (246/246), 完成.
```

```
正在更新文件: 100% (9781/9781), 完成.
正在更新文件: 100% (9777/9777), 完成.
正在更新文件: 100% (8515/8515), 完成.
正在更新文件: 100% (6380/6380), 完成.
正在更新文件: 100% (8414/8414), 完成.
正在更新文件: 100% (1904/1904), 完成.
正在更新文件: 100% (2102/2102), 完成.
正在更新文件: 100% (21250/21250), 完成.
正在更新文件: 100% (18247/18247), 完成.
正在更新文件: 100% (471/471), 完成.
正在更新文件: 100% (33/33), 完成.
正在更新文件: 100% (417/417), 完成.
正在更新文件: 100% (21/21), 完成.
正在更新文件: 100% (1090/1090), 完成.
正在更新文件: 100% (28/28), 完成.
正在更新文件: 100% (19/19), 完成.
正在更新文件: 100% (42/42), 完成.
正在更新文件: 100% (34731/34731), 完成. system/iorap正在更新文件: 74% (25776/34731)
正在更新文件: 100% (953/953), 完成.
正在更新文件: 100% (683/683), 完成.
正在更新文件: 100% (763/763), 完成.
正在更新文件: 100% (16294/16294), 完成.
正在更新文件: 100% (733/733), 完成.
Checking out: 100% (1084/1084), done in 1h1m35.863s
repo sync has finished successfully.
```

图 4.1.4.3 检出源码

整个过程将会持续比较长的时间, 大家需要耐心等待, 直到同步完成。

同步完成后, `~/rk3588_android_sdk` 目录下的内容如图 4.1.4.4 所示:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls
Android.bp  build.sh  external  libcore  pdk  platform_testing  RKTools  vendor
art  compatibility  frameworks  libnativehelper  platform_testing  sdk  WORKSPACE
bionic  cts  hardware  Makefile  prebuilts  system
bootable  dalvik  javaenv.sh  mkcombinedroot  restore_patches.sh  test
bootstrap.bash  developers  kernel  mkimage_ab.sh  rkbin  toolchain
build  development  kernel-4.19  mkimage.sh  RKDocs  tools
BUILD  device  kernel-5.10  packages  rkst  u-boot
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

图 4.1.4.4 Android SDK 工程目录

至此, RK3588 Android SDK 就安装完成了。

4.1.5 SDK 更新

正点原子 Linux 技术团队会对 SDK 软件进行更新、升级, 并以压缩包文件的形式提供给用户, 压缩包文件的命名方式为 `atk-rk3588_androidS_release_版本_发布日期.tgz` (Android13 对应为 `atk-rk3588_androidT_release_版本_发布日期.tgz`), 每次发布新版本 SDK 时会替换资料网盘中的旧版 SDK, 所以, 网盘中的 SDK 即为当前最新 SDK; 如需获取最新版本 SDK, 只能通过正点原子资料网盘进行下载; 如需获取旧版本 SDK, 请联系正点原子 Linux 技术支持获取!

每一次 SDK 版本更新将会使用 `<SDK>/.repo/manifests/ATK-Android_SDK_Note.md` 文档记录, 如果用户想要了解 SDK 版本更新记录可以查看该文档 (建议使用 Markdown 文档阅读器打开)。

4.1.6 SDK 问题反馈

用户在使用、开发过程中, 如发现了 SDK 的一些问题、软件 BUG、具体技术问题、技术咨询等, 可以通过 QQ 群或者淘宝联系正点原子 Linux 技术支持, 并向其说明问题, 后续将会安排开发人员进行相应的处理、跟踪。

4.2 SDK 软件架构介绍

本小节向用户介绍 Android SDK 软件架构。

4.2.1 Android SDK 工程目录介绍

RK3588 Android SDK 的目录结构其实就是一个 Android 源码目录结构, 包含有 art、bionic、bootable、build、compatibility、cts、dalvik、external、frameworks、hardware 等多个文件夹; 除了 Android 源码原生的目录外, 还包含瑞芯微原厂(Rockchip)提供的目录, 譬如 rkbin、RKDocs、rkst、RKTools、u-boot 等。

SDK 中每个目录或其子目录会对应一个 git 工程; 因为 Android 源码和相关文档、工具被划分成了若干 git 仓库分别进行版本管理, 它们按照功能、所属模块划分, 分别组织到不同的目录下。

接下来将向用户介绍 RK3588 Android SDK 源码目录:

顶层目录名	说明
art	Anroid RunTime(Android 运行环境), Android 5.0 之后 java 虚拟机就使用 ART 了; 区别于传统的 Dalvik 虚拟机, 旨在提高 Android 系统的流畅性
bionic	基础 C 库源代码, Android 改造的 C/C++库, 比传统的 glibc 更精简! 不受 GPL 限制, 支持 pthread_cancel(), 不支持 C++ exception 和 C++ STL 模板编程
bootable	包含一个 recovery 目录, 该目录包含 recovery 程序(恢复出厂/升级)的源代码
build	Android 的编译系统, 定义了各种编译规则, 并包含通用产品配置
compatibility	包含 Android CDD(Android 兼容性定义文档)
cts	Compatibility Test Suite, 即 Android 兼容性测试套件, CTS 是一个自动化测试套件, 包含 PC 端和设备端(被测试设备), 对手机的硬件、软件、接口、性能进行测试。只有通过 CTS 测试的 Android 手机系统, Google 才会颁发许可, 以保证不同生厂商之间的 Android 系统的兼容
dalvik	Android Dalvik 虚拟机相关内容, Android 5.0 之后已经不再使用 Dalvik 虚拟机了, 而改用 ART 作为 Android java 虚拟机。但是该目录在 Android 源码目录下依然存在, 该目录下提供了一些工具
developers	开发者目录, 为 Android 开发者提供的一些参考文档、参考实例 demo 代码
development	提供 Android 应用开发基础设施, 包括开发工具、应用例子, 提供 monkey 命令、shell 脚本和 python 脚本
device	Android 支持的各种设备及其配置, 也就是设备产品定制目录
external	Android 引入的第三方开源库, Google 会把一些比较优秀的开源库纳入到 Android 源码里面, 譬如 ppp、wpa_supplicant、libz、libcurl 等
frameworks	Android 应用框架核心层代码, 由 java 和 C++编写, 包括各种系统资源、System Service、SystemUI 等, 是 framework 开发经常打交道的目录
hardware	硬件抽象层代码, 如 gps、WiFi 等
kernel	Linux Kernel 源码目录, 不过 Android 默认不提供, 产商(譬如瑞芯微)可以将内核源码集成到该目录
libcore	Android 的 java 核心类库, 譬如 java.lang 包、libart 库、JSON 库等
libnativehelper	JNI 编程时调用的各种接口的实现代码, 如 findClass, 最终会编译成 libnativehelper.so
packages	Android 系统内置 App 的源码, 譬如设置、输入法、墙纸、屏保等
pdk	Platform Development Kit(平台开发套件)相关测试代码和工具, pdk 的目标是帮助芯片供应商(也就是半导体产商, 如瑞芯微)和原始设备制造商迁移到新的版本。从 Android 4.1 版本开始, Google 在发布新的 Android 版本之前会 Release 给部分签约的 OEM 产商和半导体产商 PDK, 这样做的目的是让 OEM 产商和半导体产商可以提前开始适配新版本 Android, 所以 PDK 本身的内容是不会公开的

platform_testing	Android 平台测试程序
prebuilts	预编译相关, 包含 x86 和 arm 架构下预编译的一些资源, 譬如 ndk 工具包、交叉编译器等
sdk	Android 的 java 层 SDK, 包含为应用程序开发人员提供的开发工具, 应用程序开发人员可以借助 sdk 中的 API 快速的进行应用开发
system	Android 底层文件系统库、应用和组件, 譬如 busybox、init 进程等
test	Android vendor 测试框架, 譬如 vts (vendor test suit)、mts (mainline test suit)、mlts。 CTS 确保 App 与 Android Framework 之间有一致的调用接口 (API), 这使得 App 开发者编写的同一个应用程序可以运行在不同系统版本 (向前兼容)、不同硬件平台、不同产商制造的不同设备上; VTS 与 CTS 类似, 通过对 vendor interface 进行测试, 确保同一个版本的 Android framework 可以运行在不同 HAL 上, 或不同 Android framework 可以运行在同一个 HAL 上。 Mainline Test Suit (MTS), MTS 由一组测试框架和测试用例组成, 旨在帮助增强主线分支代码的健壮性、可靠性和合规性。 Mlts 是 Android 机器学习测试套件
toolchain	没什么东西, 都是一些 python 测试脚本
tools	一些特殊工具, 譬如 acloud 命令用于连接谷歌云端工作站, apkzlib 用于 apk 打包
vendor	厂商定制目录, 存放了厂商相关的代码
mkcombinedroot	瑞芯微提供
rkbina	瑞星微提供, 存放了 Rockchip 相关的 Binary 和工具
RKDocs	瑞芯微提供, 存放了芯片模块开发指导文档、平台支持列表、芯片平台相关文档、Android SDK 开发指南等, 这些文档是 RK 提供给客户的参考文档
rkst	瑞芯微提供
RKTools	瑞芯微提供, 存放了 Linux 和 Windows 操作系统环境下的常用工具, 譬如镜像烧录工具等
u-boot	瑞芯微提供, 存放了 RK 基于 v2017.09 版本进行开发的 U-Boot 源码

表 4.2.1.1 RK3588 Android SDK 源码目录介绍

4.2.2 SDK 软件框图

Android 系统架构图如下所示:

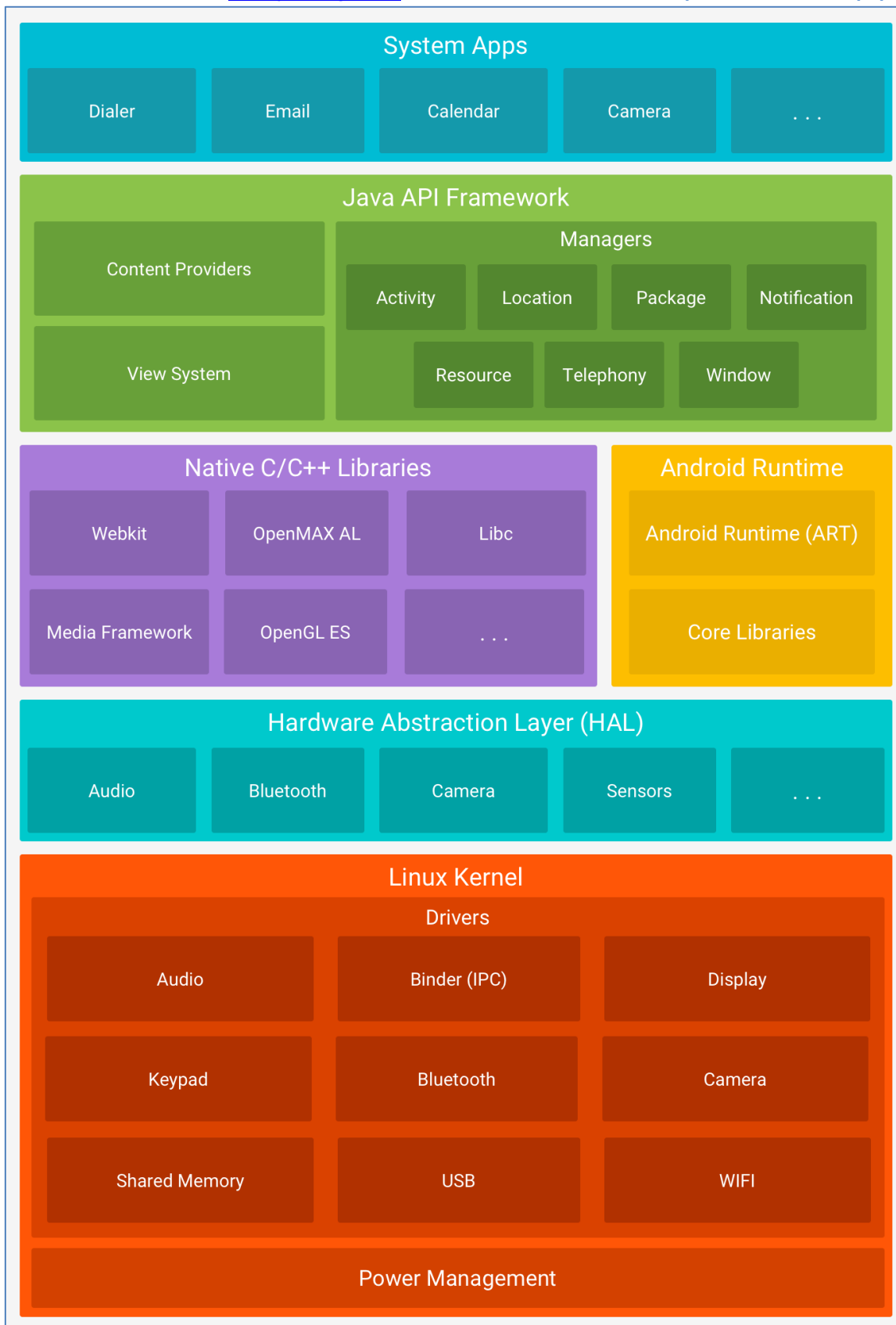


图 4.2.2.1 Android 系统架构

Android 系统架构分为五层，从上到下依次为：应用层（System Apps）、应用框架层（Java API Framework）、系统运行库层（Android Runtime & Native C/C++库）、硬件抽象层（HAL）以及 Linux 内核层（Linux Kernel）。

①、应用层

应用层是整个 Android 系统的最上层，该层包含核心的系统应用、输入法以及第三方应用等；Android 系统内置的应用程序以及非系统内置的应用程序都属于应用层，负责与用户进行直接交互，通常都是由 Java 开发。

②、应用框架层（Java API Framework）

应用框架层提供了开发 Android 应用程序所需的各种 API，应用开发人员调用框架层所提供的 API 进行应用程序开发，当然也包括系统应用；框架层是由 Java 代码编写，所以也称为 Java Framework。

除此之外，框架层还为应用程序提供一系列重要的系统服务，应用程序开发者被允许在应用中使用这些服务：

服务名称	说明
Activity Manager（活动管理器）	管理各个应用程序生命周期、并且提供通用的导航回退功能
Location Manager（位置管理器）	提供地理位置及定位功能服务
Package Manager（包管理器）	管理所有安装在 Android 系统中的应用程序，包括安装、卸载、更新等
Notification Manager（通知管理器）	使得所有应用程序都能够在状态栏显示通知信息
Resource Manager（资源管理器）	管理除了代码之外的一系列资源（非代码资源），如本地字符串、图片、布局文件、颜色文件等
Telephony Manager（电话管理器）	管理所有的移动设备功能
Window Manager（窗口管理器）	管理所有开启了窗口的程序
Content Provider（内容提供者）	使不同应用程序之间可以共享数据
View System（视图系统）	构建应用程序的基本组件，譬如列表、网格、文本框、按钮等

表 4.2.2.1 系统服务简介

③、系统运行库层（Native）

该层分为两部分：C/C++程序库和 Android 运行时库

C/C++程序库：提供给 Android 系统的各个组件使用，这些功能通过应用框架层为开发者提供服务。

Android 运行时库（Android Runtime）：运行时库分为核心库和 ART（Android 5.0 之后，Dalvik 虚拟机被 ART 取代）。核心库集成了 Java 语言核心库的大多数功能，这样开发者可以使用 java 语言来编写 Android 应用；在 ART 机制下，系统在安装应用的时候就会进行一次预编译，将字节码预先编译成机器码并存储在本地，这样在应用每次运行启动时就不需要再去执行编译了，大大提高了效率！

④、硬件抽象层（HAL/HIDL）

hal 是 hardware abstraction layer 的简写，位于 Framework 和 Linux Kernel 之间，HAL 层代码一般由 C/C++ 构成。硬件抽象层的目的在于将底层硬件抽象化，向上层提供统一、标准化的 API 接口；所以 HAL 层是上层应用对底层硬件操作屏蔽的一个软件层次，上层应用不必关心底层硬件具体是如何工作的，只需调用 HAL 层提供的统一接口即可！一般做 Android 底层开发主要就是针对 HAL 层做修改以适配具体的硬件。

另一方面，HAL 层存在的意义是为了保护知识产权，硬件产商处于利益考虑，不希望公开硬件设备相关的实现细节和参数，所以硬件产商或个人可以把不愿开源的代码（譬如一些核心

算法) 放在 HAL 层, 以动态链接库的形式提供, 而 HAL 层位于用户空间 (Linux 用户空间), 不属于 Linux 内核, 无需遵循 GPL 开源协议。

⑤、Linux Kernel

Android 系统的基础是 Linux 内核, Android 的核心系统服务基于 Linux 内核, 并在此基础上添加了部分 Android 专用的驱动, 系统的安全性、内存管理、进程管理、网络协议栈和驱动模型等都依赖于 Linux 内核。

4.2.3 SDK 版本查询

正点原子 RK3588 Android SDK 的版本可分为 RK 版本和 ATK 版本; 所谓 RK 版本, 则表示 Rockchip (瑞芯微) 发布 SDK 时所定义版本号; 而 ATK 版本则表示正点原子 Linux 技术团队发布 SDK 时所定义版本号, 每次发布新版本 SDK 时都会有一个版本号与之对应。

从我们提供的 SDK 压缩包文件名便可知道其对应的 ATK 版本以及发布日期; 也可在 Android SDK 源码根目录下, 执行如下命令查询 SDK 的 ATK 版本 (以 Android12 为例):

```
realpath .repo/manifests/atk-android12_release.xml
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ realpath .repo/manifests/atk-android12_release.xml
/home/dengtao/rk3588_android_sdk/.repo/manifests/atk-android12_release_v1.0_20240601.xml
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

图 4.2.3.1 查询 SDK 的 ATK 版本

从图中可知, 当前 SDK 的 ATK 版本为 V1.0, 发布日期为 2024-06-01。如果是 Android13, 则执行如下这条命令:

```
realpath .repo/manifests/atk-android13_release.xml
```

在 SDK 源码根目录下, 打开如下文件可查询当前 SDK 的 RK 版本:

```
vi .repo/manifests/include/rk_checkout_repository.xml
```

```
manifest>
<project name="platform/external/e2fsprogs" path="external/e2fsprogs" revision="refs/tags/android-12.1-mid-rkr14" groups="pdk"/>
<project name="platform/external/libdrm" path="external/libdrm" revision="refs/tags/android-12.1-mid-rkr14" groups="pdk"/>
<project name="platform/external/speex" path="external/speex" revision="refs/tags/android-12.1-mid-rkr14" groups="pdk"/>
<project name="platform/external/skia" path="external/skia" revision="refs/tags/android-12.1-mid-rkr14" groups="pdk"/>
<project name="platform/external/tinysalsa" path="external/tinysalsa" revision="refs/tags/android-12.1-mid-rkr14"/>
<project name="platform/build_git" path="build/make" revision="refs/tags/android-12.1-mid-rkr14" groups="pdk">
```

图 4.2.3.2 查询 SDK 的 RK 版本

从图中可知, SDK 的 RK 版本为 **android-12.1-mid-rkr14** (Android12), Android13 对应的版本为 **android-13.0-mid-rkr6**。

4.3 SDK 全自动编译

本小节向用户介绍如何一键全自动编译整个 RK3588 Android SDK, 首先进入到 SDK 根目录下, 在编译之前先执行如下命令初始化编译环境:

```
source build/envsetup.sh
```

注意: 每次打开一个新的终端编译 Android 源码之前, 都需要执行“source build/envsetup.sh”命令初始化编译环境。

接着执行 lunch 命令选择一个产品 (也就是选择一个编译目标、编译项):

```
lunch
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ lunch

You're building on Linux

Lunch menu... pick a combo:
  1. ATK_DLRK3588-user
  2. ATK_DLRK3588-userdebug
  3. aosp_arm-eng
  4. aosp_arm64-eng
  5. aosp_bluepine-userdebug
```



```
6. aosp_blueline_car-userdebug
7. aosp_bonito-userdebug
8. aosp_bonito_car-userdebug
9. aosp_bramble_car-userdebug
10. aosp_cf_arm64_auto-userdebug
11. aosp_cf_arm64_phone-userdebug
12. aosp_cf_x86_64_foldable-userdebug
13. aosp_cf_x86_64_pc-userdebug
14. aosp_cf_x86_64_phone-userdebug
15. aosp_cf_x86_64_tv-userdebug
16. aosp_cf_x86_auto-userdebug
17. aosp_cf_x86_phone-userdebug
18. aosp_cf_x86_tv-userdebug
19. aosp_coral_car-userdebug
20. aosp_crosshatch-userdebug
21. aosp_crosshatch_car-userdebug
22. aosp_crosshatch_vf-userdebug
23. aosp_flame_car-userdebug
24. aosp_oriole-userdebug
25. aosp_oriole_car-userdebug
26. aosp_raven-userdebug
27. aosp_raven_car-userdebug
28. aosp_redfin_car-userdebug
29. aosp_sargo-userdebug
30. aosp_sargo_car-userdebug
31. aosp_slider-userdebug
32. aosp_sunfish_car-userdebug
33. aosp_whitefin-userdebug
34. aosp_x86-eng
35. aosp_x86_64-eng
36. arm_krait-eng
37. arm_v7_v8-eng
38. armv8-eng
39. armv8_cortex_a55-eng
40. armv8_kryo385-eng
41. beagle_x15-userdebug
42. beagle_x15_auto-userdebug
43. fuchsia_arm64-eng
44. fuchsia_x86_64-eng
45. hikey-userdebug
46. hikey64_only-userdebug
47. hikey960-userdebug
48. hikey960_tv-userdebug
49. hikey_tv-userdebug
50. qemu_trusty_arm64-userdebug
51. rk3588_box-user
52. rk3588_box-userdebug
53. rk3588_s-user
54. rk3588_s-userdebug
55. rk3588_xr-user
56. rk3588_xr-userdebug
57. rk3588m_car-user
58. rk3588m_car-userdebug
59. rk3588m_s-user
60. rk3588m_s-userdebug
61. rk3588s_s-user
62. rk3588s_s-userdebug
63. sdk_car_arm-userdebug
64. sdk_car_arm64-userdebug
65. sdk_car_portrait_x86_64-userdebug
66. sdk_car_x86-userdebug
67. sdk_car_x86_64-userdebug
68. silvermont-eng
69. uml-userdebug
70. yukawa-userdebug
71. yukawa_sei510-userdebug
```

Which would you like? [aosp_arm-eng]

执行 lunch 命令后 (不带任何参数), 会列举出所有产品 (编译目标), 对于 ATK-DLRK3588 平台来说, 我们可以选择 ATK_DLRK3588-user 或者 ATK_DLRK3588-userdebug; ATK_DLRK3588-user 将会编译出 user 版本的 Android 系统镜像, 而 ATK_DLRK3588-userdebug 将会编译出 userdebug 版本的 Android 系统镜像。

这里以 userdebug 版本为例, 输入 “ATK_DLRK3588-userdebug” 或者对应的序号:

```
69. uhl-userdebug
70. yukawa-userdebug
71. yukawa_sei510-userdebug

Which would you like? [aosp_arm-eng] ATK_DLRK3588-userdebug
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

输入名称或对应的序号, 然后按回车

图 4.3.1 lunch 选择产品

也可以在执行 lunch 命令的时候直接指定产品名, 如下:

```
lunch ATK_DLRK3588-userdebug
```

接下来便可以进行编译了, 使用 RK 封装好的编译脚本 build.sh 进行编译。

build.sh 是 RK 封装好的一个编译脚本, 使用该脚本可以方便用户快速构建出整个 Android 系统镜像文件以及对镜像进行打包操作, 既可以一键全自动编译整个 Android SDK, 也可以单独编译 U-Boot、Linux Kernel、Android 源码等, 非常方便!

可通过执行如下命令查看 build.sh 脚本的使用方法:

```
./build.sh -h
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -h
./build.sh: 非法选项 -- h
USAGE: [-U] [-CK] [-A] [-p] [-o] [-u] [-v VERSION_NAME]
No ARGS means use default build option
WHERE: -U = build uboot
       -C = build kernel with Clang
       -K = build kernel
       -A = build android
       -p = will build packaging in IMAGE
       -o = build OTA package
       -u = build update.img
       -v = build android with 'user' or 'userdebug'
       -d = build kernel dts name
       -V = build version
       -J = build jobs
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

图 4.3.2 查看 build.sh 脚本帮助信息

如下表所示:

build.sh 脚本参数	说明	示例
-U	编译 U-Boot 源码	./build.sh -U
-C	编译 Linux Kernel 源码, 使用 clang 编译器编译; 如果需要过 GMS 认证, Google 要求需要使用 clang 编译器编译内核源码; 5.10 内核强制使用 Clang 编译器!	./build.sh -C
-K	编译 Linux kernel 源码	./build.sh -K
-A	编译 Android 源码	./build.sh -A
-p	将编译出的镜像打包到 IMAGE 目录下	./build.sh -p
-o	编译 OTA 升级包	./build.sh -o
-u	将所有镜像打包成 update.img 固件	./build.sh -u
-d	编译 Linux Kernel 时, 指定目标设备树文件	./build.sh -d
-J	指定编译线程数量, 一般这个数字越大、编译也越快, 当然具体情况看电脑配置、内存容量等	./build.sh -J24

表 4.3.1 build.sh 常用命令

后续还会给大家介绍这些参数的使用方法, 这里先对其有一个基本的了解即可!

#####**屏幕选择**#####

选择屏幕:

正点原子 ATK-DLRK3588 开发板支持以下种类屏幕:

- 正点原子 5.5 寸 1080x1920 (竖屏) MIPI 屏
 - 正点原子 5.5 寸 720x1280 (竖屏) MIPI 屏
 - 正点原子 10.1 寸 800x1280 (竖屏) MIPI 屏
 - 标准 HDMI 显示屏 (通过 HDMI 线连接)
 - 标准 DP 显示屏 (通过 type-c 转 dp 线连接)

ATK-DLRK3588 开发板上有 2 个 MIPI 屏接口, 分别为 MIPI DSI0 和 MIPI DSI1, 可连接正点原子 5.5 寸或 10.1 寸 MIPI 屏; 2 个 HDMI 输出接口, 分别为 HDMI TX0 和 HDMI TX1, 可连接标准 HDMI 显示器; 2 个全功能 type-c 接口 (支持 USB 3.1 和 DP 输出), 分别为 TYPE-C0 和 TYPE-C1, 可连接标准 DP 显示器; 用户可以在板子上找到这些显示接口, 这里就不贴图了!

用户需要根据自己购买的屏幕、配置使能对应的显示接口。

打开 **kernel-5.10/arch/arm64/boot/dts/rockchip/rk3588-atk-screen_choose.dtsi** 设备树文件:

```
vi kernel-5.10/arch/arm64/boot/dts/rockchip/rk3588-atk-screen_choose.dtsi
```

```
/*
 * RK3588平台支持4路独立的Video Port, 可实现4路独立的显示输出
 * 默认显示通路配置如下所示:
 *
 * VP0 -----> HDMI_TX0
 * VP1 -----> HDMI_TX1
 * VP2 -----> DP_TX0  DP_TX1
 * VP3 -----> DSI_TX0  DSI_TX1
 *
 *
 * 通过宏定义使能相应的显示接口:
 * ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280: 使能5.5寸720x1280 MIPI屏(MIPI DSI0)
 * ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920: 使能5.5寸1080x1920 MIPI屏(MIPI DSI0)
 * ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280: 使能10.1寸800x1280 MIPI屏(MIPI DSI0)
 *
 * ATK_LCD_TYPE_MIPI_TX1_5P5_720X1280: 使能5.5寸720x1280 MIPI屏(MIPI DSI1)
 * ATK_LCD_TYPE_MIPI_TX1_5P5_1080X1920: 使能5.5寸1080x1920 MIPI屏(MIPI DSI1)
 * ATK_LCD_TYPE_MIPI_TX1_10P1_800X1280: 使能10.1寸800x1280 MIPI屏(MIPI DSI1)
 *
 * ATK_LCD_TYPE_HDMI_TX0: 使能HDMI_TX0显示屏
 * ATK_LCD_TYPE_HDMI_TX1: 使能HDMI_TX1显示屏
 * ATK_LCD_TYPE_DP_TX0: 使能DP_TX0显示屏
 * ATK_LCD_TYPE_DP_TX1: 使能DP_TX1显示屏
 */

// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280 // from vp3
#define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280 // from vp3

// #define ATK_LCD_TYPE_MIPI_TX1_5P5_720X1280 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX1_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX1_10P1_800X1280 // from vp3

#define ATK_LCD_TYPE_HDMI_TX0 // from vp0
#define ATK_LCD_TYPE_HDMI_TX1 // from vp1
#define ATK_LCD_TYPE_DP_TX0 // from vp2
#define ATK_LCD_TYPE_DP_TX1 // from vp2
```

默认使能了 5.5 寸 1080x1920 MIPI 屏（连接在 MIPI DSI0 接口）和 2 个 HDMI 屏、2 个 DP 屏；如果用户不需要在板子上接 MIPI 屏，则注释掉所有 ATK_LCD_TYPE_MIPI 相关宏，如下：

```
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280 // from vp3

// #define ATK_LCD_TYPE_MIPI_TX1_5P5_720X1280 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX1_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX1_10P1_800X1280 // from vp3

#define ATK_LCD_TYPE_HDMI_TX0 // from vp0
#define ATK_LCD_TYPE_HDMI_TX1 // from vp1
#define ATK_LCD_TYPE_DP_TX0 // from vp2
#define ATK_LCD_TYPE_DP_TX1 // from vp2
```

如果用户需要在 MIPI DSI1 接口接 MIPI 屏，譬如 5.5 寸 1080x1920 MIPI 屏，定义如下：

```
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280 // from vp3

// #define ATK_LCD_TYPE_MIPI_TX1_5P5_720X1280 // from vp3
#define ATK_LCD_TYPE_MIPI_TX1_5P5_1080X1920 // from vp3
// #define ATK_LCD_TYPE_MIPI_TX1_10P1_800X1280 // from vp3

#define ATK_LCD_TYPE_HDMI_TX0 // from vp0
#define ATK_LCD_TYPE_HDMI_TX1 // from vp1
#define ATK_LCD_TYPE_DP_TX0 // from vp2
#define ATK_LCD_TYPE_DP_TX1 // from vp2
```

总之，用户需根据自己所使用的屏幕来配置，配置完成后保存退出！

END

在 Android SDK 根目录下，执行如下命令编译整个 Android SDK：

```
./build.sh -UKA -J10
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -UKA -J10
will build u-boot
will build kernel
will build android
-----KERNEL_VERSION:5.10
-----KERNEL_DTS:rk3588-atk-evb7-lp4-v10
Force use clang and llvm to build kernel-5.10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
start build uboot
make clean && make mrproper && make distclean && ./make.sh rk3588
```

图 4.3.3 一键全自动编译 Android SDK

- U: 表示编译 U-Boot;
- K: 表示编译 Linux Kernel;
- A: 表示编译 Android;
- J10: 指定编译线程数量为 10;

整个编译过程将会持续很长一段时间，大概需要 2~3 个小时左右，或者更长时间，具体情况取决于你的电脑性能。

在编译过程中，可能会遇到一些问题，导致编译失败！毕竟每个人的开发环境或多或少的存在一些差异，所以大家应尽量按照本文档说明进行操作，尤其是 4.1.1 小节中所要求安装的依赖软件包。除此之外，保证 Ubuntu 内存容量至少**大于或等于 16G**，如果由于内存不足导致编译失败，可以将内存容量扩大再次尝试编译（或降低-J 后面的数字）；如果电脑硬件条件不允许，已无法再继续扩大虚拟机 Ubuntu 系统的内存容量，这个时候我们可以尝试扩充 Ubuntu 系统的 swap 交换空间，扩充方法请参考：[开发板光盘 A 盘-基础资料→10、用户手册→03、辅助文档→【正点原子】Ubuntu 扩充 swap 交换空间.pdf](#)；除此之外，Ubuntu 系统的磁盘空间尽量保证至少有 400~500G。

如果没有出现意外，编译将会成功，如图 4.3.4 所示：

```

2024-03-09 10:31:40 - sparse_img.py - INFO : Total of 61424 4096-byte output blocks in 5 input chunks.
2024-03-09 10:32:32 - build_super_image.py - INFO : Done writing image out/target/product/ATK_DLRK3588/su

#### build completed successfully (14:50 (mm:ss)) ####

Build android ok!
make and copy android images
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create rockdev/Image-ATK_DLRK3588/resource.img...
done.
create rockdev/Image-ATK_DLRK3588/boot.img...
done.
create rockdev/Image-ATK_DLRK3588/boot-debug.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot.img
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot-debug.img
create rockdev/Image-ATK_DLRK3588/recovery.img...
done.
create rockdev/Image-ATK_DLRK3588/super.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/userdata.img
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
make and copy packaging in IMAGE
    
```

图 4.3.4 Android SDK 编译完成

出现了“#### build completed successfully (编译所需时间 (hh:mm:ss)) ####”字符串则表示编译成功了，那么恭喜各位！

编译完成后，生成的镜像存放在 rockdev/Image-ATK_DLRK3588/目录，如下图所示：

```

dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls -l rockdev/Image-ATK_DLRK3588/
总用量 1807528
-rwxrwxr-x 1 dengtao dengtao 1048576 3月 7 17:15 baseparameter.img
-rw-rw-r-- 1 dengtao dengtao 39993344 3月 12 17:36 boot-debug.img
-rw-rw-r-- 1 dengtao dengtao 39823360 3月 12 17:35 boot.img
-rwxrwxr-x 1 dengtao dengtao 6129 3月 7 18:28 config.cfg
-rw-rw-r-- 1 dengtao dengtao 443 3月 12 17:35 dtbo.img
-rw-rw-r-- 1 dengtao dengtao 481728 3月 12 17:20 MiniLoaderAll.bin
-rw-rw-r-- 1 dengtao dengtao 49152 3月 7 18:10 misc.img
-rw-rw-r-- 1 dengtao dengtao 638 3月 12 17:35 parameter.txt
-rw-rw-r-- 1 dengtao dengtao 49152 3月 7 18:10 pcba_small_misc.img
-rw-rw-r-- 1 dengtao dengtao 49153 3月 7 18:10 pcba_whole_misc.img
-rw-rw-r-- 1 dengtao dengtao 75446272 3月 12 17:35 recovery.img
-rw-rw-r-- 1 dengtao dengtao 697856 3月 12 17:25 resource.img
-rw-rw-r-- 1 dengtao dengtao 1689859852 3月 12 17:37 super.img
-rw-rw-r-- 1 dengtao dengtao 4194304 3月 12 17:20 uboot.img
-rwxrwxr-x 1 dengtao dengtao 4096 3月 7 17:15 vbmeta.img
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
    
```

图 4.3.5 rockdev 目录下的内容

该目录下存在多个.img 以及.bin 镜像文件，但并非所有镜像都需要烧录到开发板，下面将会介绍。

4.4 镜像介绍

本小节将对 rockdev/Image-ATK_DLRK3588/目录下的镜像文件进行简单地介绍。

镜像	说明
boot.img	Android 格式启动镜像，该镜像由多个镜像合并而成，包括 ramdisk、内核镜像 Image、内核 DTB、资源镜像 resource.img；boot.img 最终会烧录到开发板 boot 分区

boot-debug.img	与 boot.img 的区别在于: user 版本固件可以烧写这个 boot-debug.img 进行 root 权限操作
baseparameter.img	用于存储 Rockchip 平台显示分辨率、显示效果调节等配置信息, 确保关机重启后能维持和关机之前一样的效果, 并确保整个开机过程显示效果保持一致; 该镜像会烧录到 baseparameter 分区
dtbo.img	Android 10.0 及以上版本, 支持 Device Tree Overlays 功能, 用于多个产品间的兼容; 需要将 dtbo.img 镜像烧录至 dtbo 分区
MiniLoaderAll.bin	运行在 RK3588 平台 U-Boot 之前的一段 loader 代码(比 U-Boot 更早阶段的 Loader), MiniLoaderAll.bin 由 TPL 和 SPL 两部分组成, TPL 用于初始化 DDR, 运行在 SRAM; 而 SPL 运行在 DDR, 主要负责加载、引导 uboot.img。
misc.img	包含 Android BCB (Bootloader Control Block) 信息和 A/B 系统信息, 该镜像会烧写到 misc 分区。Misc 分区是一个很重要的分区, 其中包括 BCB (可用于控制系统启动流程、Android 系统与 BootLoader、recovery 交互的一种机制) 和 A/B 系统数据块。 包含 recovery-wipe 开机标识信息, 烧写后会进行 recovery, 常用
pcba_small_misc.img	包含 PCBA 开机标识信息, 烧写后会进入简易版 PCBA 模式, 不常用
pcba_whole_misc.img	包含 PCBA 开机标识信息, 烧写后会进入完整版 PCBA 模式, 不常用
parameter.txt	一个 txt 文本文件, 是 RK3588 平台的分区表(记录分区名以及每个分区它的起始地址、结束地址); 烧写镜像时, 并不需要 parameter.txt 文件烧录到 Flash 器件 (eMMC 或 NAND), 而会读取它的信息去定义分区。
recovery.img	该镜像由多个镜像合并而成, 包括 recovery-ramdisk、内核镜像 Image、DTB、DTBO 以及资源镜像 resource.img; recovery.img 用于进入 recovery 模式, recovery 模式是一种用于对设备进行修复、升级更新的模式。recovery.img 最终会烧录到开发板 recovery 分区
resource.img	RK 定义的资源镜像, 包含 logo、DTB, resource.img 不单独烧录, 而是被打包进 boot.img 和 recovery.img 中
super.img	超级镜像, 由多个镜像合并而成, 包括 odm.img、odm_dlkimg、product.img、system.img、system_ext.img、vendor.img、vendor_dlkimg 等, super.img 最终会烧录到开发板 super 分区
uboot.img	由多个镜像合并而成, 包含 trust 镜像 (ARM Trusted Firmware + OP-TEE OS)、u-boot 镜像、u-boot dtb; 在编译 U-Boot 时会将这些镜像打包成一个 uboot.img。uboot.img 最终会烧录到开发板 uboot 分区
vbmeta.img	包含 AVB (Android Verified Boot) 校验信息, 用于 AVB 校验; AVB 是 Google 设计的一套固件校验流程

表 4.4.1 RK3588 Android 系统镜像介绍

第五章将向用户介绍如何烧录镜像。

4.5 单独编译

RK3588 Android SDK 集成了 Android 源码、Linux 内核源码以及 U-Boot 源码, 本小节向用户介绍如何单独编译 U-Boot、Linux 内核以及 Android。

4.5.1 单独编译 U-Boot

通过 build.sh 脚本单独编译 U-Boot 源码, 在 SDK 根目录下执行如下命令:

```
./build.sh -U -J16
```



```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -U -J16
will build u-boot
-----KERNEL_VERSION:5.10
-----KERNEL_DTS:rk3588-atk-evb7-lp4-v10
Force use clang and llvm to build kernel-5.10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
start build uboot
make clean && make mrproper && make distclean && ./make.sh rk3588
CLEAN dts/./arch/arm/dts
```

图 4.5.1.1 编译 U-Boot(1)

```

/home/dengtao/rk3588_android_sdk
Build uboot ok!
package resource.img with charger images

Pack ./tools/images/ & ../kernel-5.10/resource.img to resource.img ...
Unpacking old image(../kernel-5.10/resource.img):
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_6.bmp
Pack to resource.img succeeded!

Packed resources:
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_6.bmp

./resource.img with battery images is ready
/home/dengtao/rk3588_android_sdk
Repacking header 2 boot...
make and copy android images
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create rockdev/Image-ATK_DLRK3588/resource.img...
done.
create rockdev/Image-ATK_DLRK3588/boot.img...
done.
create rockdev/Image-ATK_DLRK3588/boot-debug.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot.img
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot-debug.img
create rockdev/Image-ATK_DLRK3588/recovery.img...
done.
create rockdev/Image-ATK_DLRK3588/super.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/userdata.img
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$

```

图 4.5.1.2 编译 U-Boot(2)

编译完成后，会生成两个镜像文件：uboot.img 和 MiniLoaderAll.bin，镜像存放在 **rockdev/Image-ATK_DLRK3588/** 目录：

```

dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ cd rockdev/Image-ATK_DLRK3588/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ ls -l uboot.img MiniLoaderAll.bin
-rw-rw-r-- 1 dengtao dengtao 481728 3月 9 12:21 MiniLoaderAll.bin
-rw-rw-r-- 1 dengtao dengtao 4194304 3月 9 12:21 uboot.img
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$

```

图 4.5.1.3 镜像

如果用户改动过 U-Boot 源码，则需按照上面的命令重新生成 uboot.img。

4.5.2 单独编译 Linux 内核

通过 build.sh 脚本单独编译 Linux 内核，在 SDK 根目录下执行如下命令：

```

./build.sh -K -J16          # 编译内核
make bootimage              # 生成 boot.img
make recoveryimage          # 生成 recovery.img

```



```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -K -j16
will build kernel
-----KERNEL_VERSION:5.10
-----KERNEL_DTS:rk3588-atk-evb7-lp4-v10
Force use clang and llvm to build kernel-5.10
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
Start build kernel
make clean && make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 rockchip_defconfig android
rm64 rk3588-atk-evb7-lp4-v10.img -j16
Build Command: make clean && make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 rockchip_de
VM_IAS=1 ARCH=arm64 rk3588-atk-evb7-lp4-v10.img -j16
CLEAN certs
CLEAN drivers/scsi
CLEAN fs/unicode
CLEAN kernel
CLEAN lib
CLEAN net/wireless
CLEAN security/selinux
CLEAN usf
```

图 4.5.2.1 编译 Linux 内核

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ make bootimage
14:22:59 *****
14:22:59 You are building on a machine with 15.6GB of RAM
14:22:59
14:22:59 The minimum required amount of free memory is around 16GB,
14:22:59 and even with that, some configurations may not work.
14:22:59
14:22:59 If you run into segfaults or other errors, try reducing your
14:22:59 -j value.
14:22:59 *****
build/make/core/soong_config.mk:195: warning: BOARD_PLAT_PUBLIC_SEPOLICY_DIR has been deprecated. Use
build/make/core/soong_config.mk:196: warning: BOARD_PLAT_PRIVATE_SEPOLICY_DIR has been deprecated. Use
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
14:23:01 Disallowed PATH tool "python3.8" used: [ ]string{"python3.8", ".repo/repo/repo", "manifest", "
14:23:01 See https://android.googlesource.com/platform/build/+master/Changes.md#PATH_Tools for more i
[100% 1/1] Target boot image: out/target/product/ATK_DLRK3588/boot.img
#### build completed successfully (6 seconds) ####
```

图 4.5.2.2 生成 boot.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ make recoveryimage
17:34:48 *****
17:34:48 You are building on a machine with 15.6GB of RAM
17:34:48
17:34:48 The minimum required amount of free memory is around 16GB,
17:34:48 and even with that, some configurations may not work.
17:34:48
17:34:48 If you run into segfaults or other errors, try reducing your
17:34:48 -j value.
17:34:48 *****
build/make/core/soong_config.mk:195: warning: BOARD_PLAT_PUBLIC_SEPOLICY_DIR has been deprecated. Use SYSTEM
build/make/core/soong_config.mk:196: warning: BOARD_PLAT_PRIVATE_SEPOLICY_DIR has been deprecated. Use SYSTE
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
17:35:27 Disallowed PATH tool "python3.8" used: [ ]string{"python3.8", ".repo/repo/repo", "manifest", "-r", "
17:35:27 See https://android.googlesource.com/platform/build/+master/Changes.md#PATH_Tools for more informa
[100% 7/7] build out/target/product/ATK_DLRK3588/recovery.img
#### build completed successfully (05:13 (mm:ss)) ####
```

图 4.5.2.3 生成 recovery.img

编译完成后,生成 boot.img 和 recovery.img,镜像存放在 out/target/product/ATK_DLRK3588/ 目录,执行如下命令可以将其拷贝到 rockdev/Image-ATK_DLRK3588/ 目录:

```
cp out/target/product/ATK_DLRK3588/boot.img rockdev/Image-ATK_DLRK3588/  
cp out/target/product/ATK_DLRK3588/recovery.img rockdev/Image-ATK_DLRK3588/
```

因为 Linux 内核镜像(也包括内核 DTB)并不单独烧录,而是被打包进 boot.img 和 recovery.img (boot.img 和 recovery.img 镜像中都包含了内核镜像、内核 DTB)。所以,当用户改动过内核源码或设备树配置时,需要按照上面的命令重新生成 boot.img 以及 recovery.img。

4.5.3 单独编译 Android 源码

在 SDK 根目录下执行如下命令编译 Android 源码:

```
./build.sh -A -J16
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -A -J16  
will build android  
-----KERNEL_VERSION:5.10  
-----KERNEL_DTS:rk3588-atk-evb7-lp4-v10  
Force use clang and llvm to build kernel-5.10  
  
=====
```

```
PLATFORM_VERSION_CODENAME=REL  
PLATFORM_VERSION=12  
TARGET_PRODUCT=ATK_DLRK3588  
TARGET_BUILD_VARIANT=userdebug  
TARGET_BUILD_TYPE=release  
TARGET_ARCH=arm64  
TARGET_ARCH_VARIANT=armv8-a  
TARGET_CPU_VARIANT=generic  
TARGET_2ND_ARCH=arm  
TARGET_2ND_ARCH_VARIANT=armv8-a  
TARGET_2ND_CPU_VARIANT=generic  
HOST_ARCH=x86_64  
HOST_2ND_ARCH=x86  
HOST_OS=linux  
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS  
HOST_CROSS_OS=windows  
HOST_CROSS_ARCH=x86  
HOST_CROSS_2ND_ARCH=x86_64  
HOST_BUILD_TYPE=release  
BUILD_ID=SQ3A.220705.003.A1  
OUT_DIR=out  
=====
```

```
package resource.img with charger images  
  
Pack ./tools/images/ & ../kernel-5.10/resource.img to resource.img ...  
Unpacking old image(../kernel-5.10/resource.img):  
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.b  
Pack to resource.img succeeded!  
  
Packed resources:  
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.b
```

图 4.5.3.1 编译 Android 源码(1)



```
#### build completed successfully (11:04 (mm:ss)) ####
Build android ok!
make and copy android images
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create rockdev/Image-ATK_DLRK3588/resource.img...
done.
create rockdev/Image-ATK_DLRK3588/boot.img...
done.
create rockdev/Image-ATK_DLRK3588/boot-debug.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/vendor_boot-
create rockdev/Image-ATK_DLRK3588/recovery.img...
done.
create rockdev/Image-ATK_DLRK3588/super.img...
done.
skip copy images: /home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/userdata.img
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
```

图 4.5.3.2 编译 Android 源码(2)

编译完成后, 会生成多个镜像文件, 包括 boot.img、dtbo.img、super.img、recovery.img 等, 镜像存放在 rockdev/Image-ATK_DLRK3588/目录下:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls -l rockdev/Image-ATK_DLRK3588/
总用量 1807532
-rwxrwxr-x 1 dengtao dengtao 1048576 3月 7 17:15 baseparameter.img
-rw-rw-r-- 1 dengtao dengtao 39993344 3月 13 00:18 boot-debug.img
-rw-rw-r-- 1 dengtao dengtao 39823360 3月 13 00:18 boot.img
-rwxrwxr-x 1 dengtao dengtao 6129 3月 7 18:28 config.cfg
-rw-rw-r-- 1 dengtao dengtao 443 3月 13 00:18 dtbo.img
-rw-rw-r-- 1 dengtao dengtao 481728 3月 12 23:36 MiniLoaderAll.bin
-rw-rw-r-- 1 dengtao dengtao 49152 3月 7 18:10 misc.img
-rw-rw-r-- 1 dengtao dengtao 638 3月 13 00:18 parameter.txt
-rw-rw-r-- 1 dengtao dengtao 49152 3月 7 18:10 pcba_small_misc.img
-rw-rw-r-- 1 dengtao dengtao 49153 3月 7 18:10 pcba_whole_misc.img
-rw-rw-r-- 1 dengtao dengtao 75446272 3月 13 00:18 recovery.img
-rw-rw-r-- 1 dengtao dengtao 697856 3月 13 00:07 resource.img
-rw-rw-r-- 1 dengtao dengtao 1689859852 3月 13 00:19 super.img
-rw-rw-r-- 1 dengtao dengtao 4194304 3月 12 23:36 uboot.img
-rwxrwxr-x 1 dengtao dengtao 4096 3月 7 17:15 vbmeta.img
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

4.4 小节中已经对这些镜像进行了讲解, 这里不再多说! 如果用户改动过 Android 源码, 则需按照上面的命令重新编译 Android 源码 (建议先执行 make installclean 后再编译)。

编译 Android 源码之前, 需提前编译好 Linux 内核源码, 原因有两个:

①、执行 “./build.sh -A” 会生成 boot.img、recovery.img, 制作 boot.img 和 recovery.img 需提供内核镜像 Image、内核 DTB 以及 resource.img, 而这些镜像需要编译 Linux 内核后生成, 它们会被打包进 boot.img 和 recovery.img 镜像中。

②、编译 Linux 内核生成的.ko 模块 (譬如 WiFi 驱动模块) 会被打包到 vendor_dkkm.img (vendor_dkkm.img 不单独烧录, 被包含在 super.img 超级镜像中)。



4.5.4 制作 update.img 固件

使用 build.sh 脚本可以将 rockdev/Image-ATK_DLRK3588/目录下的各分立镜像打包成一个 update.img 固件, 方便用户烧写、更新升级。

在 SDK 根目录下执行如下命令制作 update.img:

```
./build.sh -u
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ./build.sh -u
will build update.img
-----KERNEL_VERSION:5.10
-----KERNEL_DTS:rk3588-atk-evb7-lp4-v10
Force use clang and llvm to build kernel-5.10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=ATK_DLRK3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====
package resoure.img with charger images

Pack ./tools/images/ & ../kernel-5.10/resource.img to resource.img ...
Unpacking old image(../kernel-5.10/resource.img):
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_1
Pack to resource.img succeeded!

Packed resources:
rk-kernel.dtb battery_0.bmp battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_1

./resource.img with battery images is ready
/home/dengtao/rk3588_android_sdk
Repacking header 2 boot...
make and copy android images
```

```

----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x29a,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x759c0,userspace=0xec
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x77000,size=0x28a,userspace=0x1,flash_address=0x0000
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x77800,size=0x400000,userspace=0x800,flash_address=0x0000400
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x477800,size=0xc000,userspace=0x18,flash_address=0x00008000
Add file: ./Image/dtbo.img
dtbo,Add file: ./Image/dtbo.img done,offset=0x483800,size=0x1bb,userspace=0x1,flash_address=0x0000a000
Add file: ./Image/vbmeta.img
vbmeta,Add file: ./Image/vbmeta.img done,offset=0x484000,size=0x1000,userspace=0x2,flash_address=0x0000c000
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x485000,size=0x25fa800,userspace=0x4bf5,flash_address=0x0000c8
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x2a7f800,size=0x47f3800,userspace=0x8fe7,flash_address
Add file: ./Image/baseparameter.img
baseparameter,Add file: ./Image/baseparameter.img done,offset=0x7273000,size=0x100000,userspace=0x200,flash
Add file: ./Image/super.img
super,Add file: ./Image/super.img done,offset=0x7373000,size=0x64b9470c,userspace=0xc9729,flash_address=0xc
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.23*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making update.img OK.
Make update image ok!
/home/dengtao/rk3588_android_sdk

```

图 4.5.4.1 生成 update.img

命令执行成功后，生成的 update.img 固件存放在 **rockdev/Image-ATK_DLRK3588/**目录，如下所示：

```

dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls -l rockdev/Image-ATK_DLRK3588/update.img
-rw-rw-r-- 1 dengtao dengtao 1811403338 3月 9 16:28 rockdev/Image-ATK_DLRK3588/update.img
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$

```

图 4.5.4.2 update.img 固件

使用烧录工具烧写镜像时，用户只需提供 update.img 即可，非常方便！

4.6 SDK 清理

build.sh 脚本并未实现清理功能，需要单独清理，包括 Android 清理、U-Boot 清理、Linux 内核清理。执行如下命令清理 U-Boot（需进入 U-Boot 源码目录）：

```

cd u-boot/
make distclean #或 make clean

```

执行如下命令清理 Linux 内核（需进入内核源码目录）：

```

cd kernel/
make distclean #或 make clean

```

在 SDK 根目录下执行如下命令清理 Android：

```

make clean # 该命令会彻底清理 需慎重使用
或
make installclean # 一般常用这个

```

执行 make clean 相当于删除 out/ 目录下的所有文件，需慎重使用！一般常用 **make installclean** 进行清理；其实也可以执行“make clean-模块”命令来清理某个特定模块，后面会给大家介绍。

第五章 镜像烧录

本章向用户介绍如何将镜像烧录至 ATK-DLRK3588 开发板并运行。

Rockchip 平台提供了多种镜像烧写方式，在第 2.9 小节中给大家进行了介绍；譬如在 Windows 下通过 RKDevTool 工具烧写、通过 SD 卡方式进行烧写、通过 FactoryTool 工具批量烧写（量产烧写工具、支持 USB 一拖多烧写）以及在 Ubuntu 系统下通过 Linux_Upgrade_Tool 工具烧写等等；总之，烧写镜像的方式有很多种，用户可以选择合适的方式进行烧写，将镜像烧写至开发板。

5.1 烧写模式介绍

2.9.1 小节中已经详细介绍了开发板烧写模式相关的问题，如果对此还有疑问，请移步前往该小节阅读！

5.2 Windows 系统下烧写

Windows 下使用瑞芯微开发工具（RKDevTool）来烧写镜像，烧写之前，先将编译生成的镜像（rockdev/Image-ATK_DLRK3588/目录下的镜像）从 Ubuntu 系统拷贝到 Windows 下，譬如拷贝到 Windows 桌面 rk3588_images 目录，包括 baseparameter.img、boot.img、dtbo.img、MiniLoaderAll.bin、misc.img、parameter.txt、recovery.img、super.img、uboot.img、vbmeta.img，如下所示：

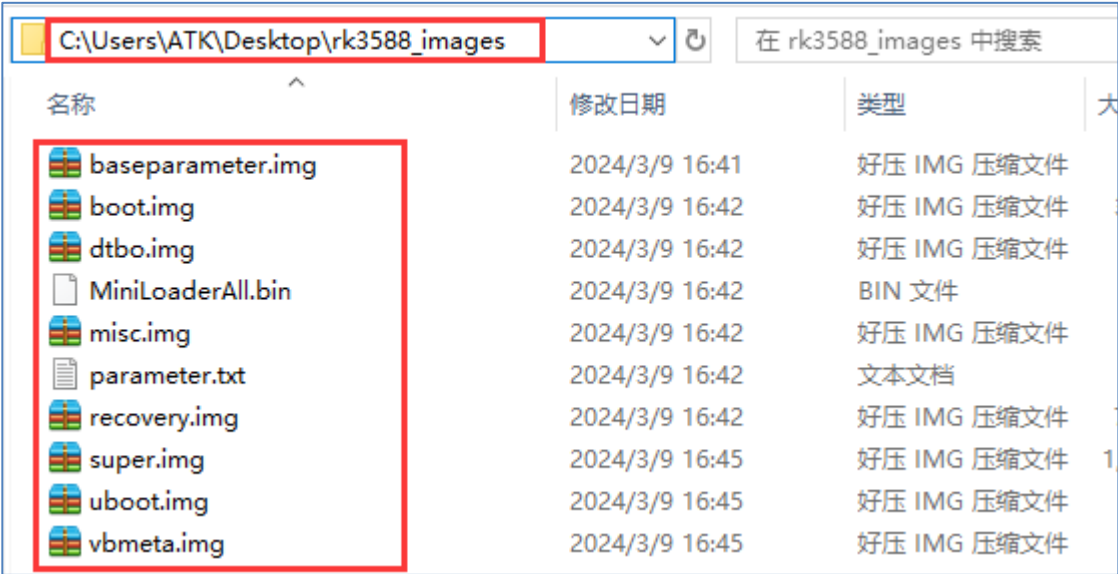


图 5.2.1 rk3588_images 目录下的文件

打开瑞芯微开发工具：



图 5.2.2 瑞芯微开发工具

5.2.1 分区表 parameter.txt 介绍

2.9.2 小节中向用户介绍了如何使用瑞芯微开发工具（RKDevTool）烧写镜像，直接导入一个.cfg 配置文件，然后点击“**执行**”按钮进行烧写；整个过程无需用户配置。但对于开发人员来说，肯定需要知道如何去配置这些信息，方便自己开发调试。

本小节讲一下分区表文件 parameter.txt，该文件是一个 txt 文本文件。前面已经给大家说过，parameter.txt 是 Rockchip 平台的分区表文件，该文件记录了每个分区的起始地址、分区大小等信息，我们来看下它的内容：

```
FIRMWARE_VER: 12.0
MACHINE_MODEL: ATK-DLRK3588
MACHINE_ID: 007
MANUFACTURER: Alientek
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: ATK_DLRK3588
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:mtdparts=rk29xxnand:0x00002000@0x00002000(security),0x00002000@0x00004000(uboot),0x00002000@0x00006000(trust),0x00002000@0x00008000(misc),0x00002000@0x0000a000(dtbo),0x00008000@0x0000c000(vbmeta),0x00014000@0x0000c800(boot),0x00036000@0x00020800(recovery),0x000ba000@0x00056800(backup),0x000c0000@0x00110800(cache),0x00008000@0x001d0800(metadata),0x00008000@0x001d8800(baseparameter),0x00614000@0x001d9000(super),-@0x007ed000(userdata:grow)
```

RK 提供了相应的文档向用户介绍 parameter.txt 文件中的各种标识，该文档所在路径为：[<SDK>/RKDocs/common/NVM/Rockchip_Introduction_Partition_CN.pdf](#)，这里只是给大家介绍 mtdparts 所定义的分区信息。

mtdparts 定义的信息如下：

```
rk29xxnand:0x00002000@0x00002000(security),0x00002000@0x00004000(uboot),0x00002000@0x00006000(trust),0x00002000@0x00008000(misc),0x00002000@0x0000a000(dtbo),0x00008000@0x0000c000(vbmeta),0x00014000@0x0000c800(boot),0x00036000@0x00020800(recovery),0x000ba000@0x00056800(backup),0x000c0000@0x00110800(cache),0x00008000@0x001d0800(metadata),0x00008000@0x001d8800(baseparameter),0x00614000@0x001d9000(super),-@0x007ed000(userdata:grow)
```

rk29xxnand 是一个标识，为了兼容性，rockchip 平台都是用 rk29xxnand 做标识。

诸如 0x00002000@0x00002000(security)、0x00002000@0x00004000(uboot)、0x00002000@0x00008000(misc)等信息用于定义分区，@符号之前的数值是分区大小，@符号之后的数值是分区的起始位置，括号里面对应的是分区的名字；所有数值的单位都是 sector（扇区），1 个 sector 为 512 字节。所以由此可知，uboot 分区的起始位置为 0x4000 sectors，大小为 0x2000 sectors（4MB）；misc 分区的起始位置为 0x8000 sectors，大小也是 0x2000 sectors。

为了性能，每个分区的起始地址需要 32KB（64 sectors）对齐，大小也需要 32KB 的整数倍。

最后一个分区需要指定 grow 参数，表示将剩余存储空间全部分配给该分区：

```
-@0x007ed000(userdata:grow)
```

userdata 分区虽然指定了起始位置，但并未指定分区大小，而是使用了“-”来代替，然后在分区名后面加入“:grow”，那么表示将剩余空间全部分配给该分区。

每个分区的作用如下表所示：

分区名	说明
security	安全分区，用于存储加密过安全数据，AVB 安全启动所需
uboot	存放 uboot.img 镜像，uboot.img 镜像会烧录到该分区
trust	trust.img 镜像会烧录到该分区，RK3588 平台并未使用该分区，它的 Trust（ATF+OPTEE）固件已经打包到 uboot.img 镜像中了
misc	misc.img 镜像会烧录到该分区，misc 分区是一个很重要的分区，主要用于 Android、U-Boot 以及 recovery 之间的通信
dtbo	存放 dtbo.img，dtbo.img 镜像会烧录到该分区

vbmata	vbmata.img 镜像会烧录到该分区, 用于 AVB 校验
boot	存放 boot.img, boot.img 镜像会烧录到该分区
recovery	存放 recovery.img, recovery.img 镜像会烧录到该分区, 用于进入 recovery 模式
backup	备份分区, 当设备无法正常启动进入 Android 系统, 用户可引导设备进入 recovery, 在 recovery 菜单栏选中“Recovery System”选项, 进行系统恢复操作, 该功能会使用到 backup 分区
cache	缓冲分区, 在系统升级过程中会使用/cache 分区, 系统升级包会下载到该分区, 主要用于缓冲系统升级 OTA 包 /cache/recovery/command /cache/recovery/log /cache/recovery/intent
metadata	此分区用于在设备使用元数据加密时存储元数据加密密钥
baseparameter	baseparameter.img 镜像会烧录到该分区
super	super.img 会烧录到该分区, 自 Android 10 之后, Android 系统支持动态分区 (dynamic partition), 它将多个系统只读分区 (包括 system、product、vendor、odm 或者其它产商自定义分区) 合并为一个 super 分区
userdata	供用户使用的分区, 此分区包含用户安装的应用和数据, 包括自定义数据

表 5.2.1.1 各分区的作用介绍

5.2.2 配置

通过 parameter.txt 分区表文件我们知道它一共定义了 14 个分区以及每个分区的名字、起始位置和大小, 根据这些信息我们可以手动配置瑞芯微开发工具, 配置完成后如下图所示:

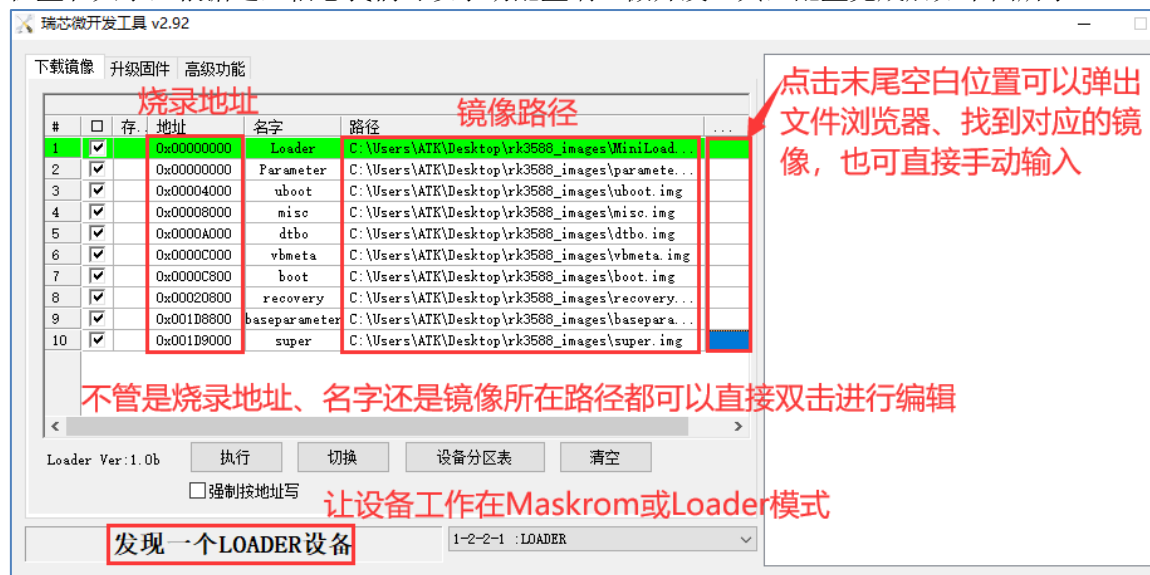


图 5.2.2.1 手动配置瑞芯微开发工具

这里需要注意几个点:

1. 烧录地址的单位也是 sectors (扇区, 一个扇区等于 512 字节);
2. 第一项对应的是 MiniLoaderAll.bin 镜像, 它的地址不用配置, 直接使用 0x0 即可, 因为 MiniLoaderAll.bin 镜像有专门的烧录地址, 无需用户配置, 而且它的名字一般都是 Loader (或小写 loader), 不要去改动它;
3. 上图中第二项对应的是分区表 parameter.txt, 同样它的地址也不用配置, 直接使用 0x0 即可, 因为 parameter.txt 分区表文件不会烧录到 Flash 存储器件中, 烧录工具会根据里面定义的分区分来创建分区表; 同样, 它的名字设置为 parameter (或者大写 Parameter) 不要去改动它, 因为烧录工具需要通过这个 “parameter” 名字来识别分区表文件。

4.除了 **MiniLoaderAll.bin** 和 **parameter.txt** 稍微特殊一点之外,其余镜像直接根据分区表进行配置即可,名字尽量使用 **parameter.txt** 文件中所定义的分区名(其实也可以直接导入 **parameter.txt** 文件,它会自动生成,我们只需配置镜像对应的路径即可)。

配置完成之后,我们还可以将这些配置信息导出、保存到一个 **.cfg** 文件中,方便下次直接导入;导出的方法很简单,步骤如图 5.2.2.2~5.2.2.3 所示:

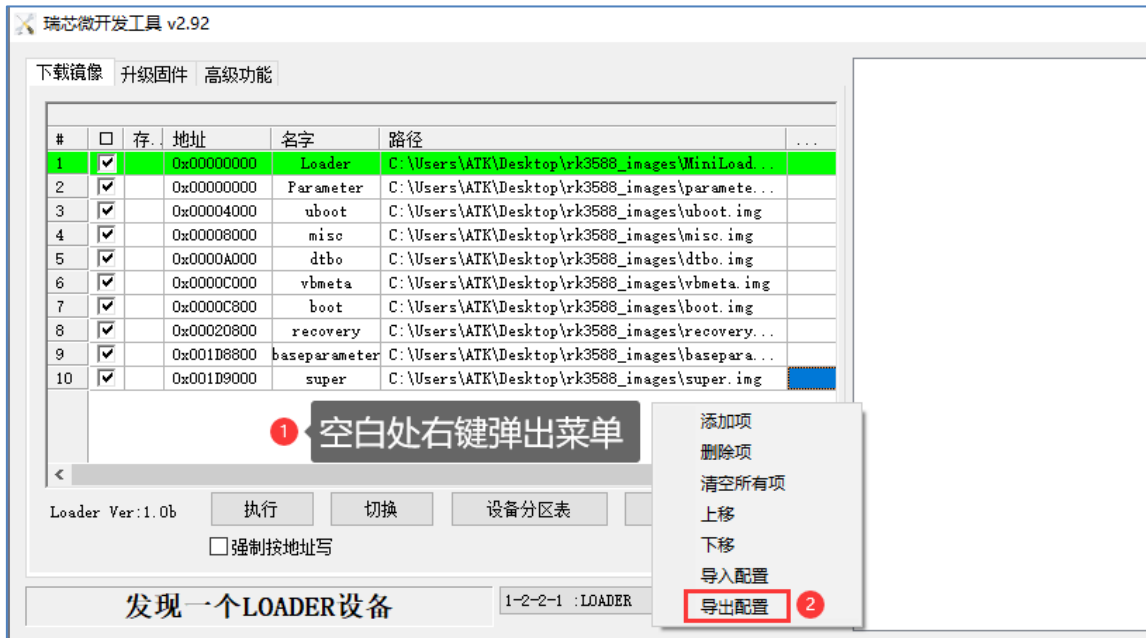


图 5.2.2.2 导出配置(1)

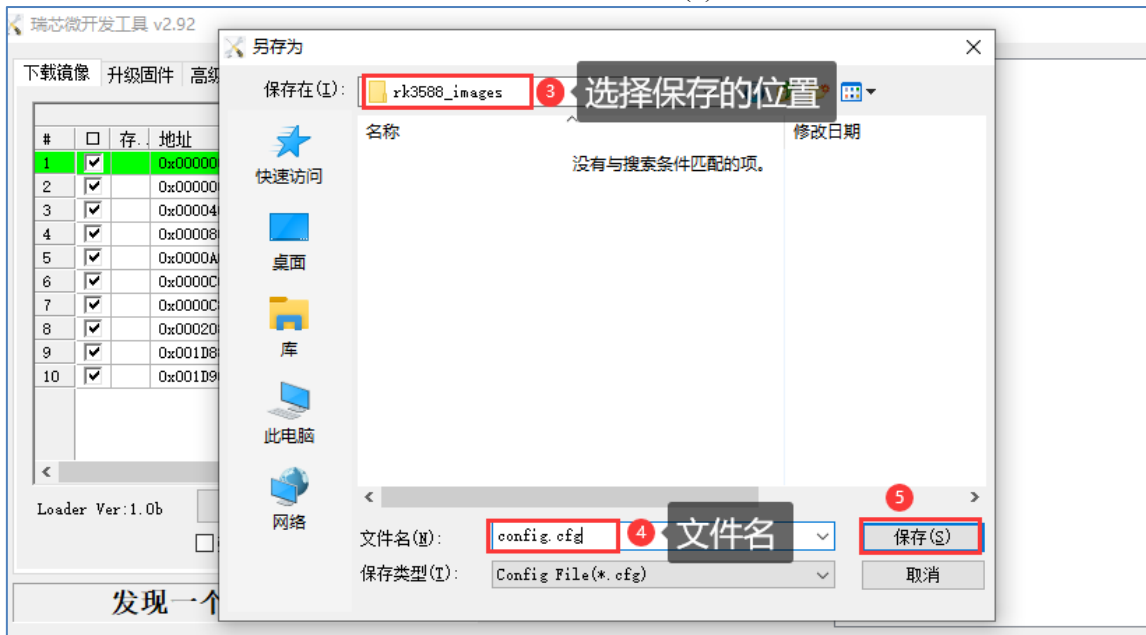


图 5.2.2.3 导出配置(2)

将配置信息导出、保存到 **config.cfg** 文件中,方便下次直接导入该配置信息,不用每次都去手动配置。

5.2.3 烧录

开发板先连接好电源适配器以及 TYPE-C0 接口,具体连接方式可以参考图 2.9.2.10 所示,这里不再重述!烧写之前,让开发板进入 Maskrom 或 Loader 模式,开发板进入 Maskrom 或 Loader 模式的方法,前面章节已经给大家讲过。

譬如通过 Loader 模式烧写镜像, 先按住开发板上的 **V+** 按键, 然后给开发板上电或复位, 此时设备便会进入 Loader 模式 (瑞芯微开发工具会提示 “发现一个 **LOADER** 设备”), 然后点击 “**执行**” 按钮烧录镜像:

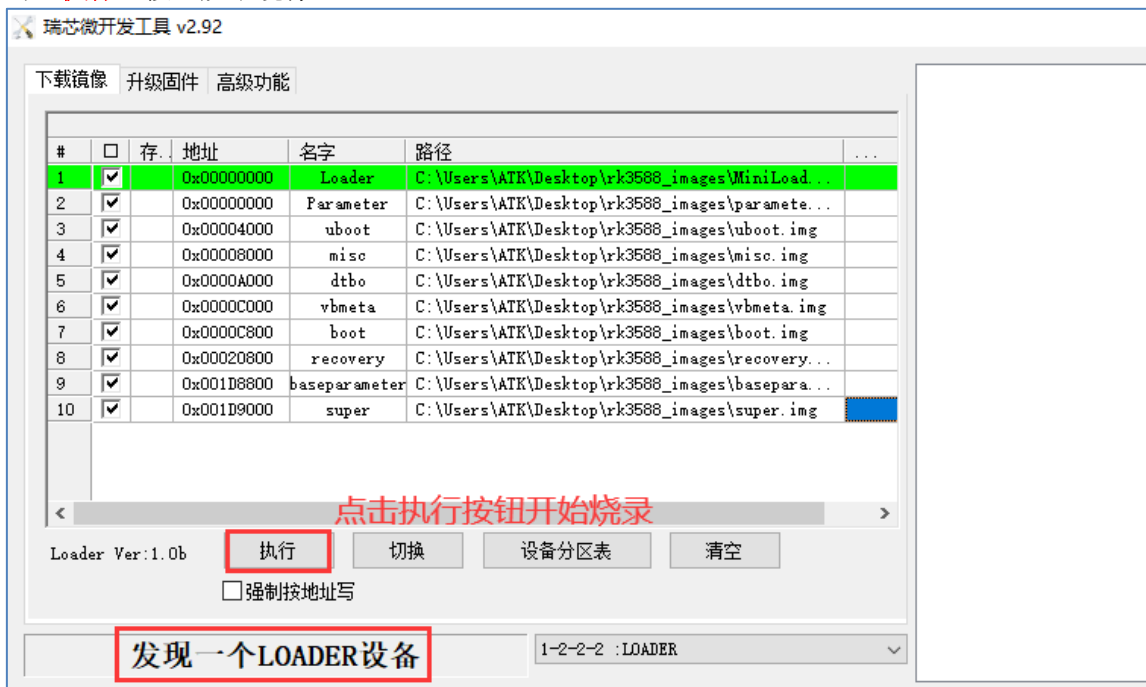


图 5.2.3.1 执行烧录

烧录过程中, 右边空白处会输出 log 信息, 如下所示:

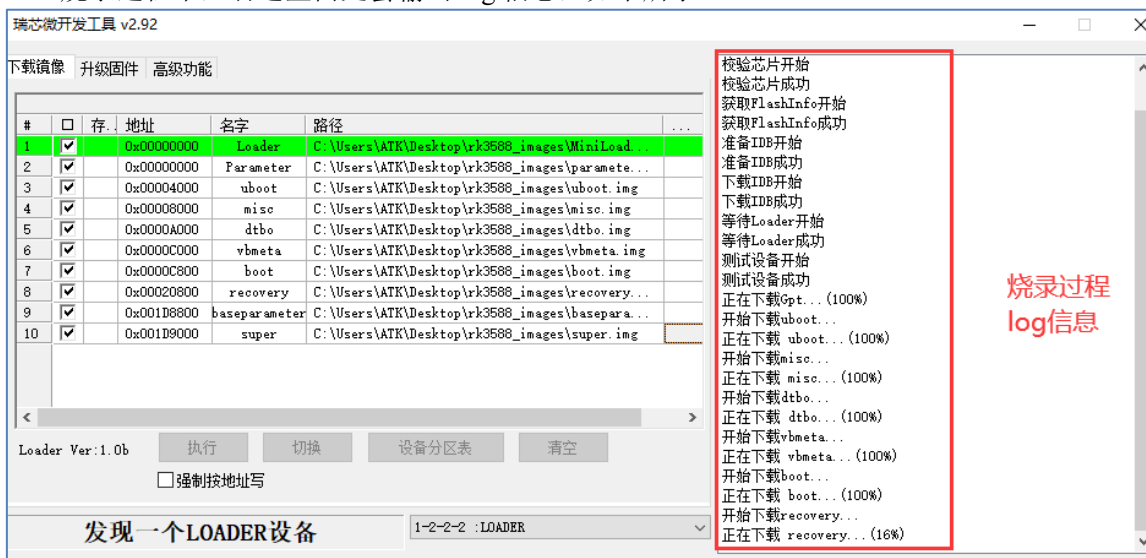


图 5.2.3.2 烧录过程 log 信息

也可单独烧录某分区, 譬如单独烧录 boot.img 到 boot 分区, 只需勾选对应的这一项即可, 如下所示:

1	☐	0x00000000	Loader	C:\Users\ATK\Desktop\rk35
2	☐	0x00000000	Parameter	C:\Users\ATK\Desktop\rk35
3	☐	0x00004000	Uboot	C:\Users\ATK\Desktop\rk35
4	☐	0x00008000	Misc	C:\Users\ATK\Desktop\rk35
5	☐	0x0000A000	Dtbo	C:\Users\ATK\Desktop\rk35
6	☐	0x0000C000	vbmata	C:\Users\ATK\Desktop\rk35
7	☒	0x0000C800	Boot	C:\Users\ATK\Desktop\rk35
8	☐	0x00020800	Recovery	C:\Users\ATK\Desktop\rk35
9	☐	0x001D8800	baseparameter	C:\Users\ATK\Desktop\rk35
10	☐	0x001D9000	Super	C:\Users\ATK\Desktop\rk35

图 5.2.3.3 单独烧录 boot.img

然后点击“**执行**”按钮进行烧录即可。这在开发、调试过程中很有用，因为有些情况下我们仅仅只是为了更新某个分区，无需重烧整个系统。需要注意的是，Maskrom 模式下更新单个分区，需要勾选 Loader 和 Parameter 一起烧录。

5.2.4 启动系统

烧录完成后会自动重启开发板，串口终端会输出信息：

```

24.991632] init: processing action (sys.boot_completed=1) from (/system/etc/init/hw/init.rc:981)
24.992317] init: starting service 'exec 14 (/bin/rm -rf /data/per_boot)'...
24.995166] init: SVC_EXEC service 'exec 14 (/bin/rm -rf /data/per_boot)' pid 1284 (uid 1000 gid 1000+0 context default) started
25.013137] type=1400 audit(1682406851.820:103): avc: denied { getattr } for comm="df" path="/linkerconfig" dev="tmpfs" ino=195
25.013817] type=1400 audit(1682406851.820:104): avc: denied { getattr } for comm="df" path="/linkerconfig" dev="tmpfs" ino=195
25.014953] type=1400 audit(1682406851.820:105): avc: denied { getattr } for comm="df" path="/data_mirror" dev="tmpfs" ino=2071
25.029035] init: Service 'exec 14 (/bin/rm -rf /data/per_boot)' (pid 1284) exited with status 0 waiting took 0.034000 seconds
25.029110] init: Sending signal 9 to service 'exec 14 (/bin/rm -rf /data/per_boot)' (pid 1284) process group...
25.029408] libprocessgroup: Successfully killed process cgroup uid 1000 pid 1284 in 0ms
25.354605] zram0: detected capacity change from 0 to 2039578624
25.382194] mkswap: Swapspace size: 1991772k, UUID=fbcbee15-f0ba-4b8b-ab83-60f7d7e815ca
25.386652] Adding 1991772k swap on /dev/block/zram0. Priority:-2 extents:1 across:1991772k SS
27.131756] type=1400 audit(1682406853.936:106): avc: denied { write } for comm="RenderThread" name="property_service" dev="tmp
permissive=0 app=com.android.launcher3
27.469184] type=1400 audit(1682406854.276:107): avc: denied { read } for comm=".update.service" name="u:object_r:serialno_prop
ermissive=0
27.469561] type=1400 audit(1682406854.276:108): avc: denied { read } for comm=".update.service" name="u:object_r:serialno_prop
ermissive=0
35.470544] vcc5v0_otg: disabling
35.470596] vcc3v3_lcd0_n: disabling
35.470614] vcc3v3_lcd1_n: disabling
35.470630] vcc3v3_pcie: disabling
36.760932] type=1400 audit(1682406863.570:109): avc: denied { read } for comm="Binder:178_2" name="wakeup4" dev="sysfs" ino=24
36.761538] type=1400 audit(1682406863.573:110): avc: denied { read } for comm="Binder:178_2" name="wakeup2" dev="sysfs" ino=16
36.762490] type=1400 audit(1682406863.573:111): avc: denied { read } for comm="Binder:178_2" name="wakeup9" dev="sysfs" ino=26
36.764383] type=1400 audit(1682406863.580:112): avc: denied { read } for comm="Binder:178_2" name="wakeup12" dev="sysfs" ino=25
36.768796] type=1400 audit(1682406863.583:113): avc: denied { read } for comm="Binder:178_2" name="wakeup8" dev="sysfs" ino=25
36.769045] type=1400 audit(1682406863.583:114): avc: denied { read } for comm="Binder:178_2" name="wakeup13" dev="sysfs" ino=25
36.769390] audit: audit_lost=1 audit_rate_limit=5 audit_backlog_limit=64
36.769426] audit: rate limit exceeded
36.842381] rockchip-vop2 fe040000.vop: [drm:vop2_crtc_atomic_disable] Crtc atomic disable vp2
36.909384] dw-mipi-dsi fe070000.dsi: [drm:dw_mipi_dsi_encoder_enable] final DSI-Link bandwidth: 432 x 4 Mbps
37.084077] rockchip-vop2 fe040000.vop: [drm:vop2_crtc_atomic_disable] Crtc atomic disable vp1
console: / $
    
```

图 5.2.4.1 进入系统

5.2.5 烧写 update.img

可以使用瑞芯微开发工具烧录 update.img 固件，首先将 4.5.4 小节生成的 update.img 拷贝到 Windows 下，然后通过瑞芯微开发工具将其烧录到开发板，烧录方法参考 2.9.3 小节。

5.3 Ubuntu 系统下烧写

本小节向用户介绍如何在 Ubuntu 下烧录镜像。在 Windows 下烧录，需要先将镜像从 Ubuntu 系统拷贝到 Windows 系统下，如此才可进行烧录。如果在 Ubuntu 下烧录，那么就不用拷贝镜像。

前面给大家讲过，在 Ubuntu 下可以使用 Linux_Upgrade_Tool 工具进行烧录，Android SDK 自带 Upgrade_Tool 烧录工具，该工具所在路径为：
<SDK>/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux.zip，如下所示：



```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ cd RKTools/linux/Linux_Upgrade_Tool/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$ ls
upgrade_tool_v2.17_for_linux.zip
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$
```

图 5.3.1 Linux_Upgrade_Tool 目录

首先将 upgrade_tool_v2.17_for_linux.zip 压缩文件解压到当前目录, 执行如下命令:

```
unzip -O GBK upgrade_tool_v2.17_for_linux.zip
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$ unzip -O GBK upgrade_tool_v2.17_for_linux.zip
Archive:  upgrade_tool_v2.17_for_linux.zip
  creating: upgrade_tool_v2.17_for_linux/
  extracting: upgrade_tool_v2.17_for_linux/config.ini
  inflating: upgrade_tool_v2.17_for_linux/revision.txt
  inflating: upgrade_tool_v2.17_for_linux/upgrade_tool
  inflating: upgrade_tool_v2.17_for_linux/命令行开发工具使用文档.pdf
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool$ cd upgrade_tool_v2.17_for_linux/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$ ls
命令行开发工具使用文档.pdf  config.ini  revision.txt  upgrade_tool
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
```

图 5.3.2 upgrade_tool_v2.17_for_linux.zip 解压

执行 unzip 命令后生成 upgrade_tool_v2.17_for_linux 文件夹, 该目录下有一份 RK 提供的使用说明文档, 详细向用户介绍了如何使用 upgrade_tool 工具, 所以详细的使用方法请参考这份文档: 《[命令行开发工具使用文档.pdf](#)》。

我们需要给 upgrade_tool 加上可执行权限, 执行如下命令:

```
chmod a+x upgrade_tool
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$ ls
命令行开发工具使用文档.pdf  config.ini  revision.txt  upgrade_tool
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$ chmod a+x upgrade_tool
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$ ls
命令行开发工具使用文档.pdf  config.ini  revision.txt  upgrade_tool
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux$
```

图 5.3.3 给 upgrade_tool 添加可执行权限

upgrade_tool 有两种运行模式: **命令行模式**和**工具模式**, 直接运行 upgrade_tool 命令, 不加任何参数则会进入到工具模式, 工具模式其实就是一个交互模式, 在交互模式下用户可以输入不同的指令、按回车执行该指令, 那么就会触发该指令所对应的操作, 譬如下载镜像、擦除镜像、读取设备信息等等。

运行 upgrade_tool 命令时加入参数 (譬如 UF UL DI DB 等等) 则会进入命令行模式, 所谓命令行模式, 指的是非交互模式; 执行 upgrade_tool 时加入参数, 通过这些参数来告诉 upgrade_tool 工具本次需要执行什么操作, 譬如烧写镜像、擦除等, 执行完任务后就会退出。

5.3.1 将 USB 设备连接到 Ubuntu

首先, 连接好硬件 (连接电源适配器以及开发板的 TYPE-C0 接口), 开发板进入 Loader 或 Maskrom 模式; 因为是在 VMware 虚拟机 Ubuntu 系统下烧写镜像, 所以需要将开发板 (USB 设备) 连接到 Ubuntu 系统。那么在 Ubuntu 系统右下角可以看到有一个名字为“**Fuzhou Rockchip Rockusb Device**”或者是“**Fuzhou Rockchip USB download gadget**”的设备 (需要将鼠标移动到图标上时才会显示出来), 如下图所示:

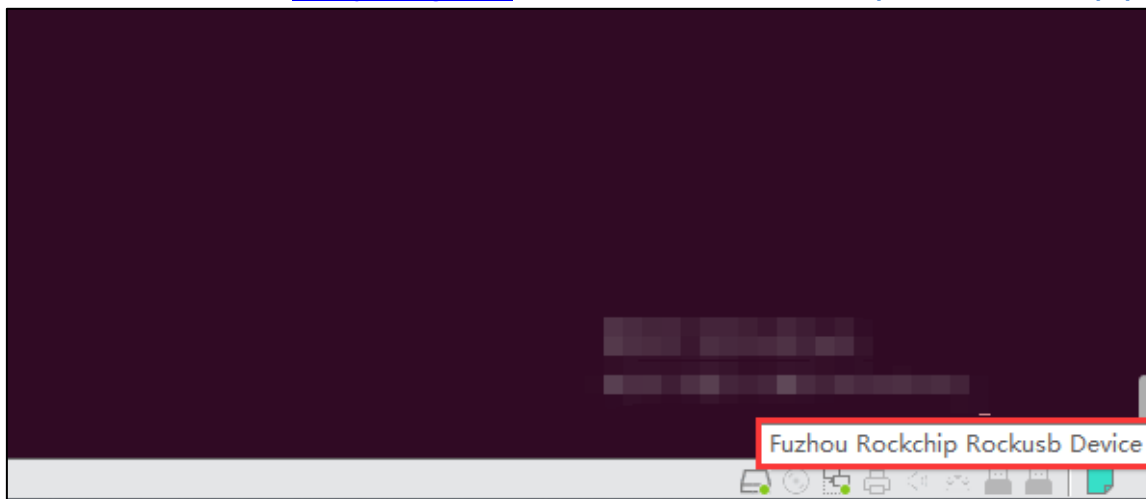


图 5.3.1.1 Rockchip Rockusb Device

同样，在“**虚拟机→可移动设备**”下面也可以找到该设备，如下图所示：

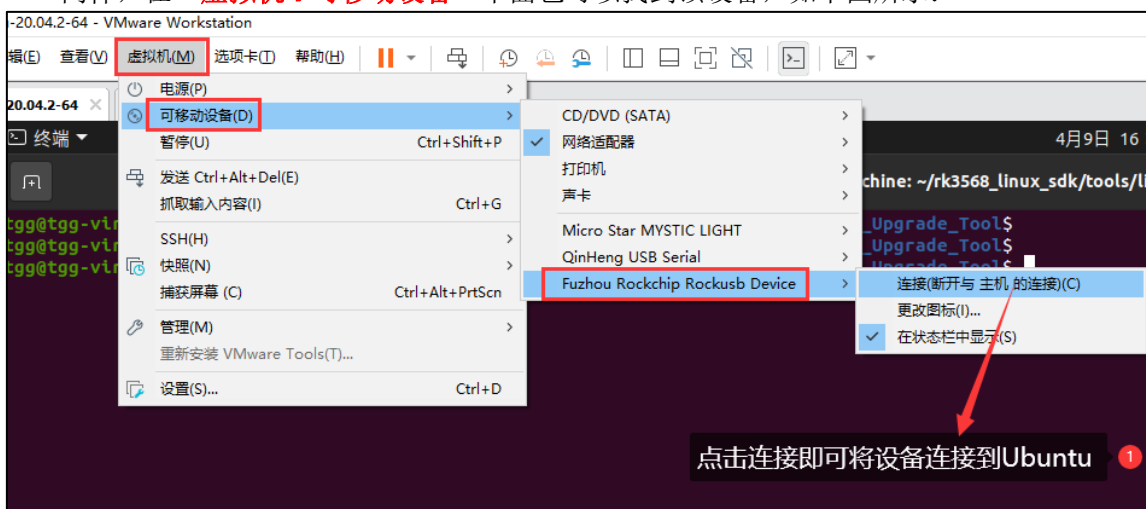


图 5.3.1.2 将设备连接到 Ubuntu(1)

也可以通过右键单击右下角的设备图标，然后选择“**连接**”：

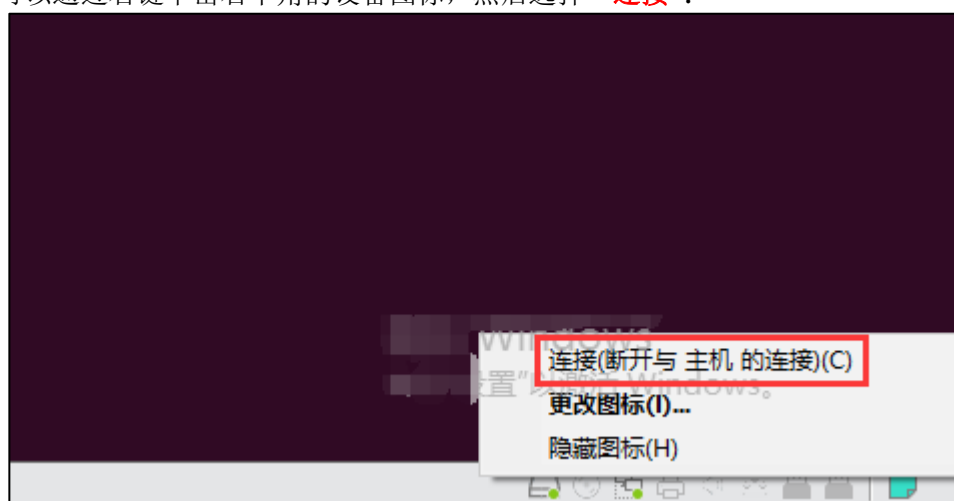


图 5.3.1.3 将设备连接到 Ubuntu(2)

通过这样操作之后，便可将开发板连接到 Ubuntu。

5.3.2 使用 upgrade_tool 工具烧写

本小节向用户介绍如何使用 upgrade_tool 工具烧录镜像, 这里只介绍命令行模式烧写。为了方便操作, 我们直接进入 rockdev/Image-ATK_DLRK3588/ 目录 (镜像存放在该目录下)。

upgrade_tool 工具支持很多指令, 使用不同的指令执行不同的操作, 譬如 CD、LD、SD、UF、DI、DB、TD、RD 等等 (**大小写都行**), 有些指令需要带参数、而有些指令则不需要带参数, 关于这些指令的作用在 RK 提供的使用文档中都有介绍。

使用 UL 指令烧写 MiniLoaderAll.bin 镜像, 使用 DI 指令烧写其它镜像 (uboot.img、boot.img、oem.img、userdata.img、rootfs.img、misc.img 等等) 以及分区表文件 parameter.txt。

先烧写 MiniLoaderAll.bin 镜像, 使用 UL 指令进行烧写 (**执行 upgrade_tool 时需要加入 sudo 获取到 root 用户权限, 否则操作会失败!**):

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool UL MiniLoaderAll.bin -noreset
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool UL MiniLoaderAll.bin -noreset
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/config.ini
Loading loader...
Support Type:RK3588      Loader ver:1.0b Loader Time:2024-03-09 12:21:43
Start to upgrade loader...
Test Device Start
Test Device Success
Check Chip Start
Check Chip Success
Get FlashInfo Start
Get FlashInfo Success
Prepare IDB Start
Prepare IDB Success
Download IDB Start
Download IDB Success
Upgrade loader ok.
```

图 5.3.2.1 烧写 MiniLoaderAll.bin

upgrade_tool 命令携带了 3 个参数, 第一个参数表示需要执行的指令, UL 指令用于烧写 Loader; 第二个参数表示 MiniLoaderAll.bin 镜像的路径; 第三个参数-noreset 表示烧写完 Loader 之后不要复位开发板 (不要复位设备)。

接下来通过 DI 指令下载 parameter.txt 分区表文件:

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -p parameter.txt
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -p parameter.txt
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/config.ini
directlba=1,first4access=1,gpt=1
Write gpt...
Write gpt ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.2 下载分区表

接下来烧录其它镜像:

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -uboot uboot.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -misc misc.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -dtbo dtbo.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -vbmeta vbmeta.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -boot boot.img
```

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -recovery recovery.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -baseparameter baseparameter.img
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -super super.img
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -uboot uboot.img
[sudo] dengtao 的密码:
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download uboot start...(0x00004000)
Write file...(100%)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.3 烧录 uboot.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -misc misc.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download misc start...(0x00008000)
Write file...(100%)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.4 烧录 misc.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -dtbo dtbo.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download dtbo start...(0x0000a000)
Write file...(100%)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.5 烧录 dtbo.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -vbmeta vbmeta.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download vbmeta start...(0x0000c000)
Write file...(100%)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.6 烧录 vbmeta.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -boot boot.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download boot start...(0x0000c800)
Write file...(100%)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.7 烧录 boot.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -recovery recovery.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download recovery start...(0x00020800)
Write file...(100%)
Download image ok.
```

图 5.3.2.8 烧录 recovery.img



```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -baseparameter baseparameter.i
mg
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download baseparameter start...(0x001d8800)
Write file...(100%)
Download image ok.
```

图 5.3.2.9 烧录 baseparameter.img

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool DI -super super.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
directlba=1,first4access=1,gpt=1
Download super start...(0x001d9000)
Download image ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.2.10 烧录 super.img

“DI-uboot”中的“uboot”指的便是 uboot 分区，譬如“-boot”指定将镜像烧录至 boot 分区、“-misc”指定将镜像烧录至 misc 分区。

如果执行命令出错，可以尝试复位、重启开发板，再次操作。

最后，当所有的镜像全部烧录完成后，我们可以执行下面这条命令复位开发板、启动系统：

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool RD
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool RD
[sudo] dengtao 的密码:
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
Reset Device OK.
```

图 5.3.2.10 复位开发板

5.3.3 烧写 update.img

首先需要在 SDK 根目录下执行“./build.sh -u”命令打包 update.img 固件，生成的 update.img 存放在 rockdev/Image-ATK_DLRK3588/目录，然后进入该目录并执行如下命令烧录 update.img 固件（使用 UF 指令烧录）：

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool UF up-
date.img
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool UF update.img
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
Loading firmware...
Support Type:3588      FW Ver:c.0.00   FW Time:2024-03-09 22:27:06
Loader ver:1.0b Loader Time:2024-03-09 12:21:43
Start to upgrade firmware...
Download Boot Start
Download Boot Success
Wait For Maskrom Start
Wait For Maskrom Success
Test Device Start
Test Device Success
Check Chip Start
Check Chip Success
Get FlashInfo Start
Get FlashInfo Success
Prepare IDB Start
Prepare IDB Success
Download IDB Start
Download IDB Success
Download Firmware Start
Download Image... (100%)
Download Firmware Success
Upgrade firmware ok.
```

图 5.3.3.1 烧写 update.img

烧录完之后会自动复位开发板、启动系统。

5.3.4 擦除操作

本小节介绍如何使用 `upgrade_tool` 工具对开发板上的 Flash 存储器件（譬如 eMMC）进行擦除操作。同样，操作之前开发板需处于 Maskrom 或 Loader 模式下，并将开发板连接到 Ubuntu。

执行如下命令可以擦除 eMMC 的所有数据（使用 EF 指令）：

```
sudo ../../RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool EF MiniLoaderAll.bin
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$ sudo ../../RKTools/
linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux/upgrade_tool EF MiniLoaderAll.bin
Using /home/dengtao/rk3588_android_sdk/RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.17_for_linux
/config.ini
Loading loader...
Start to erase flash...
Download Boot Start
Download Boot Success
Wait For Maskrom Start
Wait For Maskrom Success
Test Device Start
Test Device Success
Get FlashInfo Start
Get FlashInfo Success
Erase Flash Start
Erase Flash... (100%)
Erase Flash Success
Reset Device Start
Erase flash ok.
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/rockdev/Image-ATK_DLRK3588$
```

图 5.3.4.1 擦除所有数据

当然，也可以按地址进行扇区擦除，由用户指定擦除的起始位置和大小（使用 EL 指令）。



第六章 Android SDK 开发

本章向用户介绍如何对 RK3588 Android SDK 进行开发，包括 u-boot 开发、Linux 内核开发、Android 系统开发、Android 应用开发；用户可以基于本 SDK 进行二次开发、定制，以适配自己的 Android 产品；基于本 SDK，可以有效实现系统定制和应用移植开发，帮助用户快速开发、提高开发效率！

6.1 U-Boot 开发

U-Boot 源码存放在<SDK>/u-boot 目录，如下所示：

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ cd u-boot/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/u-boot$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/u-boot$ ls
api          common      Documentation  fit         lib         net         snapshot.commit
arch         config.mk   drivers        fs          Licenses    post        test
bl31.elf     configs     dts            include     MAINTAINERS PREUPLOAD.cfg tools
board        disk        env            Kbuild     Makefile    README     usb_update.txt
cmd          doc         examples       Kconfig    make.sh     scripts
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/u-boot$
```

图 6.1.1 U-Boot 源码目录

U-Boot 开发参考文档（由 Rockchip 瑞星微提供）所在路径为：<SDK>/RKDocs/common/u-boot/Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf，大家可以去看看！

6.1.1 U-Boot 的设备树

对于 U-Boot，RK3588 平台所使用的设备树文件为 arch/arm/dts/rk3588-evb.dts。

原生的 U-Boot 只支持 U-Boot 自己的 DTB，RK 平台在原生 U-Boot 基础上增加了 kernel DTB 机制的支持，即 U-Boot 会使用 kernel DTB 去初始化外设。这样设计的目的是为了兼容板级外设差异，譬如 power、clock、display 等。

U-Boot 设备树负责初始化存储、调试串口等基础外设；而 kernel 设备树初始化存储、调试串口之外的外设，譬如 LCD 显示、千兆网等。执行 U-Boot 代码时先用 U-Boot 的设备树完成存储、调试串口等基础外设的初始化操作，然后从存储上加载 kernel 的设备树并转而使用这份设备树继续初始化其余外设。

所以用户一般不需要去修改 U-Boot 的设备树文件（除非更改调试串口等）。

6.1.2 U-Boot 编译

在 U-Boot 源码目录下执行如下命令编译 U-Boot：

```
./make.sh rk3588
```

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/u-boot$ ./make.sh rk3588
grep: .config: 没有那个文件或目录
## make rk3588_defconfig -j32
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
SHIPPED scripts/kconfig/zconf.tab.c
SHIPPED scripts/kconfig/zconf.lex.c
SHIPPED scripts/kconfig/zconf.hash.c
HOSTCC scripts/kconfig/zconf.tab.o
In file included from scripts/kconfig/zconf.tab.c:2468:
scripts/kconfig/confdata.c: In function 'conf_write':
scripts/kconfig/confdata.c:771:19: warning: '%s' directive writing likely 7 or more bytes into a region of
771 | sprintf(newname, "%s%s", dirname, basename);
    |
scripts/kconfig/confdata.c:771:19: note: assuming directive output of 7 bytes
In file included from /usr/include/stdio.h:867,
from scripts/kconfig/zconf.tab.c:82:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:36:10: note: '__builtin___sprintf_chk' output 1 or more byte
36 | return __builtin___sprintf_chk (__s, __USE_FORTIFY_LEVEL - 1,
    |
37 |     __bos (__s), __fmt, __va_arg_pack ());
    |
In file included from scripts/kconfig/zconf.tab.c:2468:
scripts/kconfig/confdata.c:774:20: warning: '.tmpconfig.' directive writing 11 bytes into a region of si
774 | sprintf(tmpname, "%s.tmpconfig.%d", dirname, (int) getpid());
    |
In file included from /usr/include/stdio.h:867,
from scripts/kconfig/zconf.tab.c:82:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:36:10: note: '__builtin___sprintf_chk' output between 13 and
36 | return __builtin___sprintf_chk (__s, __USE_FORTIFY_LEVEL - 1,
    |
37 |     __bos (__s), __fmt, __va_arg_pack ());
    |
HOSTLD scripts/kconfig/conf
#
# configuration written to .config
#
scripts/kconfig/conf --silentoldconfig Kconfig
```

图 6.1.2.1 编译 U-Boot 源码(1)

```
Hash value: 66e30bf9e879405a49797aaa6c08ca1c41aa325443e910af42e3df309e65909b
Image 5 (fdt)
Description: U-Boot dtb
Created: Sun Mar 10 17:48:53 2024
Type: Flat Device Tree
Compression: uncompressed
Data Size: 8141 Bytes = 7.95 KiB = 0.01 MiB
Architecture: AArch64
Hash algo: sha256
Hash value: 8eccccd16d2f12c4f5639c386e51f206eb679638495283d9297497ddf0aae01a
Default Configuration: 'conf'
Configuration 0 (conf)
Description: rk3588-evb
Kernel: unavailable
Firmware: atf-1
FDT: fdt
Loadables: uboot
atf-2
atf-3
optee
*****boot_merger ver 1.31*****
Info:Pack loader ok.
pack loader okay! Input: /home/dengtao/rk3588_android_sdk/rkbin/RKBOOT/RK3588MINIALL.ini
/home/dengtao/rk3588_android_sdk/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3588_spl_loader_v1.13.113.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/dengtao/rk3588_android_sdk/rkbin/RKTRUST/RK3588TRUST.ini

Platform RK3588 is build OK, with new .config(make rk3588_defconfig -j32)
/home/dengtao/rk3588_android_sdk/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64
2024年 03月 10日 星期日 17:48:53 CST
```

图 6.1.2.2 编译 U-Boot 源码(2)

编译完成后,生成 **uboot.img** 和 **rk3588_spl_loader_v1.13.113.bin** 两个镜像文件,如下所示:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/u-boot$ ls
api          disk          Kconfig      scripts      u-boot.cfg.configs
arch         doc           lib           snapshot.commit  u-boot.dtb
bl31_0x00040000.bin  Documentation Licenses     spl          u-boot-dtb.bin
bl31_0x000f0000.bin  drivers      MAINTAINERS  System.map   u-boot.img
bl31_0xff100000.bin  dts          Makefile     tee.bin      u-boot.tds
bl31.elf       env          make.sh      test         u-boot.map
board         examples     net          tools        u-boot-nodtb.bin
cmd           fit          post         tpl          u-boot.srec
common        fs           PREUPLOAD.cfg  u-boot      u-boot.sym
config.mk     include      README       u-boot.bin  usb_update.txt
configs       Kbuild      rk3588_spl_loader_v1.13.113.bin u-boot.cfg
```

图 6.1.2.3 U-Boot 编译后生成的镜像

u-boot.img 对应<SDK>/rockdev/Image-ATK_DLRK3588/u-boot.img

rk3588_spl_loader_v1.13.113.bin 对应<SDK>/rockdev/Image-ATK_DLRK3588/MiniLoaderAll.bin (重命名)

可自行将这两个镜像拷贝至 rockdev/Image-ATK_DLRK3588/目录。

6.1.3 U-Boot 的配置

在 U-Boot 源码目录下执行如下命令打开 menuconfig 配置界面:

```
make rk3588_defconfig //选择配置文件
make menuconfig       //打开图形化配置界面
```

运行结果如下所示:

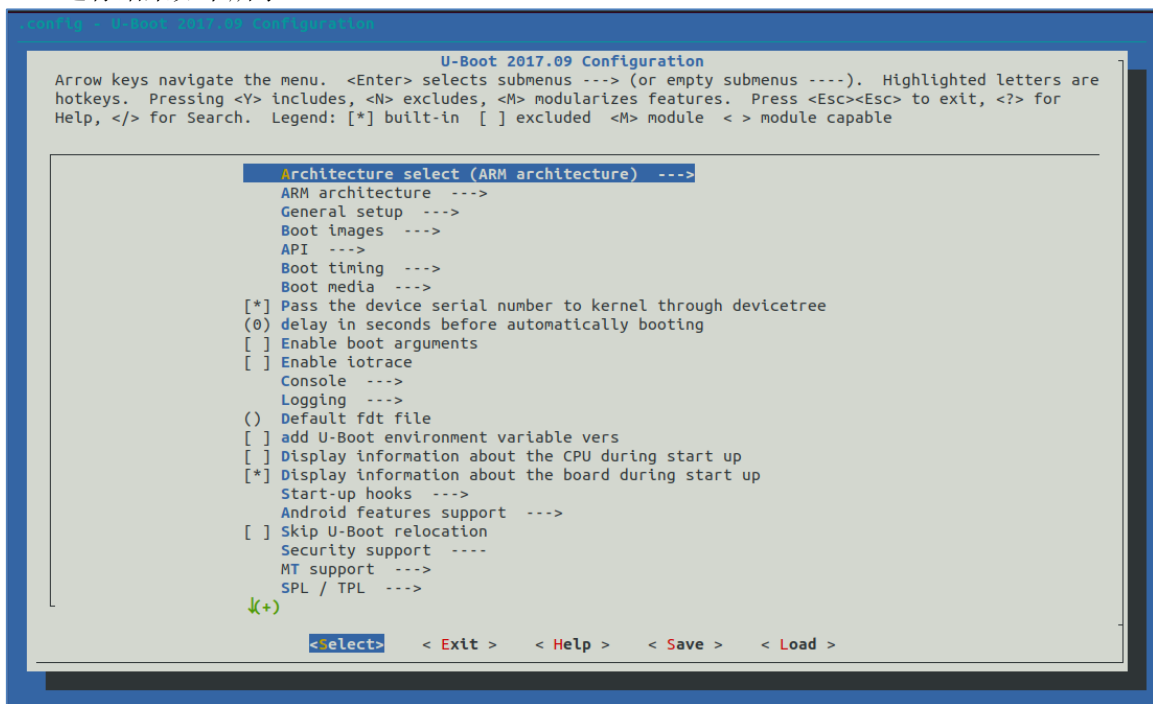


图 6.1.3.1 menuconfig 图形化配置

用户可自行对 U-Boot 进行配置, 配置完成后运行如下命令保存配置:

```
make savedefconfig //把配置信息保存到 defconfig 文件中
cp defconfig configs/rk3588_defconfig //用 defconfig 文件替换 rk3588_defconfig
```

保存配置后, 执行如下命令进行编译:

```
./make.sh rk3588 //在 U-Boot 源码目录下执行 make.sh 脚本编译
或
croot && ./build.sh -U //在 SDK 根目录下执行 build.sh 脚本编译
```

6.1.4 快捷键

RK 平台 U-Boot 支持通过串口组合键触发一些事件用于烧写、调试。RK U-Boot 运行时不会出现内核启动倒计时,它会跳过倒计时、直接启动 Linux 内核。我们可以通过串口组合键“Ctrl + C”打断它启动内核、从而进入 U-Boot 命令行模式。

除了 Ctrl + C 组合键之外,还支持如下多种组合键 (**开机或复位时多按几次,避免触发失败**):

- Ctrl + C: 进入 U-Boot 命令行模式。
- Ctrl + D: 进入 Loader 烧写模式。
- Ctrl + B: 进入 Maskrom 烧写模式。
- Ctrl + F: 进入 fastboot 模式。
- Ctrl + M: 打印 bidram/system 信息。
- Ctrl + I: 使能内核 initcall_debug。
- Ctrl + P: 打印 cmdline 信息。
- Ctrl + S: U-Boot 打印"Starting kernel..."之后进入 U-Boot 命令行。

大家可以自己测试下,这里不多说。

6.2 kernel 开发

Linux 内核源码存放在<SDK>/kernel-5.10 目录:

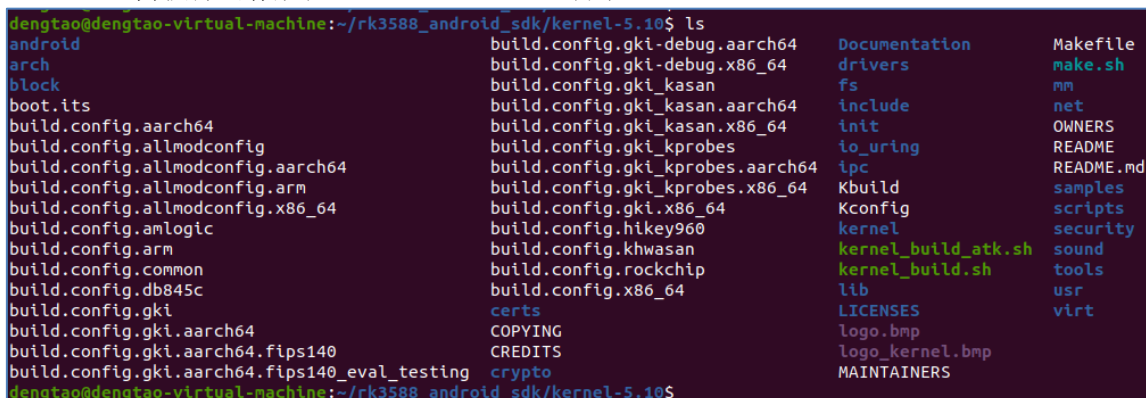


图 6.2.1 内核源码目录

6.2.1 Linux 内核编译

首先进入 Linux 内核源码目录,对于 5.10 版本内核,Android 强制要求使用 clang 编译器进行编译。

编译之前,需要将 clang 导出到 PATH 环境变量:

```
# 如果是 Android12 执行下面这条命令
export PATH=./prebuilts/clang/host/linux-x86/clang-r416183b/bin:$PATH    #对应 Android12

# 如果是 Android13 执行下面这条命令
export PATH=./prebuilts/clang/host/linux-x86/clang-r450784d/bin:$PATH    #对应 Android13
```

注意,每次打开新的终端时,都需要执行 export 导出,当然也可将其添加到 ~/.profile 文件中。

内核源码目录下有一个 make.sh 脚本,这是正点原子封装好的编译脚本,执行如下命令编译:

```
./make.sh board=ATK_DLRK3588
```

图 6.2.1.1 编译内核(1)

图 6.2.1.2 编译内核(2)

120

这上面一共有 5 条命令, 最后两条命令执行拷贝操作, 将生成的镜像从产品输出目录拷贝到 rockdev/Image-ATK_DLRK3588/目录, 方便烧录。

6.2.2 kernel 的配置

在内核源码目录下, 执行如下命令可打开 menuconfig 图形化配置界面:

```
# 如果是 Android12 执行如下两条命令
make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 \
rockchip_defconfig android-11.config atk_dlrk3588.config
make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 \
menuconfig
```

```
# 如果是 Android13 执行如下两条命令
make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 \
rockchip_defconfig android-13.config atk_dlrk3588.config
make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 \
menuconfig
```

运行结果如下所示:

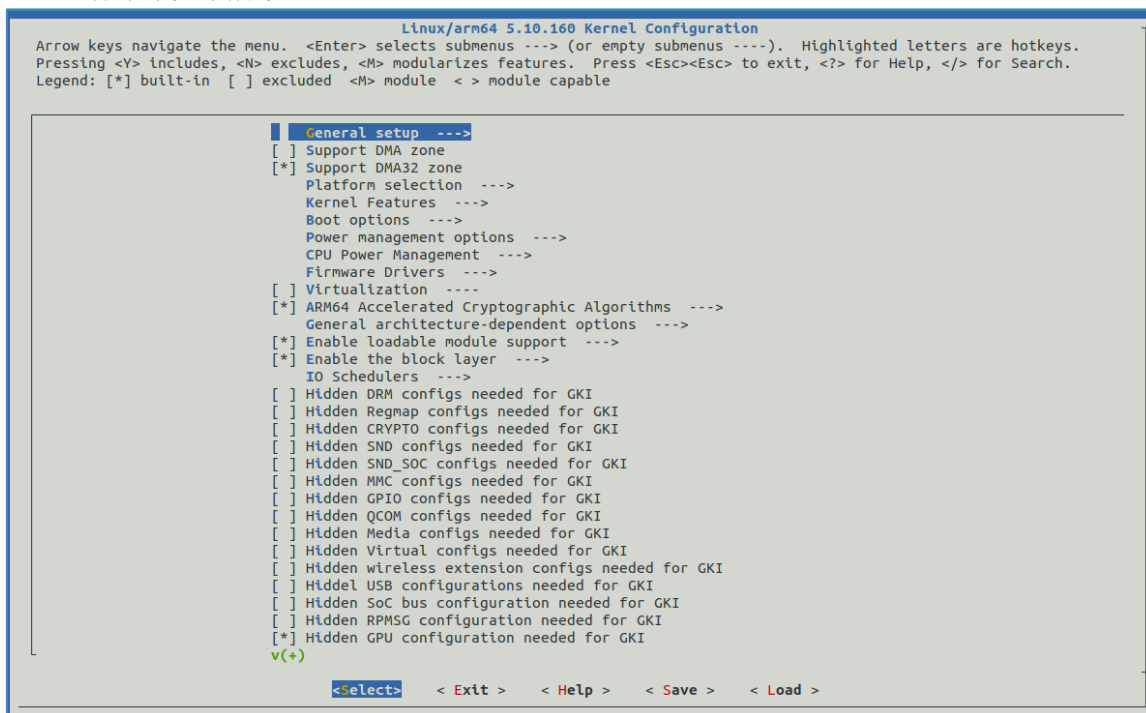


图 6.2.2.1 menuconfig 图形化配置界面

用户可自行对 Linux 内核进行配置, 配置完成后可执行如下命令保存配置:

```
make CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1 ARCH=arm64 \
savedefconfig
cp defconfig arch/arm64/configs/rockchip_defconfig
```

保存配置后, 执行如下命令编译内核:

```
croot && ./build.sh -K -J16
make bootimage -j16
make recoveryimage -j16
cp out/target/product/ATK_DLRK3588/boot.img rockdev/Image-ATK_DLRK3588/
cp out/target/product/ATK_DLRK3588/recovery.img rockdev/Image-ATK_DLRK3588/
```



6.2.3 内核设备树和 defconfig 配置文件

对于 Linux 内核，正点原子 ATK-DLRK3588 平台所使用的配置文件包括如下三个：

arch/arm64/configs/rockchip_defconfig（defconfig 默认配置文件）
 kernel/configs/android-11.config（Android 系统特定配置选项）
 kernel/configs/atk_dlrk3588.config（ATK-DLRK3588 平台特定配置选项）

Rockchip 平台使用的设备树文件都存放在 **arch/arm64/boot/dts/rockchip/**目录下。正点原子 ATK-DLRK3588 平台对应的设备树文件为 **arch/arm64/boot/dts/rockchip/rk3588-atk-evb7-lp4-v10.dts**。

rk3588-atk-evb7-lp4-v10.dts 作为顶层设备树文件、其包含有多个.dtsi 设备树，其结构如下所示：

```
rk3588-atk-evb7-lp4-v10.dts
  rk3588-atk-evb7-lp4.dtsi
    rk3588.dtsi
    rk3588-evb.dtsi
    rk3588-rk806-single.dtsi
  rk3588-atk-cameras.dtsi
  rk3588-atk-screen_choose.dtsi
  rk3588-atk-lcds.dtsi
  rk3588-android.dtsi
```

- rk3588.dtsi: 该设备树文件是 RK3588 平台级设备树文件，与具体板级硬件无关，纯 SoC 级别的设备树文件，由 RK 提供，开发者无需改动该文件！
- rk3588-evb.dtsi: 板级通用设备树文件，通常被板级设备树文件所包含；
- rk3588-rk806-single.dtsi: rk806（pmic）相关配置信息；
- rk3588-android.dtsi: 该设备树包含 Android 部分特有的配置信息；
- rk3588-atk-evb7-lp4-v10.dtsi: 板级设备树文件，该设备树文件包含板级通用设备树文件 rk3588-evb.dtsi 以及 RK3588 平台级设备树文件 rk3588.dtsi；
- rk3588-atk-screen_choose.dtsi: 该设备树用于选择需要使能的 LCD 屏，譬如 HDMI、MIPI、DP 等；
- rk3588-atk-lcds.dtsi: 该设备树包含 LCD 相关的配置信息，譬如触摸屏、背光、LCD 时序参数、显示接口等配置信息。
- rk3588-atk-cameras.dtsi: 该设备树包含摄像头相关的配置信息。

rk3588-atk-screen_choose.dtsi 文件中定义了一些控制宏，这些宏定义用于控制是否使能相应的显示接口，用户可自行修改该文件来选择需要使能的 LCD 屏，该文件的内容如下所示：

```
/*
 * RK3588平台支持4路独立的Video Port, 可实现4路独立的显示输出
 * 显示通路默认配置情况如下:
 * -----
 * | VP0 -----> HDMI_TX0 |
 * | VP1 -----> HDMI_TX1 |
 * | VP2 -----> DP_TX0   |
 * | VP3 -----> DSI_TX0   |
 * -----
 * 如需修改显示通路, 配置本文件即可!
 *
 * 通过宏定义使能相应的显示接口:
 * ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280: 使能DSI_TX0、支持正点原子5.5寸720x1280 MIPI屏
 * ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920: 使能DSI_TX0、支持正点原子5.5寸1080x1920 MIPI屏
 * ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280: 使能DSI_TX0、支持正点原子10.1寸800x1280 MIPI屏
 * ATK_LCD_TYPE_HDMI_TX0: 使能HDMI_TX0
 * ATK_LCD_TYPE_HDMI_TX1: 使能HDMI_TX1
 * ATK_LCD_TYPE_DP_TX0: 使能DP_TX0
 *
 * DSI_CONNECTOR_MIRROR_MODE: 使能DSI_TX0的前提下, 定义该宏将使能DSI_TX1、并与DSI_TX0来自于同一路VP(VP3),
 *                             VP3同时驱动DSI_TX0和DSI_TX1, 输出相同的时序并显示相同的内容, 也就是镜像模式
 *                             所以DSI_TX0和DSI_TX1接口需要连接时序参数相同的MIPI屏。
 * DP_CONNECTOR_MIRROR_MODE: 使能DP_TX0的前提下, 定义该宏将使能DP_TX1、并与DP_TX0来自于同一路VP(VP2),
 *                             VP2同时驱动DP_TX0和DP_TX1, 输出相同的时序并显示相同的内容(镜像模式)。
 */

// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280
#define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280
// #define DSI_CONNECTOR_MIRROR_MODE
#define ATK_LCD_TYPE_HDMI_TX0
#define ATK_LCD_TYPE_HDMI_TX1
#define ATK_LCD_TYPE_DP_TX0
// #define DP_CONNECTOR_MIRROR_MODE
```

图 6.2.3.1 rk3588-screen_choose.dtsi 设备树

从图中可知, 默认已经使能了 MIPI_TX0(默认支持 5.5 寸 1080x1920 MIPI 屏)、HDMI_TX0(连接 HDMI 显示器)、HDMI_TX1(连接 HDMI 显示器)以及 DP_TX0(对应开发板上的 TYPE-C0, 连接 DP 显示器)。

6.2.5 替换 logo

启动 logo 包括 u-boot 阶段 logo 以及内核阶段 logo, 这两个 logo 文件都存放在内核源码目录下:

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk/kernel$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/kernel$ pwd
/home/tgg/rk3568_android_sdk/kernel
tgg@tgg-virtual-machine:~/rk3568_android_sdk/kernel$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/kernel$ ls -l *.bmp
-rw-rw-r-- 1 tgg tgg 177700 4月 28 15:24 logo.bmp
-rw-rw-r-- 1 tgg tgg 177764 4月 28 15:24 logo_kernel.bmp
tgg@tgg-virtual-machine:~/rk3568_android_sdk/kernel$
```

图 6.2.5.1 logo 图片文件

默认 logo 如下所示:



图 6.2.5.2 U-Boot logo



图 6.2.5.3 Kernel logo

如果用户需要更改启动 logo，首先需要将你的 U-Boot logo 文件（bmp 图片）重命名为 logo.bmp、将你的内核 logo 文件（bmp 图片）重命名为 logo_kernel.bmp，然后将这两个 logo 拷贝到内核源码目录下，替换默认 logo 即可！

编译 Linux 内核时，会将 logo.bmp 和 logo_kernel.bmp 打包到 resource.img 资源镜像中。

U-Boot 运行时会把这两个 bmp 文件加载到内存中，logo.bmp 会在 U-Boot 阶段开始显示，logo_kernel.bmp 在内存中的地址会被 U-Boot 传给 Linux Kernel，在 Linux Kernel 的 DRM 驱动初始化阶段显示。

6.2.6 内核模块开发文档汇总

SDK 的 **RKDocs/common/**目录下存放了很多 RK 提供的内核模块开发文档，这些文档按照功能模块划分放在不同的子目录下、方便用户查找。

下表中对这些文档进行了简单的汇总：

模块功能	子目录	对应文档
音频	Audio	Rockchip_Developer_Guide_Audio_CN.pdf Rockchip_Developer_Guide_RK817_RK809_Codec_CN.pdf Rockchip_Developer_Guide_Audio_Call_3A_Algorithm_Integration_and_Parameter_Debugging_CN.pdf
摄像头	camera	文档存放在 common、HAL1、HAL3 子目录中
CAN	Can	Rockchip_Develop_Guide_CAN_FD_CN.pdf
时钟配置	CLK	Rockchip_Developer_Guide_Clock_CN.pdf Rockchip_Develop_Guide_Pll_Ssmod_Clock_CN.pdf Rockchip_RK3399_Developer_Guide_Clock_CN.pdf
DDR	DDR	Rockchip-Developer-Guide-DDR-CN.pdf Rockchip-Developer-Guide-DDR-Problem-Solution-CN.pdf Rockchip-Developer-Guide-DDR-Verification-Process-CN.pdf Rockchip-User-Guide-DDR-DQ-Eye-Tool-CN.pdf
JTAG/GDB 等常用调试	debug	Rockchip_Developer_Guide_DS5_CN.pdf Rockchip_Quick_Start_Linux_Perf.pdf Rockchip_Quick_Start_Linux_Streamline.pdf Rockchip_Quick_Start_Linux_Systrace.pdf Rockchip_User_Guide_J-Link_CN.pdf Rockchip_RK3399_JTAG_Configuration_CN.pdf
显示	display	Rockchip_Developer_Guide_DRM_Display_Driver_CN.pdf Rockchip_Developer_Guide_DRM_Panel_Porting_CN&EN.pdf Rockchip_Developer_Guide_DRM_Panel_Porting_CN.pdf Rockchip_Developer_Guide_HDMI_Based_on_DRM_Framework_CN&EN.pdf Rockchip_Developer_Guide_HDMI-CEC_CN.pdf Rockchip_Developer_Guide_HDMI_CN.pdf

CPU/GPU 等频率、电压调节	DVFS	Rockchip_Developer_Guide_CPUFreq_CN.pdf Rockchip_Developer_Guide_Devfreq_CN.pdf Rockchip_Developer_Guide_Linux4.4_CPUFreq_CN.pdf
以太网配置	GMAC	Rockchip_Developer_Guide_Ethernet_CN.pdf Rockchip_Developer_Guide_Linux_GMAC_CN.pdf Rockchip_Developer_Guide_Linux_GMAC_Mode_Configuration_CN.pdf Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_CN.pdf Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_EN.pdf
I2C 通信	I2C	Rockchip_Developer_Guide_I2C_CN.pdf
GPIO 电源域	IO-Domain	Rockchip_Developer_Guide_Linux_IO_DOMAIN_CN.pdf Rockchip_RK356X_Introduction_IO_Power_Domains_Configuration.pdf
IOMMU	IOMMU	Rockchip_Developer_Guide_Linux_IOMMU_CN.pdf
MMC	MMC	Rockchip_Developer_Guide_SD_Boot_CN.pdf Rockchip_Developer_Guide_SDMMC_SDIO_eMMC_CN.pdf
LED	Leds	Rockchip_Introduction_Leds_GPIO_Configuration_for_Linux4.4_CN.pdf
移动网络 4G/5G	mobile-net	Rockchip_Introduction_3G_Data_Card_USB_File_Conversion_CN.pdf Rockchip_Introduction_3G_Dongle_Configuration_CN&EN.pdf Rockchip_Introduction_4G_Module_Configuration_CN&EN.pdf
多媒体框架 MPP	MPP	Rockchip_Developer_Guide_MPP_CN.pdf
存储	NVM	Rockchip_Application_Notes_Storage_CN.pdf Rockchip_Developer_FAQ_Storage_CN.pdf Rockchip_Developer_Guide_OTP_CN.pdf Rockchip_Developer_Guide_SATA_CN.pdf Rockchip_Introduction_Partition_CN.pdf Rockchip_RK356X_Developer_Guide_SATA_CN.pdf
PCIe	PCie	Rockchip-Developer-Guide-linux4.4-PCie.pdf Rockchip_Developer_Guide_PCie_CN.pdf Rockchip_PCie_Virtualization_Developer_Guide_CN.pdf Rockchip_RK3399_Developer_Guide_PCie_CN.pdf
Pin-Ctrl 配置	PIN-Ctrl	Rockchip-Developer-Guide-Linux-Pin-Ctrl-CN.pdf
PMIC 电量计、DCDC	PMIC	Rockchip_Developer_Guide_Power_Discrete_DCDC_EN.pdf Rockchip_RK805_Developer_Guide_CN.pdf Rockchip_RK806_Developer_Guide_CN.pdf Rockchip_RK808_Developer_Guide_CN.pdf Rockchip_RK809_Developer_Guide_CN.pdf Rockchip_RK816_Developer_Guide_CN.pdf Rockchip_RK817_Developer_Guide_CN.pdf Rockchip_RK818_Developer_Guide_CN.pdf Rockchip_RK818_RK816_Developer_Guide_Fuel_Gauge_CN.pdf Rockchip_RK818_RK816_Introduction_Fuel_Gauge_Log_CN.pdf
电源、功耗	power	Rockchip_Developer_Guide_Power_Analysis_EN.pdf Rockchip_Developer_Guide_Sleep_and_Resume_CN.pdf
PWM	PWM	Rockchip_Developer_Guide_Linux_PWM_CN.pdf Rockchip_Developer_Guide_PWM_IR_CN.pdf
ADC	SARADC	Rockchip_Developer_Guide_Linux_SARADC_CN.pdf
传感器	Sensors	Rockchip_Developer_Guide_Sensors_CN.pdf
SPI	SPI	Rockchip_Developer_Guide_Linux_SPI_CN.pdf
温控	Thermal	Rockchip_Developer_Guide_Thermal_CN.pdf
TRUST	TRUST	Rockchip_Developer_Guide_Trust_CN.pdf Rockchip_Developer_Guide_Trust_EN.pdf
串口	UART	Rockchip-Developer-Guide-RT-Thread-UART.pdf Rockchip_Developer_Guide_UART_CN.pdf Rockchip_Developer_Guide_UART_FAQ_CN.pdf
USB	usb	Rockchip_Developer_Guide_Linux_USB_PHY_CN.pdf Rockchip_Developer_Guide_USB2_Compliance_Test_CN.pdf Rockchip_Developer_Guide_USB_CN.pdf Rockchip_Developer_Guide_USB_FFS_Test_Demo_CN.pdf Rockchip_Developer_Guide_USB_Gadget_UAC_CN.pdf Rockchip_Developer_Guide_USB_SQ_Test_CN.pdf Rockchip_Introduction_USB_SQ_Tool_CN.pdf Rockchip_RK3399_Developer_Guide_USB_DTS_CN.pdf Rockchip_RK356x_Developer_Guide_USB_CN.pdf Rockchip_Trouble_Shooting_Linux4.19_USB_Gadget_UVC_CN.pdf

看门狗	watchdog	Rockchip_Developer_Guide_Linux_WDT_CN.pdf
-----	----------	---

表 6.2.6.1 内核模块开发文档汇总

表中只是列举了部分文档, 并非全部, 用户可自行进入<SDK>/RKDocs/common/目录浏览!

6.3 Android 系统开发

本小节为 Android 系统开发相关内容。

6.3.1 Android 系统架构

Android 系统架构图如下所示:

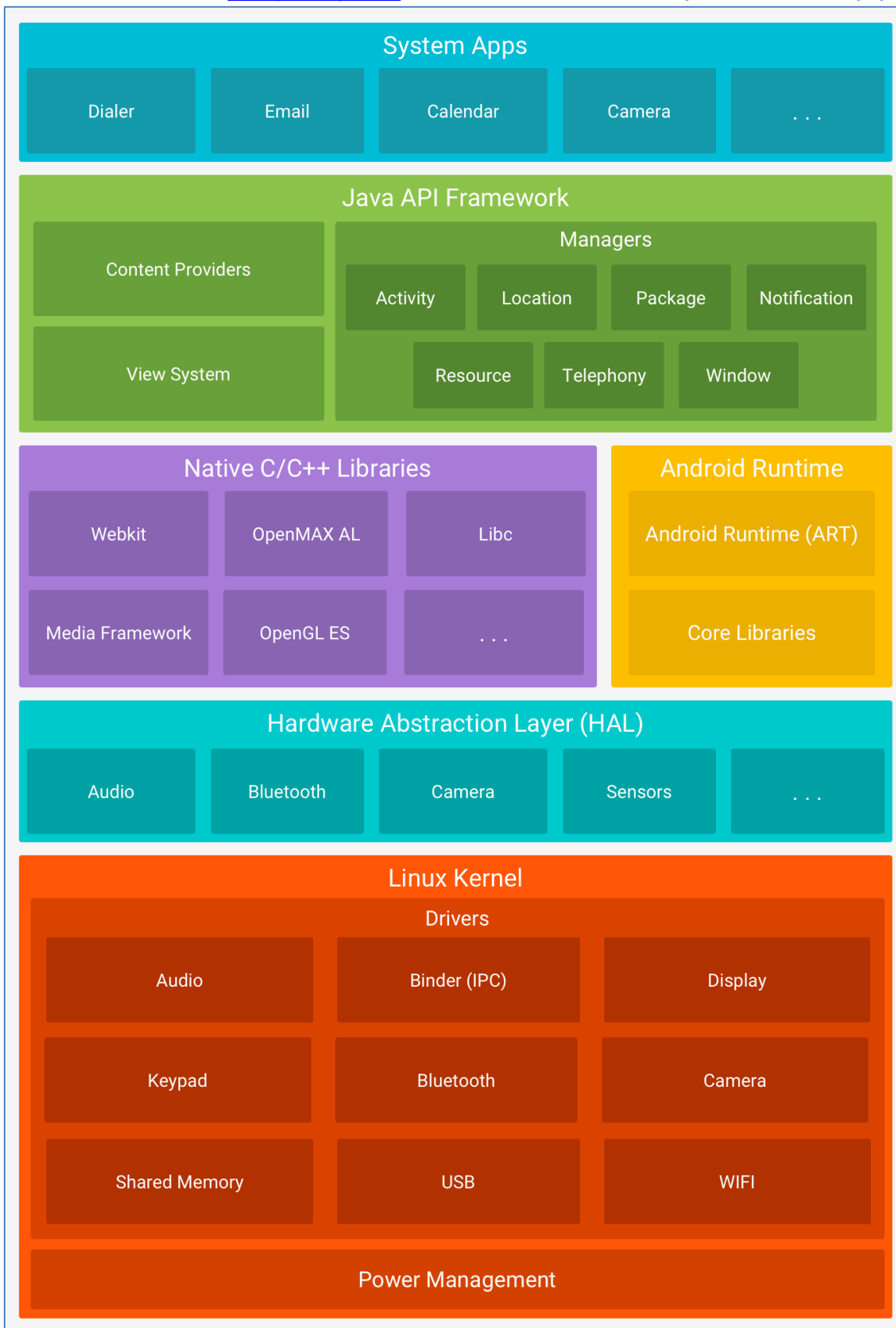


图 6.3.1.1 Android 系统架构

Android 系统架构分为五层，从上到下依次为：应用层（System Apps）、应用框架层（Java API Framework）、系统运行库层（Android Runtime & Native C/C++库）、硬件抽象层（HAL）以及 Linux 内核层（Linux Kernel）。

①、应用层

应用层是整个 Android 系统的最上层，该层包含核心的系统应用、输入法以及第三方应用等；Android 系统内置的应用程序以及非系统内置的应用程序都属于应用层，负责与用户进行直接交互，通常都是由 Java 开发。

②、应用框架层（Java API Framework）

应用框架层提供了开发 Android 应用程序所需的各种 API，应用开发人员调用框架层所提供的 API 进行应用程序开发，当然也包括系统应用；框架层是由 Java 代码编写，所以也称为 Java Framework。

除此之外，框架层还为应用程序提供一系列重要的系统服务，应用程序开发者被允许在应用中使用这些服务：

服务名称	说明
Activity Manager（活动管理器）	管理各个应用程序生命周期、并且提供通用的导航回退功能
Location Manager（位置管理器）	提供地理位置及定位功能服务
Package Manager（包管理器）	管理所有安装在 Android 系统中的应用程序，包括安装、卸载、更新等
Notification Manager（通知管理器）	使得所有应用程序都能够在状态栏显示通知信息
Resource Manager（资源管理器）	管理除了代码之外的一系列资源（非代码资源），如本地字符串、图片、布局文件、颜色文件等
Telephony Manager（电话管理器）	管理所有的移动设备功能
Window Manager（窗口管理器）	管理所有开启了窗口的程序
Content Provider（内容提供者）	使不同应用程序之间可以共享数据
View System（视图系统）	构建应用程序的基本组件，譬如列表、网格、文本框、按钮等

表 6.3.1.1 系统服务简介

③、系统运行库层（Native）

该层分为两部分：C/C++程序库和 Android 运行时库

C/C++程序库：提供给 Android 系统的各个组件使用，这些功能通过应用框架层为开发者提供服务。

Android 运行时库（Android Runtime）：运行时库分为核心库和 ART（Android 5.0 之后，Dalvik 虚拟机被 ART 取代）。核心库集成了 Java 语言核心库的大多数功能，这样开发者可以使用 java 语言来编写 Android 应用；在 ART 机制下，系统在安装应用的时候就会进行一次预编译，将字节码预先编译成机器码并存储在本地，这样在应用每次运行启动时就不需要再去执行编译了，大大提高了效率！

④、硬件抽象层（HAL/HIDL）

hal 是 hardware abstraction layer 的简写，位于 Framework 和 Linux Kernel 之间，HAL 层代码一般由 C/C++ 构成。硬件抽象层的目的在于将底层硬件抽象化，向上层提供统一、标准化的 API 接口；所以 HAL 层是上层应用对底层硬件操作屏蔽的一个软件层次，上层应用不必关心底层硬件具体是如何工作的，只需调用 HAL 层提供的统一接口即可！一般做 Android 底层开发主要就是针对 HAL 层做修改以适配具体的硬件。

另一方面，HAL 层存在的意义是为了保护知识产权，硬件产商处于利益考虑，不希望公开硬件设备相关的实现细节和参数，所以硬件产商或个人可以把不愿开源的代码（譬如一些核心

算法) 放在 HAL 层, 以动态链接库的形式提供, 而 HAL 层位于用户空间 (Linux 用户空间), 不属于 Linux 内核, 无需遵循 GPL 开源协议。

⑤、Linux Kernel

Android 系统的基础是 Linux 内核, Android 的核心系统服务基于 Linux 内核, 并在此基础上添加了部分 Android 专用的驱动, 系统的安全性、内存管理、进程管理、网络协议栈和驱动模型等都依赖于 Linux 内核。

6.3.2 Android 产品配置

在 Android SDK 根目录下执行 `lunch` 命令 (不加任何参数) 会列举出当前所配置的所有产品, 在编译 Android 源码之前, 用户需要选择其中一个产品, 譬如 `ATK_DLRK3588-userdebug`。我们应该如何去增加一个自己的产品呢?

本小节向用户介绍 Android 产品配置, 包括 Android 产品配置架构、Android 产品配置相关文件以及如何配置一个新的产品。在正式介绍产品配置前, 先了解下面几个概念:

- 目标产品: 具体指最终用户买到的 Android 设备, 譬如小米手机: `Xiaomi 13`、`Xiaomi 13 Pro`、`Xiaomi 12`、`Xiaomi 12 Pro`, Oppo 手机: `Find X6`、`Find X5`、`Find N2` 等。
- 产品系列: Android 产品开发产商在研发出一款产品后, 往往要继续在其基础上研发出新产品, 新产品往往是在老产品的硬件或软件基础上做一些升级, 这些产品 (新产品和老产品) 就构成了一个产品系列, 譬如 Oppo 的 `Find X` 系列、`Find N` 系列等。
- 目标设备: 目标设备其实可以理解为手机主板, 也就是 Android 产品的硬件主板, 它是指 Android 设备硬件配置信息的集合体, 每个 Android 产品都有它的硬件配置, 一个硬件配置可能被不同产品使用, 同一手机型号往往有高配版本和低配版本, 譬如 `8GB+64GB`、`12GB+128GB`、`12GB+256GB` 等。

在 Android 编译系统中, 每个编译项 (也叫产品编译项) 都会对应一个目标产品, 每个目标产品都会对应一个目标设备, 一个产品系列可以包含多个不同的产品, 一个目标设备可能被多个产品使用。

1. Android 产品配置架构

产品配置主要在 `<SDK>/device` 目录下进行, `device` 目录结构一般如下图所示:

```
device-|
  |-<company_name>-|
    |-<board_name>-|
      |-AndroidBoard.mk
      |-BoardConfig.mk
    |-products-|
      |-AndroidProducts.mk
      |-<first_product_name>.mk
      |-<second_product_name>.mk
```

图 6.3.2.1 device 目录结构(1)

①、一般在 `device` 目录下会创建以公司名称 (设备制造商) 命名的文件夹, 名字自己定义, 譬如 `rockchip` (瑞芯微), `alientek` (正点原子) 等;

②、接着在产商文件夹下, 创建一个 `products` 目录, 该目录用于存放该产商的所有产品配置文件 (产品 Makefile 配置文件)、`AndroidProducts.mk` 文件 (用于定义当前产商的所有产品列表) 以及其它所需文件。

③、在产商文件夹下，还会创建一个或多个以主板（或者叫设备）命名的目录，这些目录用于存放主板配置文件 BoardConfig.mk（主板 Makefile 配置文件）、AndroidBoard.mk 文件等。

除了上面这种目录结构外，常用的还有下面这种结构，如下图所示：

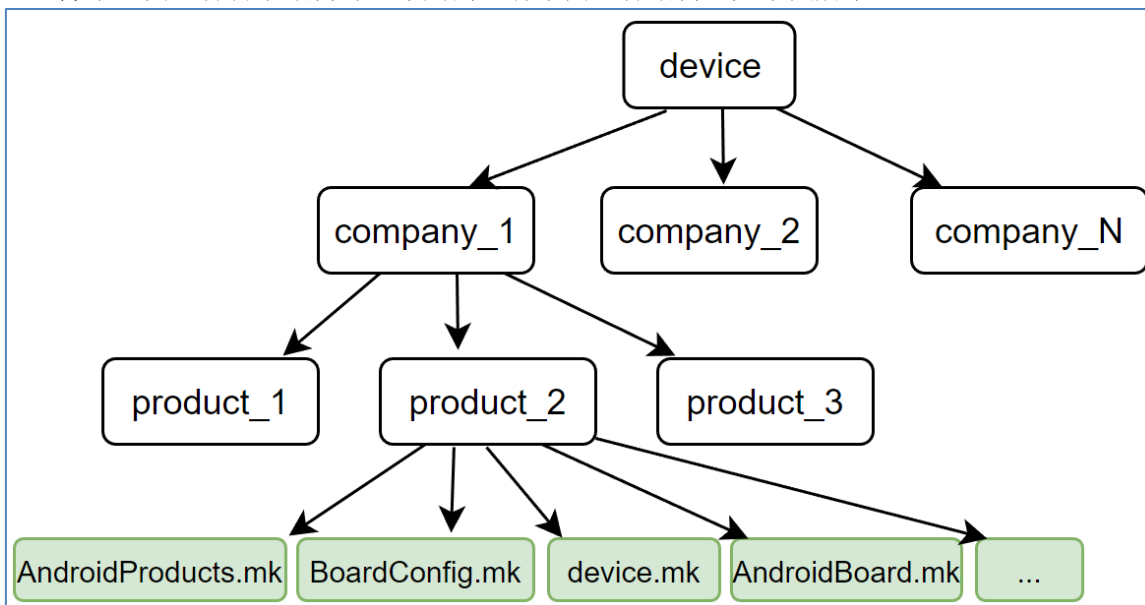


图 6.3.2.2 device 目录结构(2)

①、在 device 目录下会创建以公司名称（设备制造商）命名的文件夹，名字自己定义，譬如 rockchip（瑞芯微），alientek（正点原子）等；

②、接着在产商文件夹下，创建一个以产品命名的文件夹，名字自己定义，譬如 device/alientek/atk_dlrk3588；如果有多个产品就创建多个以产品命名的文件夹，product_1、product_2、product_3 ...；

③、在产品目录下添加 AndroidProducts.mk、BoardConfig.mk、device.mk、AndroidBoard.mk、<product_name>.mk 等一些列 mk 配置文件。

以上给大家介绍两种 device 的目录结构，当然并不是要求开发者必须按照这种结构去添加、配置一个新的产品。

RK3588 Android SDK device/目录下的内容如下所示：

```

dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls -l device/
总用量 36
drwxrwxr-x  3 dengtao dengtao 4096 3月 7 17:39 anlogic
drwxrwxr-x  3 dengtao dengtao 4096 3月 7 17:40 common
drwxrwxr-x 18 dengtao dengtao 4096 3月 7 17:40 generic
drwxrwxr-x 15 dengtao dengtao 4096 3月 7 17:14 google
drwxrwxr-x 14 dengtao dengtao 4096 3月 7 17:14 google_car
drwxrwxr-x  3 dengtao dengtao 4096 3月 7 17:14 linaro
drwxrwxr-x  4 dengtao dengtao 4096 3月 7 17:15 rockchip
drwxrwxr-x  9 dengtao dengtao 4096 3月 7 17:15 sample
drwxrwxr-x  3 dengtao dengtao 4096 3月 7 17:15 ti
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
    
```

图 6.3.2.3 device 目录

我们重点关注的便是 rockchip 目录，该目录包含了瑞芯微（Rockchip）的产品配置信息，我们来看一下这个目录的结构，如下所示：

```

rockchip
├── common
├── rk3588
│   └── Android.mk
    
```

```

├── AndroidProducts.mk
├── ATK_DLRK3588      # 正点原子 ATK-DLRK3588
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── ATK_DLRK3588.mk  # 产品配置文件
│   └── BoardConfig.mk  # 板级配置文件
├── BoardConfig.mk
├── device.mk
├── rk3588_box
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── BoardConfig.mk
│   └── rk3588_box.mk
├── rk3588m_car
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── BoardConfig.mk
│   └── rk3588m_car.mk
├── rk3588m_s
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── BoardConfig.mk
│   └── rk3588m_s.mk
├── rk3588_s
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── BoardConfig.mk
│   └── rk3588_s.mk
├── rk3588s_s
│   ├── AndroidBoard.mk
│   ├── Android.mk
│   ├── BoardConfig.mk
│   └── rk3588s_s.mk
└── rk3588_xr
    ├── AndroidBoard.mk
    ├── Android.mk
    ├── BoardConfig.mk
    └── rk3588_xr.mk
    
```

目录名	说明
common	包含 RK 平台所需通用配置文件等, 譬如通用 Makefile 文件、sh 脚本、python 脚本、xml 文件、rc 文件等
rk3588	包含瑞星微 RK3588 平台产品配置信息, 该目录下包含有多个产品配置, 譬如 ATK_DLRK3588(正点原子添加)、rk3588_box、rk3588m_car、rk3588m_s、rk3588_s、rk3588_xr 等

表 6.3.2.1 rockchip 目录说明

RK3588 平台的所有产品配置都存放在 device/rockchip/rk3588 目录:

```

dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$ ls
Android.mk      BoardConfig.mk  ota              rk3588_box      rk3588_s        wake_lock_filter.xml
AndroidProducts.mk device.mk        overlay          rk3588m_car     rk3588s_s       wifi_bt.mk
ATK_DLRK3588    dtbo_gki_fix.dts package_performance.xml rk3588m_s       rk3588_xr
bluetooth       init.rk3588.rc  public.libraries.txt rk3588.prop     sepolicy_vendor
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$
    
```

图 6.3.2.4 rk3588 目录

该目录下包含 7 个产品配置: ATK_DLRK3588(正点原子添加的产品配置)、rk3588_box、rk3588m_car、rk3588m_s、rk3588_s、rk3588s_s 以及 rk3588_xr, 每个产品目录下都有一个产品 Makefile 配置文件、板级配置文件 BoardConfig.mk、Android.mk、AndroidBoard.mk 以及其它所

需文件。device/rockchip/rk3588 目录下还存放了一个 AndroidProducts.mk 文件，用于定义该目录下的所有产品列表。

2. Android 产品配置相关.mk 文件

一、AndroidProducts.mk 文件（必须）

AndroidProducts.mk 文件用于定义产品列表，该文件的内容非常简单，一般就是定义了两个变量：PRODUCT_MAKEFILES 和 COMMON_LUNCH_CHOICES。示例如下：

```
PRODUCT_MAKEFILES := \
$(LOCAL_DIR)/rk3588_s/rk3588_s.mk \
$(LOCAL_DIR)/ATK_DLRK3588/ATK_DLRK3588.mk \
$(LOCAL_DIR)/rk3588s_s/rk3588s_s.mk \
$(LOCAL_DIR)/rk3588_box/rk3588_box.mk \
$(LOCAL_DIR)/rk3588_xr/rk3588_xr.mk \
$(LOCAL_DIR)/rk3588m_s/rk3588m_s.mk \
$(LOCAL_DIR)/rk3588m_car/rk3588m_car.mk \

COMMON_LUNCH_CHOICES := \
ATK_DLRK3588-userdebug \
ATK_DLRK3588-user \
rk3588_s-userdebug \
rk3588_s-user \
rk3588s_s-userdebug \
rk3588s_s-user \
rk3588_box-userdebug \
rk3588_box-user \
rk3588_xr-userdebug \
rk3588_xr-user \
rk3588m_s-userdebug \
rk3588m_s-user \
rk3588m_car-userdebug \
rk3588m_car-user
```

PRODUCT_MAKEFILES 变量用于记录产品的 Makefile 配置文件（.mk 文件），每一款产品都有一个对应的配置文件，也就是产品配置文件，这个产品配置文件其实就是一个 Makefile 文件，通常将其命名为<product_name>.mk，product_name 表示产品名。需要将产品配置文件路径添加到 PRODUCT_MAKEFILES 变量中保存。上述示例中，\$(LOCAL_DIR)表示当前目录，也就是 AndroidProducts.mk 文件所在目录。

COMMON_LUNCH_CHOICES 变量用于把产品（或者叫产品编译项）添加到 lunch 选择列表（执行 lunch 命令时显示的列表）中，产品编译项的名字由两部分组成：<产品名>-<版本类型>，<产品名>与<版本类型>之间使用“-”符号分隔。版本类型一共分为三种：user（用户版本）、userdebug（用户调试版本）以及 eng（工程版本）。这三者的区别如下：

版本类型	说明
user	打算作为最终发布的版本 •安装带有 user 标记的模块 •除了带有标记的模块之外，还会根据产品定义文件安装相应模块 •ro.secure=1 •ro.debuggable=0 •adb 默认处于禁用状态
userdebug	与 user 相同，但以下几点除外： •还会安装带有 debug 标记的模块 •ro.debuggable=1 •adb 默认处于启用状态
eng	•安装带有 eng 或 debug 标记的模块 •除了带有标记的模块之外，还会根据产品定义文件安装相应模块

- ro.secure=0
- ro.debuggable=1
- ro.kernel.android.checkjni=1
- adb 默认处于启用状态

表 6.3.2.2 user/userdebug/eng 三者之间的区别

注意事项:

①、AndroidProducts.mk 文件一般存放在 **<SDK>/device/公司名/产品名/**目录下、或者 **<SDK>/device/公司名/products/**目录;当然,并非一定要将 AndroidProducts.mk 文件放置在该目录下,也可按照自己的规划、需求放置在其它目录,都是没有问题的;因为执行 lunch 命令时,它会在 device 目录查找 AndroidProducts.mk 文件,将所有名为 AndroidProducts.mk 的文件全部查找出来并加载,所以它并非是在某些特定目录下查找 AndroidProducts.mk 文件,而是直接搜索整个 device 目录。

②、产品配置文件的命名方式一般为<product_name>.mk,它是一个 Makefile 文件,其中 product_name 表示产品名;Android 编译系统会根据产品名(product_name)来查找产品配置文件,它会从 PRODUCT_MAKEFILES 列表中查找是否存在<product_name>.mk;当然,产品配置文件的名字也可以不等于<product_name>.mk,但是在这种情况下,需要通过 PRODUCT_MAKEFILES 变量进行指定,方式如下:

<product_name>:<product_makefile>

<产品名>+<冒号>+<产品 Makefile 文件路径>,示例如下:

```
PRODUCT_MAKEFILES := \
aosp_cf_arm64_phone:$(LOCAL_DIR)/vsoc_arm64/phone/device.mk \
aosp_cf_x86_64_phone:$(LOCAL_DIR)/vsoc_x86_64/phone/device.mk \
aosp_cf_x86_auto:$(LOCAL_DIR)/vsoc_x86/auto/device.mk \
aosp_cf_x86_pasan:$(LOCAL_DIR)/vsoc_x86/pasan/device.mk \
aosp_cf_x86_phone:$(LOCAL_DIR)/vsoc_x86/phone/device.mk \
aosp_cf_x86_go_phone:$(LOCAL_DIR)/vsoc_x86/go_phone/device.mk \
aosp_cf_x86_gsi:$(LOCAL_DIR)/vsoc_x86/gsi/aosp_cf_x86_gsi.mk \
aosp_cf_x86_tv:$(LOCAL_DIR)/vsoc_x86/tv/device.mk

COMMON_LUNCH_CHOICES := \
aosp_cf_arm64_phone-userdebug \
aosp_cf_x86_64_phone-userdebug \
aosp_cf_x86_auto-userdebug \
aosp_cf_x86_phone-userdebug \
aosp_cf_x86_tv-userdebug
```

二、产品配置文件/产品 Makefile 文件: <product_name>.mk (必须)

产品配置文件<product_name>.mk,该文件中定义了产品的一些信息:产品名称、设备名称、产品制造商、产品型号等,示例如下:

```
PRODUCT_NAME := rk3588_s
PRODUCT_DEVICE := rk3588_s
PRODUCT_BRAND := rockchip
PRODUCT_MODEL := rk3588_s
PRODUCT_MANUFACTURER := rockchip
```

说明如下:

变量名	作用	说明
PRODUCT_NAME	产品名,将显示在“ 设置→关于手机 ”选项中	该名称要和产品编译项中的产品名保持一致,也就是 COMMON_LUNCH_CHOICES 变量所定义的产品编译项: \$(PRODUCT_NAME)-<版本类型>

PRODUCT_DEVICE	产品对应的设备名称，也就是主板名称、板子的名称；	编译系统会根据该名称查找主板配置文件 BoardConfig.mk，这个名字要和 BoardConfig.mk 文件所在目录名保持一致。 产品的输出目录也是根据该名称创建 out/target/product/\$(PRODUCT_DEVICE)
PRODUCT_BRAND	对软件进行自定义所针对的品牌（如果有），例如运营商	
PRODUCT_MODEL	最终产品的最终用户可见名称，或者叫机型名	
PRODUCT_MANUFACTURER	产品制作商的名称	会赋值给 ro.product.manufacturer 属性

表 6.3.2.3 产品 Makefile 文件中常见变量说明

再次强调，编译系统会使用 PRODUCT_DEVICE 来查找主板配置文件 BoardConfig.mk，这个名字要和 BoardConfig.mk 文件所在目录名保持一致，并且保证该目录名在 device 目录结构中唯一；因为编译系统会在 device 目录寻找（最大路径深度为 4 级）名为\$(PRODUCT_DEVICE)的目录，并加载该目录下的 BoardConfig.mk 文件。

三、BoardConfig.mk 文件（必须）

主板配置文件 BoardConfig.mk，该文件也是一个 Makefile 文件，包含了主板特定配置信息，用于定义和硬件相关的底层特性和变量，示例如下：

```
TARGET_ARCH := arm64
TARGET_ARCH_VARIANT := armv8-a
TARGET_CPU_ABI := arm64-v8a
TARGET_CPU_ABI2 :=
TARGET_CPU_VARIANT := cortex-a55
TARGET_CPU_SMP := true

TARGET_2ND_ARCH := arm
TARGET_2ND_ARCH_VARIANT := armv8-2a
TARGET_2ND_CPU_ABI := armeabi-v7a
TARGET_2ND_CPU_ABI2 := armeabi
TARGET_2ND_CPU_VARIANT := cortex-a55

TARGET_PREBUILT_KERNEL := kernel/arch/arm64/boot/Image
BOARD_PREBUILT_DTBIMAGE_DIR := kernel/arch/arm64/boot/dts/rockchip

PRODUCT_UBOOT_CONFIG ?= rk3568
PRODUCT_KERNEL_ARCH ?= arm64
PRODUCT_KERNEL_DTS ?= rk3568-evb1-ddr4-v10
PRODUCT_KERNEL_CONFIG ?= rockchip_defconfig
BOARD_NFC_SUPPORT := false
BOARD_HAS_GPS := false

BOARD_GRAVITY_SENSOR_SUPPORT := true
BOARD_COMPASS_SENSOR_SUPPORT := false
BOARD_GYROSCOPE_SENSOR_SUPPORT := true
BOARD_PROXIMITY_SENSOR_SUPPORT := true
BOARD_LIGHT_SENSOR_SUPPORT := true
BOARD_PRESSURE_SENSOR_SUPPORT := false
BOARD_TEMPERATURE_SENSOR_SUPPORT := false
BOARD_USB_HOST_SUPPORT := true
```

BoardConfig.mk 文件所在路径最大深度不能超过 4 级, 否则编译系统将无法找到该文件; 并且 BoardConfig.mk 文件所在目录的名字必须等于\$(PRODUCT_DEVICE)。

四、device.mk 文件（非必须）

一般作为通用的 Makefile 文件被产品配置文件<product_name>.mk 继承进来, 示例如下:

```
rk356x/rk3566_einkw6/rk3566_einkw6.mk:30:$(call inherit-product, device/rockchip/rk356x/device.mk)
rk356x/rk3566_einkw6/rk3566_einkw6.mk:31:$(call inherit-product, device/rockchip/common/device.mk)
rk356x/rk3566_rgo/rk3566_rgo.mk:25:$(call inherit-product, device/rockchip/rk356x/device.mk)
rk356x/rk3566_rgo/rk3566_rgo.mk:26:$(call inherit-product, device/rockchip/common/device.mk)
rk356x/rk3568_r/rk3568_r.mk:25:$(call inherit-product, device/rockchip/rk356x/device.mk)
rk356x/rk3568_r/rk3568_r.mk:26:$(call inherit-product, device/rockchip/common/device.mk)
rk356x/rk3566_32bit/rk3566_32bit.mk:25:$(call inherit-product, device/rockchip/rk356x/device.mk)
rk356x/rk3566_32bit/rk3566_32bit.mk:26:$(call inherit-product, device/rockchip/common/device.mk)
```

图 4.3.2.5 device.mk 文件

五、AndroidBoard.mk 文件（非必须）

该文件会被编译系统核心代码包含进来:

```
build/make/target/board/Android.mk:7:-include $(TARGET_DEVICE_DIR)/AndroidBoard.mk
```

```
1 #
2 # Set up product-global definitions and include product-specific rules.
3 #
4
5 LOCAL_PATH := $(call my-dir)
6
7 -include $(TARGET_DEVICE_DIR)/AndroidBoard.mk
8
```

图 4.3.2.6 包含 AndroidBoard.mk

AndroidBoard.mk 文件会被<SDK>/build/make/target/board/Android.mk 所包含, 与普通的“include”不同, 这里使用的是“-include”; 使用“include”包含一个 Makefile 文件, 如果被包含的文件不存在, 那么 make 将会报错并退出; 而使用“-include”时, 如果被包含的文件不存在、make 并不会退出, 它将会继续执行 (“-”的意思是告诉 make, 忽略此操作的错误, make 继续执行)。

所以 AndroidBoard.mk 文件并不是必须的, 可以有、也可以没有; 如果提供了该文件, 则必须将其放置在主板配置文件 BoardConfig.mk 所在目录, 也就是将 AndroidBoard.mk 和 BoardConfig.mk 放在同一个目录下。

示例如下 (device/rockchip/rk3588/ATK_DLRK3588/AndroidBoard.mk):

```
# generate fstab file for device
-include device/rockchip/common/build/rockchip/RebuildFstab.mk

# generate dtbo image for device
-include device/rockchip/common/build/rockchip/RebuildDtboImg.mk
# generate parameter.txt for device
-include device/rockchip/common/build/rockchip/RebuildParameter.mk
```

3. 配置一个新产品

本小节讲一下如何添加/创建一个新产品。

①、拷贝参考模板

首先进入<SDK>/device/rockchip/rk3588 目录, 将<SDK>/device/rockchip/rk3588/rk3588_s 作为模板 (如果是 Android13, 选择 rk3588_t 作为模板), 将其拷贝到 <SDK>/device/rockchip/rk3588/my_rk3588 (目录名可以自己定义), 如下所示:

```
# 如果是 Android12 执行如下命令
cp -r rk3588_s my_rk3588
```

如果是 Android13 选择 rk3588_t 作为模板

cp -r rk3588_t my_rk3588

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$ pwd
/home/dengtao/rk3588_android_sdk/device/rockchip/rk3588
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$ cp -r rk3588_s my_rk3588
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$ ls
Android.mk      BoardConfig.mk  my_rk3588      public.libraries.txt  rk3588.prop  sepolicy_vendor
AndroidProducts.mk  device.mk      ota            rk3588_box          rk3588_s    wake_lock_filter.xml
ATK_DLRK3588    dtbo_gki_fix.dts  overlay       rk3588m_car        rk3588s_s   wifi_bt.mk
bluetooth      init.rk3588.rc  package_performance.xml  rk3588m_s          rk3588_xr
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588$
```

图 6.3.2.7 my_rk3588

我们一般不会从零开始去配置一个新产品,因为这样工作量太大;而是以现有产品为模板,来配置一个新的产品。

②、修改 device/rockchip/rk3588/AndroidProducts.mk 文件

前面已经给大家讲过, AndroidProducts.mk 文件中定义了一个产品列表,我们需要将我们的产品 my_rk3588 添加到这个产品列表中, 如下所示:

```
PRODUCT_MAKEFILES := \
$(LOCAL_DIR)/my_rk3588/my_rk3588.mk \
$(LOCAL_DIR)/rk3588_s/rk3588_s.mk \
$(LOCAL_DIR)/ATK_DLRK3588/ATK_DLRK3588.mk \
$(LOCAL_DIR)/rk3588s_s/rk3588s_s.mk \
$(LOCAL_DIR)/rk3588_box/rk3588_box.mk \
$(LOCAL_DIR)/rk3588_xr/rk3588_xr.mk \
$(LOCAL_DIR)/rk3588m_s/rk3588m_s.mk \
$(LOCAL_DIR)/rk3588m_car/rk3588m_car.mk \

COMMON_LUNCH_CHOICES := \
my_rk3588-userdebug \
my_rk3588-user \
ATK_DLRK3588-userdebug \
ATK_DLRK3588-user \
rk3588_s-userdebug \
rk3588_s-user \
rk3588s_s-userdebug \
rk3588s_s-user \
rk3588_box-userdebug \
rk3588_box-user \
rk3588_xr-userdebug \
rk3588_xr-user \
rk3588m_s-userdebug \
rk3588m_s-user \
rk3588m_car-userdebug \
rk3588m_car-user
```

加粗部分便是我们需要添加的内容。这两个变量所表示的含义,上文都已介绍过,这里便不再多说!

③、修改 my_rk3588.mk

首先进入到 device/rockchip/rk3588/my_rk3588 目录,将该目录下的 rk3588_s.mk 文件重命名为 my_rk3588.mk (如果是 Android13,则将 rk3588_t.mk 重命名为 my_rk3588.mk), 如下所示:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$ ls
AndroidBoard.mk  bt_vendor.conf  config.cfg_ab_gki  ota            resolution_white.xml
Android.mk      config.cfg      dt-overlay.in      recovery.fstab  rk3588_s.mk
BoardConfig.mk  config.cfg_ab  media_profiles_default.xml  recovery.fstab_AB
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$ mv rk3588_s.mk my_rk3588.mk
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$ ls
AndroidBoard.mk  bt_vendor.conf  config.cfg_ab_gki  my_rk3588.mk  recovery.fstab_AB
Android.mk      config.cfg      dt-overlay.in      ota            resolution_white.xml
BoardConfig.mk  config.cfg_ab  media_profiles_default.xml  recovery.fstab
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$
```

图 6.3.2.8 将 rk3588_s.mk 重命名为 my_rk3588.mk

my_rk3588.mk 就是产品的 Makefile 配置文件（产品配置文件），并对其进行修改，如下所示：

示：

```
PRODUCT_NAME := my_rk3588
PRODUCT_DEVICE := my_rk3588
PRODUCT_BRAND := rockchip
PRODUCT_MODEL := my_rk3588
PRODUCT_MANUFACTURER := rockchip

# First lunching is S, api_level is 31
PRODUCT_SHIPPING_API_LEVEL := 31
PRODUCT_DTBO_TEMPLATE := $(LOCAL_PATH)/dt-overlay.in

include device/rockchip/common/build/rockchip/DynamicPartitions.mk
include device/rockchip/rk3588/rk3588_s/BoardConfig.mk
include device/rockchip/common/BoardConfig.mk
$(call inherit-product, device/rockchip/rk3588/device.mk)
$(call inherit-product, device/rockchip/common/device.mk)
$(call inherit-product, frameworks/native/build/tablet-10in-xhdpi-2048-dalvik-heap.mk)

DEVICE_PACKAGE_OVERLAYS += $(LOCAL_PATH)/../overlay

PRODUCT_CHARACTERISTICS := tablet

PRODUCT_NAME := my_rk3588
PRODUCT_DEVICE := my_rk3588
PRODUCT_BRAND := rockchip
PRODUCT_MODEL := my_rk3588
PRODUCT_MANUFACTURER := rockchip
PRODUCT_AAPT_PREF_CONFIG := xhdpi

## add Rockchip properties
#
PRODUCT_PROPERTY_OVERRIDES += ro.sf.lcd_density=320
PRODUCT_PROPERTY_OVERRIDES += ro.wifi.sleep.power.down=true
PRODUCT_PROPERTY_OVERRIDES += persist.wifi.sleep.delay.ms=0
PRODUCT_PROPERTY_OVERRIDES += persist.bt.power.down=true
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.primary=DSI
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.extend=HDMI-A,eDP
PRODUCT_PROPERTY_OVERRIDES += sys.mouse.presentation=1
```

图 6.3.2.9 my_rk3588.mk 文件内容

这些变量前面已经讲过，重点需要注意的便是 PRODUCT_NAME 和 PRODUCT_DEVICE。PRODUCT_NAME 便是产品名，它需要和产品编译项中的产品名保持一致，也就是 COMMON_LUNCH_CHOICES 变量所定义的产品编译项：

\$(PRODUCT_NAME)-<版本类型>

除此之外，产品配置文件的名字也必须是 \$(PRODUCT_NAME).mk，除非在 PRODUCT_MAKEFILES 变量中指定。

PRODUCT_DEVICE 表示设备名，也就是主板名，编译系统会使用 PRODUCT_DEVICE 来查找主板配置文件 BoardConfig.mk，PRODUCT_DEVICE 要和 BoardConfig.mk 文件所在目录名保持一致。

④、修改路径信息

我们是以 rk3588_s（Android13 则对应为 rk3588_t）为模板创建了一个新产品，我们需要将带有“rk3588_s”相关的路径修改为 my_rk3588，如下所示：

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$ grep -nR rk3588_s
my_rk3588.mk:22:include device/rockchip/rk3588/rk3588_s/BoardConfig.mk
BoardConfig.mk:32:    TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/rk3588_s/recovery.fstab_AB
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/my_rk3588$
```

图 6.3.2.10 与 rk3588_s 目录相关的文件

只需修改 my_rk3588.mk 和 BoardConfig.mk 文件即可。

my_rk3588.mk 文件修改如下所示：



```

17 # First lunching is S, api_level is 31
18 PRODUCT_SHIPPING_API_LEVEL := 31
19 PRODUCT_DTBO_TEMPLATE := $(LOCAL_PATH)/dt-overlay.in
20
21 include device/rockchip/common/build/rockchip/DynamicPartitions.mk
22 include device/rockchip/rk3588/my_rk3588/BoardConfig.mk
23 include device/rockchip/common/BoardConfig.mk
24 $(call inherit-product, device/rockchip/rk3588/device.mk)
25 $(call inherit-product, device/rockchip/common/device.mk)
26 $(call inherit-product, frameworks/native/build/tablet-10in-xhdpi-2048-dalvik-heap.mk)
27
28 DEVICE_PACKAGE_OVERLAYS += $(LOCAL_PATH)/../overlay
29

```

图 6.3.2.11 修改 my_rk3588.mk 文件

BoardConfig.mk 文件修改如下所示:

```

23 BOARD_GRAVITY_SENSOR_SUPPORT := true
24 BOARD_COMPASS_SENSOR_SUPPORT := true
25 BOARD_SENSOR_COMPASS_AK8963-64 := true
26 BOARD_GYROSCOPE_SENSOR_SUPPORT := true
27 BOARD_PROXIMITY_SENSOR_SUPPORT := true
28 BOARD_LIGHT_SENSOR_SUPPORT := true
29
30 ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
31     include device/rockchip/common/BoardConfig_AB.mk
32     TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/my_rk3588/recovery.fstab_AB
33 endif
34

```

图 6.3.2.12 修改 BoardConfig.mk 文件

至此，我们就添加了一个新产品 my_rk3588。

⑤、lunch 选择新产品 my_rk3588

回到<SDK>根目录，执行 lunch 命令，此时可以看到我们添加的新产品 my_rk3588，如下所示：



```

40. armv8_kryo385-eng
41. beagle_x15-userdebug
42. beagle_x15_auto-userdebug
43. fuchsia_arm64-eng
44. fuchsia_x86_64-eng
45. hikey-userdebug
46. hikey64_only-userdebug
47. hikey960-userdebug
48. hikey960_tv-userdebug
49. hikey_tv-userdebug
50. my_rk3588-user
51. my_rk3588-userdebug
52. qemu_trusty_armv8-userdebug
53. rk3588_box-user
54. rk3588_box-userdebug
55. rk3588_s-user
56. rk3588_s-userdebug
57. rk3588_xr-user
58. rk3588_xr-userdebug
59. rk3588m_car-user
60. rk3588m_car-userdebug
61. rk3588m_s-user
62. rk3588m_s-userdebug
63. rk3588s_s-user
64. rk3588s_s-userdebug
65. sdk_car_arm-userdebug
66. sdk_car_arm64-userdebug
67. sdk_car_portrait_x86_64-userdebug
68. sdk_car_x86-userdebug
69. sdk_car_x86_64-userdebug
70. silvermont-eng
71. uml-userdebug
72. yukawa-userdebug
73. yukawa_sei510-userdebug

Which would you like? [aosp_arm-eng]

```

图 6.3.2.13 lunch 选择产品(1)

输入名字或者对应的数字编号、按回车即可选择该产品:

```

69. sdk_car_x86_64-userdebug
70. silvermont-eng
71. uml-userdebug
72. yukawa-userdebug
73. yukawa_sei510-userdebug

Which would you like? [aosp_arm-eng] my_rk3588-userdebug 输入名字或对应的数字编号
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=my_rk3588
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-a
TARGET_2ND_CPU_VARIANT=generic
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-97-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SQ3A.220705.003.A1
OUT_DIR=out
=====

```

图 6.3.2.14 lunch 选择产品(2)

4. Android 产品配置中涉及到的各种变量

Android 产品配置中涉及到很多的变量，有些是通用变量，譬如 PRODUCT_NAME、PRODUCT_DEVICE、PRODUCT_BRAND、PRODUCT_MODEL、PRODUCT_MANUFACTURER 等；有些是自定义变量，有些则是功能变量（表示该变量有特殊功能），譬如 PRODUCT_COPY_FILES、PRODUCT_PROPERTY_OVERRIDES、PRODUCT_PACKAGES 等，如下所示：

变量名	作用
PRODUCT_COPY_FILES	用于完成拷贝，将源路径下的文件复制到目标路径
PRODUCT_PROPERTY_OVERRIDES	用于设置系统属性（覆盖）
PRODUCT_PACKAGES	将要安装的 APK 和模块列表

表 6.3.2.4 常见功能变量说明

PRODUCT_COPY_FILES 示例：

```
PRODUCT_COPY_FILES += vendor/rockchip/common/phone/etc/spn-conf.xml:system/etc/spn-conf.xml
```

表示将 vendor/rockchip/common/phone/etc/spn-conf.xml 拷贝到产品输出目录（out/target/product/\$(PRODUCT_DEVICE)）下 system/etc/spn-conf.xml。

PRODUCT_PROPERTY_OVERRIDES 示例：

```
PRODUCT_PROPERTY_OVERRIDES += persist.bt.power.down=true
```

定义属性 persist.bt.power.down=true。

PRODUCT_PACKAGES 示例：

```
PRODUCT_PACKAGES += \
    rkaiq_tool_server \
    rkaiq_3A_server
```

编译并安装 rkaiq_tool_server 模块和 rkaiq_3A_server 模块。

6.3.3 Android 源码编译

可以使用 RK 封装好的脚本 build.sh 编译 Android 源码，也可以直接通过 make 命令编译 Android 源码，本小节向用户介绍使用 make 编译 Android 源码。在编译之前，需要完成下面两件事：

- ①、初始化编译环境：source build/envsetup.sh
- ②、lunch 选择产品：lunch ATK_DLRK3588-userdebug 或 lunch ATK_DLRK3588-user

1. 常见编译命令

在 Android 源码目录下有一个 Makefile 文件，如下所示：

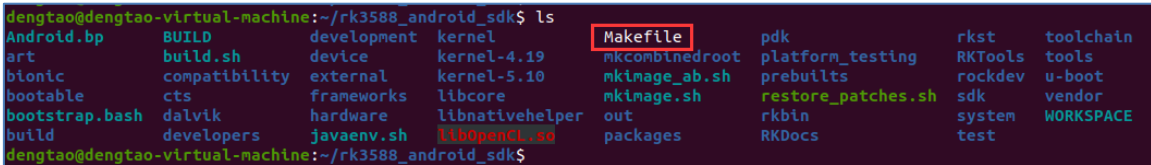


图 6.3.3.1 Android 源码 Makefile 文件

这个 Makefile 文件就是 Android 源码的 Makefile，所以我们可以通过 make 命令编译 Android 源码，既可以编译整个 Android 源码，也可单独编译某个特定模块。

①、镜像编译命令

编译命令	作用
make	不加任何参数，表示编译整个 Android 源码，编译过程产生的中间文件以及最终镜像将会输出到

	out/target/product/\$(PRODUCT_DEVICE)目录, 我们把这个目录称为产品输出目录
make bootimage	编译 boot.img 镜像
make systemimage	编译 system.img 镜像
make systemextimage	编译 system_ext.img 镜像
make system_dlkimage	编译 system_dlk.img 镜像
make vendorimage	编译 vendor.img 镜像
make vendor_dlkimage	编译 vendor_dlk.img 镜像
make productimage	编译 product.img 镜像
make odimage	编译 odm.img 镜像
make odm_dlkimage	编译 odm_dlk.img 镜像
make systemextimage	编译 system_ext.img 镜像
make recoveryimage	编译 recovery.img 镜像
make cacheimage	编译 cache.img 镜像
make userdataimage	编译 userdata.img 镜像
make vbmetaimage	编译 vbmeta.img 镜像
make superimage	编译 super.img 镜像

表 6.3.3.1 make 镜像编译命令

②、模块编译命令

几个主要模块的编译指令:

模块	make 命令
init	make init
zygote	make app_process
system_server	make services
java framework	make framework
framework 资源	make framework-res
jni framework	make libandroid_runtime
binder	make libbinder
Settings	make Settings
.....	make xxx

表 6.3.3.2 make 模块编译命令

每个模块都可以执行 “**make <module_name>**” 命令单独编译。

③、清除命令

有几个清除命令:

make 命令	说明
make clean	彻底清除, 相当于删除 out/目录下的所有内容
make clobber	效果等同于 make clean, 也是删除 out 目录下的所有内容
make installclean (常用)	删除 out/target/product/\$(PRODUCT_DEVICE)/目录下的所有 img 镜像文件、所有.zip 文件, 删除 system、vendor、product、data、recovery、odm、ramdisk、root、cache 等目录, 以及其它一些文件, 这里没法给大家一一列举, 后续会单独介绍该命令。
make dataclean	相当于执行如下操作: rm -rf out/target/product/\$(PRODUCT_DEVICE)/data/*
make clean-<module>	清除单个特定模块

表 6.3.3.3 make 清除命令

④、build/envsetup.sh 脚本提供的命令介绍

在编译 Android 源码之前, 需要执行 “**source build/envsetup.sh**” 命令初始化编译环境, 每次打开一个新的终端时都要执行一次。接着执行 **lunch** 命令选择一个产品, 这里需要注意: 这个 **lunch** 命令并不是 Ubuntu 系统提供的, 而是执行 “**source build/envsetup.sh**” 命令后才会出现, 也就是说 **lunch** 命令是由 **build/envsetup.sh** 脚本提供, 而非 Ubuntu 系统自带。

如果没有执行 “**source build/envsetup.sh**” 命令, 则不会出现 **lunch** 命令。

build/envsetup.sh 脚本除了提供 **lunch** 命令之外, 还提供了很多其它的命令, 譬如 **m**、**mm**、**mmm**、**mma**、**mmaa**、**hmm** 以及其它命令。可以执行 “**hmm**” 命令来查看帮助信息, 如下所示:

hmm

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk$ hmm
Run "m help" for help with the build system itself.

Invoke ". build/envsetup.sh" from your shell to add the following functions to your environment:
- lunch:      lunch <product_name>-<build_variant>
               Selects <product_name> as the product to build, and <build_variant> as the variant to
               build, and stores those selections in the environment to be read by subsequent
               invocations of 'm' etc.
- tapas:      tapas [<App1> <App2> ...] [arm|x86|mips|arm64|x86_64|mips64] [eng|userdebug|user]
- croot:      Changes directory to the top of the tree, or a subdirectory thereof.
- m:          Makes from the top of the tree.
- mm:         Builds and installs all of the modules in the current directory, and their
               dependencies.
- mmm:        Builds and installs all of the modules in the supplied directories, and their
               dependencies.
               To limit the modules being built use the syntax: mmm dir1:target1,target2.
- mma:        Same as 'mm'
- mmaa:       Same as 'mmm'
- provision:  Flash device with all required partitions. Options will be passed on to fastboot.
- cgrep:      Greps on all local C/C++ files.
- ggrep:      Greps on all local Gradle files.
- gogrep:     Greps on all local Go files.
- jgrep:      Greps on all local Java files.
- resgrep:    Greps on all local res/*.xml files.
- mangrep:    Greps on all local AndroidManifest.xml files.
- mgrep:      Greps on all local Makefiles and *.bp files.
- owngrep:    Greps on all local OWNERS files.
- sepgrep:    Greps on all local seppolicy files.
- sgrep:      Greps on all local source files.
- godir:      Go to the directory containing a file.
- allmod:     List all modules.
- gomod:      Go to the directory containing a module.
- pathmod:    Get the directory containing a module.
- refreshmod: Refresh list of modules for allmod/gomod.

Environment options:
- SANITIZE_HOST: Set to 'address' to use ASAN for all host modules.
- ANDROID_QUIET_BUILD: set to 'true' to display only the essential messages.

Look at the source to view more functions. The complete list is:
addcompletions add_lunch_combo allmod build_build_var_cache cgrep check_product check_type check_variant choosecombo choosepr
able coredump_setup cproj _croot croot destroy_build_var_cache enable_zsh_completion get_abs_build_var getbugreports get_buil
rdpath gettargetarch gettop ggrep godir gogrep gomod hmm isviewserverstarted jgrep key_back key_home key_menu _lunch lunch
nch_menu provision qpid rcgrep refreshmod resgrep runhat runtest sepgrep setpaths set_sequence_number set_stuff_for_environe
opviewserver systemstack tapas tracedmdump treegrep _trigger_build validate_current_shell _wrap_build
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
```

图 6.3.3.2 执行 **hmm** 查看帮助信息

文本信息:

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk$ hmm
Run "m help" for help with the build system itself.

Invoke ". build/envsetup.sh" from your shell to add the following functions to
your environment:
- lunch:      lunch <product_name>-<build_variant>
               Selects <product_name> as the product to build, and <build_variant>
               as the variant to
               build, and stores those selections in the environment to be read by
               subsequent
               invocations of 'm' etc.
- tapas:      tapas [<App1> <App2> ...] [arm|x86|mips|arm64|x86_64|mips64]
               [eng|userdebug|user]
- croot:      Changes directory to the top of the tree, or a subdirectory thereof.
- m:          Makes from the top of the tree.
- mm:         Builds and installs all of the modules in the current directory, and
               their
               dependencies.
- mmm:        Builds and installs all of the modules in the supplied directories,
               and their
```

```
dependencies.
To limit the modules being built use the syntax: mmm
dir/:target1,target2.
- mma: Same as 'mm'
- mmma: Same as 'mmm'
- provision: Flash device with all required partitions. Options will be passed
on to fastboot.
- cgrep: Greps on all local C/C++ files.
- ggrep: Greps on all local Gradle files.
- gogrep: Greps on all local Go files.
- jgrep: Greps on all local Java files.
- resgrep: Greps on all local res/*.xml files.
- mangrep: Greps on all local AndroidManifest.xml files.
- mgrep: Greps on all local Makefiles and *.bp files.
- owngrep: Greps on all local OWNERS files.
- sepgrep: Greps on all local sepolicy files.
- sgrep: Greps on all local source files.
- godir: Go to the directory containing a file.
- allmod: List all modules.
- gomod: Go to the directory containing a module.
- pathmod: Get the directory containing a module.
- refreshmod: Refresh list of modules for allmod/gomod.

Environment options:
- SANITIZE_HOST: Set to 'address' to use ASAN for all host modules.
- ANDROID_QUIET_BUILD: set to 'true' to display only the essential messages.

Look at the source to view more functions. The complete list is:
addcompletions add_lunch_combo allmod build_build_var_cache cgrep check_product
check_type check_variant choosecombo chooseproduct choosetype choosevariant
_complete_android_module_names core coredump_enable coredump_setup cproj_croot
croot destroy_build_var_cache enable_zsh_completion get_abs_build_var
getbugreports get_build_var getlastsscreenshot get_make_command getprebuilt
getsscreenshotpath getsdcardpath gettargetarch gettop ggrep godir gogrep gomod hmm
is_isviewserverstarted jgrep key_back key_home key_menu _lunch lunch m make
mangrep mgrep mm mma mmm mmma owngrep pathmod pez printconfig print_lunch_menu
provision qpid rcgrep refreshmod resgrep runhat runtest sepgrep setpaths
set_sequence_number set_stuff_for_environment sgrep should_add_completion
smoketest source_vendorsetup startviewserver stopviewserver systemstack tapas
tracedmdump treegrep _trigger_build validate_current_shell _wrap_build
```

由此可见, build/envsetup.sh 脚本提供了很多命令, 接下来简单介绍一下。

编译指令:

命令	说明
m	在 Android 源码根目录下执行编译操作, 也就是在 Android 源码根目录下执行 make 命令, 譬如: m -j16 m bootimage -j16 m systemimage -j16 但无需用户在 Android 源码根目录下执行 m 命令, 可以在 Android 源码的任何子目录下执行 m 命令, 相当于: croot; make
mm	编译当前目录下的所有模块, 但不包含依赖
mmm <module_path>	编译指定目录下的所有模块, 但不包含依赖
mma	编译当前目录下的所有模块, 包含依赖
mma <module_name>	编译指定目录下的所有模块, 包含依赖

表 6.3.3.4 m/mm/mmm 编译命令

Android 源码非常庞大, 直接使用 grep/ag 来搜索代码, 会耗费大量时间。根据具体需求, 可以选择合适的检索指令, 有效节省代码搜索时间, 提高准确度。

搜索命令:

命令	说明
cgrep	对所有 C/C++ 文件执行 grep 搜索操作
jgrep	对所有 java 文件执行 grep 搜索操作
ggrep	对所有 gradle 文件执行 grep 搜索操作
mgrep	对所有 Makefile 文件以及 *.bp 文件执行 grep 搜索操作
sepgrep	对所有 sepolicy 文件执行 grep 搜索操作
resgrep	对所有 res/*.xml 文件执行 grep 搜索操作
gogrep	对所有 go 文件执行 grep 搜索操作
mangrep	对所有 AndroidManifest.xml 文件执行 grep 搜索操作
...	...

表 6.3.3.4 grep 搜索命令

其它命令:

命令	说明
croot	执行 croot 命令可以切换到 Android 源码根目录（顶层目录），需要在 Android 源码根目录或者子目录下执行
hmm	查看帮助信息
check_product	检查产品是否有效
get_abs_build_var	获取某个编译变量的值
get_build_var	获取某个编译变量的值
printconfig	打印当前选择的产品对应的配置信息
...	...

表 6.3.3.5 其它命令

2. 编译整个 Android 源码

在 Android 源码根目录下执行 make 命令，不带任何参数（-jX 参数除外，它表示编译线程数量），将会编译整个 Android 源码：

```
make -j16
```

或者执行 m 命令（可在 Android 源码的任何子目录下执行）：

```
m -j16
```

Android 编译生成的所有文件都是和源码分离的，所有编译过程中生成的中间文件和最终结果都存放在 out 目录。out 目录结构如下所示：

```
out/
├── Android.mk
├── build_date.txt
├── build_error
├── casecheck.txt
├── CaseCheck.txt
├── CleanSpec.mk
├── ninja_build
├── soong
├── soong_ui
├── commit_id.xml
├── dumpvars-build_error
├── host      #当前编译主机（Ubuntu 主机）需要用到的工具和库，一般都是 x86 架构
│   ├── common
│   └── linux-x86
├── target   #存放目标机器运行所需的各种二进制文件、库文件、镜像等等
│   └── common
```

```
|
|   └── product
|   ...
```

host 和 **target** 是我们需要关注的两个子目录, host 目录下存放了当前编译主机 (也就是我们的 Ubuntu 主机) 需要用到的各种文件, 譬如编译主机需要使用到的工具、库等; target 目录下存放了为目标机生成的各种文件, 包括各种中间文件、二进制文件、Android APK、java 库、.so 库、配置文件以及最终 img 镜像等等。

host 目录结构如下:

```
host/
├── common
│   └── obj      #包含 JAVA 库
├── linux-x86
│   ├── bin      #二进制程序, 譬如 mkbootimg、adb 等
│   ├── com.android.i18n
│   ├── com.android.tzdata
│   ├── fake_packages
│   ├── framework #JAVA 库, .jar 包
│   ├── lib       #.so 动态链接库
│   ├── lib64     #64 位 .so 动态链接库
│   ├── obj       #编译产生的中间文件 (目标文件, 64 位)
│   ├── obj32     #编译产生的中间文件 (目标文件, 32 位)
└── usr
```

target 目录结构如下所示:

```
target/
├── common
│   └── obj
│   └── R
├── product
└── ATK_DLRK3588 #该目录的名字由 PRODUCT_DEVICE 变量决定
```

重点关注 ATK_DLRK3588 目录, 这个目录便是产品输出目录:

```
tree -L 1 out/target/product/ATK_DLRK3588/
out/target/product/ATK_DLRK3588/
├── boot-debug.img #支持 root 权限操作、支持 debug 的 boot.img 镜像
├── boot.img       #boot.img, 将会烧写到开发板 boot 分区
├── cache          #用来创建 cache.img
├── cache.img      #将会烧写到开发板 cache 分区, 被挂载到 Android 根文件系统/cache
目录, RK 平台未使用
├── clean_steps.mk
├── data          #用来创建 userdata.img, 被挂载到 Android 根文件系统/data 目
录, RK 平台未使用
├── debug_ramdisk #用来创建 ramdisk-debug.img
├── dtb.img       #内核 DTB
├── fake_packages
├── gen
├── kernel        #内核镜像 Image
├── obj           #该目录用于存放目标文件 (或者叫中间文件), 64 位
├── obj_arm       #该目录用于存放目标文件 (或者叫中间文件), 32 位
├── odm           #用来创建 odm.img
├── odm.img       #odm.img, 被挂载到 Android 根文件系统/odm 目录
├── parameter.txt #RK 平台分区表文件
├── previous_build_config.mk
├── product       #用来创建 product.img
├── product.img   #product.img, 被挂载到 Android 根文件系统/product 目录
├── ramdisk       #用来创建 ramdisk.img
├── ramdisk-debug.img #支持 debug 调试功能的 ramdisk.img
└── ramdisk.img   #普通模式下的 ramdisk.img, 会将其打包进 boot.img
```



```

|—— ramdisk-recovery.img #recovery 模式下的 ramdisk.img, 会将其打包进 recovery.img
|—— rebuild-dtbo.img      #dtbo.img
|—— recovery              #用来创建 ramdisk-recovery.img
|—— recovery.img          #recovery.img, 将会烧写到开发板 recovery 分区
|—— root                  #Android 根文件系统目录
|—— super_empty.img
|—— super.img             #super.img, 稀疏镜像格式, 将会烧写到开发板 super 分区
|—— system               #用来创建 system.img
|—— system_ext            #用来创建 system_ext.img
|—— system_ext.img        #被挂载到 Android 根文件系统/system_ext 目录
|—— system.img           #被挂载到 Android 根文件系统/system 目录
|—— testcases
|—— vendor                #用来创建 vendor.img
|—— vendor.img            #被挂载到 Android 根文件系统/vendor 目录

```

obj 和 obj_arm 两个目录用于存放目标文件, 也就是中间文件, 分为 64 位平台和 32 位平台, obj 存放 64 位平台目标文件, obj_arm 存放 32 位平台目标文件。目录结构如下所示:

```

obj/
|—— APPS                  #存放 Android 应用, 各种 APK, 以及编译产生的中间文件
|—— CONFIG
|—— ETC                  #etc 的中间编译文件, 一般都会最终生成到/system/etc 或/vendor/etc
|—— EXECUTABLES          #各种可执行文件、以及编译过程产生的中间文件, 譬如.o 文件
|—— FAKE
|—— JAVA_LIBRARIES       #Java jar 库文件以及编译过程产生的中间文件
|—— NATIVE_TESTS
|—— NOTICE_FILES
|—— NOTICE_ODM.txt
|—— NOTICE_ODM.xml
|—— NOTICE_ODM.xml.gz
|—— NOTICE_PRODUCT.txt
|—— NOTICE_PRODUCT.xml
|—— NOTICE_PRODUCT.xml.gz
|—— NOTICE_SYSTEM_EXT.txt
|—— NOTICE_SYSTEM_EXT.xml
|—— NOTICE_SYSTEM_EXT.xml.gz
|—— NOTICE.txt
|—— NOTICE_VENDOR.txt
|—— NOTICE_VENDOR.xml
|—— NOTICE_VENDOR.xml.gz
|—— NOTICE.xml
|—— NOTICE.xml.gz
|—— PACKAGING            #镜像打包时的中间文件
|—— RENDERSCRIPT_BITCODE
|—— SHARED_LIBRARIES     #.so 动态库文件以及编译过程产生的中间文件
|—— STATIC_LIBRARIES     #.a 静态库文件以及编译过程产生的中间文件

```

```

obj_arm/
|—— EXECUTABLES          #可执行文件以及编译过程产生的中间文件, 譬如.o 文件
|—— NATIVE_TESTS
|—— RENDERSCRIPT_BITCODE
|—— SHARED_LIBRARIES     #.so 动态库文件以及编译过程产生的中间文件
|—— STATIC_LIBRARIES     #.a 静态库文件以及编译过程产生的中间文件

```

关于 out 目录就给大家介绍这么多!

3. 编译模块

Android 源码非常庞大! 其中包含了许许多多的模块, 每个模块都可通过相应命令进行编译。通过 `make` 命令或者 `build/envsetup.sh` 脚本提供的模块编译命令 (`mm/mma/mmm/mmma`) 都可对指定模块进行编译。

①、Android.mk 和 Android.bp 文件介绍

Android 使用 `Android.mk` 或 `Android.bp` 文件对 Android 源码进行模块化编译, 每一个模块目录下都有一个 `Android.mk` 或 `Android.bp` 文件。

早期的 Android 都是采用 `Android.mk` 来编译源码, `Android.mk` 就是 `Makefile` 文件, `Android.mk` 语法遵从 `Makefile`。但是随着 Android 版本更新, 工程越来越大, 包含的模块也越来越多, 由于 `make` 在编译时表现出效率不够高、增量编译速度慢等问题, 于是 Google 在 Android 7.0 之后引入了 Soong 构建系统, 旨在取代 `make`; 它利用 `Kati` 和 `Ninja` 构建系统组件来加速 Android 源码的编译。

并且希望采用 `Android.bp` 来替换 `Android.mk`, 因为 `Android.bp` 文件很简单, 它们不包含任何条件语句, 也不包含控制流语句, 所以 `Android.bp` 文件更加方便 `ninja` 文件的产生 (`Soong` 构建系统会将所有的 `Android.mk` 以及 `Android.bp` 文件转换成 `ninja` 文件, 最后通过 `Ninja` 工具来解析 `ninja` 文件执行最终的编译)。

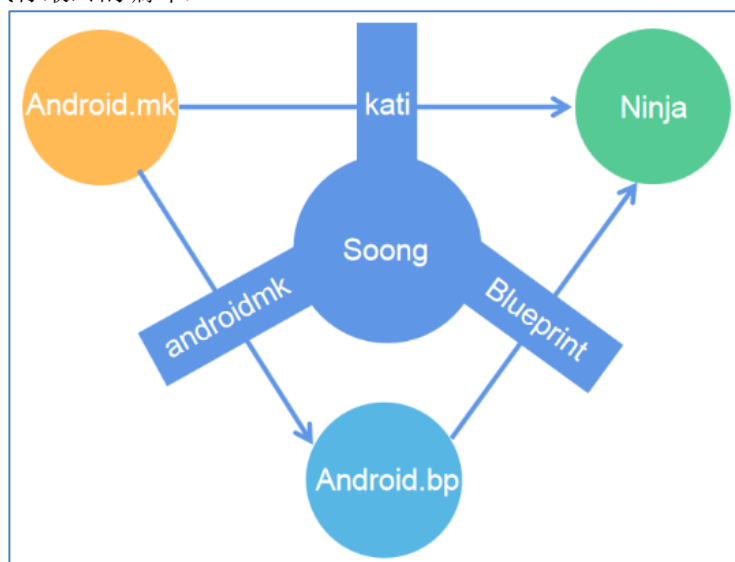


图 6.3.3.38 关系图

- `Kati`: 它可以将 `Android.mk` 文件转换为 `ninja` 文件;
- `androidmk`: 它可以将 `Android.mk` 文件转换为 `Android.bp` 文件;
- `Blueprint`: 它可以将 `Android.bp` 文件转换为 `ninja` 文件。

在 Android 源码 `out` 目录下, 确实可以看到有很多的 `ninja` 文件, 如下所示:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ ls -l out/*.ninja
-rw-rw-r-- 1 dengtao dengtao 84 3月 13 11:44 out/build-ATK_DLRK3588-cleanspec.ninja
-rw-rw-r-- 1 dengtao dengtao 958792485 3月 13 11:45 out/build-ATK_DLRK3588.ninja
-rw-rw-r-- 1 dengtao dengtao 167385 3月 8 21:15 out/build-ATK_DLRK3588-package.ninja
-rw-rw-r-- 1 dengtao dengtao 160 3月 13 11:46 out/combined-ATK_DLRK3588.ninja
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

图 6.3.3.39 out 目录下的 `ninja` 文件

从上文可知, `Android.bp` 转换成 `ninja` 文件的效率远远高于 `Android.mk` 文件, 所以 `Android.mk` 文件会逐渐被 `Android.bp` 文件取代; 但是因为在引入 `Android.bp` 之前, Android 源码中已经存在了大量的 `Android.mk` 文件, 所以要完全替代 `Android.mk` 并不是一朝一夕的事情, 并不是一下就能全部替换完成, 所以导致 Android 源码中既存在 `Android.mk`、也存在 `Android.bp` 文件。

如果大家需要在 Android 源码中添加一个自己的模块, 那么推荐大家使用 Android.bp 文件来编写模块的编译规则。

②、模块编译命令

通过 make 命令编译指定模块, 命令格式如下:

```
make <module_name>
```

module_name 表示需要编译的模块名称; 也可以通过 build/envsetup.sh 脚本提供的编译命令 (mm/mma/mmm/mmma) 进行编译, 譬如 mmm:

```
mmm <module_path>
```

module_path 表示模块所在路径。

下面列举了几个主要模块的编译命令:

模块	make 命令	mmm 命令
init	make init	mmm system/core/init
zygtoe	make app_process	mmm frameworks/base/cmds/app_process
system_server	make services	mmm frameworks/base/services
java framework	make framework	mmm frameworks/base
framework 资源	make framework-res	mmm frameworks/base/core/res
jni framework	make libandroid runtime	mmm frameworks/base/core/jni
binder	make libbinder	mmm frameworks/native/libs/binder
Settings	make Settings	mmm packages/apps/Settings

“make <module_name>” 命令中的 module_name 模块名该如何确定? 这个模块名其实就定义在 Android.mk 或 Android.bp 文件中, 所以要查看模块目录下的 Android.mk 或 Android.bp 文件才能确认模块名。

譬如以 zygtoe 模块为例, 该模块使用 Android.bp 文件定义编译规则, 打开 frameworks/base/cmds/app_process/Android.bp 文件, 如下所示:



```

1 cc_binary {
2     name: "app_process",
3
4     srcs: ["app_main.cpp"],
5
6     multilib: [
7         lib32: {
8             version_script: ":art_sigchain_version_script32.txt",
9             suffix: "32",
10        },
11        lib64: {
12            version_script: ":art_sigchain_version_script64.txt",
13            suffix: "64",
14        },
15    ],
16    ldflags: ["-Wl,--export-dynamic"],
17
18    shared_libs: [
19        "libandroid_runtime",
20        "libbinder",
21        "libcutils",
22        "libdl",
23        "libhidlibbase",
24        "liblog",
25        "libnativeloader",
26        "libutils",
27
28        // This is a list of libraries that need to be included in order to avoid
29        // bad apps. This prevents a library from having a mismatch when resolving
30        // new/delete from an app shared library.
31        // See b/21032018 for more details.
32        "libwilhelm",
33    ],
34
35    whole_static_libs: ["libsigchain"],
36
37    compile_multilib: "both",
38
39    cflags: [
40        "-Wall",
41        "-Werror",
42    ],
43 }

```

name属性定义了模块名

cc_binary表示当前模块是一个可执行程序模块，也就是最终会编译出一个可执行文件

图 6.3.3.40 Android.bp 查看模块名

由此可见，该模块的名称为“app_process”，执行“make app_process”命令可编译该模块。

注意，模块名必须是全局唯一的，不能出现模块名相同的两个模块，否则编译会报错。

cc_binary 表示当前模块会编译成可执行程序。关于 Android.bp 文件的语法规则，本文不做介绍，网上资料一大把！

再来看个 Android.mk 的例子，譬如 vendor/rockchip/common/apps/RKVRPlayer 模块，该模块使用 Android.mk 定义编译规则，打开该目录下的 Android.mk 文件，如下所示：

```

1 #####
2 # Rk4kVideoPlayer
3 LOCAL_PATH := $(call my-dir)
4 include $(CLEAR_VARS)
5 LOCAL_MODULE := RKVRPlayer
6 LOCAL_MODULE_CLASS := APPS
7 LOCAL_MODULE_TAGS := optional
8 LOCAL_BUILT_MODULE_STEM := package.apk
9 LOCAL_MODULE_SUFFIX := $(COMMON_ANDROID_PACKAGE_SUFFIX)
10 #LOCAL_PRIVILEGED_MODULE :=
11 LOCAL_CERTIFICATE := PRESIGNED
12 #LOCAL_OVERRIDES_PACKAGES :=
13 LOCAL_SRC_FILES := $(LOCAL_MODULE).apk
14 #LOCAL_REQUIRED_MODULES :=
15 #LOCAL_PREBUILT_JNI_LIBS :=
16 include $(BUILD_PREBUILT)

```

LOCAL_MODULE用于定义模块名

图 6.3.3.41 Android.mk 查看模块名

Android.mk 文件中的 LOCAL_MODULE 变量就是定义了当前模块的模块名(与 Android.bp 文件中的 name 属性意义相同), 所以执行 “make RKVRPlayer” 命令可对该模块进行编译。同样, 关于 Android.mk 文件语法规则, 本文不做介绍, 网上资料一大把!

使用 “make <module_name>” 和 “mmm <module_path>” 编译模块还是有一些区别的, “make <module_name>” 命令仅用于编译指定模块, 而 “mmm <module_path>” 可以编译指定路径下的所有模块(也包括其子目录下的模块), 因为一个 Android.mk 或 Android.bp 文件中可以定义多个模块的编译规则, 并非只能定义一个! 这个需要注意。

4. 清除命令

下表列举了几个常见的清除命令:

make 命令	说明
make clean	彻底清除, 相当于删除 out/目录下的所有内容
make clobber	效果等同于 make clean, 也是删除 out 目录下的所有内容
make installclean	见下文
make dataclean	相当于执行如下操作: rm -rf out/target/product/\$(PRODUCT_DEVICE)/data/*
make clean-<module_name>	清理指定模块, 参数 module_name 表示模块名

下面一一进行介绍:

①、make clean

彻底清除, 相当于删除整个 out/目录下的所有内容。

②、make clobber

效果等同于 make clean, 也是删除 out/目录下的所有内容。一般不会轻易执行 “make clean” 或 “make clobber” 命令, 因为它们会删除 out/目录下的所有内容, 包括编译过程中构建的所有文件、所有中间文件以及最终镜像, 一旦删除了, 也就意味着需要从零开始重新编译, 过程将会花费很长时间。

③、make installclean

该命令会删除所有已安装的文件, 包括产品输出目录下的所有 *.img 文件、所有 .zip 压缩文件、内核镜像 kernel, 以及其它文件, 但会保留中间文件, 具体情况如下图所示 (build/soong/ui/build/cleanbuild.go):

```

91     removeGlobs(ctx,
92                 hostOut("apex"),
93                 hostOut("obj/NOTICE_FILES"),
94                 hostOut("obj/PACKAGING"),
95                 hostOut("coverage"),
96                 hostOut("cts"),
97                 hostOut("nativetest*"),
98                 hostOut("sdk"),
99                 hostOut("sdk_addon"),
100                hostOut("testcases"),
101                hostOut("vts"),
102                hostOut("vts10"),
103                hostOut("vts-core"),
104                productOut("*.img"),
105                productOut("*.zip"),
106                productOut("android-info.txt"),
107                productOut("apex"),
108                productOut("kernel"),
109                productOut("data"),
110                productOut("skin"),
111                productOut("obj/NOTICE_FILES"),
112                productOut("obj/PACKAGING"),
113                productOut("ramdisk"),
114                productOut("debug_ramdisk"),
115                productOut("vendor-ramdisk"),
116                productOut("vendor-ramdisk-debug.cpio.gz"),
117                productOut("vendor_debug_ramdisk"),
118                productOut("test_harness_ramdisk"),
119                productOut("recovery"),
120                productOut("root"),
121                productOut("system"),
122                productOut("system_other"),
123                productOut("vendor"),
124                productOut("product"),
125                productOut("system_ext"),
126                productOut("oem"),
127                productOut("obj/FAKE"),
128                productOut("breakpad"),
129                productOut("cache"),
130                productOut("coverage"),
131                productOut("installer"),
132                productOut("odm"),
133                productOut("sysloader"),
134                productOut("testcases"))
135 }

```

图 6.3.3.42 make installclean 清除文件列表

该列表定义在 build/soong/ui/build/cleanbuild.go 文件中的 installClean 函数中，表示执行“make installclean”命令时需要删除的文件。hostOut 表示 out/host/linux-x86/目录，所以 hostOut(“cts”)就表示 out/host/linux-x86/cts；productOut 表示产品输出目录 out/target/product/ATK_DLRK3588/，所以 productOut(“*.img”)表示产品输出目录下的所有*.img 镜像文件。

整个 Android 源码编译完成后，如果有代码更新，一般先执行“make installclean”之后再执行“make”。

④、make dataclean

相当于删除产品输出目录下的 data/目录中的所有内容。

⑤、make clean-<module_name>

删除指定模块，module_name 表示模块名。该命令会删除模块编译产生的所有中间文件（也就是 object 目标文件）以及编译得到的最终文件，譬如可执行程序、库等。

譬如清除 init 模块：

```
make clean-init
```

6.3.4 关闭 SELinux

6.3.4.1 SELinux/SEAndroid 简介

SELinux 全称 Security-Enhanced Linux，在 Android 系统上一般称为 SEAndroid。SEAndroid 是 Google 在 Android 4.4 正式推出的一套以 SELinux 为基础核心的系统安全机制。

SELinux 的主要作用是强化系统的安全性、保护系统资源免受未经授权的访问和攻击。

SELinux 想要解决的是安全问题，简单的说就是：在 SELinux 安全机制下，指定的进程只能访问策略规则授权给它的特定资源，执行特定的操作，而在规则之外的资源不能访问、不能操作；通过对进程的权限控制，从而避免进程越权操作，进而达到系统安全要求！

相比传统 Linux 中的自主访问控制 (Discretionary Access control, 简称 DAC) 机制, SELinux 是一种强制访问控制 (Mandatory Access control, 简称 MAC) 机制, SELinux 提供了更细粒度、更灵活的安全策略, 以保护系统和数据的安全性, 可以根据不同的场景和需求进行定制, 以满足各种安全需求。

在 SELinux 之前, Linux 采用的是 DAC 权限控制机制, 基本概念就是: 系统只提供基本的权限验证, 完整的访问控制由开发者自己控制 (譬如直接通过 `chmod` 就可以修改权限)。基于用户和组, 进程的权限取决于调用这个进程的用户的权限, 将用户划分为 Owner、Group、Other 三大类, 按照这三类用户对权限进行定义。

- 资源访问者: 系统中运行的各种进程;
- 被访问的资源一般都是文件;
- 资源的访问权限分为 read、write、execute 三种。

如果进程是由 root 超级用户启动, 那么该进程便拥有 root 用户权限, 意味着该进程可以在系统中做任何事情, 这个是比较危险的, 极其不安全!

SELinux 采用 MAC 权限控制机制, 要求系统对任何进程的每一项访问都进行严格的限制和验证, 具体的限制和验证策略 (policy) 由开发者给出, 即需要明确规定某个进程可以访问哪些资源 (Android 系统中的资源类型并不局限于文件, 还包括网络端口、属性等), 可以对这些资源执行哪些操作。

MAC 可以明显弥补 DAC 的缺陷, 譬如限制 root 权限, 即使你拥有 root 权限, 如果无法通过 MAC 验证, 那么同样无法执行相关操作。另外, DAC 的权限控制粒度非常大, 不够细、不够灵活; 而 SELinux MAC 提供了更细粒度、更灵活的安全策略, 以保护系统和数据的安全性, 可以根据不同的场景和需求进行定制, 以满足各种安全需求。



Android 系统并未摒弃传统 Linux 中的 DAC 权限检查机制, 而是在 DAC 的基础上增加了 MAC (SELinux 是 MAC 的一种实现) 权限检查机制。Android 系统中, 进程访问某一资源时, 会先做 DAC 检查, 如果未通过 DAC 权限检查, 则直接操作失败; 通过 DAC 检查之后, 再执行 SELinux MAC 权限检查!

6.3.4.2 关闭 SELinux

本小节介绍如何关闭 SELinux, 关闭 SELinux 的方法有如下几种:

①、通过 `setenforce` 命令关闭

通过 Android 系统命令行终端执行如下命令关闭 SELinux:

```
setenforce 0
```

执行如下命令开启 SELinux:

这种方法只用于临时修改，系统重启或复位后失效！

②、通过 CONFIG_SECURITY_SELINUX 变量控制 SELinux 开启或关闭

打开 device/rockchip/rk3588/ATK_DLRK3588/BoardConfig.mk 文件，添加如下这行：

```
BOARD_SELINUX_ENFORCING := false
```

BOARD_SELINUX_ENFORCING 变量设置为 true 表示开启 SELinux，设置为 false 表示关闭 SELinux。

```
ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
    include device/rockchip/common/BoardConfig_AB.mk
    TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/ATK_DLRK3588/recovery.fstab_AB
endif

PRODUCT_UBOOT_CONFIG := rk3588
PRODUCT_KERNEL_DTS := rk3588-atk-evb7-lp4-v10
BOARD_GSENSOR_MXC6655XA_SUPPORT := true
BOARD_CAMERA_SUPPORT_EXT := true
BOARD_HS_ETHERNET := true
BOARD_INCLUDE_DTB_IN_BOOTIMG := true
PRODUCT_KERNEL_CONFIG := rockchip_defconfig android-13.config atk_dlrk3588.config
BOARD_HAS_RK_4G_MODEM := true
BOARD_HDMI_IN_SUPPORT := true

BOARD_SELINUX_ENFORCING := false
```

修改完成后保存退出，重新编译 Android 源码（建议先执行“make installclean”后再 make 编译），将编译得到的镜像烧录到开发板。

6.3.5 应用预安装功能

Android 系统上的应用预安装功能，主要是指产品配置时，根据产商要求，将事先准备好的第三方应用预制进 Android 系统。

预安装分为：

1. 安装不可卸载应用；
2. 安装可永久卸载应用；
3. 安装卸载后、恢复出厂设置后自动恢复的应用。

①、配置和使用

首先执行“source build/envsetup.sh”初始化编译环境，然后执行 lunch 命令选择产品。

执行 lunch 选择产品后，通过执行“get_build_var TARGET_DEVICE_DIR”命令找到对应的目标设备文件夹（譬如 device/rockchip/rk3588/ATK_DLRK3588）：

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ get_build_var TARGET_DEVICE_DIR
device/rockchip/rk3588/ATK_DLRK3588
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$
```

进入到该目录下，该目录下有 preinstall、preinstall_del 以及 preinstall_del_forever 三个文件夹，如下图所示（如果没有则自己创建）：



```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588$ ls -l
总用量 120
-rw-rw-r-- 1 dengtao dengtao 298 3月 7 18:28 AndroidBoard.mk
-rw-rw-r-- 1 dengtao dengtao 80 3月 7 18:28 Android.mk
-rw-rw-r-- 1 dengtao dengtao 2011 3月 7 18:28 ATK_DLRK3588.mk
drwxrwxr-x 2 dengtao dengtao 4096 3月 7 18:28 bluetooth
-rw-rw-r-- 1 dengtao dengtao 1444 3月 7 18:28 BoardConfig.mk
-rw-rw-r-- 1 dengtao dengtao 152 3月 7 18:28 bt_vendor.conf
-rwxrwxr-x 1 dengtao dengtao 6129 3月 7 18:28 config.cfg
-rw-rw-r-- 1 dengtao dengtao 7959 3月 7 18:28 config.cfg_ab
-rw-rw-r-- 1 dengtao dengtao 11009 3月 7 18:28 config.cfg_ab_gki
-rw-rw-r-- 1 dengtao dengtao 239 3月 7 18:28 dt-overlay.in
-rw-rw-r-- 1 dengtao dengtao 25503 3月 7 18:28 media_profiles_default.xml
drwxrwxr-x 3 dengtao dengtao 4096 3月 7 18:28 ota
drwxrwxr-x 3 dengtao dengtao 4096 3月 7 18:28 overlay
drwxrwxr-x 2 dengtao dengtao 4096 3月 13 12:12 preinstall
drwxrwxr-x 2 dengtao dengtao 4096 3月 13 12:12 preinstall_del
drwxrwxr-x 2 dengtao dengtao 4096 3月 13 12:12 preinstall_del_forever
-rw-rw-r-- 1 dengtao dengtao 3475 3月 7 18:28 recovery.fstab
-rw-rw-r-- 1 dengtao dengtao 2000 3月 7 18:28 recovery.fstab_AB
-rw-rw-r-- 1 dengtao dengtao 5757 3月 7 18:28 resolution_white.xml
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588$
```

这三个文件夹分别为:

preinstall: 存放不可卸载的应用

preinstall_del_forever: 存放可永久卸载的应用

preinstall_del: 存放卸载后、恢复出厂设置可自动恢复的应用

将需要预置的 APK 应用放入对应的文件夹即可, 注意, apk 文件名尽量使用英文, 避免空格。譬如将 Android_DevCheck.apk 预置进 Android 系统, 要求该应用可卸载, 并且卸载后、恢复出厂设置可自动恢复; 所以需要将 APK 放入 preinstall_del/目录, 如下所示:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588$ cd preinstall_del/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588/preinstall_del$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588/preinstall_del$ ls
Android_DevCheck.apk
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/rk3588/ATK_DLRK3588/preinstall_del$
```

②、编译结果

重新编译 Android 源码, 编译完成后, 将会在 \$OUT/odm 目录下生成一个名为 bundled_uninstall_back-app 的文件夹, 如下所示 (OUT 指的是产品输出目录):

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/out/target/product/ATK_DLRK3588/odm$ pwd
/home/dengtao/rk3588_android_sdk/out/target/product/ATK_DLRK3588/odm
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/out/target/product/ATK_DLRK3588/odm$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/out/target/product/ATK_DLRK3588/odm$ ls
bundled_uninstall_back-app  etc  lib
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/out/target/product/ATK_DLRK3588/odm$
```

卸载后、恢复出厂设置可自动恢复的应用将会安装到 odm/bundled_uninstall_back-app 目录, 而 odm 文件夹会被创建为 odm.img (odm.img 合并到 super.img, super.img 烧录到开发板 super 分区, 系统启动后, odm.img 会被挂载到 Android 根文件系统/odm 目录)。

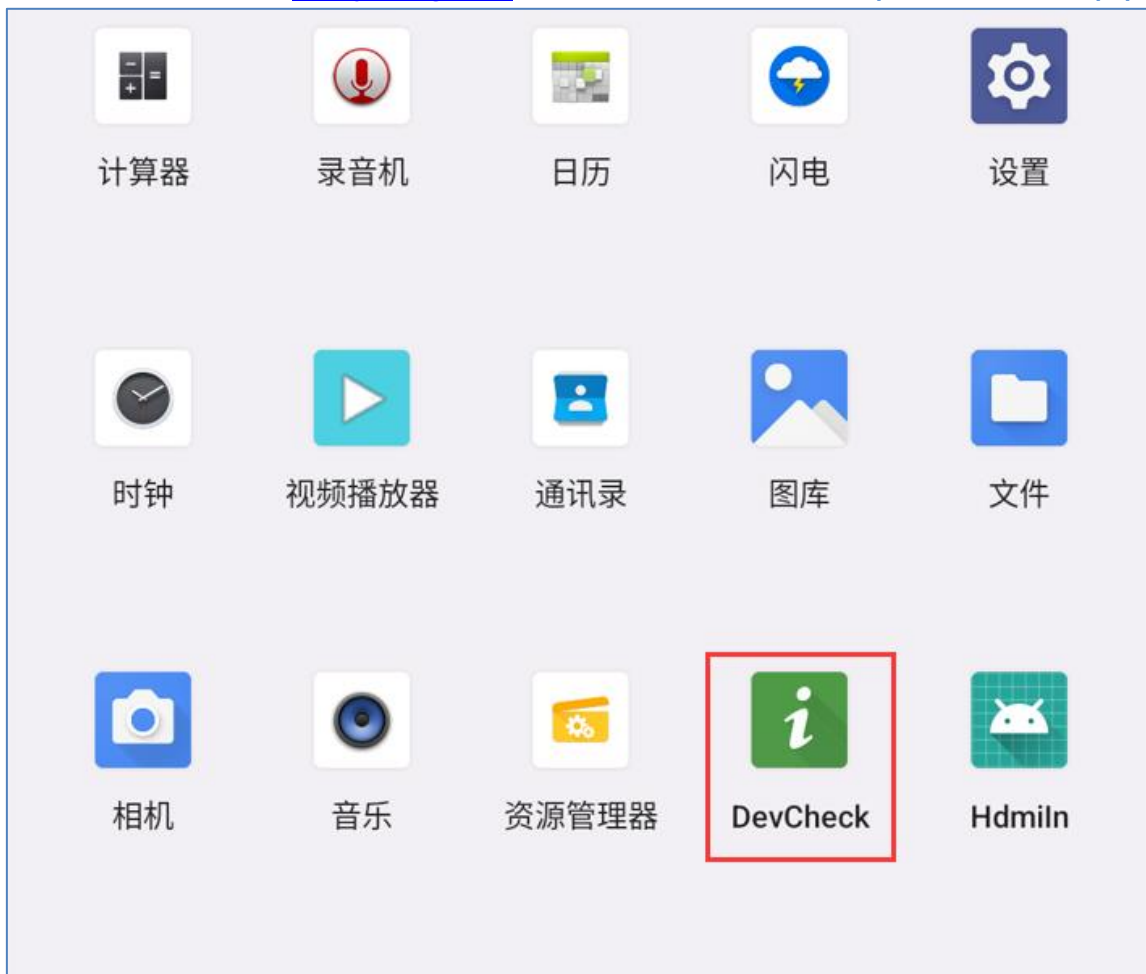
除了 bundled_uninstall_back-app 目录之外, 还有如下两个目录:

bundled_uninstall_gone-app: 可永久卸载的应用安装到该目录下;

bundled_persist-app: 不可卸载的应用安装到该目录下。

③、烧录镜像、启动 Android 系统

重新烧写镜像, 然后启动 Android 系统, 进入系统后便会发现预置的 Android_DevCheck 应用, 如下所示:



6.3.6 修改默认语言和时区信息

修改系统默认的语言和时区信息，主要是修改如下两个属性：

```
persist.sys.locale
persist.sys.timezone
```

第一个属性 `persist.sys.locale` 表示当前系统语言，第二个属性 `persist.sys.timezone` 表示当前系统时区。

打开 `device/rockchip/rk3588/rk3588.prop` 属性配置文件，添加如下两行内容：

```
persist.sys.locale=zh-Hans-CN
persist.sys.timezone=Asia/Shanghai

persist.sys.locale=zh-Hans-CN
persist.sys.timezone=Asia/Shanghai
~
~
```

zh-Hans-CN：表示简体中文，譬如 `en-US` 表示美国英语、`ja-JP` 表示日本日语、`ko-KR` 表示韩国韩语、`de-DE` 表示德国德语、`ru-RU` 俄罗斯俄语等。

Asia/Shanghai：表示中国上海（所在时区），譬如 `America/New_York` 表示美国纽约、`Europe/London` 表示欧洲伦敦、`Asia/Jerusalem` 表示亚洲热路撒冷、`Asia/Dubai` 表示亚洲迪拜、`Asia/Hong_Kong` 表示中国香港、`Asia/Tokyo` 日本东京、`Asia/Seoul` 韩国首尔等。

修改完成后保存退出，重新编译 Android 源码（最好先执行“`make installclean`”后再 `make` 编译），将编译得到的镜像烧录到开发板。

6.3.7 设置系统默认不锁屏、不熄屏

修改文件:

device/rockchip/rk3588/overlay/frameworks/base/packages/SettingsProvider/res/values/defaults.xml

打开该文件, 添加如下信息:

```
<integer name="def_screen_off_timeout">0x7fffffff</integer>
<bool name="def_lockscreen_disabled">true</bool>
```

def_screen_off_timeout 参数表示息屏时间, 单位为 ms(毫秒), 将其修改为最大值 0x7fffffff, 表示永不熄屏。

def_lockscreen_disabled 参数用于控制是否锁屏, 如果设置为 true 表示不会锁屏, 如果设置为 false, 表示会锁屏。

```
18 -->
19 <resources>
20     <integer name="def_screen_off_timeout">0x7fffffff</integer>
21     <bool name="def_lockscreen_disabled">true</bool>
22     <!-- Initial value for the Settings.Secure.IMMERSIVE_MODE_CONFIRMATIONS setting,
23          which is a comma separated list of packages that no longer need confirmation
24          for immersive mode.
25          Override to disable immersive mode confirmation for certain packages. -->
```

修改完成后保存退出, 重新编译 Android 源码 (最好先执行 “make installclean” 后再 make 编译), 将编译得到的镜像烧录到开发板。

6.3.8 修改默认背光亮度

修改文件:

device/rockchip/common/overlay/frameworks/base/core/res/res/values/config.xml

找到该文件中的 config_screenBrightnessSettingDefault 参数, 如下所示:

```
20 <resources>
21
22 <!-- This device is not "voice capable"; it's data-only. -->
23 <bool name="config_voice_capable">false</bool>
24
25 <!-- The name of the package that will hold the SMS role by default. -->
26 <!-- Set it empty here -->
27 <string name="config_defaultSms" translatable="false"></string>
28
29 <!-- This device does not allow sms service. -->
30 <bool name="config_sms_capable">false</bool>
31
32 <integer name="config_screenBrightnessSettingDefault">255</integer>
33
34 <!-- If this is true, the screen will come on when you unplug usb/power/whatever. -->
35 <bool name="config_unplugTurnsOnScreen">true</bool>
36
```

config_screenBrightnessSettingDefault 参数表示屏幕亮度, 取值为 0~255, 默认为最大亮度 255, 用户可根据自己的需求进行修改。

修改完成后保存退出, 重新编译 Android 源码 (最好先执行 “make installclean” 后再 make 编译), 将编译得到的镜像烧录到开发板。

6.3.9 修改启动 logo

启动 logo 包括 u-boot 阶段 logo 以及内核阶段 logo, 这两个 logo 文件都存放在内核源码目录下:

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk$ cd kernel-5.10/
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/kernel-5.10$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/kernel-5.10$ ls -l *.bmp
-rw-rw-r-- 1 dengtao dengtao 177760 3月  7 21:33 logo.bmp
-rw-rw-r-- 1 dengtao dengtao 177760 3月  7 21:33 logo_kernel.bmp
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/kernel-5.10$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/kernel-5.10$
```

logo.bmp 是 U-Boot 阶段 logo 图片, logo_kernel.bmp 是内核阶段 logo 图片, 都是 bmp 图片; 正点原子出厂系统默认启动 logo 如下所示:



U-Boot logo



Kernel logo

如果需要替换默认 logo，首先将 U-Boot logo 图片重命名为 logo.bmp、将内核 logo 图片重命名为 logo_kernel.bmp，然后将这两个 bmp 图片文件拷贝到内核源码目录下，完成替换，然后重新编译内核。

编译内核时，会将 logo.bmp 和 logo_kernel.bmp 打包到 resource.img 中，然后再把 resource.img 打包到 boot.img。

U-Boot 启动的时候会把这两个 bmp 文件加载到内存中，logo.bmp 会在 U-Boot 阶段开始显示，logo_kernel.bmp 在内存中的地址会被 U-Boot 传给 Linux Kernel，在 Linux Kernel 的 DRM 驱动初始化阶段显示。

6.3.10 修改开机动画

如果用户需要修改 Android 系统默认的开机动画，可以参考本小节内容进行修改。

Android 的开机动画是由一系列连续的 PNG 图片作为帧组成的动画形式，不是一张 GIF 图片。将各帧 PNG 图片以压缩方式进行保存（压缩方式要求是存储压缩），并将保存的文件命名为 bootanimation.zip，这个 bootanimation.zip 压缩文件就是开机动画文件。

①、制作开机动画文件 bootanimation.zip

首先准备一组连续的 PNG 图片（可以将视频的每一帧导出生成一组连续的 PNG 图片，譬如通过格式工厂软件就可以进行导出；当然也可以自己手动制作每一帧对应的 PNG 图片），将使用这组图片做成开机动画。

将这组连续的 PNG 图片按照显示顺序以数字形式进行命名, 如果是 100 张 PNG 图片, 则可将这组图片命名为 0001.png~0100.png, 当然也可以是 001.png~100.png, 只需保证文件名对应的数字个数相同即可!

在 Windows 下创建一个名为 bootanimation 的文件夹, 并在该目录下创建一个或多个命名为 partN (N 表示数字, 譬如 0、1、2、3 等) 的目录, 开机动画可以分为多个阶段, 每个 partN 目录用于存放不同阶段所要播放的 PNG 图片, 每个 partN 目录下都可以存放一部分图片 (一张或多张图片):

第一阶段动画图片存放在 part0 目录;

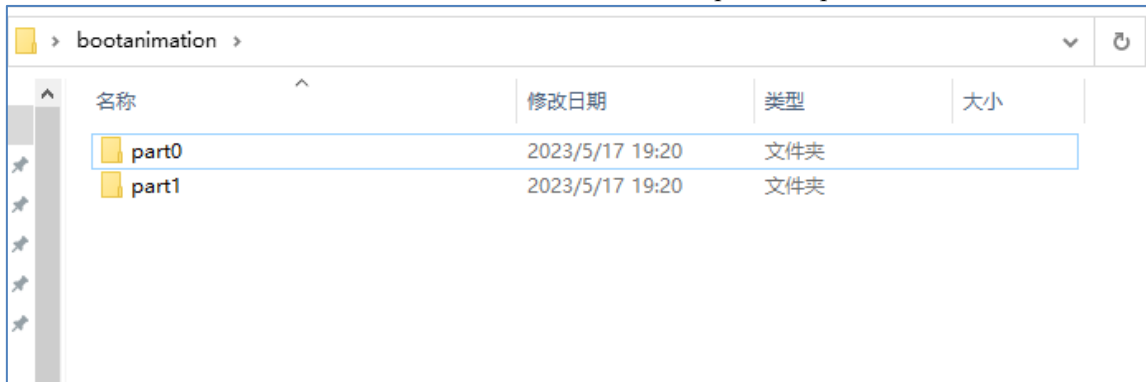
第二阶段动画图片存放在 part1 目录;

第三阶段动画图片存放在 part2 目录;

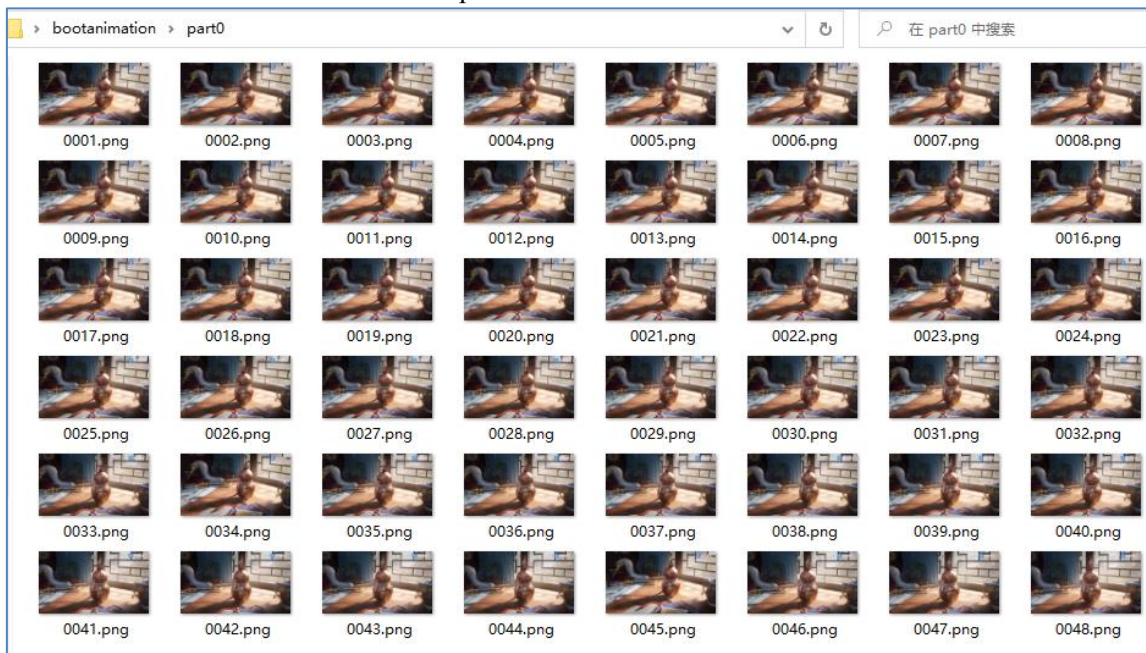
等以此类推!

如果只是循环播放所有的图片, 只需创建 part0 即可; 如果需要先播放一部分图片, 然后再循环播放另一部分图片, 也就是说有两个阶段; 那么这就需要创建 part0 和 part1 两个文件夹, part0 目录存放第一阶段需要播放的图片, part1 目录用于存放第二阶段需要播放的图片。

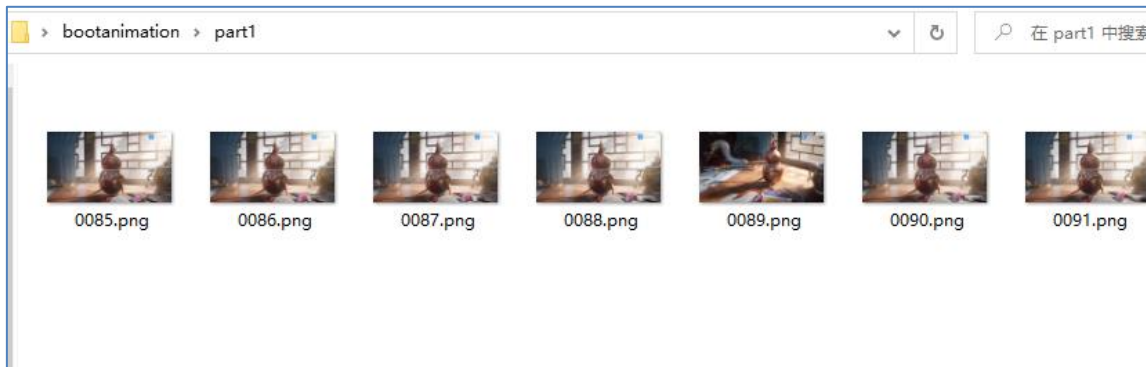
假设我们要制作包含两个阶段的开机动画, 首先创建 part0 和 part1 两个目录, 如下所示:



将第一阶段需要播放的图片放入 part0 目录, 如下:



本例中一共有 84 张 PNG 图片放置在 part0 目录; 然后将第二阶段需要播放的图片放入 part1 目录, 如下:



本例中一共有 7 张图片放置在 part1 目录。

接下来我们需要编写一个 desc.txt 文件，该文件称为动画属性描述文件。desc.txt 文件用于设置动画像素大小、帧数、循环次数以及各阶段对应的文件夹等信息。desc.txt 文件的内容如下所示：

```
480 427 20
p 1 0 part0
p 0 10 part1
```

480、427 这两个数字代表图片的像素宽度和高度；

20 表示帧数，也就是 1 秒钟播放图片的数量；20 也就表示一秒钟播放 20 张图片；

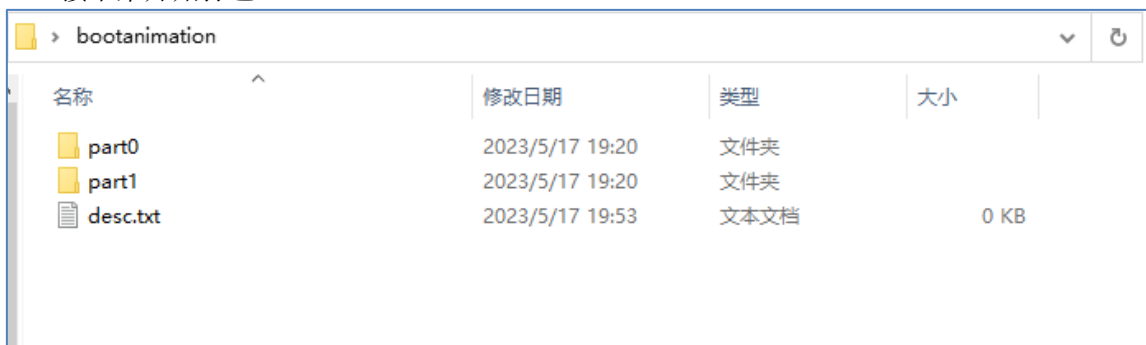
p 1 0 part0：这行中，第一位字母 p 表示播放的意思，第二个数字 1 表示本阶段播放的次数（本阶段播放一次），也就是本阶段循环的次数；第三个数字 0 表示间隔时间，譬如在循环播放的情况下，播放完一轮后、需要间隔多少时间重新播放下一轮，单位是一个帧的持续时间，譬如 20 帧，那么一帧的持续时间就是 $1 \text{ 秒} / 20 = 50\text{ms}$ 。最后一个参数 part0 表示本阶段播放的图片存放在 port0 目录。

p 0 10 part1：与第二行所表示的含义是相同的，第一个字母 p 表示播放的意思，第二个数字 0 表示本阶段循环的次数（0 表示本阶段无限循环）；第三个数字 10 表示间隔时间；最后一个参数 part1 表示本阶段播放的图片存放在 port1 目录。

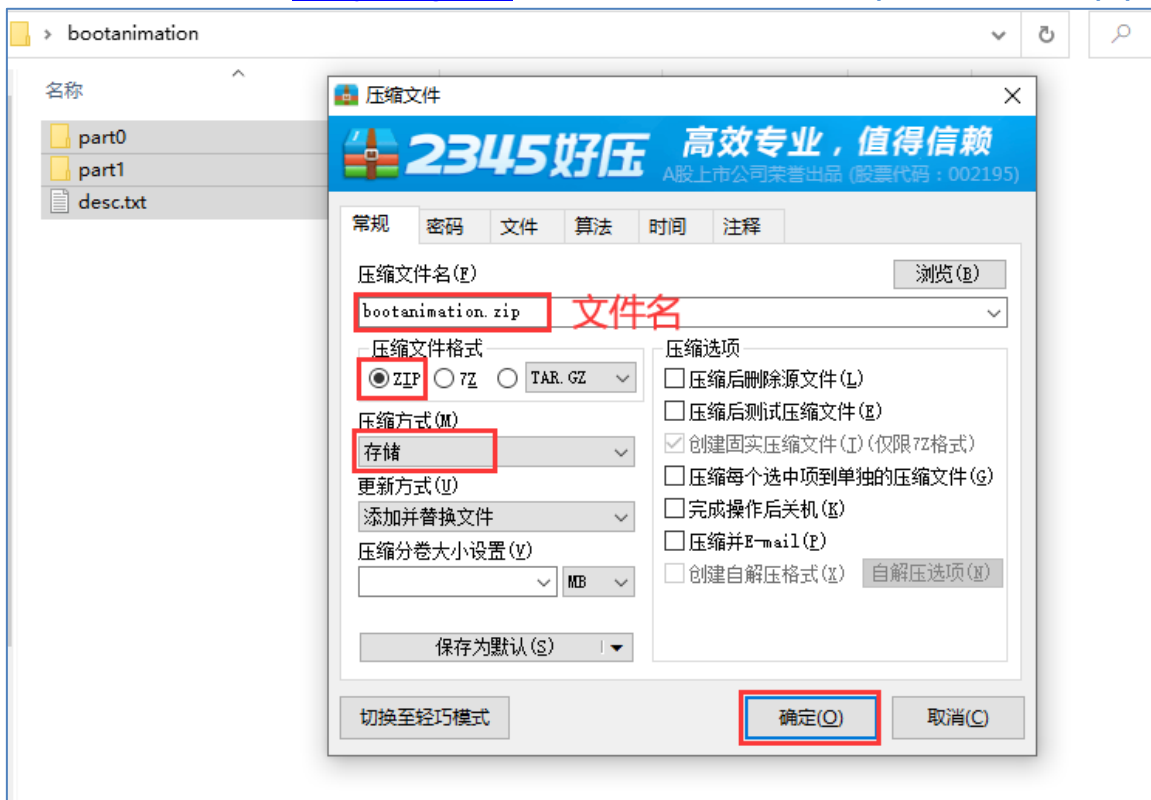
我们来看一下整个动画属性描述文件 desc.txt 所表示的含义：

首先以每秒钟 20 张的帧率播放一次 part0 目录下的所有图片，然后再以每秒钟 20 张的帧率无线循环播放 part1 目录下所有图片。

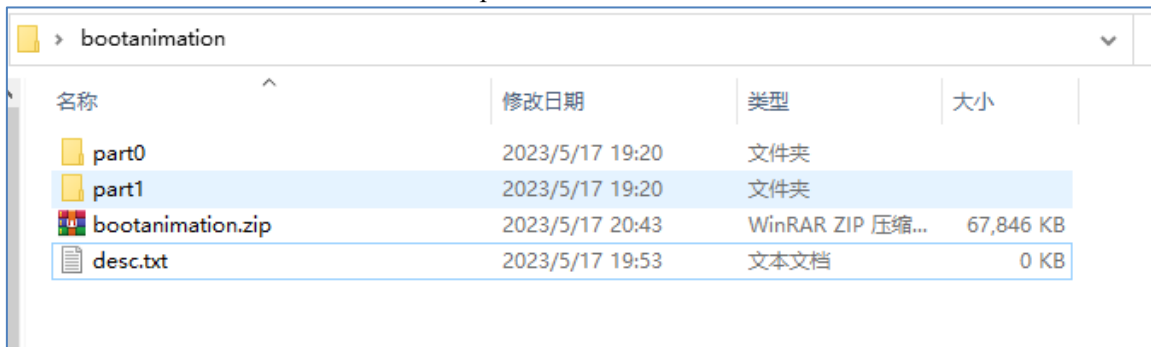
接下来开始打包：



同时选中 part0、part1 以及 desc.txt，然后鼠标右键单击选择“添加到压缩文件”，压缩方式选择“存储”，如下所示：



这样开机动画文件 bootanimation.zip 就制作好了。



②、产品配置

将制作好的 bootanimation.zip 文件拷贝到 RK3588 Android 源码 device/rockchip/common/bootshutdown/目录下，如下所示：

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$ pwd
/home/dengtao/rk3588_android_sdk/device/rockchip/common/bootshutdown
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$ ls
bootanimation.zip bootshutdown.mk
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$
```

并且将 bootanimation.zip 文件的权限设置为 755，如下：

```
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$ chmod 755 bootanimation.zip
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$ ls -l
总用量 5712
-rwxr-xr-x 1 dengtao dengtao 5843557 3月 13 14:24 bootanimation.zip
-rwxrwxr-x 1 dengtao dengtao 520 3月 7 17:15 bootshutdown.mk
dengtao@dengtao-virtual-machine:~/rk3588_android_sdk/device/rockchip/common/bootshutdown$
```

接下来打开板级配置文件 device/rockchip/rk3588/ATK_DLRK3588/BoardConfig.mk，添加下面这一行内容：

BOOT_SHUTDOWN_ANIMATION_RINGING := true

```
include device/rockchip/rk3588/BoardConfig.mk
BUILD_WITH_GO_OPT := false

# AB image definition
BOARD_USES_AB_IMAGE := false
BOARD_ROCKCHIP_VIRTUAL_AB_ENABLE := false

BOARD_GRAVITY_SENSOR_SUPPORT := true
BOARD_GYROSCOPE_SENSOR_SUPPORT := true

BOOT_SHUTDOWN_ANIMATION_RINGING := true

ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
    include device/rockchip/common/BoardConfig_AB.mk
    TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/ATK_DLRK3588/recovery.fstab_AB
endif

PRODUCT_UBOOT_CONFIG := rk3588
PRODUCT_KERNEL_DTS := rk3588-atk-evb7-lp4-v10
```

修改完成后保存退出，然后重新编译 Android 源码（建议先执行“make installclean”命令后再 make 编译），然后将编译得到的镜像烧录到开发板。

③、编译 Android 源码、烧录镜像

将编译得到的镜像烧录到开发板，重新启动开发板、进入 Android 系统后发现开机动画已经更改。

RK 参考文档：

RKDocs/android/Rockchip_Introduction_Android_Power_On_Off_Animation_and_Tone_Customization_CN&EN.pdf

6.3.11 修改主屏/副屏

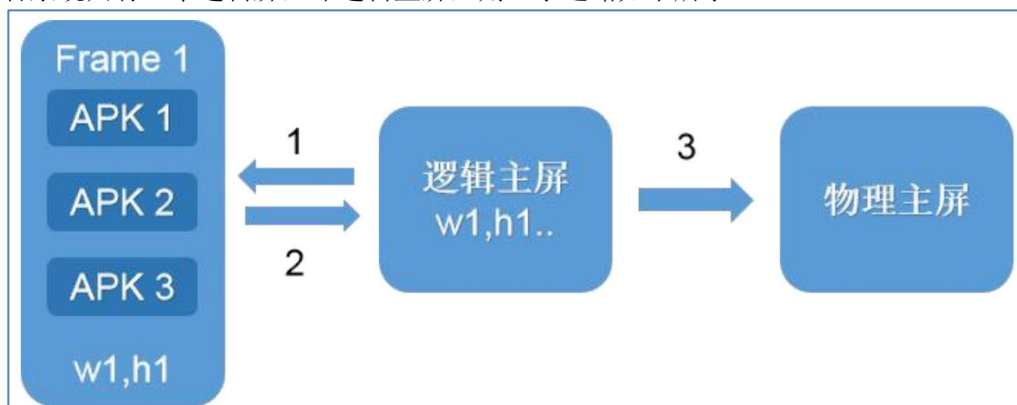
RK 参考文档：

<SDK>/RKDocs/common/display/Rockchip_Developer_Guide_Dual_Display_Rotation_Direction_Debugging_CN.pdf

Android 系统逻辑屏分为主屏、副屏以及虚拟屏（暂不考虑），对应到实际的物理屏幕；主屏只有一个，而副屏可以有多个。系统中的逻辑屏属性与显示通路会根据显示需求而存在差异，譬如多屏同显与多屏异显。

①、只有一个逻辑屏

若系统只有一个逻辑屏，即逻辑主屏，则显示通路如下所示：

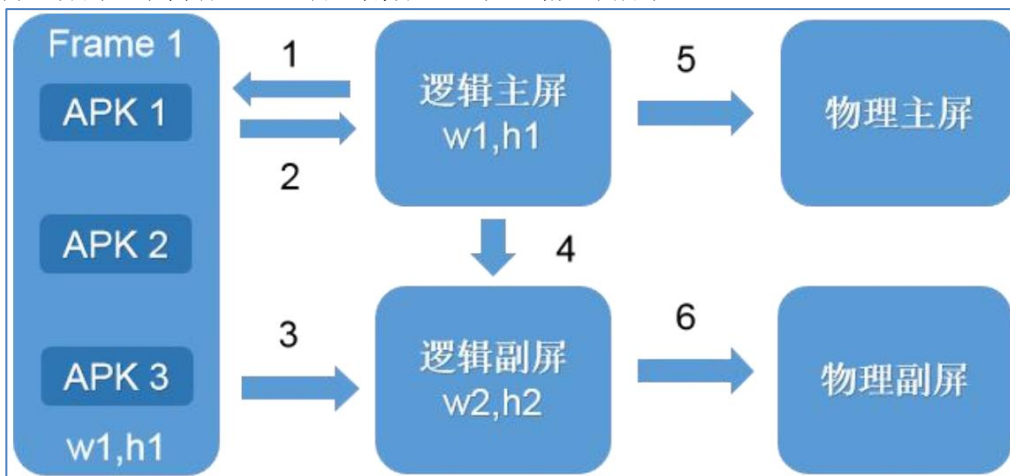


- 1: APK 获取逻辑主屏的属性信息（宽/高/刷新率），作为 APK 的渲染布局；
- 2: 多个 APK 将显示请求提交给逻辑主屏，主屏根据层级结构进行混合；
- 3: 逻辑主屏将混合的结果送显至实际的物理主屏，默认情况下，逻辑主屏的属性信息与物理主屏一致；

备注：由于 APK 通过主屏的属性信息进行渲染绘图的，所以在只有一个逻辑主屏的情况下，显示效果是正常的，不存在拉伸等情况。

②、多屏同显

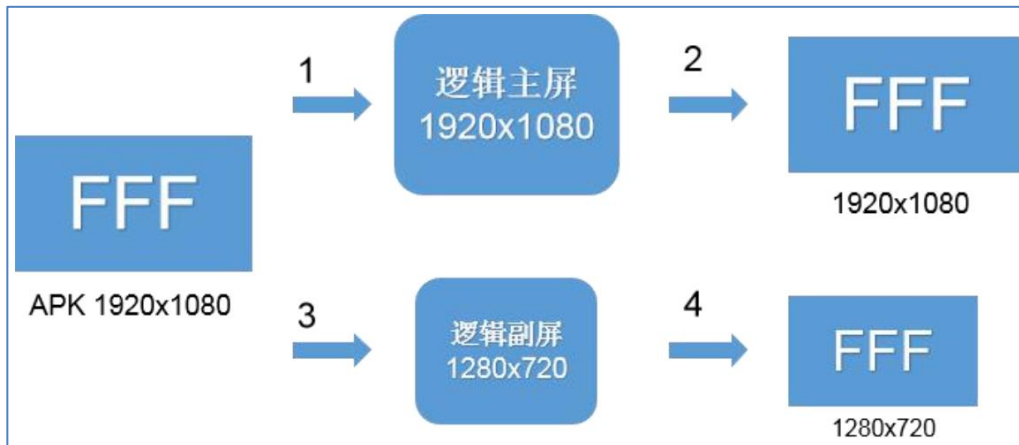
多屏同显，譬如 RK3588 平台支持双屏同显、三屏同显以及四屏同显，实际上是逻辑副屏将逻辑主屏的显示内容送显至物理副屏，显示通路如下所示：



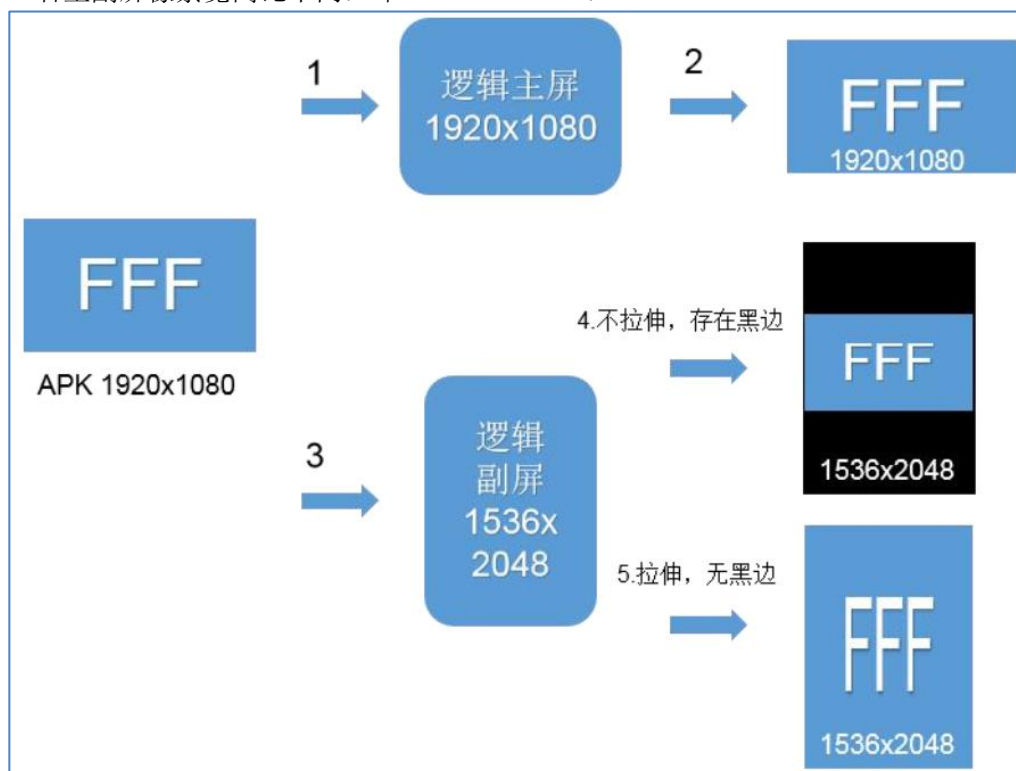
- 1: APK 获取 逻辑主屏 的属性信息（宽/高/刷新率），作为 APK 的渲染布局；
- 2: 多个 APK 将显示请求提交给 逻辑主屏，主屏根据层级结构进行混合；
- 3: 多个 APK 将显示请求提交给 逻辑副屏，副屏根据层级结构进行混合；
- 4: 3 步骤其实可以等效为逻辑主屏将显示内容提交给逻辑副屏；
- 5: 逻辑主屏 将混合的结果送显至实际的物理主屏，默认情况下，逻辑主屏的属性信息与物理主屏一致；
- 6: 逻辑副屏 将混合的结果送显至实际的物理副屏；

多屏同显的情况下，逻辑副屏就有可能出现黑边或拉伸的情况，具体显示情况分为如下两种：

- 主副屏像素宽高比相同：即 $w1/h1 = w2/h2$ ：

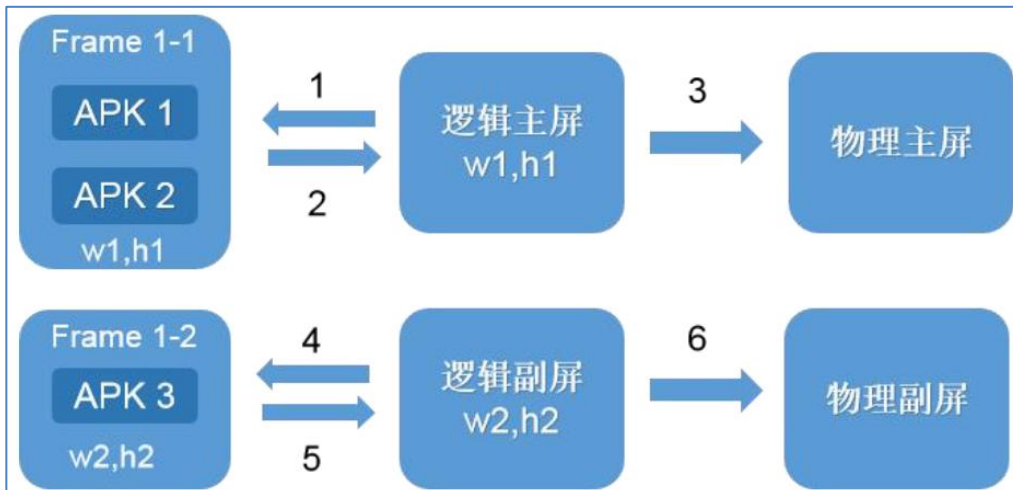


- 若主副屏像素宽高比不同：即 $w1/h1 \neq w2/h2$ ：



③、多屏异显

多屏异显，实际上是 APK 根据逻辑屏的属性信息独立绘制，并独立送显，区别于多屏同显：



- 1: APK1, APK2 获取 逻辑主屏 的属性信息（宽/高/刷新率），作为 APK1,APK2 的渲染布局；
- 2: APK1, APK2 将显示请求提交给 逻辑主屏，主屏根据层级结构进行混合；
- 3: 逻辑主屏 将混合的结果送显至实际的物理主屏，默认情况下，逻辑主屏的属性信息与物理主屏一致；
- 4: APK3 获取 逻辑副屏 的属性信息（宽/高/刷新率），作为 APK3 的渲染布局；
- 5: APK3 将显示请求提交给 逻辑副屏，副屏根据层级结构进行混合；
- 6: 逻辑副屏 将混合的结果送显至实际的物理副屏，默认情况下，逻辑副屏的属性信息与物理副屏一致；

多屏异显由于 APK 送显内容为独立绘制，故 APK 渲染布局始终与需要显示的物理屏幕属性一致，故不存在拉伸或黑边的情况，所有屏幕均正常显示，并且显示的内容取决于各自的 APK。

④、物理横屏与物理竖屏

- 物理横屏:



即实际屏幕宽度大于高度的屏幕, 例如 1920x1080, 1280x720 等。

- 物理竖屏:



即实际屏幕高度大于宽度的屏幕, 例如 1536x2048, 1080x1920 等。

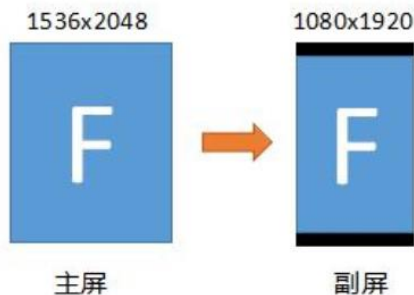
⑤、属性介绍

接下来介绍四个属性: **persist.sys.rotation.efull-1**、**persist.sys.rotation.einit-1**、**vendor.hwc.device.primary**、**vendor.hwc.device.extend**。用户可根据实际项目需求添加这些属性。

- **persist.sys.rotation.efull-1** 属性用于控制副屏是否全屏显示, 可取值为 **false** 和 **true**:

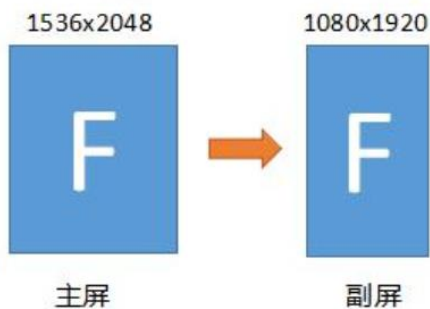
false:

副屏图像参考主屏宽高比进行缩放, 若宽高比不一致, 则出现黑边。

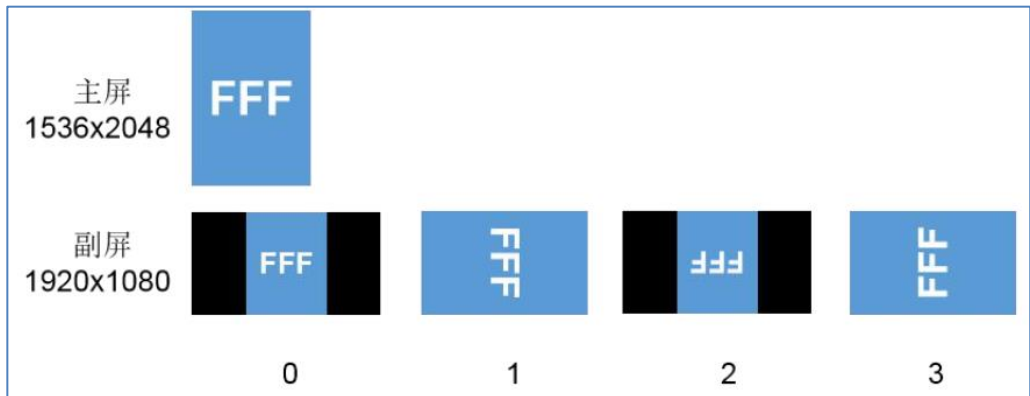


true:

副屏图像强制全屏显示, 凸显出现拉伸。



● persist.sys.rotation.einit-1 属性用于设置副屏的显示方向，可取值为 0/1/2/3，分别表示将副屏显示方向旋转（顺时针）0 度、90 度、180 度、270 度。



● vendor.hwc.device.primary 属性用于设置主屏，譬如：

vendor.hwc.device.primary=DSI	#设置 MIPI 屏为主屏
vendor.hwc.device.primary=HDMI-A	#设置 HDMI 屏为主屏
vendor.hwc.device.primary=DP	#设置 DP 屏为主屏

● vendor.hwc.device.extend 属性用于设置副屏，譬如：

vendor.hwc.device.extend=DSI	#设置 MIPI 屏为副屏
vendor.hwc.device.extend=HDMI-A	#设置 HDMI 屏为副屏
vendor.hwc.device.extend=DP	#设置 DP 屏为副屏

6.3.12 修改屏幕方向

正点原子 ATK-DLRK3588 开发板支持多种 LCD 屏，包括 5.5 寸 MIPI 屏（720p 和 1080p）、10.1 寸 800x1280 MIPI 屏、DP 显示屏以及 HDMI 显示屏。正点原子 5.5/10.1 寸 MIPI 屏是物理竖屏（像素宽度小于像素高度），DP、HDMI 显示器一般都是物理横屏。

Android 系统既支持横屏显示、也支持竖屏显示。在开发过程中，我们使用的屏幕可能是物理横屏或物理竖屏，如果想要将物理横屏修改为竖屏显示或者将物理竖屏修改为横屏显示，譬如将 MIPI 屏修改为横屏显示、或者将 DP/HDMI 显示屏修改为竖屏显示，那么这种情况就需要在软件上对其进行配置、修改屏幕方向，将显示方向旋转 90 度或 270 度。

本小节我们以 5.5 寸 MIPI 屏为例，向用户介绍如何将其修改为横屏显示，分为如下两部分内容：

- 1、MIPI 屏作为主屏时如何旋转显示方向；
- 2、MIPI 屏作为副屏时如何旋转显示方向；

6.3.12.1 修改主屏显示方向

①、选择 LCD 屏

进入内核源码目录，打开 arch/arm64/boot/dts/rockchip/rk3588-atk-screen_choose.dtsi 设备树文件，选择 LCD 屏，如下所示：



```
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280
#define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280
// #define DSI_CONNECTOR_MIRROR_MODE
// #define ATK_LCD_TYPE_HDMI_TX0
// #define ATK_LCD_TYPE_HDMI_TX1
// #define ATK_LCD_TYPE_DP_TX0
// #define DP_CONNECTOR_MIRROR_MODE
```

我们以 MIPI DSI0 接口连接 5.5 寸 1080p MIPI 屏为例，只需定义 ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920 宏即可！修改完成后保存退出。

②、配置 Android 系统主屏显示方向

回到 Android 源码根目录，打开板级配置文件 device/rockchip/rk3588/BoardConfig.mk，找到 SF_PRIMARY_DISPLAY_ORIENTATION 变量，将其修改为 90，如下所示：

```
SF_PRIMARY_DISPLAY_ORIENTATION := 90
```

```
21 PRODUCT_UBOOT_CONFIG ?= rk3588
22 PRODUCT_KERNEL_ARCH ?= arm64
23 PRODUCT_KERNEL_DTS ?= rk3588-evb1-lp4-v10
24 PRODUCT_KERNEL_CONFIG ?= rockchip_defconfig pcie_wifi.config
25
26 #BOARD_AVB_ENABLE := true
27 # used for fstab_generator, sdmmc controller address
28 PRODUCT_BOOT_DEVICE := fe2e0000.mmc
29 PRODUCT_SDMMC_DEVICE := fe2c0000.mmc
30
31 SF_PRIMARY_DISPLAY_ORIENTATION := 90
32
33 # Disable emulator for "make dist" until there is a 64-bit qemu kernel
34 BUILD_EMULATOR := false
35 TARGET_BOARD_PLATFORM := rk3588
36 TARGET_BOARD_PLATFORM_GPU := mali-G610
37 TARGET_RK_GRALLOC_VERSION := 4
38 BOARD_USE_DRM := true
39
```

该变量可取值为 0/90/180/270，分别表示将主屏显示方向旋转（顺时针）0 度、90 度、180 度以及 270 度。

修改完成后保存退出。

③、将 MIPI 屏设置为主屏

打开 device/rockchip/rk3588/ATK_DLRK3588/ATK_DLRK3588.mk 产品配置文件，找到下面两行：

```
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.primary=DSI
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.extend=HDMI-A,eDP
```

```
54 PRODUCT_PROPERTY_OVERRIDES += ro.sf.lcd_density=320
55 PRODUCT_PROPERTY_OVERRIDES += ro.wifi.sleep.power.down=true
56 PRODUCT_PROPERTY_OVERRIDES += persist.wifi.sleep.delay.ms=0
57 PRODUCT_PROPERTY_OVERRIDES += persist.bt.power.down=true
58 PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.primary=DSI
59 PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.extend=HDMI-A,eDP
60 PRODUCT_PROPERTY_OVERRIDES += sys.mouse.presentation=1
```

vendor.hwc.device.primary=DSI 表示将 MIPI 屏设置为主屏。

vendor.hwc.device.extend=HDMI-A,eDP 表示将 HDMI 显示屏和 eDP 显示屏作为副屏。

这一步可以不用做，如果只使能了一个屏（MIPI 屏），该屏默认就是主屏。

修改完成后保存退出。

④、配置触摸方向

进入到内核源码目录, 打开 arch/arm64/boot/dts/rockchip/rk3588-atk-lcds.dtsi 设备树文件, 找到触摸屏设备节点, 如下所示:

```

155 /***** 触摸屏 *****/
156 &i2c5 {
157     dsi0_touch: gt911@14 {
158         compatible = "goodix,gt9xx";
159         reg = <0x14>;
160         pinctrl-names = "default";
161         pinctrl-0 = <&touch0_gpio>;
162         touch-gpio = <&gpio3 RK_PC0 IRQ_TYPE_LEVEL_LOW>;
163         reset-gpio = <&gpio3 RK_PC1 GPIO_ACTIVE_HIGH>;
164         max-x = <1080>;
165         max-y = <1920>;
166         tp-size = <911>;
167         tp-supply = <&vcc5v0_sys>;
168         wakeup-source;
169         goodix-ts-name = "dsi0_ts_gt9xx";
170         status = "disabled";
171     };
172 };
173
174 &i2c6 {
175     dsi1_touch: gt911@14 {
176         compatible = "goodix,gt9xx";
177         reg = <0x14>;
178         pinctrl-names = "default";
179         pinctrl-0 = <&touch1_gpio>;
180         touch-gpio = <&gpio3 RK_PD5 IRQ_TYPE_LEVEL_LOW>;
181         reset-gpio = <&gpio3 RK_PA6 GPIO_ACTIVE_HIGH>;
182         max-x = <1080>;
183         max-y = <1920>;
184         tp-size = <911>;
185         tp-supply = <&vcc5v0_sys>;
186         wakeup-source;
187         goodix-ts-name = "dsi1_ts_gt9xx";
188         status = "disabled";
189     };
190 };
191 /***** END *****/

```

MIPI DSI0对应的触摸屏节点

MIPI DSI1对应的触摸屏节点

将下面两个属性添加到 MIPI DSI0 对应的触摸屏设备节点中:

```

touchscreen-inverted-x;
touchscreen-swapped-x-y;

```

```

/***** 触摸屏 *****/
&i2c5 {
    dsi0_touch: gt911@14 {
        compatible = "goodix,gt9xx";
        reg = <0x14>;
        pinctrl-names = "default";
        pinctrl-0 = <&touch0_gpio>;
        touch-gpio = <&gpio3 RK_PC0 IRQ_TYPE_LEVEL_LOW>;
        reset-gpio = <&gpio3 RK_PC1 GPIO_ACTIVE_HIGH>;
        max-x = <1080>;
        max-y = <1920>;
        tp-size = <911>;
        tp-supply = <&vcc5v0_sys>;
        wakeup-source;
        goodix-ts-name = "dsi0_ts_gt9xx";
        status = "disabled";
        touchscreen-inverted-x;
        touchscreen-swapped-x-y;
    };
};

```

第一个属性 `touchscreen-inverted-x` 表示将 X 轴坐标进行反转 (反向) 处理, 如果需要将 Y 轴坐标进行反转处理, 则需要定义 `touchscreen-inverted-y` 属性。

第二个属性 `touchscreen-swapped-x-y` 表示将 X 轴和 Y 轴进行交换。

不同的旋转角度, X 轴和 Y 轴的处理方式是不同的, 这个需要你自己去理解, 用手比划下大概就懂了, 没什么可说的。

修改完成后保存退出。

⑤、编译、烧录镜像

切换到 Android 源码根目录，执行如下命令编译 Android 源码、内核源码：

```
make installclean -j16
./build.sh -KAu -J16
```

编译完成后，将 rockdev/Image-ATK_DLRK3588/目录下的镜像烧写到开发板。

6.3.12.2 修改副屏显示方向

同样也按照①~⑤步骤进行配置。

①、选择 LCD 屏

进入内核源码目录，打开 arch/arm64/boot/dts/rockchip/rk3588-atk-screen_choose.dtsi 设备树文件，选择 LCD 屏，如下所示：

```
// #define ATK_LCD_TYPE_MIPI_TX0_5P5_720X1280
#define ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920
// #define ATK_LCD_TYPE_MIPI_TX0_10P1_800X1280
// #define DSI_CONNECTOR_MIRROR_MODE
#define ATK_LCD_TYPE_HDMI_TX0
// #define ATK_LCD_TYPE_HDMI_TX1
// #define ATK_LCD_TYPE_DP_TX0
// #define DP_CONNECTOR_MIRROR_MODE
```

这里我们以 HDMI_TX0 和 MIPI DSI0 为例，HDMI_TX0 作为主屏、MIPI DSI0 连接 5.5 寸 1080p MIPI 屏作为副屏，需要定义 ATK_LCD_TYPE_MIPI_TX0_5P5_1080X1920 和 ATK_LCD_TYPE_HDMI_TX0 两个宏！修改完成后保存退出。

②、配置 Android 系统副屏显示方向

回到 Android 源码根目录，打开 device/rockchip/rk3588/rk3588.prop 属性配置文件，添加如下内容：

```
persist.sys.rotation.efull-1=false
persist.sys.rotation.einit-1=1
```

```
persist.sys.locale=zh-Hans-CN
persist.sys.timezone=Asia/Shanghai

persist.sys.rotation.efull-1=false
persist.sys.rotation.einit-1=1
~
```

persist.sys.rotation.einit-1 属性用于设置副屏的显示方向，可取值为 0、1、2、3，分别表示将副屏显示方向旋转（顺时针）0 度、90 度、180 度以及 270 度。

persist.sys.rotation.efull-1 属性在<6.3.11-修改主屏/副屏>小节中有介绍，这里不再多说！修改完成后保存退出。

③、将 MIPI 屏设置为副屏

打开 device/rockchip/rk3588/ATK_DLRK3588/ATK_DLRK3588.mk 文件，修改 vendor.hwc.device.primary 和 vendor.hwc.device.extend 这两个属性的内容，修改完成后如下所示：

```
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.primary=HDMI-A
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.extend=DSI
```



```
PRODUCT_PROPERTY_OVERRIDES += ro.sf.lcd_density=320
PRODUCT_PROPERTY_OVERRIDES += ro.wifi.sleep.power.down=true
PRODUCT_PROPERTY_OVERRIDES += persist.wifi.sleep.delay.ms=0
PRODUCT_PROPERTY_OVERRIDES += persist.bt.power.down=true
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.primary=HDMI-A
PRODUCT_PROPERTY_OVERRIDES += vendor.hwc.device.extend=DSI
```

vendor.hwc.device.primary=HDMI-A 表示将 HDMI 屏作为主屏；
 vendor.hwc.device.extend=DSI 表示将 DSI（MIPI 屏）作为副屏。
 修改完成后保存退出。

④、编译、烧录镜像

参考 6.3.12.1 小节中的第⑤点。

6.3.12.3 recovery 模式屏幕方向旋转

如果需要修改 recovery 模式下屏幕方向，可打开 device/rockchip/rk3588/ATK_DLRK3588/BoardConfig.mk 文件，添加 TARGET_RECOVERY_DEFAULT_ROTATION 变量，如下所示：

```
include device/rockchip/rk3588/BoardConfig.mk
BUILD_WITH_GO_OPT := false

# AB image definition
BOARD_USES_AB_IMAGE := false
BOARD_ROCKCHIP_VIRTUAL_AB_ENABLE := false

BOARD_GRAVITY_SENSOR_SUPPORT := true
BOARD_GYROSCOPE_SENSOR_SUPPORT := true

TARGET_RECOVERY_DEFAULT_ROTATION := ROTATION_NONE

ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
  include device/rockchip/common/BoardConfig_AB.mk
  TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/ATK_DLRK3588/recovery.fstab_AB
endif
```

可通过 TARGET_RECOVERY_DEFAULT_ROTATION 变量指定 recovery 模式下屏幕显示方向，默认值为 ROTATION_NONE 表示不旋转显示方向；可取值为：ROTATION_NONE、ROTATION_RIGHT、ROTATION_DOWN、ROTATION_LEFT，分别表示将显示方向旋转 0、90、180、270 度。

修改完成后保存退出，重新编译 Android 源码。

6.3.13 添加自定义模块

Android 源码非常庞大！其中包含了大量的模块，本小节向用户介绍如何在 Android 源码中添加一个自己的模块，本例以一个 C/C++ 可执行程序模块为例。

①、添加模块

我们以 vendor 目录为例，首先进入 Android 源码根目录下的 vendor 目录，并在该目录下创建一个以公司名称命名的文件夹，譬如“alientek”，如下所示：

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk$ cd vendor/
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$ ls
rockchip  widevine
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$ mkdir alientek
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$ ls
alientek  rockchip  widevine
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$
```

进入 alientek 目录，创建一个 C 或 C++ 源文件，如下所示：

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor$ cd alientek/
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ ls
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ touch my_test.c
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ ls
my_test.c
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$
```

并打开 my_test.c 源文件，编写一个测试代码，本例测试代码如下：

```
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

接下来需要在当前目录下创建一个 Android.bp 或者 Android.mk 文件，推荐使用 Android.bp。Android 使用 Android.mk 或 Android.bp 文件对 Android 源码进行模块化编译，[<6.3.3.3-模块编译>](#)小节中已经给大家介绍过，这里不再多说。

创建 Android.bp 文件，如下所示：

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ ls
my_test.c
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ touch Android.bp
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ ls
Android.bp  my_test.c
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$
```

打开 Android.bp 文件，添加如下内容：

```
cc_binary {
    name: "my_c_module",
    srcs: ["my_test.c"],
}
```

```
cc_binary {
    name: "my_c_module",
    srcs: ["my_test.c"],
}
~
~
```

cc_binary 表示该模块是一个 C/C++ 可执行程序模块，最终会编译出一个二进制可执行程序；name 属性指定模块的名字为 my_c_module；srcs 属性用于指定编译该模块所需的源文件；该文件的内容非常简单。当然除了这两个属性之外，Android.bp 文件还支持很多其它属性，关于 Android.bp 文件的语法规则，本文不做介绍，自行百度！

编写完之后保存退出，然后回到 Android 源码根目录下（直接执行 croot 命令可切换至根目录）：

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ ls
Android.bp  my_test.c
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$
tgg@tgg-virtual-machine:~/rk3568_android_sdk/vendor/alientek$ croot
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
```

②、编译模块

切换至 Android 源码根目录下，然后通过“make my_c_module”命令编译我们的模块：

```
make my_c_module -j12
```

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
tgg@tgg-virtual-machine:~/rk3568_android_sdk$ make my_c_module -j12
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-53-generic-x86_64-Ubuntu-20.04.2-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RD2A.211001.002
OUT_DIR=out
=====
10:33:14 Disallowed PATH tool "python3.8" used: [ ]string{"python3.8", ".repo/repo/repo", "manifest", "-r", "-o",
10:33:14 See https://android.googlesource.com/platform/build/+/-master/Changes.md#PATH_Tools for more information.
[100% 2/2] Install: out/target/product/rk3568_r/system/bin/my_c_module
#### build completed successfully (3 seconds) ####
```

make 后面的参数 my_c_module 便是模块名称；编译得到的可执行程序最终会被安装到 \$OUT/system/bin/my_c_module，如下所示：

```
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
tgg@tgg-virtual-machine:~/rk3568_android_sdk$ file out/target/product/rk3568_r/system/bin/my_c_module
out/target/product/rk3568_r/system/bin/my_c_module: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
dynamically linked, interpreter /system/bin/linker64, BuildID[md5/uuid]=f4feb3a70cfb230cb5d70b2392c9aee8, stripp
d
tgg@tgg-virtual-machine:~/rk3568_android_sdk$
```

6.3.14 获取 root 权限

参考 RK 提供的文档：[RKDocs/android/patches/root/Android_root.pdf](#)。

按照 android_root.pdf 文档要求修改后重新编译 Android 源码并将镜像烧录到开发板。

启动开发板进入 Android 系统，使用 adb 安装 root 检查 apk：
RKDocs/android/patches/root/RootChecker.apk。

APK 安装成功后，打开该 APP，如果提示“Root check pass”，如下图所示，说明 root 权限检查通过！



6.3.15 多屏异显异触

参考 RK 提供的文档: [RKDocs/android/patches/customized_functions/Android11 异显开发说明.zip](#)。

6.3.16 多屏异声

参考 RK 提供的文档: [RKDocs/android/patches/customized_functions/Dual_Audio_v1.0.zip](#)。

6.3.17 多屏显示

参考正点原子提供的文档: [开发板光盘 A 盘-基础资料→10、用户手册→03、辅助文档→【正点原子】ATK-DLRK3588 开发板多屏显示参考手册.pdf](#)。

6.4 Android 调试

Android 系统调试相关工具介绍。

6.4.1 ADB

参考正点原子提供的文档: [开发板光盘 A 盘-基础资料→10、用户手册→03、辅助文档→【正点原子】adb 工具使用说明.pdf](#)。

6.4.2 logcat

Android 日志系统提供了记录和查看系统调试信息的功能。日志都是从各种软件和一些系统的缓冲区中记录下来的,缓冲区可以通过 logcat 命令行工具来查看和使用,logcat 是调试程序阶段使用最多的工具,该工具主要是通过打印日志来显示程序的运行情况,由于要打印的日志量非常大,通常需要对其进行过滤等操作。

①、logcat 命令使用方式

用 logcat 命令来查看系统日志缓冲区的内容,基本用法如下所示:



```
logcat [<option>] [<filter-spec>]
```

如需通过 adb shell 运行 logcat, 则如下:

```
[adb] shell logcat [<option>] [<filter-spec>]
```

示例如下:

```
adb shell
logcat
```

直接运行 logcat 不加任何参数将会打印默认日志缓冲区 (main/system/crash) 中的所有日志信息, 信息量非常大! 通常不会这么做, 往往需要进行过滤。

②、常用的日志过滤方式

● 控制日志输出优先级, 示例如下:

```
adb shell
logcat *:W
```

示例说明: 显示优先级为 warning 或更高的日志信息。优先级是以下字母之一 (按照从最低到最高优先级的顺序排列):

- **V**: 详细 (最低优先级)
- **D**: 调试
- **I**: 信息
- **W**: 警告
- **E**: 错误
- **F**: 严重错误
- **S**: 静默 (最高优先级, 绝不会输出任何内容)

● 控制日志标签和输出优先级, 示例如下:

```
adb shell
logcat ActivityManager:I MyApp:D *:S
```

示例说明: 该表达式仅输出标签为 “ActivityManager”、优先级不低于 “信息(I)” 的日志信息, 以及标签为 “MyApp”、优先级不低于 “调试(D)” 的日志信息。除了这两种日志信息之外的所有其它信息均不会输出。

● 只输出特定标签的日志信息, 示例如下:

```
adb shell
logcat WishTV:* *:S
```

或者

```
adb shell
logcat -s WishTV
```

示例说明: 只输出标签为 “WishTV” 的日志信息。

● 只输出指定优先级和标签的日志信息, 示例如下:

```
adb shell
logcat WishTV:I *:S
```

示例说明: 只输出标签为 “WishTV”、优先级不低于 I 的日志消息。

● 查看指定缓冲区的日志信息

Android 日志信息缓冲区分为默认日志缓冲区和备用日志缓冲区，默认日志缓冲区包括：system、main 以及 crash，除了默认缓冲区之外，还包括几个备用缓冲区。

- **radio**：查看包含无线装置/电话相关消息的缓冲区。
- **events**：查看已经过解译的二进制系统事件缓冲区消息。
- **main**：查看主日志缓冲区（默认），不包含系统和崩溃日志消息。
- **system**：查看系统日志缓冲区（默认）。
- **crash**：查看崩溃日志缓冲区（默认）。
- **all**：查看所有缓冲区。
- **default**：报告 **main**、**system** 和 **crash** 缓冲区。

logcat 命令可以通过 **-b** 参数来查看指定缓冲区中的日志信息（也可以加入多个 **-b** 选项指定多个缓冲区），示例如下：

```
adb shell
logcat -b radio
```

示例说明：只查看包含无线装置和电话相关信息（radio）的日志缓冲区。

```
adb shell
logcat -b system *:W
```

示例说明：只查看系统日志缓冲区（system）中的所有优先级不低于 W 的日志信息。

③、清空日志信息

logcat 命令通过 **-c** 参数来清空日志缓冲区，也就是清空日志信息。示例如下：

```
adb shell
logcat -c
```

示例说明：清空默认缓冲区（system/main/crash）。

```
adb shell
logcat -b all -c
```

示例说明：清空所有缓冲区。

```
adb shell
logcat -b system -c
```

示例说明：清空 system 系统日志缓冲区。

6.5 Android 应用开发

参考正点原子提供的文档：[开发板光盘 A 盘-基础资料→10、用户手册→02、开发文档→【正点原子】ATK-DLRK3588 Android 应用开发。](#)

6.7 安全启动/安全机制开发

参考 RK 提供的文档：

RKDocs/common/NVM/Rockchip_Developer_Guide_OTP_CN.pdf

RKDocs/common/u-boot/Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf

<SDK>/RKDocs/common/security

RKDocs/common/security/

└── patch

- |—— RK3399_Efuse_Operation_Instructions_V1.00_EN.pdf
- |—— RK356X_SecurityBoot_And_AVB_instructions_CN.pdf
- |—— RK356X_SecurityBoot_And_AVB_instructions_EN.pdf
- |—— RK3588_SecurityBoot_And_AVB_instructions_CN.pdf
- |—— RK3588_SecurityBoot_And_AVB_instructions_EN.pdf
- |—— Rockchip_Developer_Guide_Crypto_HWRNG_CN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_CN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_EN.pdf
- |—— Rockchip_Developer_Guide_TEE_SDK_CN.pdf
- |—— Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_CN.pdf
- |—— Rockchip Vendor Storage Application Note.pdf

6.8 A/B 系统开发

参考 RK 提供的文档:

<SDK>/RKDocs/common/u-boot

- |—— Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf
- |—— Rockchip-Developer-Guide-Linux-AB-System.pdf

<SDK>/RKDocs/common/security

- |—— RK3399_Efuse_Operation_Instructions_V1.00_EN.pdf
- |—— RK356X_SecurityBoot_And_AVB_instructions_CN.pdf
- |—— RK356X_SecurityBoot_And_AVB_instructions_EN.pdf
- |—— RK3588_SecurityBoot_And_AVB_instructions_CN.pdf
- |—— RK3588_SecurityBoot_And_AVB_instructions_EN.pdf
- |—— Rockchip_Developer_Guide_Crypto_HWRNG_CN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_CN.pdf
- |—— Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_EN.pdf
- |—— Rockchip_Developer_Guide_TEE_SDK_CN.pdf
- |—— Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_CN.pdf
- |—— Rockchip Vendor Storage Application Note.pdf

6.9 系统升级开发

系统升级方案, 参考 RK 提供的文档:

<SDK>/RKDocs/common/NVM

- |—— Rockchip_Application_Notes_Storage_CN.pdf

<SDK>/RKDocs/android

- |—— Rockchip_Developer_Guide_Android_Recovery_CN.pdf